civil-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `civil_engineering_ai999x.about()` for the same answer.

Every capability civil-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import civil_engineering_ai999x

result = civil_engineering_ai999x.solve('rectangular_channel_section', b='3 m', y='1.2 m')
result.show()
# A = 3.6 m^2, P = 5.4 m, R = 0.666667 m, T = 3 m, Dh = 1.2 m
```

By topic

topic	capabilities
Foundations	Vertical stress below the centre of a uniformly loaded circle, Vertical stress below a uniformly loaded rectangle (Newmark), Vertical stress below a uniformly loaded strip, Ultimate bearing capacity of a strip footing, Rankine active earth pressure and thrust on a wall, Rankine passive earth pressure and thrust, Coulomb active thrust on a rough wall with sloping backfill, Coulomb passive thrust on a rough wall, Terzaghi bearing capacity with stated factors
Groundwater	Darcy flow across soil layers in series; equivalent conductivities, Hydraulic conductivity from a falling-head test, Transmissivity from steady pumping (Thiem, confined aquifer), Hydraulic conductivity from steady pumping (unconfined aquifer), Theis drawdown around a pumped well, Transmissivity and storativity by the Cooper-Jacob straight line, Drawdown from several wells by superposition (Theis), Unconfined flow between two water bodies with recharge (Dupuit)
Hydrology	SCS curve-number runoff depth, Direct runoff hydrograph by unit-hydrograph convolution, Unit hydrograph from a storm by least-squares deconvolution, Areal rainfall by Thiessen polygons, Risk of exceedance over a design life, Return period for an acceptable risk over a design life, Gumbel (EV1) design flood, Log-Pearson type III design flood, Return periods by the Weibull plotting position, Nash-Sutcliffe efficiency, RMSE and percent bias, Kling-Gupta efficiency, Horton infiltration rate and cumulative infiltration, Green-Ampt cumulative infiltration under ponding, Phi-index of a storm, Muskingum flood routing, Peak runoff by the rational method, Time of concentration by the Kirpich formula
Open-channel hydraulics	Rectangular channel: area, wetted perimeter, hydraulic radius, Trapezoidal channel: area, wetted perimeter, hydraulic radius, Part-full circular conduit: area, wetted perimeter, hydraulic radius, Manning velocity from hydraulic radius, Manning discharge in a rectangular channel, Manning discharge in a trapezoidal channel, Manning flow in a part-full circular pipe, Normal depth in a rectangular channel, Normal depth in a trapezoidal channel, Chezy velocity, Chezy coefficient from Manning's n, Manning's n from a Chezy coefficient, Froude number of flow in a rectangular channel, Critical depth in a rectangular channel, Critical depth in a trapezoidal channel, Critical depth in a circular conduit, Specific energy and alternate depth in a rectangular channel, Hydraulic jump in a rectangular channel, Length of a gradually varied flow profile (rectangular channel), Sharp-crested rectangular weir discharge, V-notch (triangular) weir discharge, Broad-crested weir discharge, Discharge through a small orifice under a head

topic	capabilities
Pipe flow and water networks	Hazen-Williams head loss in a pipe, Pipe flow from a head loss (Hazen-Williams), Flow division among parallel pipes (Hazen-Williams), Pipe network flows by the Hardy Cross method (Hazen-Williams), Three-reservoir problem (Hazen-Williams)
Soil mechanics	Soil phase relations from void ratio and water content, Soil properties from a weighed sample, Relative density of a granular soil, Liquid limit from Casagrande cup tests, Effective sizes and gradation coefficients from a sieve analysis, Total, pore-water and effective vertical stress at a depth, Primary consolidation settlement of a normally consolidated clay, Primary consolidation settlement of an overconsolidated clay, Consolidation settlement of a thick clay layer integrated over depth, Average degree of consolidation after a time (Terzaghi), Time to reach a degree of consolidation, Coefficient of consolidation from a consolidation time, Excess pore pressure at a depth during consolidation, Major principal stress at failure (Mohr-Coulomb), Cohesion and friction angle from triaxial tests, Cohesion and friction angle from direct shear tests, Optimum water content and maximum dry density from a compaction test, Factor of safety of an infinite slope, Plasticity, liquidity and consistency indices, Secondary compression (creep) settlement of a clay layer
Surveying	Closed traverse: misclosure and Bowditch (compass rule) adjustment, Reduced levels by the rise-and-fall method, Elements of a simple horizontal circular curve, Elevations on a vertical parabolic curve, Earthwork volume from cross-section areas, Area between a survey line and a boundary from offsets

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
rectangular_channel_section	Rectangular channel: area, wetted perimeter, hydraulic radius	b, y	A, P, R, T, Dh
trapezoidal_channel_section	Trapezoidal channel: area, wetted perimeter, hydraulic radius	b, y, z	A, P, R, T, Dh
circular_channel_section	Part-full circular conduit: area, wetted perimeter, hydraulic radius	D, y	A, P, R, T, Dh, theta
manning_velocity	Manning velocity from hydraulic radius	R, S, n	V
manning_rectangular_discharge	Manning discharge in a rectangular channel	b, y, S, n	Q, V, A, R
manning_trapezoidal_discharge	Manning discharge in a trapezoidal channel	b, y, z, S, n	Q, V, A, R
manning_partially_full_pipe	Manning flow in a part-full circular pipe	D, y, S, n	Q, V, A, R, Q_full, ratio
normal_depth_rectangular	Normal depth in a rectangular channel	Q, b, S, n	yn, V, Fr
normal_depth_trapezoidal	Normal depth in a trapezoidal channel	Q, b, z, S, n	yn, V, A, Fr
chezy_velocity	Chezy velocity	C, R, S	V
chezy_from_manning	Chezy coefficient from Manning's n	n, R	C
manning_from_chezy	Manning's n from a Chezy coefficient	C, R	n
channel_froude_number	Froude number of flow in a rectangular channel	Q, b, y	Fr, V, regime
critical_depth_rectangular	Critical depth in a rectangular channel	Q, b	yc, Vc, Emin, q
critical_depth_trapezoidal	Critical depth in a trapezoidal channel	Q, b, z	yc, Ac, Vc, Emin

id	operation	inputs	outputs
critical_depth_circular	Critical depth in a circular conduit	Q, D	yc, Ac, Vc, Emin
alternate_depth_rectangular	Specific energy and alternate depth in a rectangular channel	Q, b, y	E, y_alt, yc, Fr, regime
hydraulic_jump_rectangular	Hydraulic jump in a rectangular channel	Q, b, y1	y2, Fr1, Fr2, dE, power_loss, ratio
gradually_varied_flow_length	Length of a gradually varied flow profile (rectangular channel)	Q, b, S0, n, y1, y2	L, yn, yc, profile
rectangular_weir_discharge	Sharp-crested rectangular weir discharge	Cd, b, H, contractions	Q, L_eff
v_notch_weir_discharge	V-notch (triangular) weir discharge	Cd, theta, H	Q
broad_crested_weir_discharge	Broad-crested weir discharge	Cd, b, H	Q, yc
orifice_discharge	Discharge through a small orifice under a head	Cd, d, h	Q, V_ideal, area
hazen_williams_head_loss	Hazen-Williams head loss in a pipe	Q, D, L, C	hf, V, Sf
hazen_williams_flow	Pipe flow from a head loss (Hazen-Williams)	hf, D, L, C	Q, V
parallel_pipes_hazen_williams	Flow division among parallel pipes (Hazen-Williams)	Q, L, D, C	hf, flows
hardy_cross_network	Pipe network flows by the Hardy Cross method (Hazen-Williams)	pipes, L, D, C, demands	flows, head_loss, loops, iterations
three_reservoir_problem	Three-reservoir problem (Hazen-Williams)	z, L, D, C	h_junction, flows
scs_curve_number_runoff	SCS curve-number runoff depth	P, CN, lam	Q, S, Ia, C
unit_hydrograph_convolution	Direct runoff hydrograph by unit-hydrograph convolution	excess, uh, baseflow	DRO, total, peak, peak_period
unit_hydrograph_derivation	Unit hydrograph from a storm by least-squares deconvolution	dro, excess	uh, rms_residual, peak
thiessen_polygon_rainfall	Areal rainfall by Thiessen polygons	gx, gy, rain, bx, by	mean, areas, weights, arithmetic_mean, catchment_area
flood_exceedance_risk	Risk of exceedance over a design life	T, n	risk, p_annual, p_none
design_return_period_for_risk	Return period for an acceptable risk over a design life	R, n	T, p_annual
gumbel_design_flood	Gumbel (EV1) design flood	peaks, T	Q_T, K_T, mean, std
log_pearson_iii_design_flood	Log-Pearson type III design flood	peaks, T	Q_T, K_T, mean_log, std_log, skew
weibull_plotting_positions	Return periods by the Weibull plotting position	peaks	ranked, T, P

id	operation	inputs	outputs
nash_sutcliffe_efficiency	Nash-Sutcliffe efficiency, RMSE and percent bias	obs, sim	NSE, RMSE, PBIAS
kling_gupta_efficiency	Kling-Gupta efficiency	obs, sim	KGE, r, alpha, beta
horton_infiltration	Horton infiltration rate and cumulative infiltration	f0, fc, k, t	f, F, f_mm_h, F_mm
green_ampt_infiltration	Green-Ampt cumulative infiltration under ponding	K, psi, dtheta, t	F, f, F_mm, f_mm_h
phi_index	Phi-index of a storm	rain, dt, R	phi, excess, contributing
muskingum_routing	Muskingum flood routing	inflow, K, X, dt, O1	outflow, C0, C1, C2, peak_outflow, attenuation
darcy_flow_layered_soil	Darcy flow across soil layers in series; equivalent conductivities	H, K, A, dh	Q, q, Kv, Kh, heads
falling_head_permeability	Hydraulic conductivity from a falling-head test	d, D, L, h1, h2, t	K, K_cm_s
thiem_confined_transmissivity	Transmissivity from steady pumping (Thiem, confined aquifer)	Q, r1, r2, s1, s2, b	T, T_m2_d, K
thiem_unconfined_conductivity	Hydraulic conductivity from steady pumping (unconfined aquifer)	Q, r1, r2, h1, h2	K, K_m_d
theis_drawdown	Theis drawdown around a pumped well	Q, T, S, r, t	s, u, W
cooper_jacob_analysis	Transmissivity and storativity by the Cooper-Jacob straight line	times, drawdowns, Q, r	T, S, ds_per_cycle, t0_min, T_m2_d
well_field_drawdown	Drawdown from several wells by superposition (Theis)	wx, wy, Qs, T, S, t, x0, y0	s, parts
dupuit_flow_with_recharge	Unconfined flow between two water bodies with recharge (Dupuit)	K, h1, h2, L, W	q0, qL, x_divide, h_max
soil_phase_relations	Soil phase relations from void ratio and water content	Gs, e, w	n, S, gamma_d, gamma, gamma_sat, gamma_sub
soil_phase_from_lab_masses	Soil properties from a weighed sample	M, Md, V, Gs	w, rho, rho_d, e, n, S, gamma, gamma_d
relative_density_from_unit_weights	Relative density of a granular soil	gd, gd_min, gd_max	Dr, Dr_percent
liquid_limit_flow_curve	Liquid limit from Casagrande cup tests	blows, w, PL	LL, flow_index, PI
grain_size_coefficients	Effective sizes and gradation coefficients from a sieve analysis	sizes, passing	D10, D30, D60, Cu, Cc
effective_stress_at_depth	Total, pore-water and effective vertical stress at a depth	H, gam, zw, z	sigma, u, sigma_eff, sigma_eff_kPa
consolidation_settlement_nc	Primary consolidation settlement of a normally consolidated clay	Cc, e0, H, s0, ds	Sc, de, e_final

id	operation	inputs	outputs
consolidation_settlement_oc	Primary consolidation settlement of an overconsolidated clay	Cc, Cs, e0, H, s0, ds, sp	Sc, de, OCR, case
thick_clay_layer_settlement	Consolidation settlement of a thick clay layer integrated over depth	H, s_top, gsub, ds, Cc, e0	S, S_one_layer
degree_of_consolidation	Average degree of consolidation after a time (Terzaghi)	cv, t, Hdr, Sc	Tv, U, U_percent, St
consolidation_time	Time to reach a degree of consolidation	cv, Hdr, U	Tv, t, t_days
coefficient_of_consolidation	Coefficient of consolidation from a consolidation time	Hdr, t, U	cv, Tv, cv_m2_yr
excess_pore_pressure_isochrone	Excess pore pressure at a depth during consolidation	u0, cv, t, Hdr, z	u, Uz, Tv
stress_below_circular_load	Vertical stress below the centre of a uniformly loaded circle	q, a, z	dsigma, I
stress_below_rectangular_load	Vertical stress below a uniformly loaded rectangle (Newmark)	q, B, L, z, position	dsigma, I
stress_below_strip_load	Vertical stress below a uniformly loaded strip	q, B, z, x	dsigma, I
strip_footing_bearing_capacity	Ultimate bearing capacity of a strip footing	c, phi, gam, B, Df, FS	qu, qa, Nc, Nq, Ngamma
rankine_active_thrust	Rankine active earth pressure and thrust on a wall	phi, gam, H, c, q	Ka, sigma_base, Pa, z_crack, ybar
rankine_passive_thrust	Rankine passive earth pressure and thrust	phi, gam, H, c, q	Kp, sigma_base, Pp, ybar
coulomb_active_thrust	Coulomb active thrust on a rough wall with sloping backfill	phi, delta, beta, gam, H	Ka, Pa, Pa_h, Pa_v
coulomb_passive_thrust	Coulomb passive thrust on a rough wall	phi, delta, beta, gam, H	Kp, Pp, Pp_h, Pp_v
mohr_coulomb_failure_stress	Major principal stress at failure (Mohr-Coulomb)	s3, c, phi	s1, sigma_f, tau_f, theta
triaxial_strength_parameters	Cohesion and friction angle from triaxial tests	s3, s1	phi, c, alpha
direct_shear_strength_parameters	Cohesion and friction angle from direct shear tests	sigma, tau	phi, c, tan_phi
proctor_compaction_optimum	Optimum water content and maximum dry density from a compaction test	w, rho_d, Gs	w_opt, rho_dmax, rho_zav
traverse_bowditch_adjustment	Closed traverse: misclosure and Bowditch (compass rule) adjustment	az, d, E0, N0	misclosure_E, misclosure_N, linear_misclosure, precision, E, N, area
levelling_rise_and_fall	Reduced levels by the rise-and-fall method	readings, sights, RL0	RL, rise, fall, last_RL, check

id	operation	inputs	outputs
horizontal_circular_curve	Elements of a simple horizontal circular curve	R, Delta, PI	T, L, LC, E, M, PC, PT
vertical_parabolic_curve	Elevations on a vertical parabolic curve	g1, g2, L, PVI, elev_PVI, x	elev, x_turn, elev_turn, r, BVC, elev_BVC
earthwork_volume_end_areas	Earthwork volume from cross-section areas	areas, d	V_end_area, V_prismoidal
area_from_offsets	Area between a survey line and a boundary from offsets	offsets, d	A_trapezoidal, A_simpson
rational_method_peak_runoff	Peak runoff by the rational method	C, i, A	Q, Q_Ls
kirpich_time_of_concentration	Time of concentration by the Kirpich formula	L, S	tc, tc_min, tc_h
infinite_slope_factor_of_safety	Factor of safety of an infinite slope	c, phi, gam, z, beta, m	FS, tau, sigma_eff
terzaghi_bearing_capacity	Terzaghi bearing capacity with stated factors	c, gam, B, Df, Nc, Nq, Ngamma, shape, FS	qu, qa, q_net
plasticity_liquidity_index	Plasticity, liquidity and consistency indices	LL, PL, w	PI, A_line_PI, LI, CI
secondary_compression_settlement	Secondary compression (creep) settlement of a clay layer	Ca, H, ep, t1, t2	Ss, Ss_mm, C_alpha_mod

Rectangular channel: area, wetted perimeter, hydraulic radius, `rectangular_channel_section`

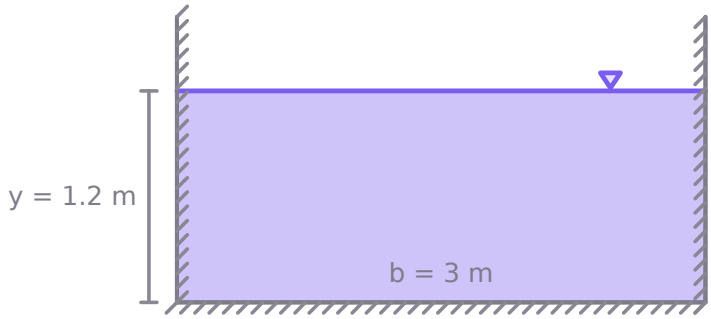
Flow area, wetted perimeter, hydraulic radius, top width and hydraulic depth of a rectangular channel flowing at depth y.

$$A = b\,y, \quad P = b + 2y, \quad R = \frac{A}{P}$$

input	meaning	condition
`b`	bottom width	
`y`	flow depth	

output	meaning
`A`	flow area (m^2)
`P`	wetted perimeter (m)
`R`	hydraulic radius A/P (m)
`T`	top width (m)
`Dh`	hydraulic depth A/T (m)

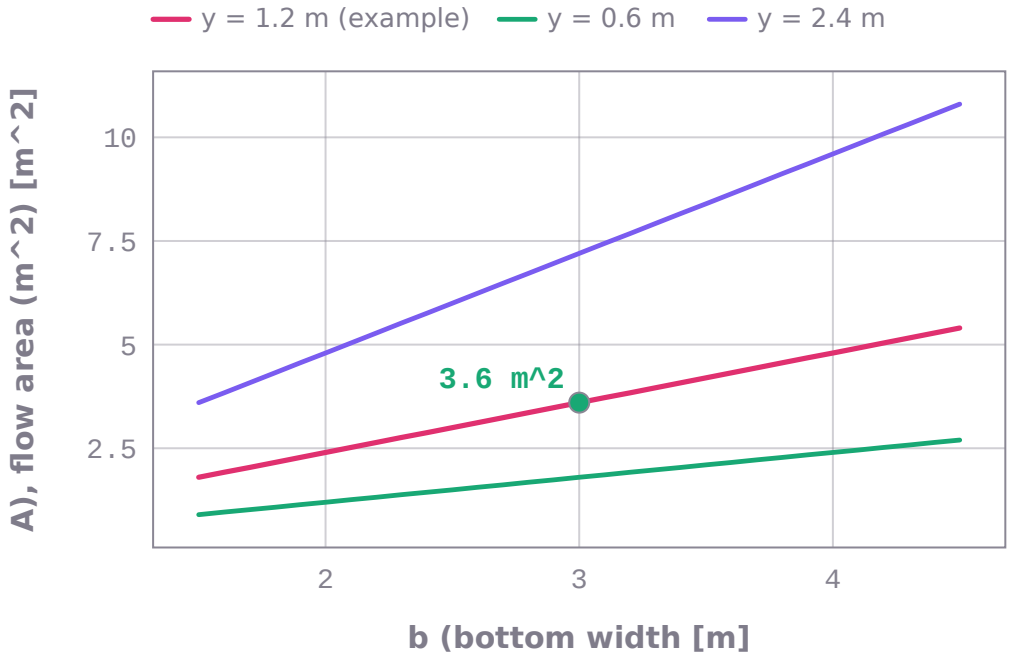
- **Method:** $A = b$ y, $P = b + 2y$, $R = A/P$, $T = b$, $D = A/T$
- **Assumes:** $b > 0$; $y > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A = 3.6 m²
P = 5.4 m
R = 0.6667 m

T = 3 m
D_h = 1.2 m

Rectangular channel (b = 3.0 m, y = 1.2 m) drawn to scale with its water. Marked: the returned A = 3.6 m², P = 5.4 m, R = 0.6667 m, T = 3 m, D_h = 1.2 m.



A (flow area (m^2)) against b (bottom width) from 1.5 m to 4.5 m. One curve for each value of y (flow depth): 1.2 m, 0.6 m, 2.4 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, b = 3 m, A = 3.6 m^2.

```
civil_engineering_ai999x.solve('rectangular_channel_section', b=3.0, y=1.2)
# -> A = 3.6 m^2, P = 5.4 m, R = 0.666667 m, T = 3 m, Dh = 1.2 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('rectangular_channel_section', b=0.6, y=0.25)
# -> A = 0.15 m^2, P = 1.1 m, R = 0.136364 m, T = 0.6 m, Dh = 0.25 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rectangular channel section with width = 3 m and depth = 1.2 m.	SOLVED	A = 3.6 m^2, P = 5.4 m, R = 0.666667 m, T = 3 m, Dh = 1.2 m

question	outcome	result
Rectangular channel section with width = 0.6 m and depth = 0.25 m.	SOLVED	A = 0.15 m^2, P = 1.1 m, R = 0.136364 m, T = 0.6 m, Dh = 0.25 m
Rectangular channel section with width = 3 m.	NEEDS_INPUT	rectangular_channel_section needs y (flow depth). Example: solve('rectangular_channel_section', b='3 m', y='1.2 m').
Rectangular channel section with width = 3 m and depth = 1.2 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Rectangular channel section with width = 3 m and depth = -1 m.	INVALID_INPUT	Rectangular channel: area, wetted perimeter, hydraulic radius: y must be positive.

Trapezoidal channel: area, wetted perimeter, hydraulic radius, `trapezoidal_channel_section`

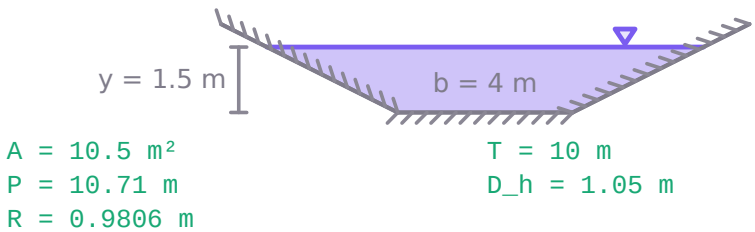
Flow area, wetted perimeter, hydraulic radius, top width and hydraulic depth of a trapezoidal channel with bottom width b and side slopes z (horizontal : vertical); $b = 0$ is a triangular channel.

$$A = (b + zy)\,y, \quad P = b + 2y\sqrt{1 + z^2}, \quad T = b + 2zy$$

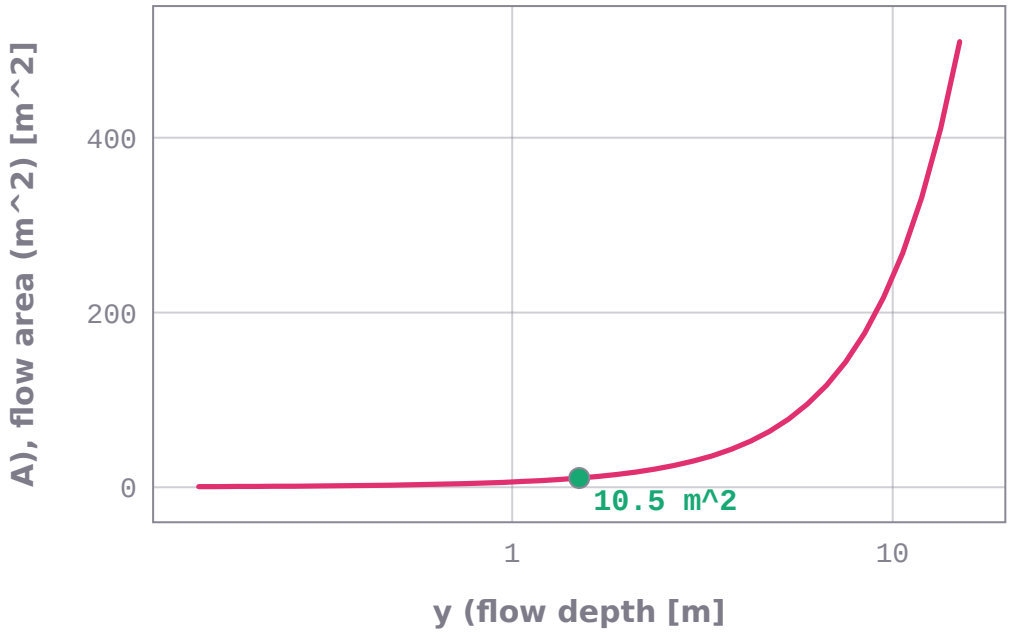
input	meaning	condition
`b`	bottom width	
`y`	flow depth	
`z`	side slope, horizontal per unit vertical	

output	meaning
`A`	flow area (m^2)
`P`	wetted perimeter (m)
`R`	hydraulic radius A/P (m)
`T`	top width (m)
`Dh`	hydraulic depth A/T (m)

- **Method:** A = (b + z y) y, $P = b + 2$ y sqrt(1 + z^2), $T = b + 2$ z y
- **Assumes:** $y > 0$; $b \geq 0$; $z \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Trapezoidal channel (b = 4.0 m, y = 1.5 m, z = 2.0 (H:1V)) drawn to scale with its water. Marked: the returned A = 10.5 m², P = 10.71 m, R = 0.9806 m, T = 10 m, D_h = 1.05 m.



A (flow area (m^2)) against y (flow depth) from 0.15 m to 15 m on a logarithmic axis. Other inputs as in the worked example: b = 4.0 m, z = 2.0. 41 points computed by civil-engineering-ai999x; marked: the worked example, y = 1.5 m, A = 10.5 m^2.

```
civil_engineering_ai999x.solve('trapezoidal_channel_section', b=4.0, y=1.5, z=2.0)
# -> A = 10.5 m^2, P = 10.7082 m, R = 0.980557 m, T = 10 m, Dh = 1.05 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('trapezoidal_channel_section', b=0.0, y=0.8, z=1.0)
# -> A = 0.64 m^2, P = 2.26274 m, R = 0.282843 m, T = 1.6 m, Dh = 0.4 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Trapezoidal channel section with width = 4 m, depth = 1.5 m and side slope = 2.	SOLVED	A = 10.5 m^2, P = 10.7082 m, R = 0.980557 m, T = 10 m, Dh = 1.05 m
Trapezoidal channel section with width = 0 m, depth = 0.8 m and side slope = 1.	SOLVED	A = 0.64 m^2, P = 2.26274 m, R = 0.282843 m, T = 1.6 m, Dh = 0.4 m
Trapezoidal channel section with width = 4 m and depth = 1.5 m.	NEEDS_INPUT	trapezoidal_channel_section needs z (side slope, horizontal per unit vertical). Example: solve('trapezoidal_channel_sect
Trapezoidal channel section with width = 4 m, depth = 1.5 m and side slope = 2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Trapezoidal channel section with width = 4 m, depth = 1.5 m and side slope = -1.	INVALID_INPUT	Trapezoidal channel: area, wetted perimeter, hydraulic radius: need b >= 0, z >= 0 and not both zero.

Part-full circular conduit: area, wetted perimeter, hydraulic radius,

`circular_channel_section`

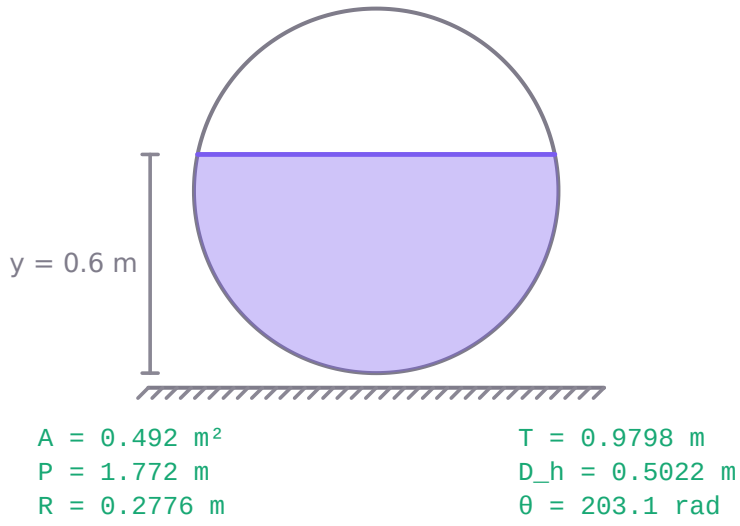
Flow area, wetted perimeter, hydraulic radius, top width and hydraulic depth of a circular pipe or culvert flowing part full at depth y, with the central angle of the wetted arc.

$$\theta = 2\cos^{-1}\left(1 - \frac{2y}{D}\right), \quad A = \frac{D^2}{8}(\theta - \sin\theta), \quad P = \frac{D\theta}{2}$$

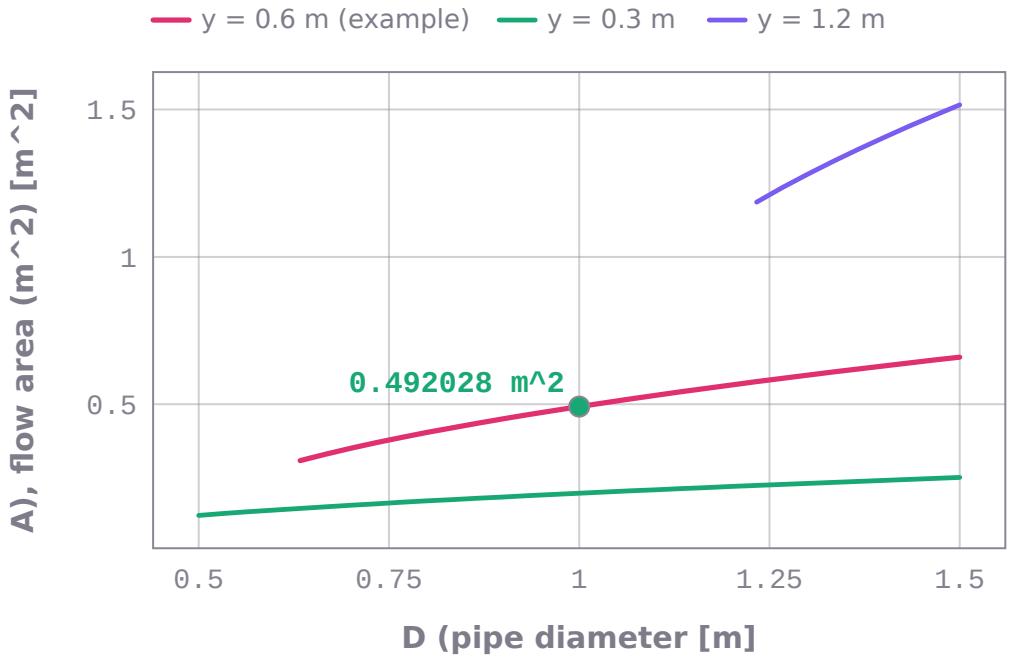
input	meaning	condition
`D`	pipe diameter	
`y`	flow depth	

output	meaning
`A`	flow area (m^2)
`P`	wetted perimeter (m)
`R`	hydraulic radius A/P (m)
`T`	top width (m)
`Dh`	hydraulic depth A/T (m)
`theta`	central angle of the wetted arc (degrees)

- **Method:** $\theta = 2$ acos(1 - 2y/D), $A = D^2$ (theta - sin theta)/8, $P = D\,\theta/2$, $T = D$ sin(theta/2)
- **Assumes:** $0 < y < D$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Circular conduit flowing part full (D = 1.0 m, y = 0.6 m), to scale. Marked: the returned A = 0.492 m², P = 1.772 m, R = 0.2776 m, T = 0.9798 m, D_h = 0.5022 m, θ = 203.1 rad.



A (flow area (m^2)) against D (pipe diameter) from 0.5 m to 1.5 m. One curve for each value of y (flow depth): 0.6 m, 0.3 m, 1.2 m. 27 points per curve computed by civil-engineering-ai999x; marked: the worked example, D = 1 m, A = 0.492028 m^2.

```
civil_engineering_ai999x.solve('circular_channel_section', D=1.0, y=0.6)
# -> A = 0.492028 m^2, P = 1.77215 m, R = 0.277644 m, T = 0.979796 m, Dh = 0.502174 m,
theta = 203.074   (SOLVED)
```

```
civil_engineering_ai999x.solve('circular_channel_section', D=0.45, y=0.1)
# -> A = 0.0263165 m^2, P = 0.441794 m, R = 0.0595673 m, T = 0.374166 m, Dh = 0.0703338 m,
theta = 112.502   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Circular channel section with diameter = 1 m and depth = 0.6 m.	SOLVED	A = 0.492028 m^2, P = 1.77215 m, R = 0.277644 m, T = 0.979796 m, Dh = 0.502174 m, theta = 203.074
Circular channel section with diameter = 0.45 m and depth = 0.1 m.	SOLVED	A = 0.0263165 m^2, P = 0.441794 m, R = 0.0595673 m, T = 0.374166 m, Dh = 0.0703338 m, theta = 112.502
Circular channel section with diameter = 1 m.	NEEDS_INPUT	circular_channel_section needs y (flow depth). Example: solve('circular_channel_section', D='1 m', y='0.6 m').
Circular channel section with diameter = 1 m and depth = 0.6 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Circular channel section with diameter = 1 m and depth = 1.2 m.	INVALID_INPUT	Part-full circular conduit: area, wetted perimeter, hydraulic radius: the depth must be below the diameter (a part-full

Manning velocity from hydraulic radius, `manning_velocity`

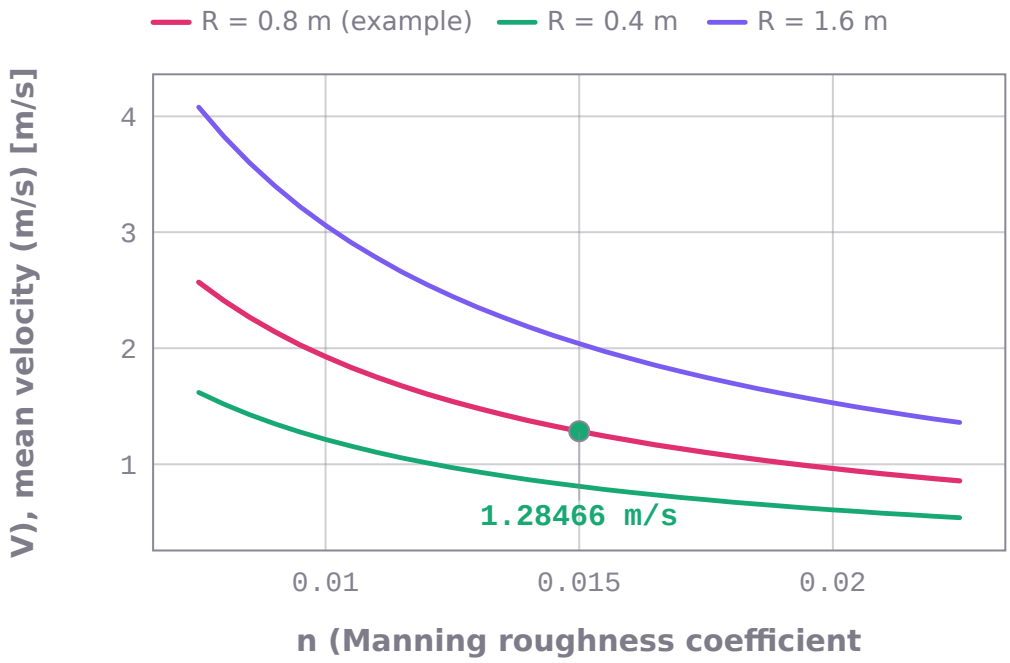
Mean velocity of uniform flow from Manning's equation (SI) for a hydraulic radius R, slope S and roughness n.

$$V = \frac{1}{n} R^{2/3} S^{1/2}$$

input	meaning	condition
R	hydraulic radius	
S	channel (energy) slope, m/m	
n	Manning roughness coefficient	

output	meaning
V	mean velocity (m/s)

- **Method:** $V = R^{2/3}\, S^{1/2} / n$
- **Assumes:** $R > 0$; $0 < S < 1$; $0.005 \le n \le 0.2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (mean velocity (m/s)) against n (Manning roughness coefficient) from 0.0075 to 0.0225. One curve for each value of R (hydraulic radius): 0.8 m, 0.4 m, 1.6 m. Other inputs as in the worked example: S = 0.0005. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, n = 0.015, V = 1.28466 m/s.

```
civil_engineering_ai999x.solve('manning_velocity', R=0.8, S=0.0005, n=0.015)
# -> V = 1.28466 m/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('manning_velocity', R=2.5, S=0.0002, n=0.035)
# -> V = 0.744287 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Manning velocity with hydraulic radius = 0.8 m, slope = 0.0005 and Manning n = 0.015.	SOLVED	V = 1.28466 m/s

question	outcome	result
Manning velocity with hydraulic radius = 2.5 m, slope = 0.0002 and Manning n = 0.035.	SOLVED	V = 0.744287 m/s
Manning velocity with hydraulic radius = 0.8 m and slope = 0.0005.	NEEDS_INPUT	manning_velocity needs n (Manning roughness coefficient). Example: solve('manning_velocity', R='0.8 m', S=0.0005, n=0.01
Manning velocity with hydraulic radius = 0.8 m, slope = 0.0005 and Manning n = 0.015 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Manning velocity with hydraulic radius = 0.8 m, slope = 0.0005 and Manning n = 0.5.	INVALID_INPUT	Manning velocity from hydraulic radius: Manning's n must lie between 0.005 and 0.2.

Manning discharge in a rectangular channel, `manning_rectangular_discharge`

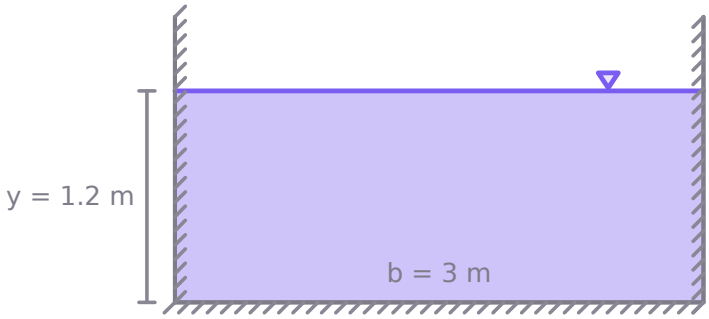
Uniform-flow discharge and velocity in a rectangular channel from Manning's equation.

$$Q = \frac{1}{n} A R^{2/3} S^{1/2}, \quad A = by,\ R = \frac{by}{b + 2y}$$

input	meaning	condition
b	bottom width	
y	flow depth	
S	bed slope, m/m	
n	Manning roughness coefficient	

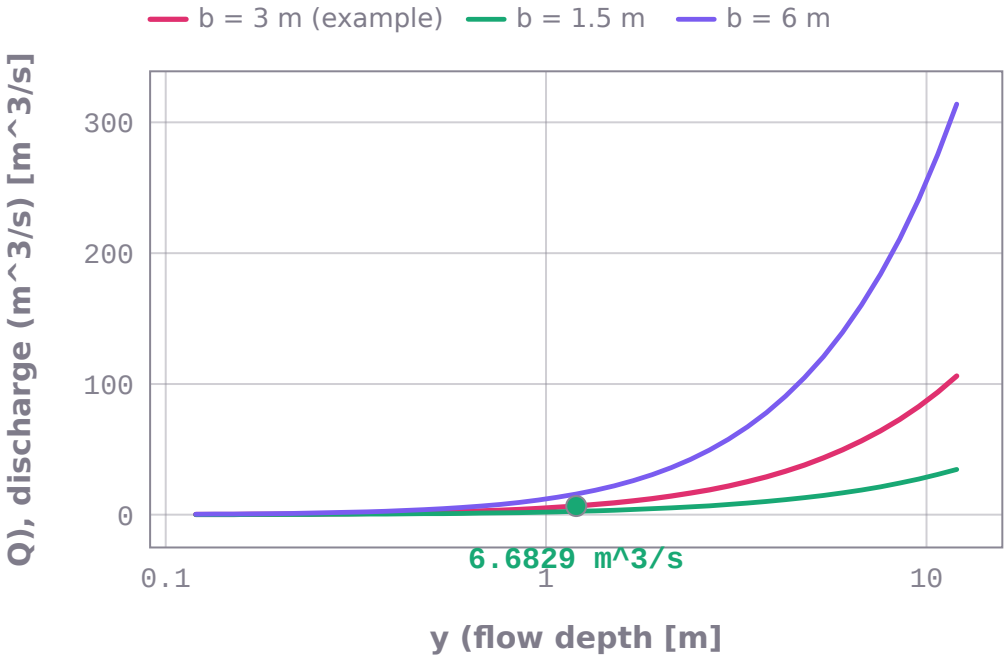
output	meaning
Q	discharge (m^3/s)
V	mean velocity (m/s)
A	flow area (m^2)
R	hydraulic radius (m)

- **Method:** $A = b$ y, $R = A/(b + 2y)$, $Q = A\, R^{2/3}\, S^{1/2}/n$
- **Assumes:** b, $y > 0$; $0 < S < 1$; $0.005 \le n \le 0.2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q = 6.683 m³/s
V = 1.856 m/s
A = 3.6 m²
R = 0.6667 m

Uniform flow in a rectangular channel (b = 3.0 m, y = 1.2 m, S = 0.001, n = 0.013), section to scale. Marked: the returned Q = 6.683 m³/s, V = 1.856 m/s, A = 3.6 m², R = 0.6667 m (Manning).



Q (discharge (m^3/s)) against y (flow depth) from 0.12 m to 12 m on a logarithmic axis. One curve for each value of b (bottom width): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: S = 0.001, n = 0.013. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, y = 1.2 m, Q = 6.6829 m^3/s.

```
civil_engineering_ai999x.solve('manning_rectangular_discharge', b=3.0, y=1.2, S=0.001,
n=0.013)
# -> Q = 6.6829 m^3/s, V = 1.85636 m/s, A = 3.6 m^2, R = 0.666667 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('manning_rectangular_discharge', b=6.0, y=2.0, S=0.0004,
n=0.03)
# -> Q = 9.03395 m^3/s, V = 0.752829 m/s, A = 12 m^2, R = 1.2 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Manning discharge in a rectangular channel with width = 3	SOLVED	Q = 6.6829 m^3/s, V = 1.85636 m/s, A = 3.6 m^2, R = 0.666667 m

question	outcome	result
m, depth = 1.2 m, slope = 0.001 and Manning n = 0.013.		
Manning discharge in a rectangular channel with width = 6 m, depth = 2 m, slope = 0.0004 and Manning n = 0.03.	SOLVED	Q = 9.03395 m^3/s, V = 0.752829 m/s, A = 12 m^2, R = 1.2 m
Manning discharge in a rectangular channel with width = 3 m, depth = 1.2 m and slope = 0.001.	NEEDS_INPUT	manning_rectangular_discharge needs n (Manning roughness coefficient). Example: solve('manning_rectangular_discharge', b
Manning discharge in a rectangular channel with width = 3 m, depth = 1.2 m, slope = 0.001 and Manning n = 0.013 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Manning discharge in a trapezoidal channel, `manning_trapezoidal_discharge`

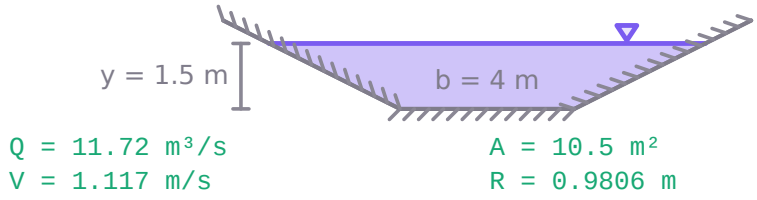
Uniform-flow discharge and velocity in a trapezoidal channel (side slopes z H:1V) from Manning's equation.

$$Q = \frac{1}{n} A R^{2/3} S^{1/2}, \quad A = (b + z y) y,\ P = b + 2y\sqrt{1 + z^2}$$

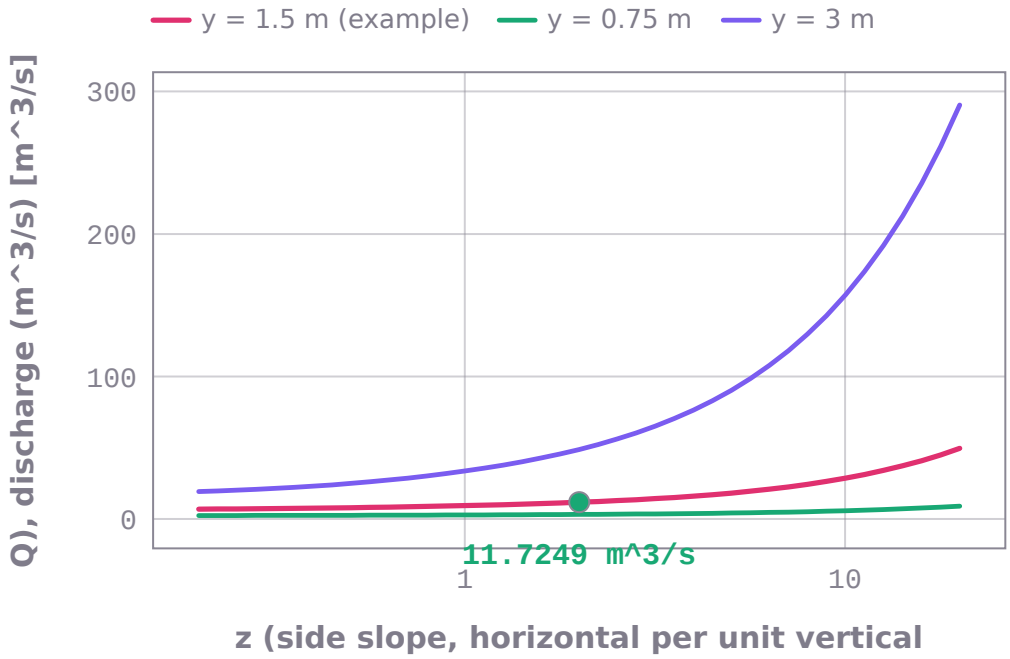
input	meaning	condition
b	bottom width	
y	flow depth	
z	side slope, horizontal per unit vertical	
S	bed slope, m/m	
n	Manning roughness coefficient	

output	meaning
Q	discharge (m^3/s)
V	mean velocity (m/s)
A	flow area (m^2)
R	hydraulic radius (m)

- **Method:** A = (b + z y) y, R = A/(b + 2 y sqrt(1 + z^2)), $Q = A\,R^{2/3}\,S^{1/2}/n$
- **Assumes:** $y > 0$; $b \geq 0$; $z \geq 0$; $0 < S < 1$; $0.005 \leq n \leq 0.2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Uniform flow in a trapezoidal channel (b = 4.0 m, y = 1.5 m, z = 2.0 (H:1V), S = 0.0008, n = 0.025), section to scale. Marked: the returned Q = 11.72 m³/s, V = 1.117 m/s, A = 10.5 m², R = 0.9806 m (Manning).



Q (discharge (m^3/s)) against z (side slope, horizontal per unit vertical) from 0.2 to 20 on a logarithmic axis. One curve for each value of y (flow depth): 1.5 m, 0.75 m, 3 m. Other inputs as in the worked example: b = 4.0 m, S = 0.0008, n = 0.025. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, z = 2, Q = 11.7249 m^3/s.

```
civil_engineering_ai999x.solve('manning_trapezoidal_discharge', b=4.0, y=1.5, z=2.0,
S=0.0008, n=0.025)
# -> Q = 11.7249 m^3/s, V = 1.11666 m/s, A = 10.5 m^2, R = 0.980557 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('manning_trapezoidal_discharge', b=2.0, y=0.9, z=1.5,
S=0.002, n=0.016)
# -> Q = 5.82609 m^3/s, V = 1.93237 m/s, A = 3.015 m^2, R = 0.574834 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Manning discharge in a trapezoidal channel with width = 4 m, depth = 1.5 m, side slope = 2, slope = 0.0008 and Manning n = 0.025.	SOLVED	Q = 11.7249 m^3/s, V = 1.11666 m/s, A = 10.5 m^2, R = 0.980557 m
Manning discharge in a trapezoidal channel with width = 2 m, depth = 0.9 m, side slope = 1.5, slope = 0.002 and Manning n = 0.016.	SOLVED	Q = 5.82609 m^3/s, V = 1.93237 m/s, A = 3.015 m^2, R = 0.574834 m

question	outcome	result
Manning discharge in a trapezoidal channel with width = 4 m, depth = 1.5 m, side slope = 2 and slope = 0.0008.	NEEDS_INPUT	manning_trapezoidal_discharge needs n (Manning roughness coefficient). Example: solve('manning_trapezoidal_discharge', b
Manning discharge in a trapezoidal channel with width = 4 m, depth = 1.5 m, side slope = 2, slope = 0.0008 and Manning n = 0.025 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Manning flow in a part-full circular pipe, `manning_partially_full_pipe`

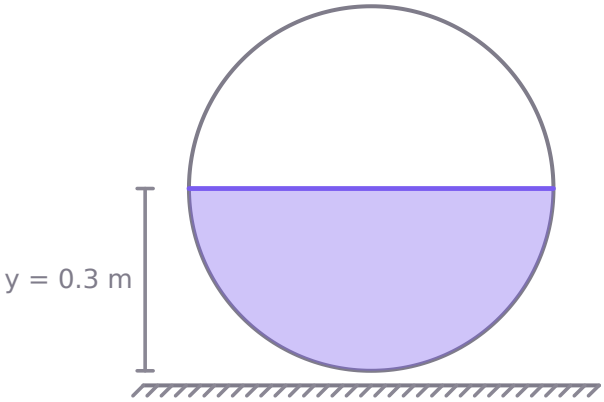
Gravity (open-channel) discharge and velocity in a circular sewer or culvert flowing at depth y, from Manning's equation, with the full-bore capacity and the ratio Q/Q_full (n taken as constant with depth).

$$Q = \frac{1}{n} A R^{2/3} S^{1/2}, \quad Q_{full} = \frac{1}{n} \frac{\pi D^2}{4} \left(\frac{D}{4}\right)^{2/3} S^{1/2}$$

input	meaning	condition
`D`	pipe diameter	
`y`	flow depth	
`S`	pipe slope, m/m	
`n`	Manning roughness coefficient	

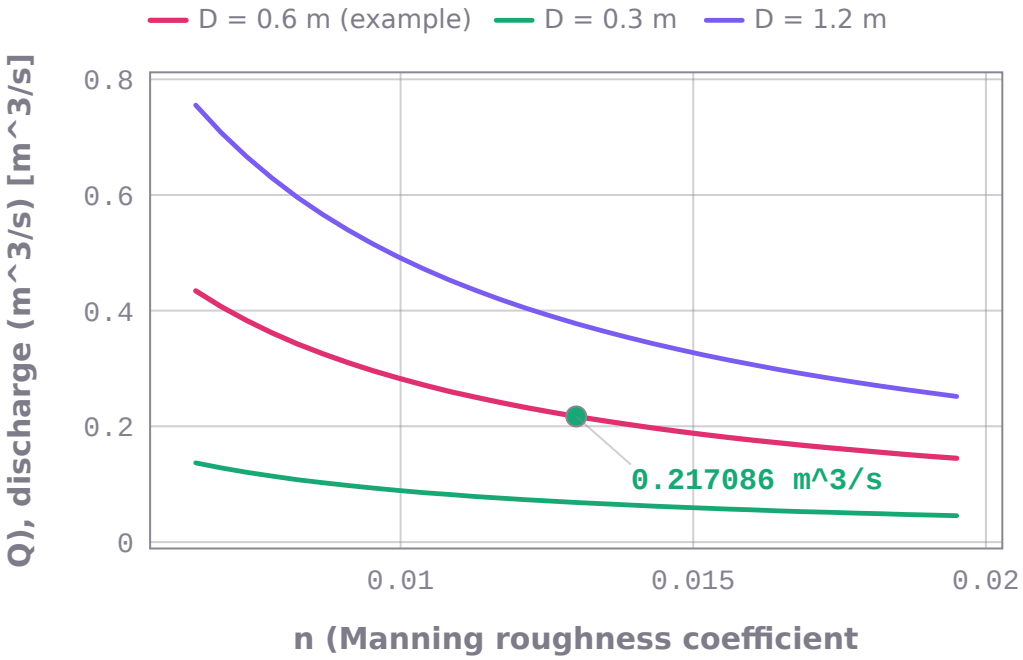
output	meaning
`Q`	discharge (m^3/s)
`V`	mean velocity (m/s)
`A`	flow area (m^2)
`R`	hydraulic radius (m)
`Q_full`	full-bore capacity (m^3/s)
`ratio`	Q/Q_full

- **Method:** circular-segment geometry, $Q = A\,R^{2/3}\,S^{1/2}/n$; Q_full with $R = D/4$
- **Assumes:** $0 < y \le D$; $0 < S < 1$; n constant with depth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q = 0.2171 m³/s
V = 1.536 m/s
A = 0.1414 m²
R = 0.15 m
Q full = 0.4342 m³/s
Q/Q full = 0.5

A pipe flowing part full (D = 0.6 m, y = 0.3 m, S = 0.005, n = 0.013), section to scale. Marked: the returned Q = 0.2171 m³/s, V = 1.536 m/s, A = 0.1414 m², R = 0.15 m, Q full = 0.4342 m³/s, Q/Q full = 0.5 (Manning).



Q (discharge (m^3/s)) against n (Manning roughness coefficient) from 0.0065 to 0.0195. One curve for each value of D (pipe diameter): 0.6 m, 0.3 m, 1.2 m. Other inputs as in the worked example: y = 0.3 m, S = 0.005. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, n = 0.013, Q = 0.217086 m^3/s.

```
civil_engineering_ai999x.solve('manning_partially_full_pipe', D=0.6, y=0.3, S=0.005,
n=0.013)
# -> Q = 0.217086 m^3/s, V = 1.53557 m/s, A = 0.141372 m^2, R = 0.15 m, Q_full = 0.434172
m^3/s, ratio = 0.5   (SOLVED)
```

```
civil_engineering_ai999x.solve('manning_partially_full_pipe', D=1.2, y=0.9, S=0.001,
n=0.015)
# -> Q = 0.974343 m^3/s, V = 1.07086 m/s, A = 0.909867 m^2, R = 0.362025 m, Q_full = 1.0685
m^3/s, ratio = 0.911878   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Manning flow in a partially full pipe with diameter = 0.6 m, depth = 0.3 m, slope = 0.005 and Manning n = 0.013.	SOLVED	Q = 0.217086 m^3/s, V = 1.53557 m/s, A = 0.141372 m^2, R = 0.15 m, Q_full = 0.434172 m^3/s, ratio = 0.5
Manning flow in a partially full pipe with diameter = 1.2 m, depth = 0.9 m, slope = 0.001 and Manning n = 0.015.	SOLVED	Q = 0.974343 m^3/s, V = 1.07086 m/s, A = 0.909867 m^2, R = 0.362025 m, Q_full = 1.0685 m^3/s, ratio = 0.911878
Manning flow in a partially full pipe with diameter = 0.6 m, depth = 0.3 m and slope = 0.005.	NEEDS_INPUT	manning_partially_full_pipe needs n (Manning roughness coefficient). Example: solve('manning_partially_full_pipe', D='0.
Manning flow in a partially full pipe with diameter = 0.6 m, depth = 0.3 m, slope = 0.005 and Manning n = 0.013 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Manning flow in a partially full pipe with diameter = 0.6 m, depth = 0.8 m, slope = 0.005 and Manning n = 0.013.	INVALID_INPUT	Manning flow in a part-full circular pipe: the depth cannot exceed the diameter.

Normal depth in a rectangular channel, `normal_depth_rectangular`

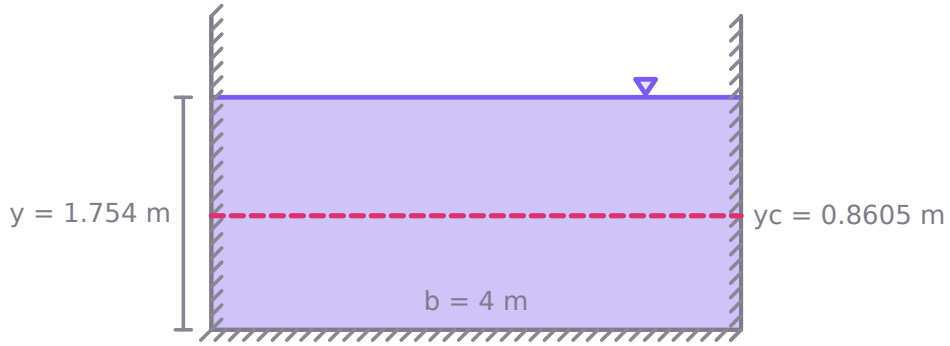
The uniform-flow (normal) depth that carries a discharge Q in a rectangular channel (Manning), with the velocity and Froude number at that depth.

$$Q = \frac{1}{n}\, by\left(\frac{by}{b+2y}\right)^{2/3} S^{1/2}$$

input	meaning	condition
`Q`	discharge	
`b`	bottom width	
`S`	bed slope, m/m	
`n`	Manning roughness coefficient	

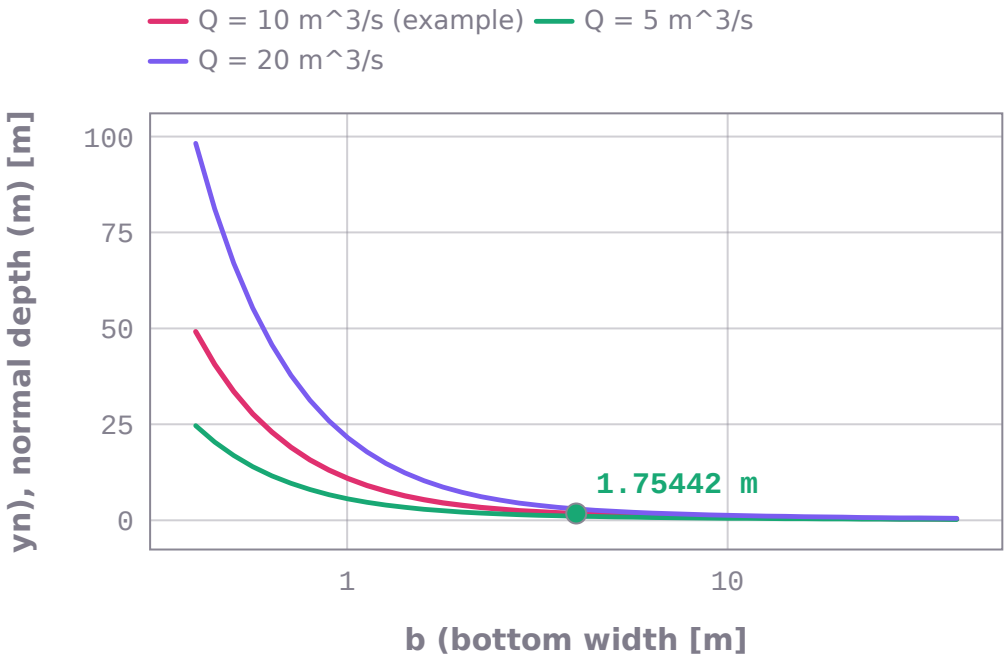
output	meaning
`yn`	normal depth (m)
`V`	velocity at normal depth (m/s)
`Fr`	Froude number at normal depth

- **Method:** bisection on Manning's Q(y) = Q (Q increases with y)
- **Assumes:** Q, $b > 0$; $0 < S < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




```
yn = 1.754 m
V = 1.425 m/s
Fr = 0.3435
```

Rectangular channel carrying Q = 10.0 m³/s, b = 4.0 m, S = 0.0005, n = 0.015, drawn to scale at the returned normal depth yn = 1.75442 m (V = 1.42498 m/s, Fr = 0.343484); dashed: the critical depth yc = 0.860473 m from the package's critical-depth capability.



yn (normal depth (m)) against b (bottom width) from 0.4 m to 40 m on a logarithmic axis. One curve for each value of Q (discharge): 10 m^3/s, 5 m^3/s, 20 m^3/s. Other inputs as in the worked example: S = 0.0005, n = 0.015. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, b = 4 m, yn = 1.75442 m.

```
civil_engineering_ai999x.solve('normal_depth_rectangular', Q=10.0, b=4.0, S=0.0005,
n=0.015)
# -> yn = 1.75442 m, V = 1.42498 m/s, Fr = 0.343484   (SOLVED)
```

```
civil_engineering_ai999x.solve('normal_depth_rectangular', Q=2.5, b=1.5, S=0.002, n=0.025)
# -> yn = 1.48279 m, V = 1.12401 m/s, Fr = 0.294711   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal depth in a rectangular channel with discharge = 10 m^3/s, width = 4 m, slope = 0.0005 and Manning n = 0.015.	SOLVED	yn = 1.75442 m, V = 1.42498 m/s, Fr = 0.343484
Normal depth in a rectangular channel with discharge = 2.5 m^3/s, width = 1.5 m, slope = 0.002 and Manning n = 0.025.	SOLVED	yn = 1.48279 m, V = 1.12401 m/s, Fr = 0.294711
Normal depth in a rectangular channel with discharge = 10 m^3/s, width = 4 m and slope = 0.0005.	NEEDS_INPUT	normal_depth_rectangular needs n (Manning roughness coefficient). Example: solve('normal_depth_rectangular', Q='10 m^3/s
Normal depth in a rectangular channel with discharge = 10 m^3/s, width = 4 m, slope = 0.0005 and Manning n = 0.015 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Normal depth in a trapezoidal channel, `normal_depth_trapezoidal`

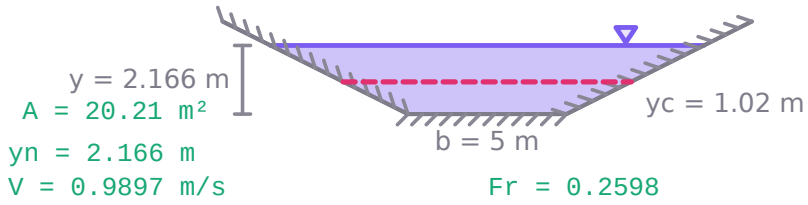
The uniform-flow (normal) depth that carries a discharge Q in a trapezoidal channel (Manning), with the velocity, area and Froude number (on the hydraulic depth A/T).

$$Q = \frac{1}{n} A R^{2/3} S^{1/2}, \quad A = (b + zy)y,\ P = b + 2y\sqrt{1+z^2}$$

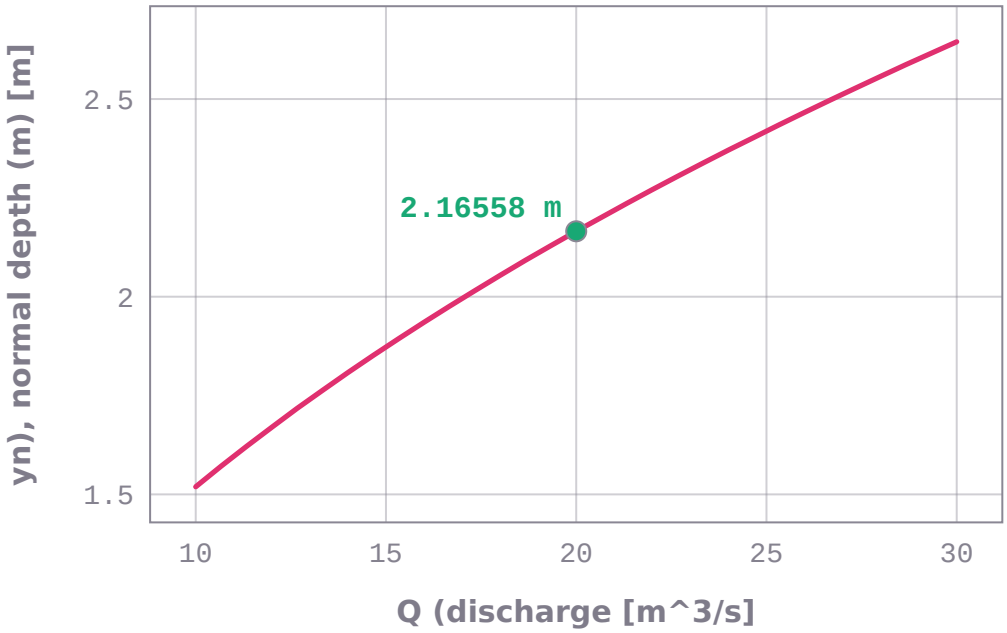
input	meaning	condition
`Q`	discharge	
`b`	bottom width	
`z`	side slope, horizontal per unit vertical	
`S`	bed slope, m/m	
`n`	Manning roughness coefficient	

output	meaning
`yn`	normal depth (m)
`V`	velocity at normal depth (m/s)
`A`	flow area (m^2)
`Fr`	Froude number on the hydraulic depth

- **Method:** bisection on Manning's Q(y) = Q
- **Assumes:** $Q > 0$; $b \geq 0$; $z \geq 0$; $0 < S < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Trapezoidal channel carrying Q = 20.0 m³/s, b = 5.0 m, z = 2.0 (H:1V), S = 0.0004, n = 0.025, drawn to scale at the returned normal depth yn = 2.16558 m (V = 0.989735 m/s, A = 20.2074 m², Fr = 0.259831); dashed: the critical depth yc = 1.01997 m from the package's critical-depth capability.



yn (normal depth (m)) against Q (discharge) from 10 m^3/s to 30 m^3/s. Other inputs as in the worked example: b = 5.0 m, z = 2.0, S = 0.0004, n = 0.025. 31 points computed by civil-engineering-ai999x; marked: the worked example, Q = 20 m^3/s, yn = 2.16558 m.

```
civil_engineering_ai999x.solve('normal_depth_trapezoidal', Q=20.0, b=5.0, z=2.0, S=0.0004, n=0.025)
# -> yn = 2.16558 m, V = 0.989735 m/s, A = 20.2074 m^2, Fr = 0.259831   (SOLVED)
```

```
civil_engineering_ai999x.solve('normal_depth_trapezoidal', Q=1.2, b=1.0, z=1.0, S=0.003, n=0.018)
# -> yn = 0.53853 m, V = 1.44832 m/s, A = 0.828545 m^2, Fr = 0.732145   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal depth in a trapezoidal channel with discharge = 20 m^3/s, width = 5 m, side slope = 2, slope = 0.0004 and Manning n = 0.025.	SOLVED	yn = 2.16558 m, V = 0.989735 m/s, A = 20.2074 m^2, Fr = 0.259831
Normal depth in a trapezoidal channel with discharge = 1.2 m^3/s, width = 1 m, side slope = 1, slope = 0.003 and Manning n = 0.018.	SOLVED	yn = 0.53853 m, V = 1.44832 m/s, A = 0.828545 m^2, Fr = 0.732145

question	outcome	result
Normal depth in a trapezoidal channel with discharge = 20 m^3/s, width = 5 m, side slope = 2 and slope = 0.0004.	NEEDS_INPUT	normal_depth_trapezoidal needs n (Manning roughness coefficient). Example: solve('normal_depth_trapezoidal', Q='20 m^3/s
Normal depth in a trapezoidal channel with discharge = 20 m^3/s, width = 5 m, side slope = 2, slope = 0.0004 and Manning n = 0.025 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Chezy velocity, `chezy_velocity`

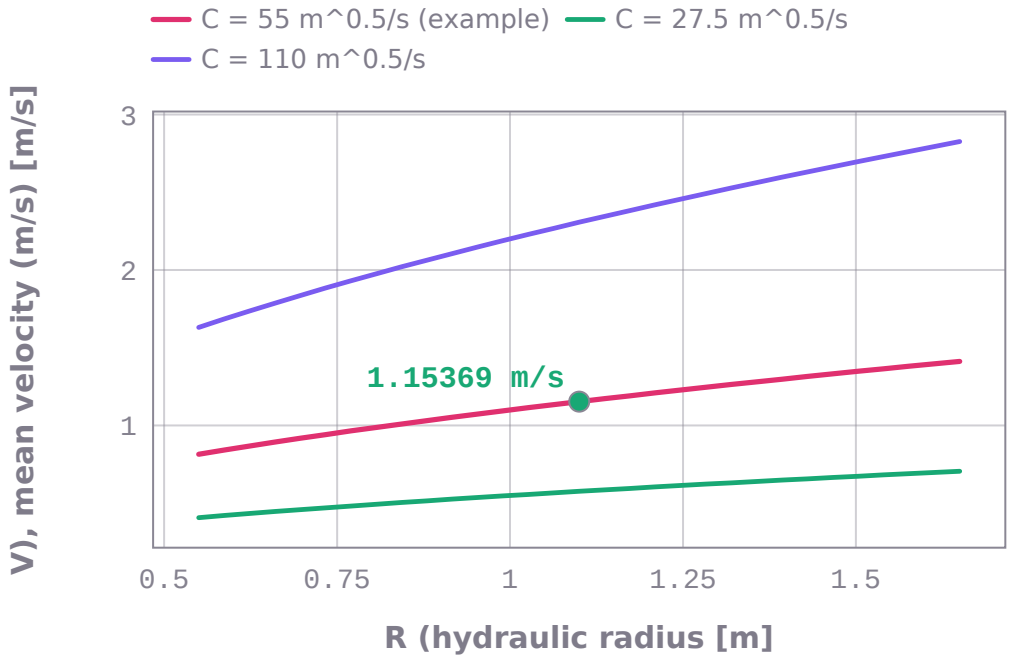
Mean velocity of uniform flow from the Chezy equation for a Chezy coefficient C $\left(m^{1/2}/s\right)$, hydraulic radius R and slope S.

$$V = C\sqrt{RS}$$

input	meaning	condition
C	Chezy coefficient $\left(m^{1/2}/s\right)$	
R	hydraulic radius	
S	channel (energy) slope, m/m	

output	meaning
V	mean velocity (m/s)

- **Method:** $V = C$ sqrt(R S)
- **Assumes:** C, $R > 0$; $0 < S < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (mean velocity (m/s)) against R (hydraulic radius) from 0.55 m to 1.65 m. One curve for each value of C (Chezy coefficient (m^(1/2)/s)): 55 m^0.5/s, 27.5 m^0.5/s, 110 m^0.5/s. Other inputs as in the worked example: S = 0.0004. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, R = 1.1 m, V = 1.15369 m/s.

```
civil_engineering_ai999x.solve('chezy_velocity', C=55.0, R=1.1, S=0.0004)
# -> V = 1.15369 m/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('chezy_velocity', C=40.0, R=0.5, S=0.002)
# -> V = 1.26491 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chezy velocity with Chezy C = 55 m^0.5/s, hydraulic radius = 1.1 m and slope = 0.0004.	SOLVED	V = 1.15369 m/s
Chezy velocity with Chezy C = 40 m^0.5/s, hydraulic radius = 0.5 m and slope = 0.002.	SOLVED	V = 1.26491 m/s
Chezy velocity with Chezy C = 55 m^0.5/s and hydraulic radius = 1.1 m.	NEEDS_INPUT	chezy_velocity needs S (channel (energy) slope, m/m). Example: solve('chezy_velocity', C='55 m^0.5/s', R='1.1 m', S=0.00
Chezy velocity with Chezy C = 55 m^0.5/s, hydraulic radius = 1.1 m and slope = 0.0004 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Chezy coefficient from Manning's n, `chezy_from_manning`

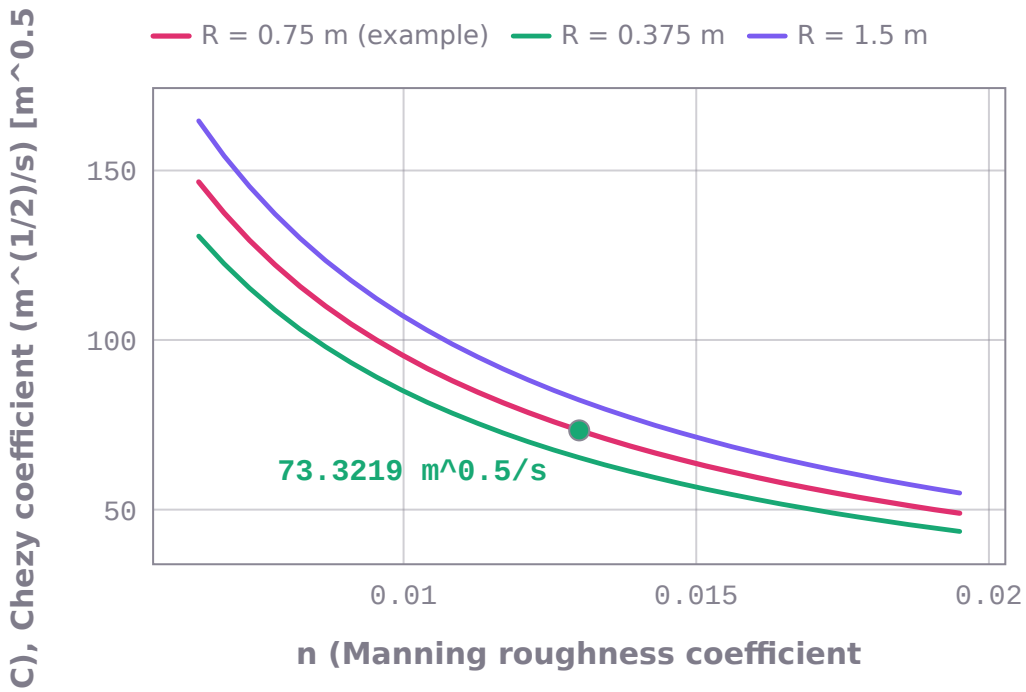
The Chezy coefficient equivalent to a Manning roughness n at hydraulic radius R (SI units).

$$C = \frac{R^{1/6}}{n}$$

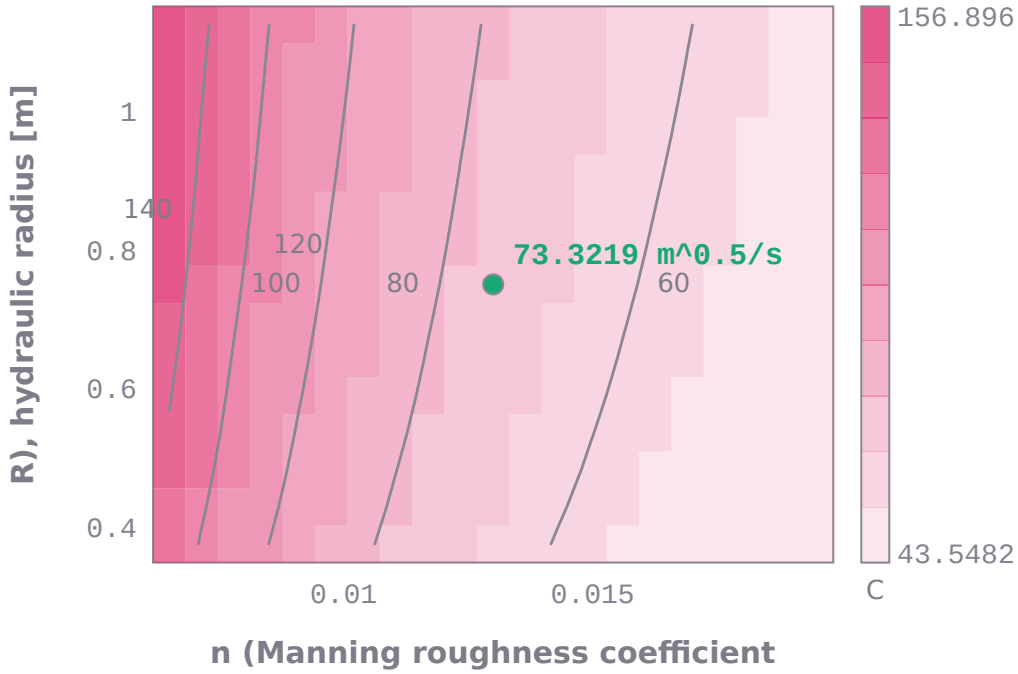
input	meaning	condition
`n`	Manning roughness coefficient	
`R`	hydraulic radius	

output	meaning
`C`	Chezy coefficient $(m^{1/2}/s)$

- **Method:** $C = R^{1/6}/n$
- **Assumes:** $0.005 \le n \le 0.2$; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (Chezy coefficient (m^(1/2)/s)) against n (Manning roughness coefficient) from 0.0065 to 0.0195. One curve for each value of R (hydraulic radius): 0.75 m, 0.375 m, 1.5 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, n = 0.013, C = 73.3219 m^0.5/s.



C (Chezy coefficient (m^(1/2)/s)) over n (Manning roughness coefficient) from 0.0065 to 0.0195 and R (hydraulic radius) from 0.375 m to 1.125 m: each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 43.5482 m^0.5/s to dark 156.896 m^0.5/s); iso-lines at C = 60, 80, 100, 120, 140 m^0.5/s. Marked: the worked example, n = 0.013, R = 0.75 m, C = 73.3219 m^0.5/s.

```
civil_engineering_ai999x.solve('chezy_from_manning', n=0.013, R=0.75)
# -> C = 73.3219 m^0.5/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('chezy_from_manning', n=0.03, R=2.0)
# -> C = 37.4154 m^0.5/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chezy coefficient from manning with Manning n = 0.013 and hydraulic radius = 0.75 m.	SOLVED	C = 73.3219 m^0.5/s
Chezy coefficient from manning with Manning n = 0.03 and hydraulic radius = 2 m.	SOLVED	C = 37.4154 m^0.5/s
Chezy coefficient from manning with Manning n = 0.013.	NEEDS_INPUT	chezy_from_manning needs R (hydraulic radius). Example: solve('chezy_from_manning', n=0.013, R='0.75 m').
Chezy coefficient from manning with Manning n = 0.013 and hydraulic radius = 0.75 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Manning's n from a Chezy coefficient, `manning_from_chezy`

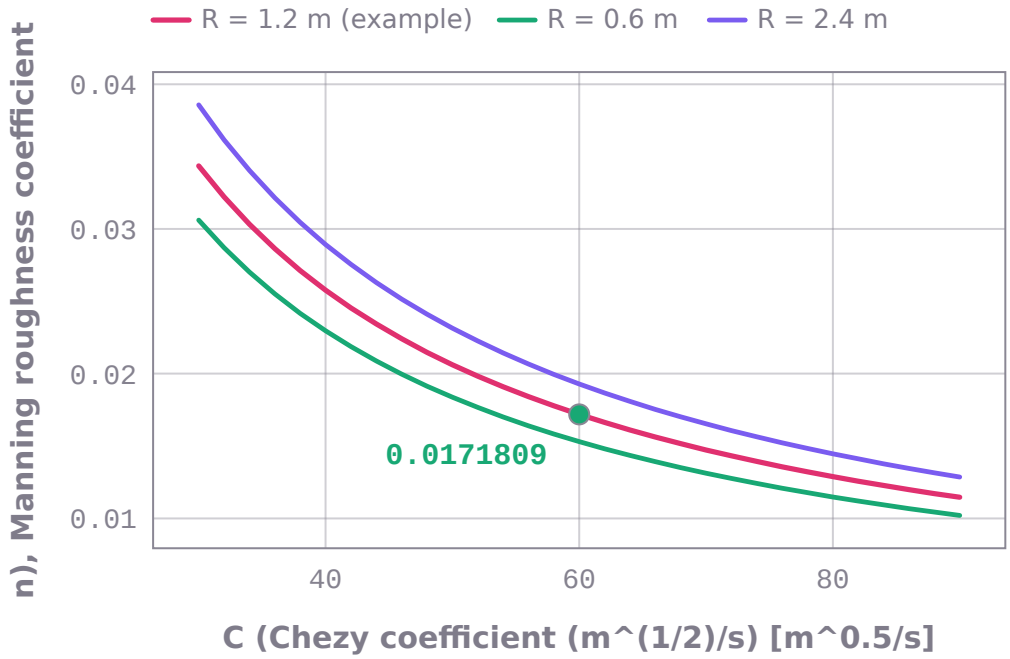
The Manning roughness n equivalent to a Chezy coefficient C at hydraulic radius R (SI units).

$$n = \frac{R^{1/6}}{C}$$

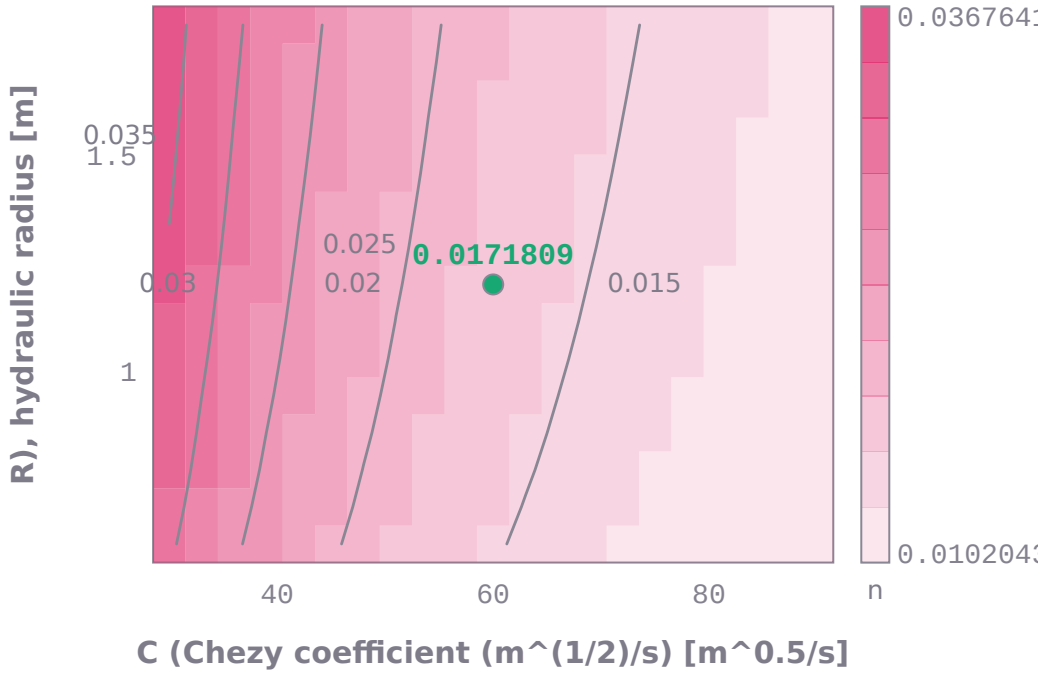
input	meaning	condition
`C`	Chezy coefficient $(m^{1/2}/s)$	
`R`	hydraulic radius	

output	meaning
`n`	Manning roughness coefficient

- **Method:** $n = R^{1/6}/C$
- **Assumes:** C, $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (Manning roughness coefficient) against C (Chezy coefficient (m^(1/2)/s)) from 30 m^0.5/s to 90 m^0.5/s. One curve for each value of R (hydraulic radius): 1.2 m, 0.6 m, 2.4 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, C = 60 m^0.5/s, n = 0.0171809.



n (Manning roughness coefficient) over C (Chezy coefficient (m^(1/2)/s)) from 30 m^0.5/s to 90 m^0.5/s and R (hydraulic radius) from 0.6 m to 1.8 m: each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 0.0102043 to dark 0.0367641); iso-lines at n = 0.015, 0.02, 0.025, 0.03, 0.035. Marked: the worked example, C = 60 m^0.5/s, R = 1.2 m, n = 0.0171809.

```
civil_engineering_ai999x.solve('manning_from_chezy', C=60.0, R=1.2)
# -> n = 0.0171809   (SOLVED)
```

```
civil_engineering_ai999x.solve('manning_from_chezy', C=35.0, R=0.4)
# -> n = 0.024525   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Manning n from chezy with Chezy C = 60 m^0.5/s and hydraulic radius = 1.2 m.	SOLVED	n = 0.0171809
Manning n from chezy with Chezy C = 35 m^0.5/s and hydraulic radius = 0.4 m.	SOLVED	n = 0.024525
Manning n from chezy with Chezy C = 60 m^0.5/s.	NEEDS_INPUT	manning_from_chezy needs R (hydraulic radius). Example: solve('manning_from_chezy', C='60 m^0.5/s', R='1.2 m').
Manning n from chezy with Chezy C = 60 m^0.5/s and hydraulic radius = 1.2 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Froude number of flow in a rectangular channel, `channel_froude_number`

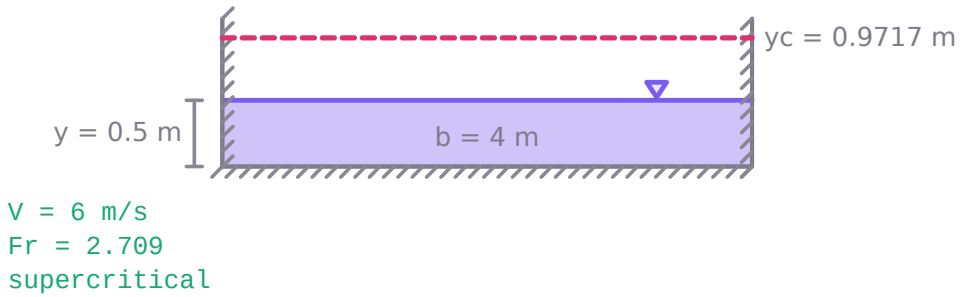
Froude number V/sqrt(g y) of flow at depth y in a rectangular channel, and whether the flow is subcritical, critical or supercritical.

$$Fr = \frac{V}{\sqrt{g\,y}}$$

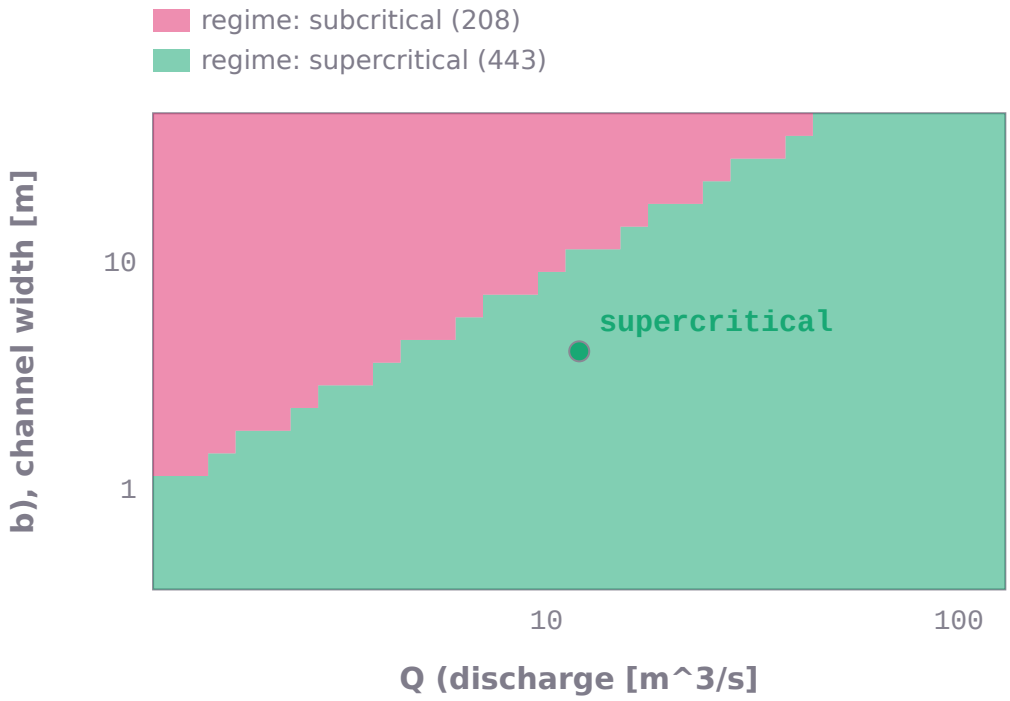
input	meaning	condition
`Q`	discharge	
`b`	channel width	
`y`	flow depth	

output	meaning
`Fr`	Froude number
`V`	mean velocity (m/s)
`regime`	flow regime: subcritical, critical or supercritical

- **Method:** V = Q/(b y), $Fr = V / sqrt$(g y)
- **Assumes:** Q, b, $y > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rectangular channel carrying Q = 12.0 m³/s, b = 4.0 m at depth y = 0.5 m (to scale); dashed: the critical depth yc = 0.971683 m from the package's critical-depth capability. Marked: the returned V = 6 m/s, Fr = 2.70914, regime: supercritical.



flow regime: subcritical, critical or supercritical (regime) over Q (discharge) from 1.2 m^3/s to 120 m^3/s and b (channel width) from 0.4 m to 40 m (logarithmic axes): each of the 31 × 21 cells is one run of civil-engineering-ai999x, coloured by the verdict it returned, subcritical: 208 cells, supercritical: 443 cells. Other inputs as in the worked example: y = 0.5 m. Marked: the worked example, Q = 12 m^3/s, b = 4 m → supercritical.

```
civil_engineering_ai999x.solve('channel_froude_number', Q=12.0, b=4.0, y=0.5)
# -> Fr = 2.70914, V = 6 m/s, regime = supercritical   (SOLVED)
```

```
civil_engineering_ai999x.solve('channel_froude_number', Q=3.0, b=2.5, y=1.4)
# -> Fr = 0.231289, V = 0.857143 m/s, regime = subcritical   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Froude number in a rectangular channel with discharge = 12 m^3/s, width = 4 m and depth = 0.5 m.	SOLVED	Fr = 2.70914, V = 6 m/s, regime = supercritical
Froude number in a rectangular channel with discharge = 3 m^3/s, width = 2.5 m and depth = 1.4 m.	SOLVED	Fr = 0.231289, V = 0.857143 m/s, regime = subcritical

question	outcome	result
Froude number in a rectangular channel with discharge = 12 m^3/s and width = 4 m.	NEEDS_INPUT	channel_froude_number needs y (flow depth). Example: solve('channel_froude_number', Q='12 m^3/s', b='4 m', y='0.5 m').
Froude number in a rectangular channel with discharge = 12 m^3/s, width = 4 m and depth = 0.5 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Critical depth in a rectangular channel, `critical_depth_rectangular`

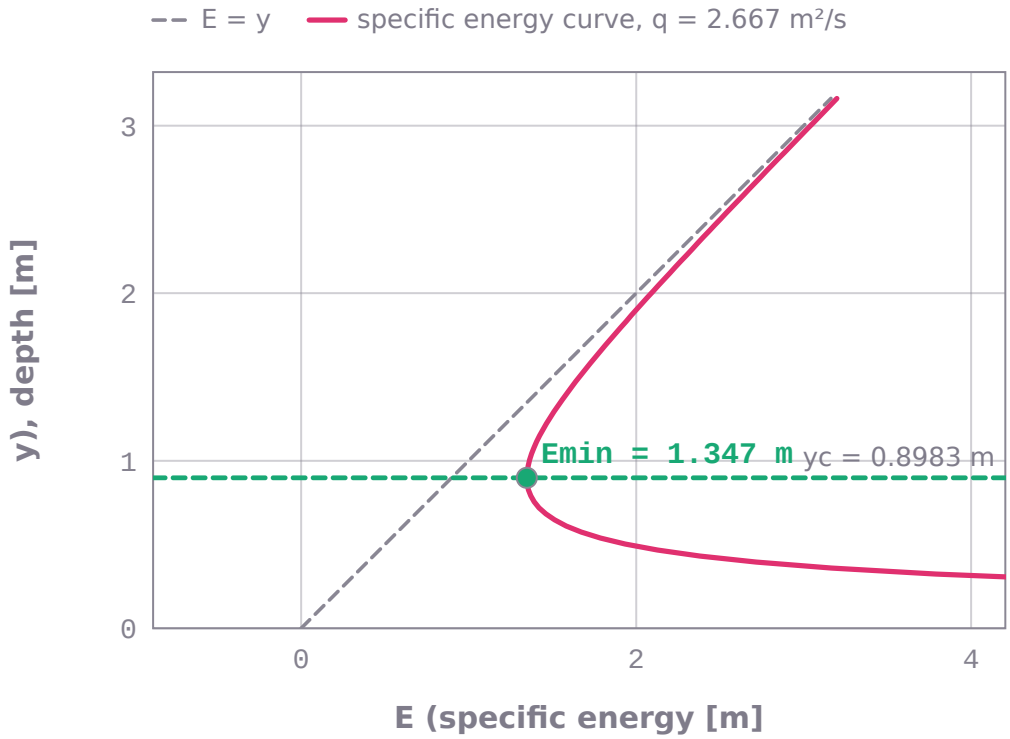
Critical depth, critical velocity and minimum specific energy for a discharge Q in a rectangular channel.

$$y_c = \left(\frac{q^2}{g}\right)^{1/3}, \quad E_{min} = \tfrac{3}{2} y_c$$

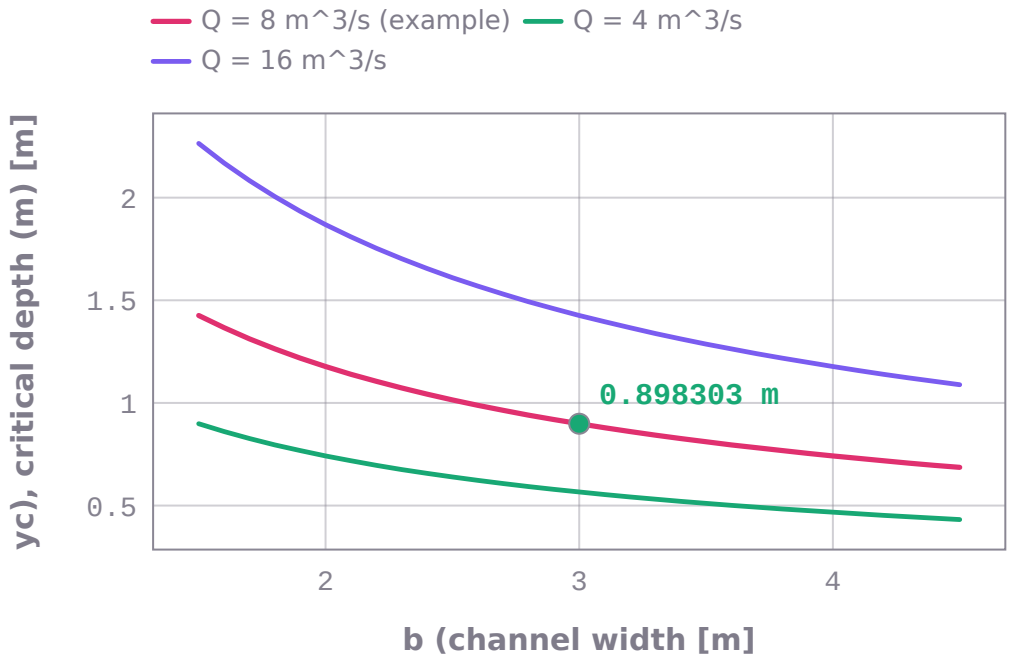
input	meaning	condition
`Q`	discharge	
`b`	channel width	

output	meaning
`yc`	critical depth (m)
`Vc`	critical velocity (m/s)
`Emin`	minimum specific energy 1.5 yc (m)
`q`	discharge per unit width (m^2/s)

- **Method:** $yc = (q^2/g)^{1/3}$, $q = Q/b$
- **Assumes:** Q, $b > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Specific-energy diagram of a rectangular channel carrying Q = 8.0 m³/s in b = 3.0 m: each point is the specific energy the package returns at a depth (81 depths). Marked: the returned critical depth yc = 0.898303 m, minimum specific energy Emin = 1.34745 m, critical velocity 2.96856 m/s, unit discharge q = 2.66667 m²/s.



yc (critical depth (m)) against b (channel width) from 1.5 m to 4.5 m. One curve for each value of Q (discharge): 8 m^3/s, 4 m^3/s, 16 m^3/s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, b = 3 m, yc = 0.898303 m.

```
civil_engineering_ai999x.solve('critical_depth_rectangular', Q=8.0, b=3.0)
# -> yc = 0.898303 m, Vc = 2.96856 m/s, Emin = 1.34745 m, q = 2.66667 m^2/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('critical_depth_rectangular', Q=0.5, b=1.2)
# -> yc = 0.260597 m, Vc = 1.59889 m/s, Emin = 0.390896 m, q = 0.416667 m^2/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical depth in a rectangular channel with discharge = 8 m^3/s and width = 3 m.	SOLVED	yc = 0.898303 m, Vc = 2.96856 m/s, Emin = 1.34745 m, q = 2.66667 m^2/s
Critical depth in a rectangular channel with discharge = 0.5 m^3/s and width = 1.2 m.	SOLVED	yc = 0.260597 m, Vc = 1.59889 m/s, Emin = 0.390896 m, q = 0.416667 m^2/s
Critical depth in a rectangular channel with discharge = 8 m^3/s.	NEEDS_INPUT	critical_depth_rectangular needs b (channel width). Example: solve('critical_depth_rectangular', Q='8 m^3/s', b='3 m').
Critical depth in a rectangular channel with discharge = 8 m^3/s and width = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Critical depth in a trapezoidal channel, `critical_depth_trapezoidal`

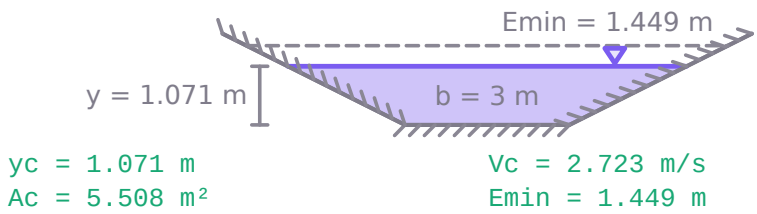
Critical depth, area, velocity and minimum specific energy for a discharge Q in a trapezoidal (or triangular, b = 0) channel, from $Q^2\,T = g\,A^3$.

$$\frac{Q^2\,T}{g\,A^3} = 1, \quad A = (b + zy)y,\; T = b + 2zy$$

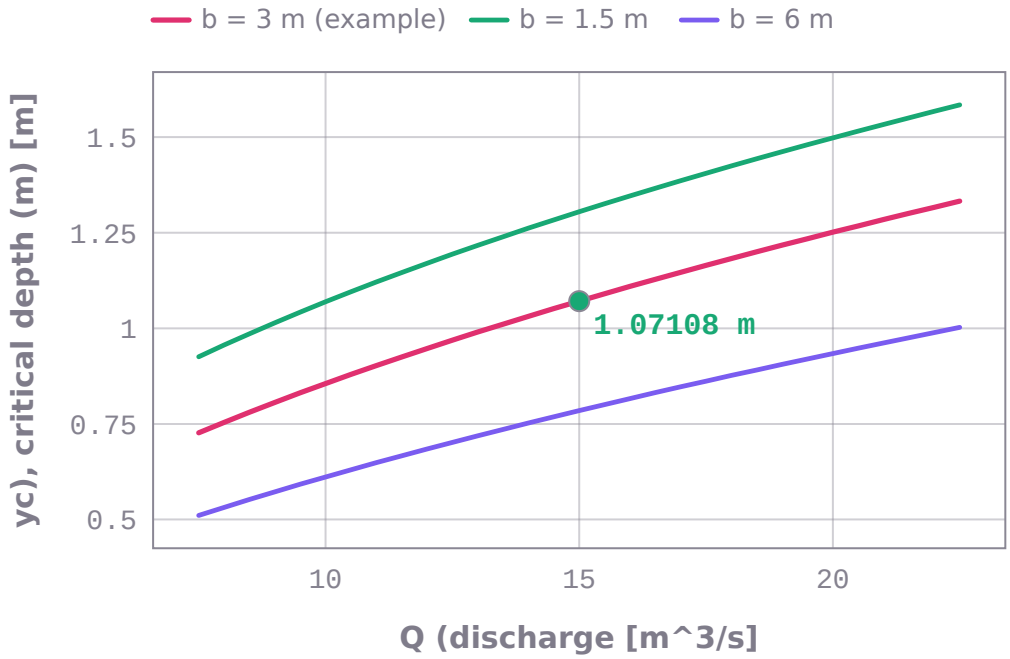
input	meaning	condition
`Q`	discharge	
`b`	bottom width	
`z`	side slope, horizontal per unit vertical	

output	meaning
`yc`	critical depth (m)
`Ac`	critical flow area (m^2)
`Vc`	critical velocity (m/s)
`Emin`	minimum specific energy (m)

- **Method:** bisection on Q^2 T(y) = g A(y)^3
- **Assumes:** $Q > 0$; $b \geq 0$; $z > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Trapezoidal channel carrying Q = 15.0 m³/s, b = 3.0 m, z = 2.0 (H:1V), drawn to scale at the returned critical depth yc = 1.07108 m. Marked: Ac = 5.50766 m², Vc = 2.72348 m/s, minimum specific energy Emin = 1.44913 m (dashed level).



yc (critical depth (m)) against Q (discharge) from 7.5 m^3/s to 22.5 m^3/s. One curve for each value of b (bottom width): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: z = 2.0. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, Q = 15 m^3/s, yc = 1.07108 m.

```
civil_engineering_ai999x.solve('critical_depth_trapezoidal', Q=15.0, b=3.0, z=2.0)
# -> yc = 1.07108 m, Ac = 5.50766 m^2, Vc = 2.72348 m/s, Emin = 1.44913 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('critical_depth_trapezoidal', Q=0.8, b=0.0, z=1.0)
# -> yc = 0.665439 m, Ac = 0.442809 m^2, Vc = 1.80665 m/s, Emin = 0.831799 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical depth in a trapezoidal channel with discharge = 15 m^3/s, width = 3 m and side slope = 2.	SOLVED	yc = 1.07108 m, Ac = 5.50766 m^2, Vc = 2.72348 m/s, Emin = 1.44913 m
Critical depth in a trapezoidal channel with discharge = 0.8 m^3/s, width = 0 m and side slope = 1.	SOLVED	yc = 0.665439 m, Ac = 0.442809 m^2, Vc = 1.80665 m/s, Emin = 0.831799 m
Critical depth in a trapezoidal channel with discharge = 15 m^3/s and width = 3 m.	NEEDS_INPUT	critical_depth_trapezoidal needs z (side slope, horizontal per unit vertical). Example: solve('critical_depth_trapezoida
Critical depth in a trapezoidal channel with discharge = 15 m^3/s, width = 3 m and side slope = 2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Critical depth in a circular conduit, `critical_depth_circular`

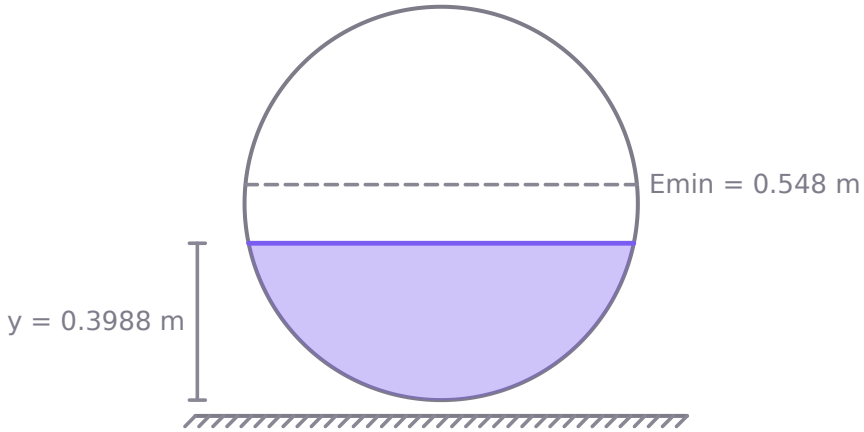
Critical depth, area, velocity and minimum specific energy for a discharge Q in a circular pipe or culvert flowing part full (refused when the critical depth would exceed 0.95 D).

$$\frac{Q^2 T}{g A^3} = 1, \quad A = \frac{D^2}{8}(\theta - \sin\theta),\ T = D \sin\frac{\theta}{2}$$

input	meaning	condition
`Q`	discharge	
`D`	pipe diameter	

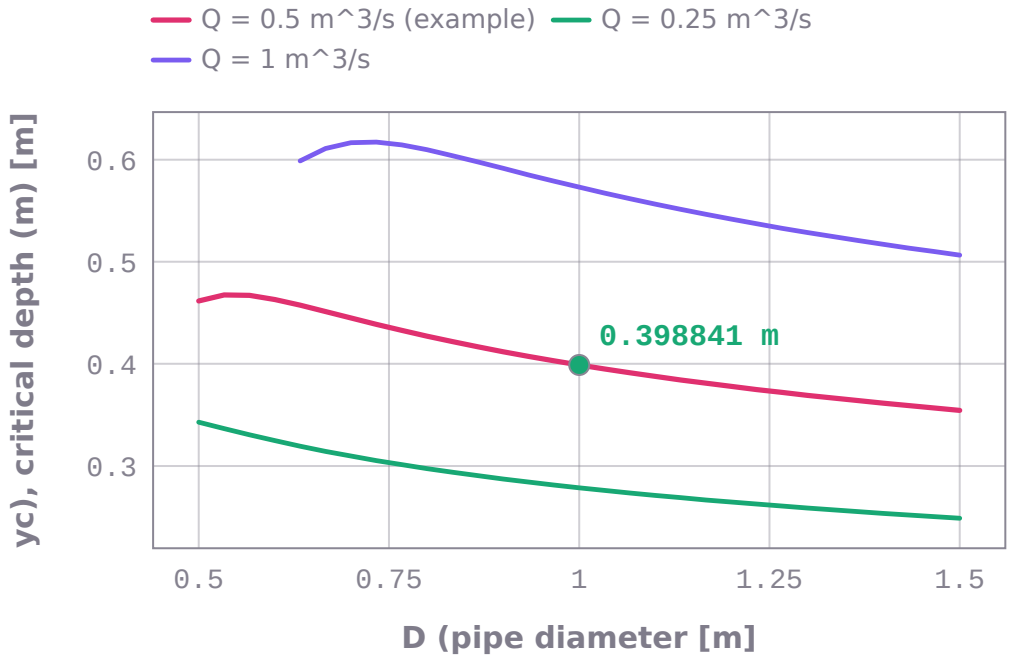
output	meaning
`yc`	critical depth (m)
`Ac`	critical flow area (m^2)
`Vc`	critical velocity (m/s)
`Emin`	minimum specific energy (m)

- **Method:** bisection on Q^2 T(y) = g A(y)^3 with circular-segment geometry
- **Assumes:** Q, $D > 0$; critical depth below 0.95 D
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




```
yc = 0.3988 m          Vc = 1.711 m/s
Ac = 0.2922 m²         Emin = 0.548 m
```

Circular conduit carrying Q = 0.5 m³/s, D = 1.0 m, to scale at the returned critical depth yc = 0.398841 m. Marked: Ac = 0.292235 m², Vc = 1.71095 m/s, Emin = 0.548044 m (dashed level).



yc (critical depth (m)) against D (pipe diameter) from 0.5 m to 1.5 m. One curve for each value of Q (discharge): 0.5 m^3/s, 0.25 m^3/s, 1 m^3/s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, D = 1 m, yc = 0.398841 m.

```
civil_engineering_ai999x.solve('critical_depth_circular', Q=0.5, D=1.0)
# -> yc = 0.398841 m, Ac = 0.292235 m^2, Vc = 1.71095 m/s, Emin = 0.548044 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('critical_depth_circular', Q=0.05, D=0.4)
# -> yc = 0.158555 m, Ac = 0.0463732 m^2, Vc = 1.07821 m/s, Emin = 0.217807 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical depth in a circular pipe with discharge = 0.5 m^3/s and diameter = 1 m.	SOLVED	yc = 0.398841 m, Ac = 0.292235 m^2, Vc = 1.71095 m/s, Emin = 0.548044 m
Critical depth in a circular pipe with discharge = 0.05 m^3/s and diameter = 0.4 m.	SOLVED	yc = 0.158555 m, Ac = 0.0463732 m^2, Vc = 1.07821 m/s, Emin = 0.217807 m
Critical depth in a circular pipe with discharge = 0.5 m^3/s.	NEEDS_INPUT	critical_depth_circular needs D (pipe diameter). Example: solve('critical_depth_circular', Q='0.5 m^3/s', D='1 m').
Critical depth in a circular pipe with discharge = 0.5 m^3/s and diameter = 1 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Critical depth in a circular pipe with discharge = 20 m^3/s and diameter = 1 m.	INVALID_INPUT	Critical depth in a circular conduit: the critical depth would exceed 0.95 D: the conduit is too small for this discharg

Specific energy and alternate depth in a rectangular channel, `alternate_depth_rectangular`

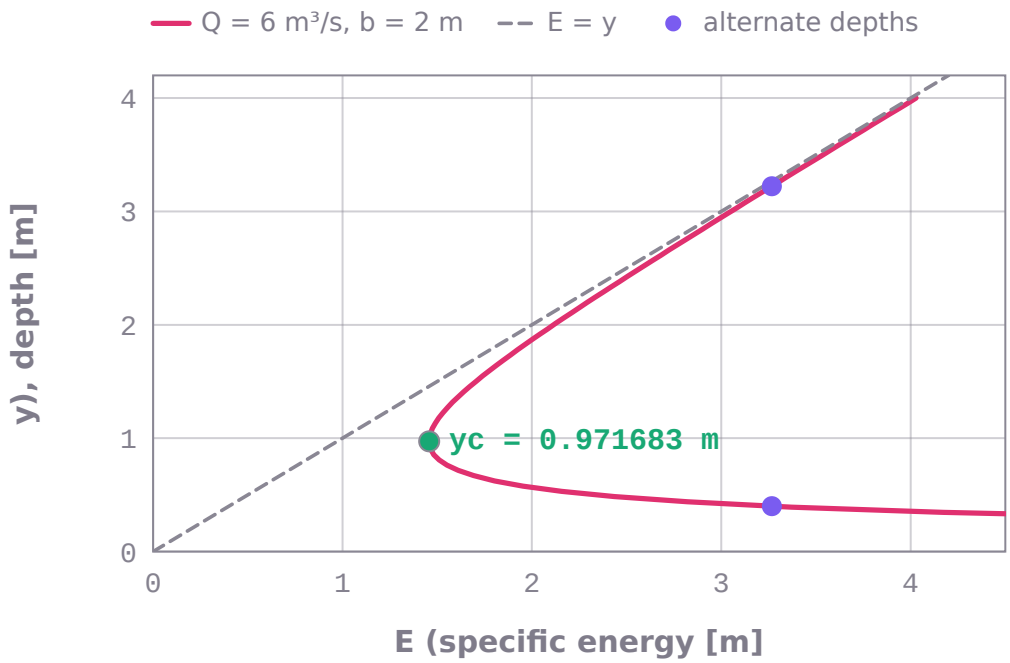
The specific energy of flow at depth y in a rectangular channel and the alternate depth with the same specific energy on the other side of critical depth.

$$E = y + \frac{q^2}{2gy^2}, \quad y_2^3 - E\,y_2^2 + \frac{q^2}{2g} = 0$$

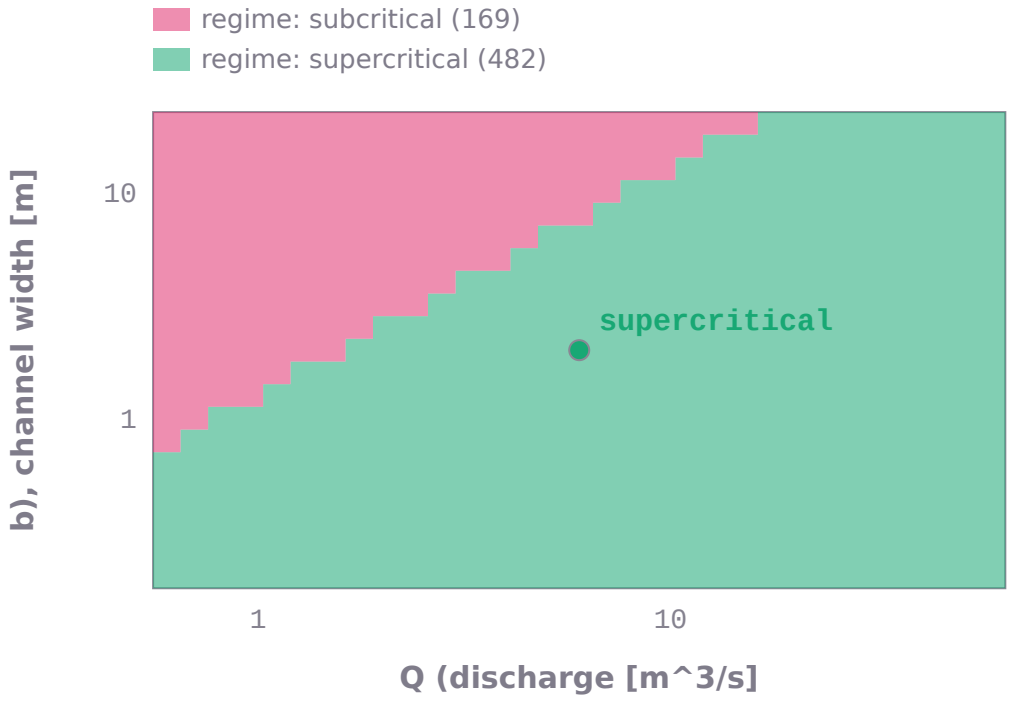
input	meaning	condition
`Q`	discharge	
`b`	channel width	
`y`	flow depth	

output	meaning
`E`	specific energy $y + V^2/2g$ (m)
`y_alt`	alternate depth (m)
`yc`	critical depth (m)
`Fr`	Froude number at depth y
`regime`	flow regime at depth y

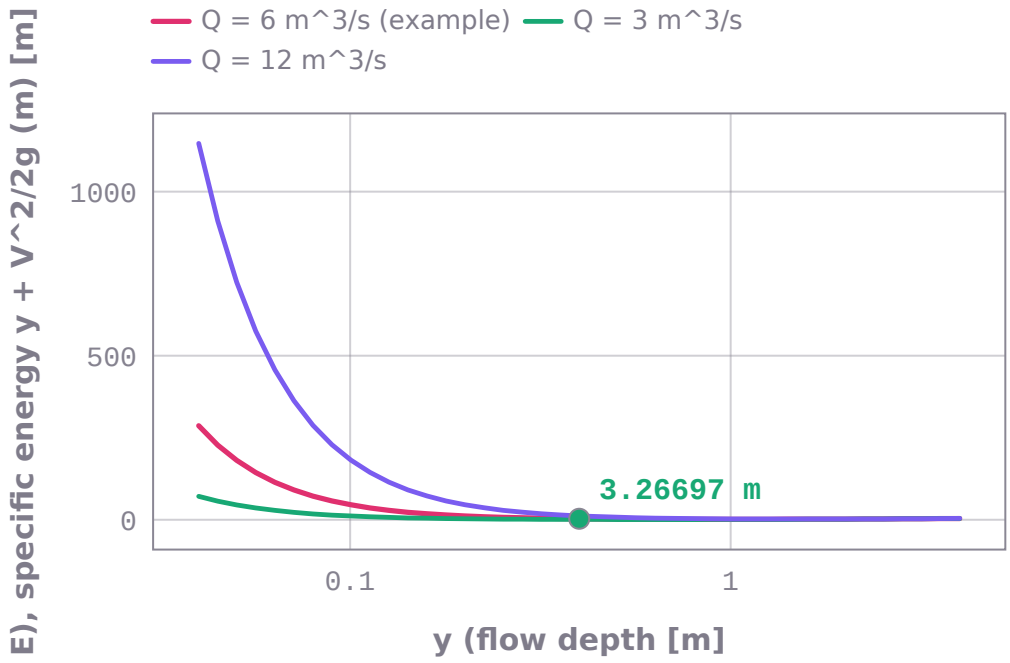
- **Method:** E = y + q^2/(2 g y^2); the other positive root of $y^3 - E\,y^2 + q^2/(2g) = 0$ by bisection
- **Assumes:** Q, b, $y > 0$; y is not the critical depth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Specific-energy diagram of a rectangular channel carrying Q = 6 m³/s in b = 2 m (E computed by the package at 81 depths). Depth y = 0.4 m (supercritical, Fr = 3.78614) has E = 3.26697 m and the alternate depth 3.22281 m; marked: critical depth yc = 0.971683 m at the minimum energy E = 1.45752 m.



flow regime at depth y (regime) over Q (discharge) from 0.6 m^3/s to 60 m^3/s and b (channel width) from 0.2 m to 20 m (logarithmic axes): each of the 31 × 21 cells is one run of civil-engineering-ai999x, coloured by the verdict it returned, subcritical: 169 cells, supercritical: 482 cells. Other inputs as in the worked example: y = 0.4 m. Marked: the worked example, Q = 6 m^3/s, b = 2 m → supercritical.



E (specific energy y + V^2/2g (m)) against y (flow depth) from 0.04 m to 4 m on a logarithmic axis. One curve for each value of Q (discharge): 6 m^3/s, 3 m^3/s, 12 m^3/s. Other inputs as in the worked example: b = 2.0 m. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, y = 0.4 m, E = 3.26697 m.

```
civil_engineering_ai999x.solve('alternate_depth_rectangular', Q=6.0, b=2.0, y=0.4)
# -> E = 3.26697 m, y_alt = 3.22281 m, yc = 0.971683 m, Fr = 3.78614, regime =
supercritical   (SOLVED)
```

```
civil_engineering_ai999x.solve('alternate_depth_rectangular', Q=6.0, b=2.0, y=1.8)
# -> E = 1.94158 m, y_alt = 0.580547 m, yc = 0.971683 m, Fr = 0.396623, regime =
subcritical   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Specific energy and alternate depth with discharge = 6 m^3/s, width = 2 m and depth = 0.4 m.	SOLVED	E = 3.26697 m, y_alt = 3.22281 m, yc = 0.971683 m, Fr = 3.78614, regime = supercritical
Specific energy and alternate depth with discharge = 6 m^3/s, width = 2 m and depth = 1.8 m.	SOLVED	E = 1.94158 m, y_alt = 0.580547 m, yc = 0.971683 m, Fr = 0.396623, regime = subcritical
Specific energy and alternate depth with discharge = 6 m^3/s and width = 2 m.	NEEDS_INPUT	alternate_depth_rectangular needs y (flow depth). Example: solve('alternate_depth_rectangular', Q='6 m^3/s', b='2 m', y=
Specific energy and alternate depth with discharge = 6 m^3/s, width = 2 m and depth = 0.4 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Hydraulic jump in a rectangular channel, `hydraulic_jump_rectangular`

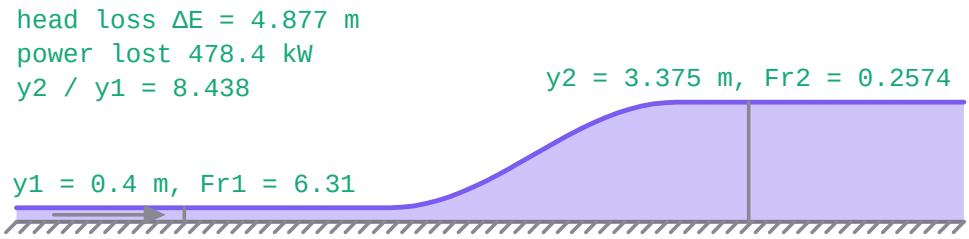
Sequent (conjugate) depth, energy loss and dissipated power of a hydraulic jump in a horizontal rectangular channel (Belanger's equation from the momentum balance).

$$\frac{y_2}{y_1} = \tfrac{1}{2}\left(\sqrt{1+8Fr_1^2}-1\right), \quad \Delta E = \frac{(y_2-y_1)^3}{4y_1y_2}$$

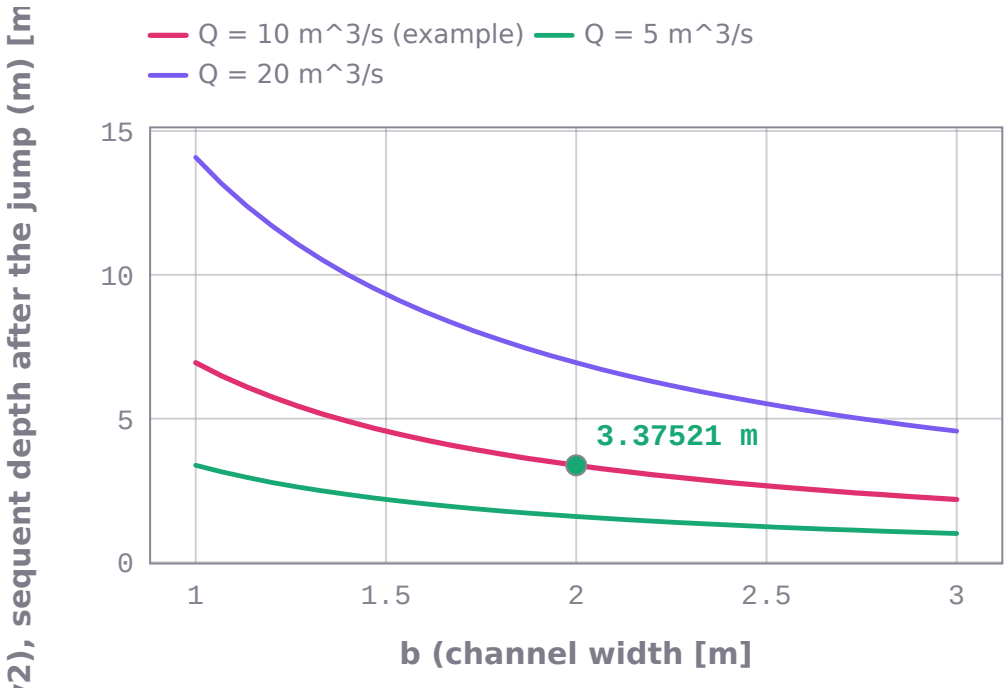
input	meaning	condition
`Q`	discharge	
`b`	channel width	
`y1`	upstream (supercritical) depth	

output	meaning
`y2`	sequent depth after the jump (m)
`Fr1`	upstream Froude number
`Fr2`	downstream Froude number
`dE`	head lost in the jump (m)
`power_loss`	power dissipated gamma Q dE (W)
`ratio`	depth ratio y_2/y_1

- **Method:** $y_2 = (y_1/2)$(sqrt($1 + 8$ Fr1^2) - 1), dE = (y2 - y1)^3/(4 y1 y2)
- **Assumes:** horizontal, frictionless bed; $Fr1 > 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hydraulic jump in a rectangular channel (Q = 10.0 m³/s, b = 2.0 m) from the supercritical depth y1 = 0.4 m to the returned sequent depth y2 = 3.37521 m, depths to scale; the shape and length of the rise are schematic (not computed). Marked: Fr1 = 6.31023, Fr2 = 0.257445, head loss 4.87676 m, power lost 478.41 kW, depth ratio 8.43802.



y2 (sequent depth after the jump (m)) against b (channel width) from 1 m to 3 m. One curve for each value of Q (discharge): 10 m^3/s, 5 m^3/s, 20 m^3/s. Other inputs as in the worked example: y1 = 0.4 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, b = 2 m, y2 = 3.37521 m.

```
civil_engineering_ai999x.solve('hydraulic_jump_rectangular', Q=10.0, b=2.0, y1=0.4)
# -> y2 = 3.37521 m, Fr1 = 6.31023, Fr2 = 0.257445, dE = 4.87676 m, power_loss = 478410 W,
ratio = 8.43802   (SOLVED)
```

```
civil_engineering_ai999x.solve('hydraulic_jump_rectangular', Q=3.0, b=1.5, y1=0.25)
# -> y2 = 1.68542 m, Fr1 = 5.10841, Fr2 = 0.291834, dE = 1.75479 m, power_loss = 51643.5 W,
ratio = 6.74166   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydraulic jump in a rectangular channel with discharge = 10 m^3/s, width = 2 m and upstream depth = 0.4 m.	SOLVED	y2 = 3.37521 m, Fr1 = 6.31023, Fr2 = 0.257445, dE = 4.87676 m, power_loss = 478410 W, ratio = 8.43802
Hydraulic jump in a rectangular channel with discharge = 3 m^3/s, width = 1.5 m and upstream depth = 0.25 m.	SOLVED	y2 = 1.68542 m, Fr1 = 5.10841, Fr2 = 0.291834, dE = 1.75479 m, power_loss = 51643.5 W, ratio = 6.74166
Hydraulic jump in a rectangular channel with discharge = 10 m^3/s and width = 2 m.	NEEDS_INPUT	hydraulic_jump_rectangular needs y1 (upstream (supercritical) depth). Example: solve('hydraulic_jump_rectangular', Q='10
Hydraulic jump in a rectangular channel with discharge = 10 m^3/s, width = 2 m and upstream depth = 0.4 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Hydraulic jump in a rectangular channel with discharge = 1 m^3/s, width = 2 m and upstream depth = 1 m.	INVALID_INPUT	Hydraulic jump in a rectangular channel: the upstream flow must be supercritical (Fr1 > 1) for a hydraulic jump.

Length of a gradually varied flow profile (rectangular channel),

`gradually_varied_flow_length`

Distance along a prismatic rectangular channel between two sections of a gradually varied flow profile, by integrating $dx/dy = (1 - Fr^2)/(S_0 - Sf)$ exactly (not the direct-step approximation), with the profile type (M1, M2, M3, S1, S2, S3). Positive L: the section at depth y2 lies downstream of the one at depth y1.

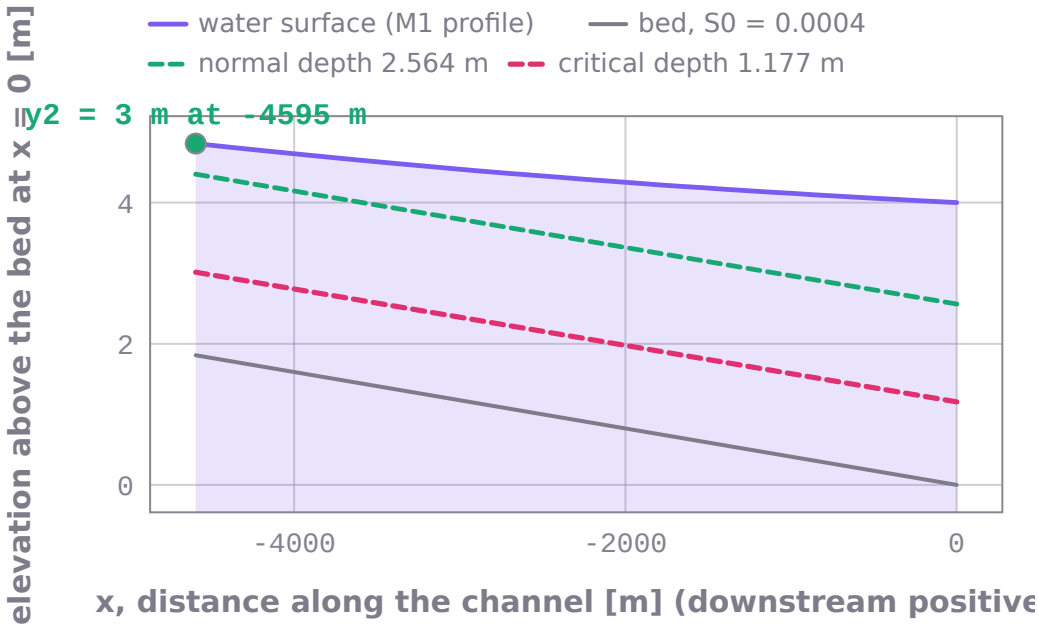
$$\frac{dx}{dy} = \frac{1 - Fr^2}{S_0 - S_f}, \quad S_f = \frac{n^2 V^2}{R^{4/3}}$$

input	meaning	condition
`Q`	discharge	
`b`	channel width	
`S0`	bed slope, m/m (positive)	
`n`	Manning roughness coefficient	
`y1`	depth at the first section	
`y2`	depth at the second section	

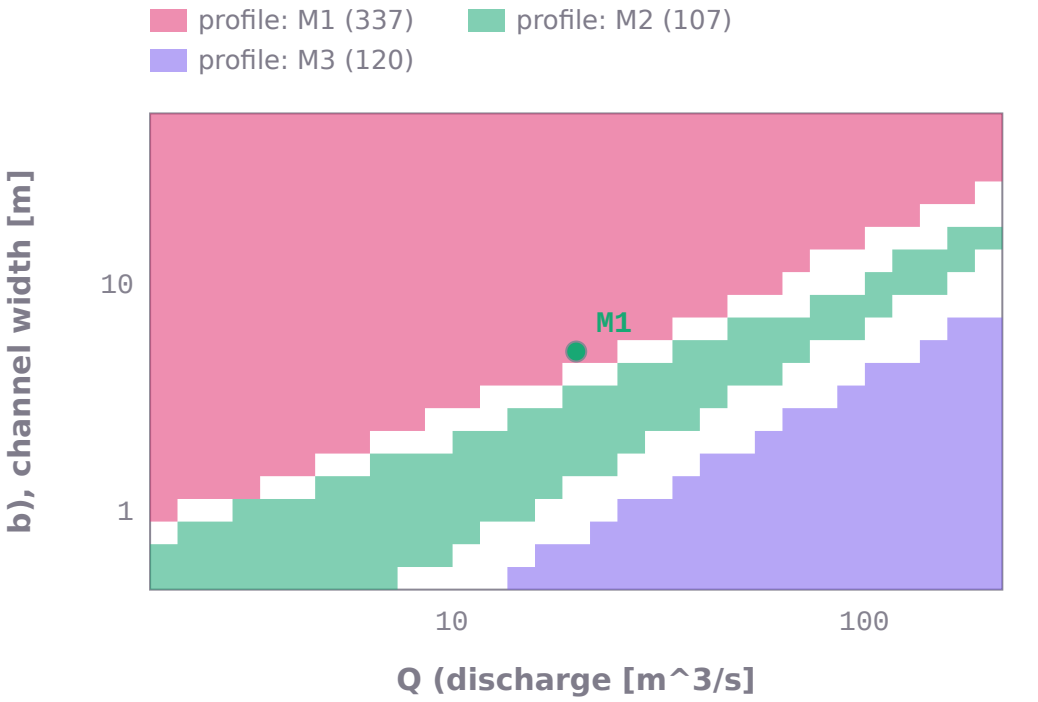
output	meaning
`L`	distance from section 1 to section 2, positive downstream (m)
`yn`	normal depth (m)
`yc`	critical depth (m)
`profile`	profile type (M1 ... S3)

- **Method:** x2 - x1 = integral of $(1 - Fr^2)/(S_0 - Sf)$ dy from y1 to y2, Sf from Manning (tanh-sinh quadrature)

- **Assumes:** $S_0 > 0$ (mild or steep slope); prismatic rectangular channel; both depths in one zone, 2% from yn and yc
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gradually varied flow profile in a rectangular channel (Q = 20.0 m³/s, b = 5.0 m, S0 = 0.0004, n = 0.015) from y1 = 4 m at x = 0: the position of 30 intermediate depths is the length the package returns from y1 to that depth. Marked: the returned length L = -4594.59 m to y2 = 3 m, normal depth 2.56382 m, critical depth 1.17711 m, profile type M1.



profile type (M1 ... S3) (profile) over Q (discharge) from 2 m^3/s to 200 m^3/s and b (channel width) from 0.5 m to 50 m (logarithmic axes): each of the 31 × 21 cells is one run of civil-engineering-ai999x, coloured by the verdict it returned, M1: 337 cells, M2: 107 cells, M3: 120 cells; blank cells: inputs the capability refuses. Other inputs as in the worked example: S0 = 0.0004, n = 0.015, y1 = 4.0 m, y2 = 3.0 m. Marked: the worked example, Q = 20 m^3/s, b = 5 m → M1.

```
civil_engineering_ai999x.solve('gradually_varied_flow_length', Q=20.0, b=5.0, S0=0.0004,
n=0.015, y1=4.0, y2=3.0)
# -> L = -4594.59 m, yn = 2.56382 m, yc = 1.17711 m, profile = M1   (SOLVED)
```

```
civil_engineering_ai999x.solve('gradually_varied_flow_length', Q=20.0, b=5.0, S0=0.0004,
n=0.015, y1=2.4, y2=1.6)
# -> L = 2438.49 m, yn = 2.56382 m, yc = 1.17711 m, profile = M2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gradually varied flow length with discharge = 20 m^3/s, width = 5 m, slope = 0.0004, Manning n = 0.015, first depth = 4 m and second depth = 3 m.	SOLVED	L = -4594.59 m, yn = 2.56382 m, yc = 1.17711 m, profile = M1
Gradually varied flow length with discharge = 20 m^3/s, width = 5 m, slope = 0.0004, Manning n = 0.015, first depth = 2.4 m and second depth = 1.6 m.	SOLVED	L = 2438.49 m, yn = 2.56382 m, yc = 1.17711 m, profile = M2
Gradually varied flow length with discharge = 20 m^3/s, width = 5 m, slope = 0.0004, Manning n = 0.015 and first depth = 4 m.	NEEDS_INPUT	gradually_varied_flow_length needs y2 (depth at the second section). Example: solve('gradually_varied_flow_length', Q='2
Gradually varied flow length with discharge = 20 m^3/s, width = 5 m, slope = 0.0004, Manning n = 0.015, first depth = 4 m and second depth = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Gradually varied flow length with discharge = 20 m^3/s, width = 5 m, slope = 0.0004, Manning n = 0.015, first depth = 3 m and second depth = 2 m.	INVALID_INPUT	Length of a gradually varied flow profile (rectangular channel): the depths must stay on one side of (and 2% away from)

Sharp-crested rectangular weir discharge, `rectangular_weir_discharge`

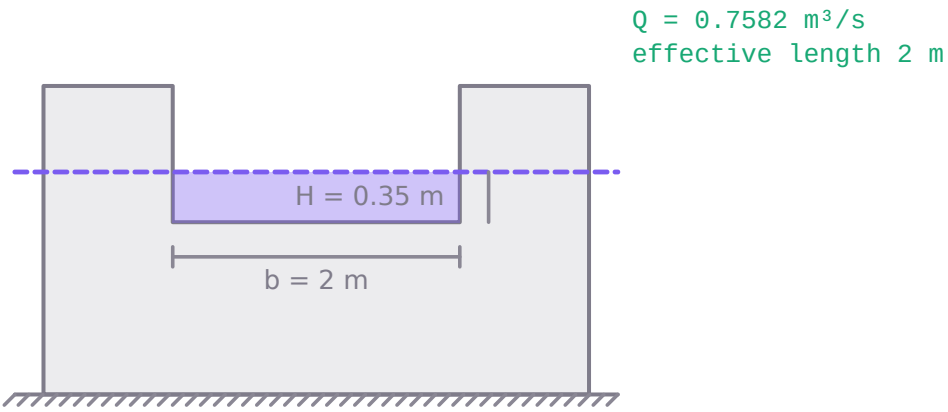
Discharge over a sharp-crested rectangular weir for a stated discharge coefficient Cd and head H, with Francis' allowance of 0.1 H per end contraction (0, 1 or 2 contractions).

$$Q = \tfrac{2}{3}\, C_d\, L_e \sqrt{2g}\, H^{3/2}, \quad L_e = b - 0.1\, n H$$

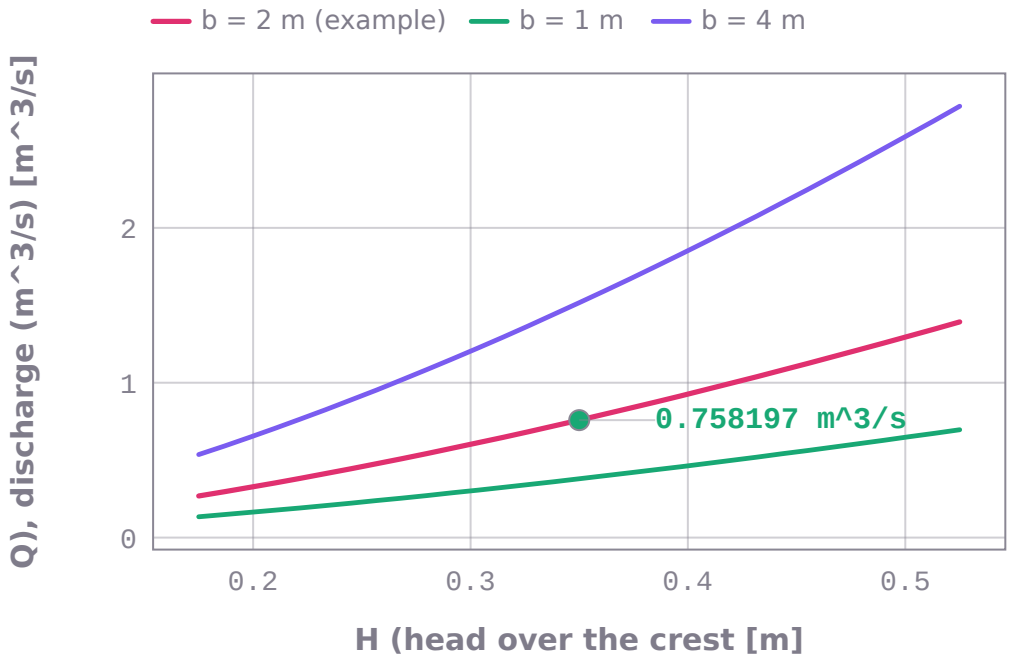
input	meaning	condition
`Cd`	discharge coefficient (stated, e.g. 0.62)	
`b`	crest length	
`H`	head over the crest	
`contractions`	number of end contractions (0, 1 or 2; default 0)	

output	meaning
`Q`	discharge (m^3/s)
`L_eff`	effective crest length (m)

- **Method:** $Q = (2/3)$ Cd L_eff sqrt(2g) $H^{3/2}$, L_eff = b - 0.1 n H
- **Assumes:** Cd stated; free (unsubmerged) nappe; $b - 0.1$ n $H > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Front view of a sharp-crested rectangular weir, notch width b = 2 m, head H = 0.35 m above the crest (Cd = 0.62, end contractions 0); notch and head to scale, plate outline schematic. Marked: the returned discharge Q = 0.758197 m³/s and effective crest length 2 m.



Q (discharge (m^3/s)) against H (head over the crest) from 0.175 m to 0.525 m. One curve for each value of b (crest length): 2 m, 1 m, 4 m. Other inputs as in the worked example: Cd = 0.62. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, H = 0.35 m, Q = 0.758197 m^3/s.

```
civil_engineering_ai999x.solve('rectangular_weir_discharge', Cd=0.62, b=2.0, H=0.35)
# -> Q = 0.758197 m^3/s, L_eff = 2 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('rectangular_weir_discharge', Cd=0.6, b=1.5, H=0.4,
contractions=2)
# -> Q = 0.636484 m^3/s, L_eff = 1.42 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rectangular weir discharge with discharge coefficient = 0.62, crest length = 2 m and head = 0.35 m.	SOLVED	Q = 0.758197 m^3/s, L_eff = 2 m
Rectangular weir discharge with discharge coefficient = 0.6, crest length = 1.5 m, head = 0.4 m and end contractions = 2.	SOLVED	Q = 0.636484 m^3/s, L_eff = 1.42 m
Rectangular weir discharge with discharge coefficient = 0.62 and crest length = 2 m.	NEEDS_INPUT	rectangular_weir_discharge needs H (head over the crest). Example: solve('rectangular_weir_discharge', Cd=0.62, b='2 m',
Rectangular weir discharge with discharge coefficient = 0.62, crest length = 2 m and head = 0.35 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Rectangular weir discharge with discharge coefficient = 1.4, crest length = 2 m and head = 0.35 m.	INVALID_INPUT	Sharp-crested rectangular weir discharge: the discharge coefficient must not exceed 1.

V-notch (triangular) weir discharge, `v_notch_weir_discharge`

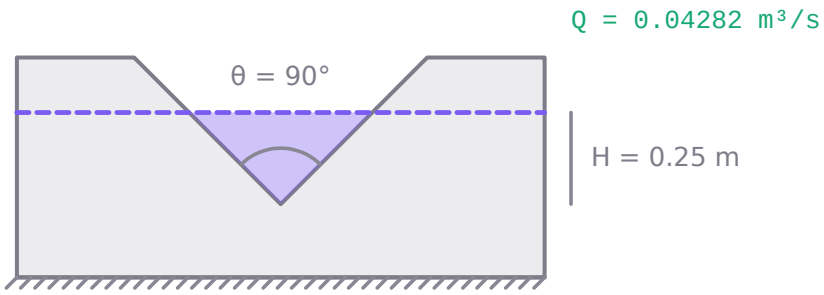
Discharge over a sharp-crested triangular (V-notch) weir of notch angle theta for a stated discharge coefficient.

$$Q = \tfrac{8}{15}\, C_d \sqrt{2g}\, \tan\frac{\theta}{2}\, H^{5/2}$$

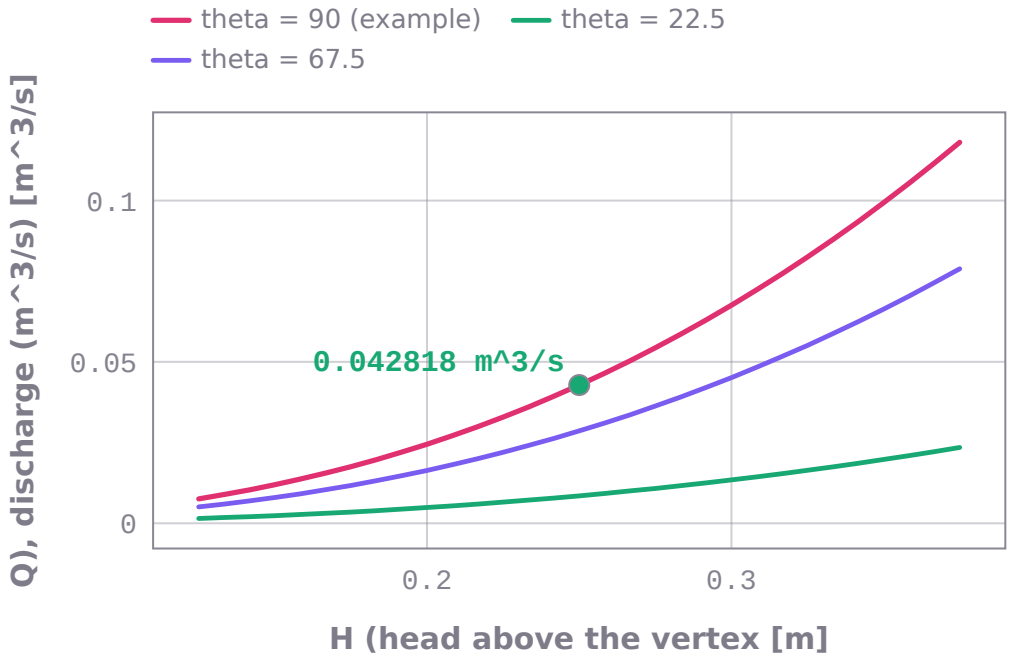
input	meaning	condition
`Cd`	discharge coefficient (stated, e.g. 0.58)	
`theta`	notch angle (degrees)	
`H`	head above the vertex	

output	meaning
`Q`	discharge (m^3/s)

- **Method:** $Q = (8/15)$ Cd sqrt(2g) tan(theta/2) $H^{5/2}$
- **Assumes:** Cd stated; $0 < \theta < 180$ degrees; free nappe
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Front view of a triangular (V-notch) weir with notch angle θ = 90° and head H = 0.25 m above the vertex (Cd = 0.58), notch and head to scale. Marked: the returned discharge Q = 0.042818 m³/s.



Q (discharge (m^3/s)) against H (head above the vertex) from 0.125 m to 0.375 m. One curve for each value of theta (notch angle (degrees)): 90, 22.5, 67.5. Other inputs as in the worked example: Cd = 0.58. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, H = 0.25 m, Q = 0.042818 m^3/s.

```
civil_engineering_ai999x.solve('v_notch_weir_discharge', Cd=0.58, theta=90.0, H=0.25)
# -> Q = 0.042818 m^3/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('v_notch_weir_discharge', Cd=0.6, theta=60.0, H=0.15)
# -> Q = 0.00713127 m^3/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
V notch weir discharge with discharge coefficient = 0.58, notch	SOLVED	Q = 0.042818 m^3/s

question	outcome	result
angle = 90 degrees and head = 0.25 m.		
V notch weir discharge with discharge coefficient = 0.6, notch angle = 60 degrees and head = 0.15 m.	SOLVED	Q = 0.00713127 m^3/s
V notch weir discharge with discharge coefficient = 0.58 and notch angle = 90 degrees.	NEEDS_INPUT	v_notch_weir_discharge needs H (head above the vertex). Example: solve('v_notch_weir_discharge', Cd=0.58, theta=90, H='0
V notch weir discharge with discharge coefficient = 0.58, notch angle = 90 degrees and head = 0.25 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
V notch weir discharge with discharge coefficient = 0.58, notch angle = 200 degrees and head = 0.25 m.	INVALID_INPUT	V-notch (triangular) weir discharge: the notch angle must be below 180 degrees.

Broad-crested weir discharge, `broad_crested_weir_discharge`

Discharge over a broad-crested weir for a stated discharge coefficient (critical depth $2H/3$ on the crest, approach velocity head included in H).

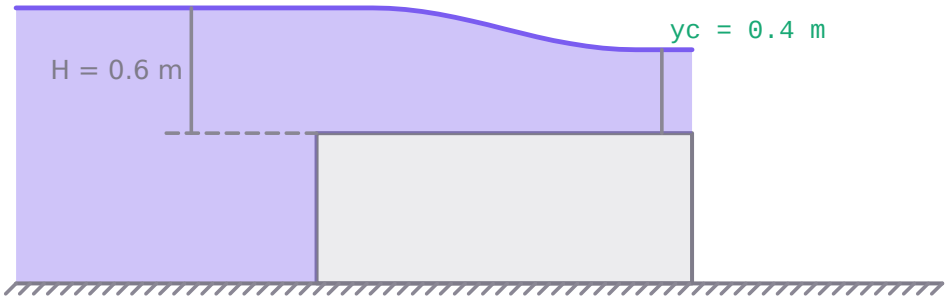
$$Q = C_d\, b \sqrt{g} \left(\tfrac{2}{3} H\right)^{3/2}$$

input	meaning	condition
`Cd`	discharge coefficient (stated, e.g. 0.95)	
`b`	crest width	
`H`	upstream head over the crest	

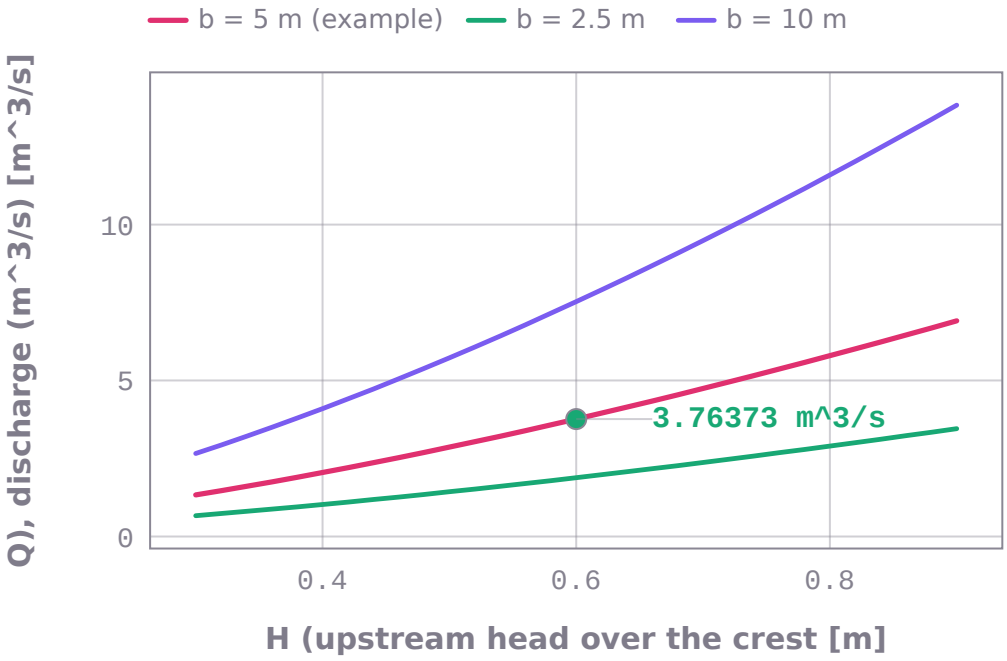
output	meaning
`Q`	discharge (m^3/s)
`yc`	critical depth on the crest $2H/3$ (m)

- **Method:** $Q = Cd$ b sqrt(g) $(2H/3)^{3/2}$
- **Assumes:** Cd stated; critical flow on the crest; H includes the approach velocity head
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

Q = 3.764 m³/s (b = 5 m)



Side view of a broad-crested weir with head H = 0.6 m above the crest (b = 5.0 m, Cd = 0.95); head and the returned critical depth over the crest yc = 0.4 m to scale, crest height and length and the shape of the drawdown schematic. Marked: the returned discharge Q = 3.76373 m³/s.



Q (discharge (m^3/s)) against H (upstream head over the crest) from 0.3 m to 0.9 m. One curve for each value of b (crest width): 5 m, 2.5 m, 10 m. Other inputs as in the worked example: Cd = 0.95. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, H = 0.6 m, Q = 3.76373 m^3/s.

```
civil_engineering_ai999x.solve('broad_crested_weir_discharge', Cd=0.95, b=5.0, H=0.6)
# -> Q = 3.76373 m^3/s, yc = 0.4 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('broad_crested_weir_discharge', Cd=1.0, b=2.0, H=0.3)
# -> Q = 0.560286 m^3/s, yc = 0.2 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Broad crested weir discharge with discharge coefficient = 0.95, crest width = 5 m and head = 0.6 m.	SOLVED	Q = 3.76373 m^3/s, yc = 0.4 m
Broad crested weir discharge with discharge coefficient = 1, crest width = 2 m and head = 0.3 m.	SOLVED	Q = 0.560286 m^3/s, yc = 0.2 m
Broad crested weir discharge with discharge coefficient = 0.95 and crest width = 5 m.	NEEDS_INPUT	broad_crested_weir_discharge needs H (upstream head over the crest). Example: solve('broad_crested_weir_discharge', Cd=0
Broad crested weir discharge with discharge coefficient = 0.95, crest width = 5 m and head = 0.6 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Discharge through a small orifice under a head, `orifice_discharge`

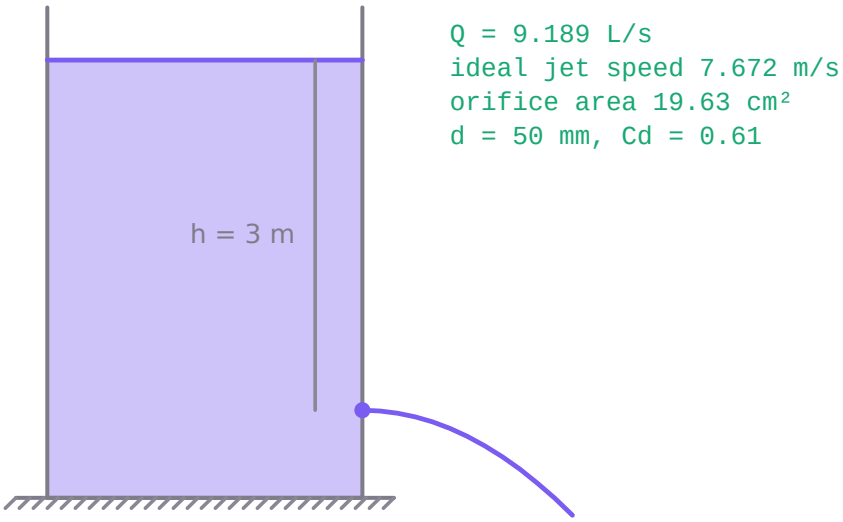
Discharge through a small sharp-edged orifice of diameter d under a constant head h for a stated discharge coefficient (Torricelli's velocity times Cd times the orifice area).

$$Q = C_d \, \frac{\pi d^2}{4} \sqrt{2gh}$$

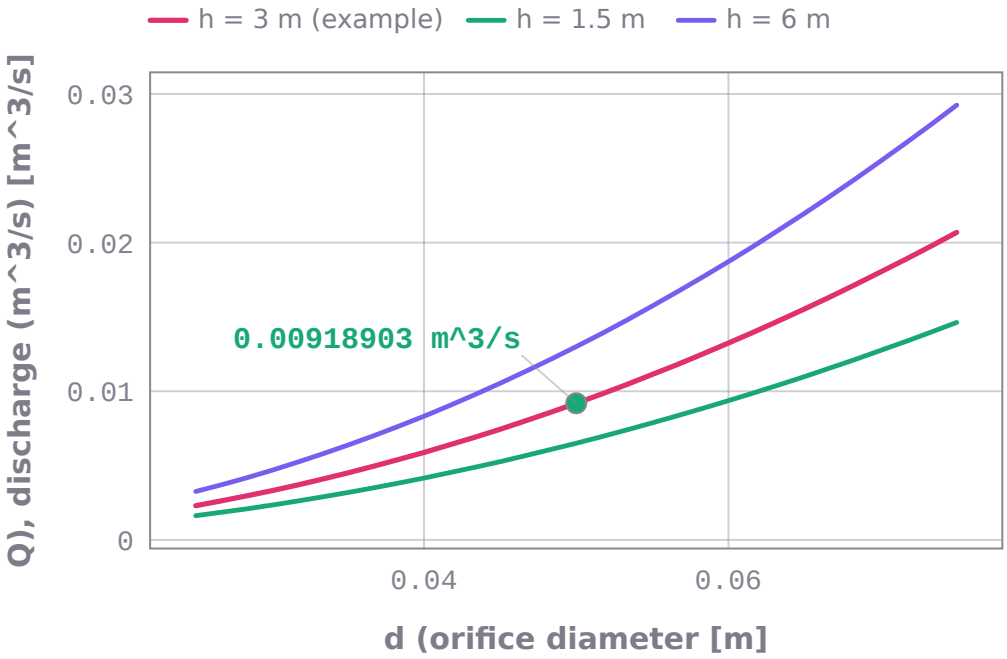
input	meaning	condition
`Cd`	discharge coefficient (stated, e.g. 0.61)	
`d`	orifice diameter	
`h`	head above the orifice centre	

output	meaning
`Q`	discharge (m^3/s)
`V_ideal`	ideal jet velocity sqrt(2 g h) (m/s)
`area`	orifice area (m^2)

- **Method:** $Q = Cd$ (pi d^2/4) sqrt(2 g h)
- **Assumes:** small orifice (d much smaller than h); free jet; Cd stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A tank discharging through a sharp-edged orifice of diameter 50 mm (Cd = 0.61) under a head h = 3 m (head to scale; tank and jet schematic). Marked: the returned discharge Q = 9.18903 L/s, ideal jet velocity 7.67203 m/s and orifice area 19.635 cm².



Q (discharge (m^3/s)) against d (orifice diameter) from 0.025 m to 0.075 m. One curve for each value of h (head above the orifice centre): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: Cd = 0.61. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, d = 0.05 m, Q = 0.00918903 m^3/s.

```
civil_engineering_ai999x.solve('orifice_discharge', Cd=0.61, d=0.05, h=3.0)
# -> Q = 0.00918903 m^3/s, V_ideal = 7.67203 m/s, area = 0.0019635 m^2   (SOLVED)
```

```
civil_engineering_ai999x.solve('orifice_discharge', Cd=0.6, d=0.1, h=1.2)
# -> Q = 0.0228655 m^3/s, V_ideal = 4.85222 m/s, area = 0.00785398 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Orifice discharge with discharge coefficient = 0.61, diameter = 50	SOLVED	Q = 0.00918903 m^3/s, V_ideal = 7.67203 m/s, area = 0.0019635 m^2

question	outcome	result
mm and head = 3 m.		
Orifice discharge with discharge coefficient = 0.6, diameter = 100 mm and head = 1.2 m.	SOLVED	Q = 0.0228655 m^3/s, V_ideal = 4.85222 m/s, area = 0.00785398 m^2
Orifice discharge with discharge coefficient = 0.61 and diameter = 50 mm.	NEEDS_INPUT	orifice_discharge needs h (head above the orifice centre). Example: solve('orifice_discharge', Cd=0.61, d='0.05 m', h='3
Orifice discharge with discharge coefficient = 0.61, diameter = 50 mm and head = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Orifice discharge with discharge coefficient = 1.3, diameter = 50 mm and head = 3 m.	INVALID_INPUT	Discharge through a small orifice under a head: the discharge coefficient must not exceed 1.

Hazen-Williams head loss in a pipe, `hazen_williams_head_loss`

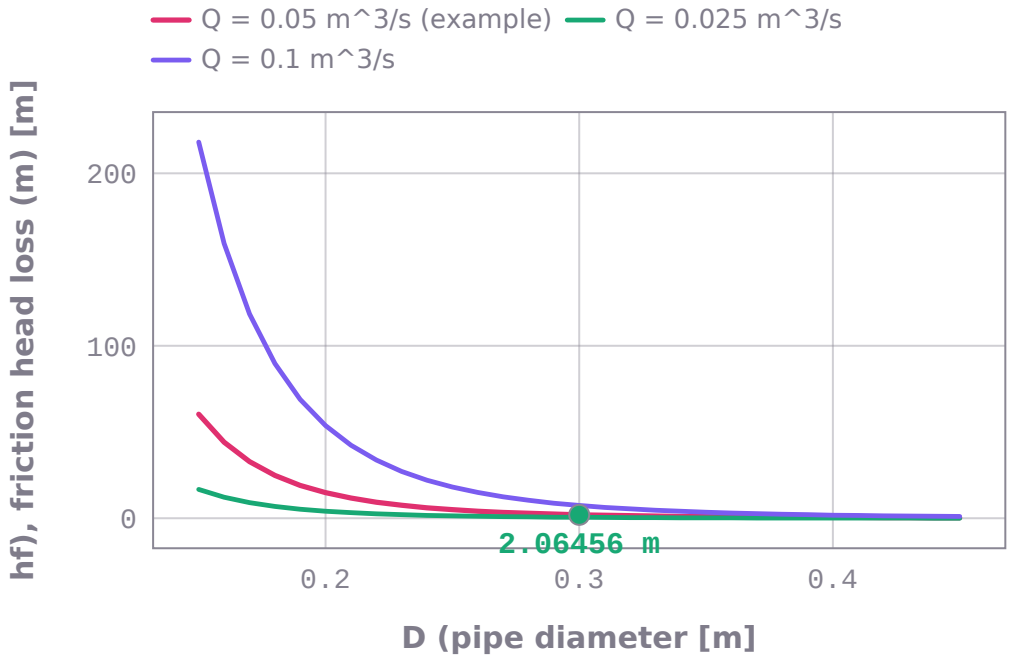
Friction head loss, velocity and friction slope for water flowing at Q through a pipe, from the Hazen-Williams formula (SI form used by EPANET).

$$h_f = \frac{10.67\,L\,Q^{1.852}}{C^{1.852}\,D^{4.871}}$$

input	meaning	condition
`Q`	discharge	
`D`	pipe diameter	
`L`	pipe length	
`C`	Hazen-Williams roughness coefficient	

output	meaning
`hf`	friction head loss (m)
`V`	mean velocity (m/s)
`Sf`	friction slope hf/L (m/m)

- **Method:** $hf = 10.6668$ L Q^1.852/(C^1.852 D^4.871)
- **Assumes:** water near 15 C, turbulent flow; $40 \le C \le 160$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



hf (friction head loss (m)) against D (pipe diameter) from 0.15 m to 0.45 m. One curve for each value of Q (discharge): 0.05 m^3/s, 0.025 m^3/s, 0.1 m^3/s. Other inputs as in the worked example: L = 1000.0 m, C = 120.0. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, D = 0.3 m, hf = 2.06456 m.

```
civil_engineering_ai999x.solve('hazen_williams_head_loss', Q=0.05, D=0.3, L=1000.0,
C=120.0)
# -> hf = 2.06456 m, V = 0.707355 m/s, Sf = 0.00206456   (SOLVED)
```

```
civil_engineering_ai999x.solve('hazen_williams_head_loss', Q=0.008, D=0.1, L=250.0,
C=130.0)
# -> hf = 3.15119 m, V = 1.01859 m/s, Sf = 0.0126048   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hazen williams head loss with discharge = 50 L/s, diameter = 300 mm, length = 1000 m and Hazen-Williams C = 120.	SOLVED	hf = 2.06456 m, V = 0.707355 m/s, Sf = 0.00206456
Hazen williams head loss with discharge = 8 L/s, diameter = 100 mm, length = 250 m and Hazen-Williams C = 130.	SOLVED	hf = 3.15119 m, V = 1.01859 m/s, Sf = 0.0126048
Hazen williams head loss with discharge = 50 L/s, diameter = 300 mm and length = 1000 m.	NEEDS_INPUT	hazen_williams_head_loss needs C (Hazen-Williams roughness coefficient). Example: solve('hazen_williams_head_loss', Q='0
Hazen williams head loss with discharge = 50 L/s, diameter = 300 mm, length = 1000 m and Hazen-Williams C = 120 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Hazen williams head loss with discharge = 50 L/s, diameter = 300 mm, length = 1000 m and Hazen-Williams C = 400.	INVALID_INPUT	Hazen-Williams head loss in a pipe: the Hazen-Williams C must lie between 40 and 160.

Pipe flow from a head loss (Hazen-Williams), `hazen_williams_flow`

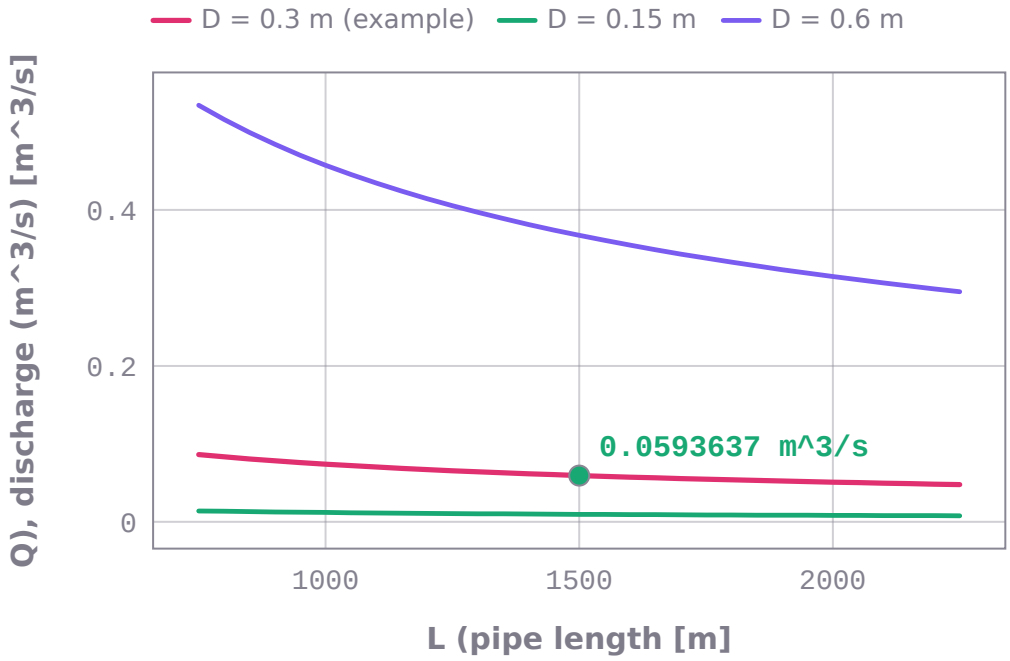
The discharge a pipe carries under a given friction head loss (for example between two reservoirs), from the Hazen-Williams formula.

$$Q = \left(\frac{h_f \, C^{1.852} D^{4.871}}{10.67 \, L} \right)^{0.54}$$

input	meaning	condition
`hf`	friction head loss	
`D`	pipe diameter	
`L`	pipe length	
`C`	Hazen-Williams roughness coefficient	

output	meaning
`Q`	discharge (m^3/s)
`V`	mean velocity (m/s)

- **Method:** Q = (hf $C^{1.852}$ D^4.871/(10.6668 L))^$(1/1.852)$
- **Assumes:** $40 \le C \le 160$; $hf > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (discharge (m^3/s)) against L (pipe length) from 750 m to 2250 m. One curve for each value of D (pipe diameter): 0.3 m, 0.15 m, 0.6 m. Other inputs as in the worked example: hf = 5.0 m, C = 110.0. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, L = 1500 m, Q = 0.0593637 m^3/s.

```
civil_engineering_ai999x.solve('hazen_williams_flow', hf=5.0, D=0.3, L=1500.0, C=110.0)
# -> Q = 0.0593637 m^3/s, V = 0.839824 m/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('hazen_williams_flow', hf=12.0, D=0.15, L=800.0, C=130.0)
# -> Q = 0.0255288 m^3/s, V = 1.44463 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flow from head loss by hazen williams with head loss = 5 m, diameter = 300 mm, length = 1500 m and Hazen-Williams C = 110.	SOLVED	Q = 0.0593637 m^3/s, V = 0.839824 m/s
Flow from head loss by hazen williams with head loss = 12 m, diameter = 150 mm, length = 800 m and Hazen-Williams C = 130.	SOLVED	Q = 0.0255288 m^3/s, V = 1.44463 m/s
Flow from head loss by hazen williams with head loss = 5 m, diameter = 300 mm and length = 1500 m.	NEEDS_INPUT	hazen_williams_flow needs C (Hazen-Williams roughness coefficient). Example: solve('hazen_williams_flow', hf='5 m', D='0
Flow from head loss by hazen williams with head loss = 5 m, diameter = 300 mm, length = 1500 m and Hazen-Williams C = 110 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Flow division among parallel pipes (Hazen-Williams), `parallel_pipes_hazen_williams`

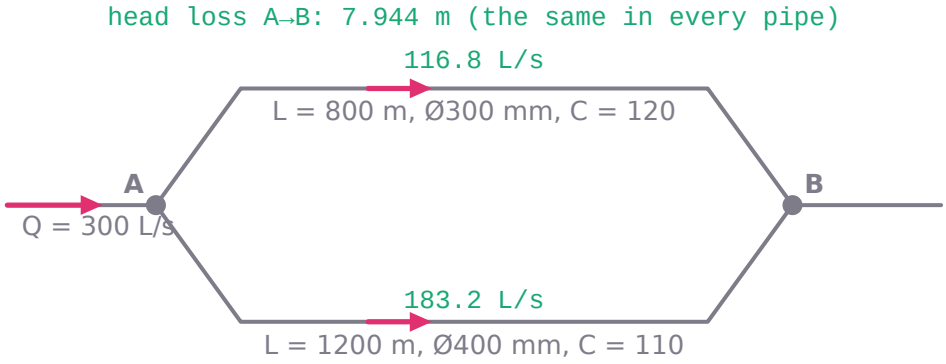
How a total flow Q divides among two or more pipes in parallel between the same two junctions, and the common head loss, from the Hazen-Williams formula.

$$Q = \sum_i \left(\frac{h_f}{k_i}\right)^{0.54}, \quad k_i = \frac{10.67 L_i}{C_i^{1.852} D_i^{4.871}}$$

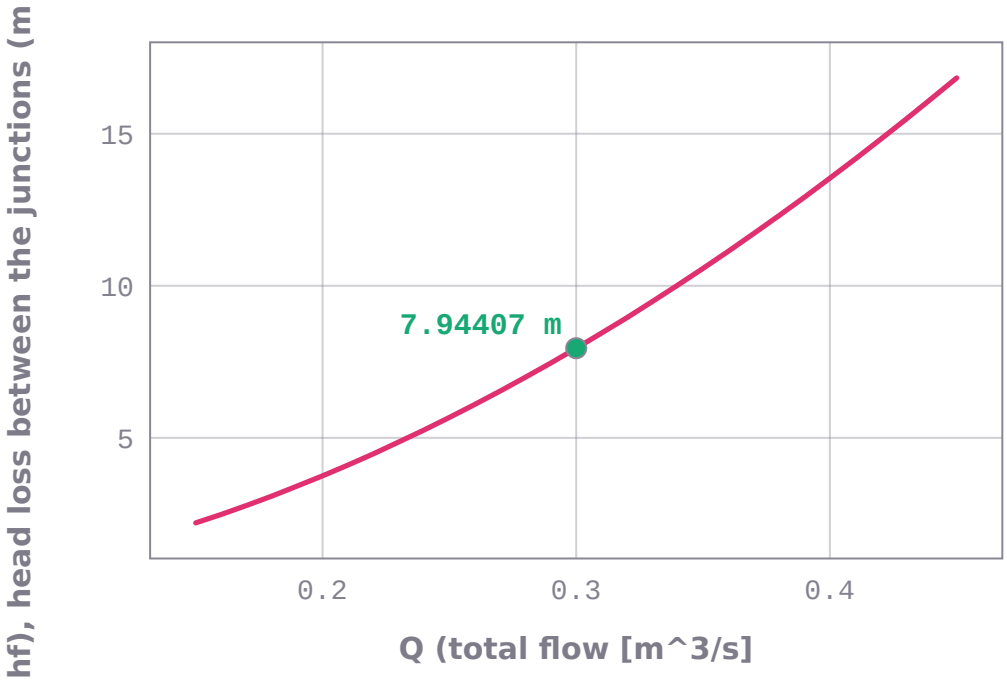
input	meaning	condition
`Q`	total flow	
`L`	pipe lengths in m (one per pipe)	
`D`	pipe diameters in mm (one per pipe)	
`C`	Hazen-Williams C of each pipe	

output	meaning
`hf`	head loss between the junctions (m)
`flows`	flow in each pipe (m^3/s)

- **Method:** hf = (Q / sum $k_i^{-1/1.852}$)^1.852 with $k_i = 10.6668$ L_i/(C_i^1.852 D_i^4.871); $Q_i = (hf/k_i)^{1/1.852}$
- **Assumes:** the same head loss across every pipe; $40 \le C \le 160$; lists of equal length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Pipes in parallel between A and B sharing a total flow Q = 300 L/s (lengths [800, 1200] m, diameters [300, 400] mm, Hazen–Williams C [120, 110]); layout schematic. Marked: the returned split 116.759, 183.241 L/s and the common head loss 7.94407 m.



hf (head loss between the junctions (m)) against Q (total flow) from 0.15 m^3/s to 0.45 m^3/s. Other inputs as in the worked example: L = [800, 1200], D = [300, 400], C = [120, 110]. 31 points computed by civil-engineering-ai999x; marked: the worked example, Q = 0.3 m^3/s, hf = 7.94407 m.

```
civil_engineering_ai999x.solve('parallel_pipes_hazen_williams', Q=0.3, L=[800, 1200], D=
[300, 400], C=[120, 110])
# -> hf = 7.94407 m, flows = [0.116758581689, 0.183241418311] m^3/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('parallel_pipes_hazen_williams', Q=0.15, L=[500, 650, 900],
D=[200, 250, 300], C=[130, 120, 100])
# -> hf = 3.64983 m, flows = [0.0368759234991, 0.0531302920946, 0.0599937844063] m^3/s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallel pipes with total flow = 0.3 m^3/s, lengths = [800, 1200], diameters = [300, 400] and C factors = [120, 110].	SOLVED	hf = 7.94407 m, flows = [0.116758581689, 0.183241418311] m^3/s
Parallel pipes with total flow = 0.15 m^3/s, lengths = [500, 650, 900], diameters = [200, 250, 300] and C factors = [130, 120, 100].	SOLVED	hf = 3.64983 m, flows = [0.0368759234991, 0.0531302920946, 0.0599937844063] m^3/s
Parallel pipes with total flow = 0.3 m^3/s, lengths = [800, 1200] and diameters = [300, 400].	NEEDS_INPUT	parallel_pipes_hazen_williams needs C (Hazen-Williams C of each pipe). Example: solve('parallel_pipes_hazen_williams', Q
Parallel pipes with total flow = 0.3 m^3/s, lengths = [800, 1200], diameters = [300, 400] and C factors = [120, 110] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Pipe network flows by the Hardy Cross method (Hazen-Williams), `hardy_cross_network`

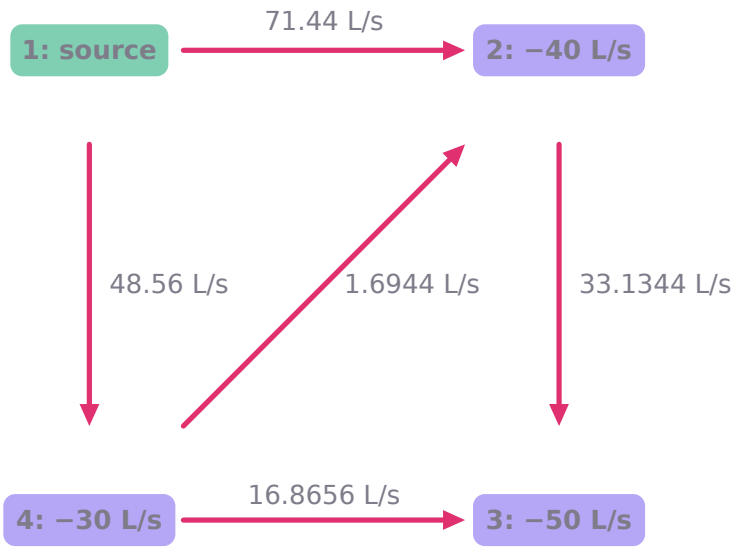
Steady flows in every pipe of a looped water network and the head loss from the source to every node, by Hardy Cross loop corrections (finished with simultaneous loop corrections to full precision), Hazen-Williams friction. Nodes are numbered 1..N; node 1 is the source and supplies the total demand.

$$\Delta Q = -\frac{\sum k\,Q|Q|^{0.852}}{1.852\sum k|Q|^{0.852}}, \quad k = \frac{10.67L}{C^{1.852}D^{4.871}}$$

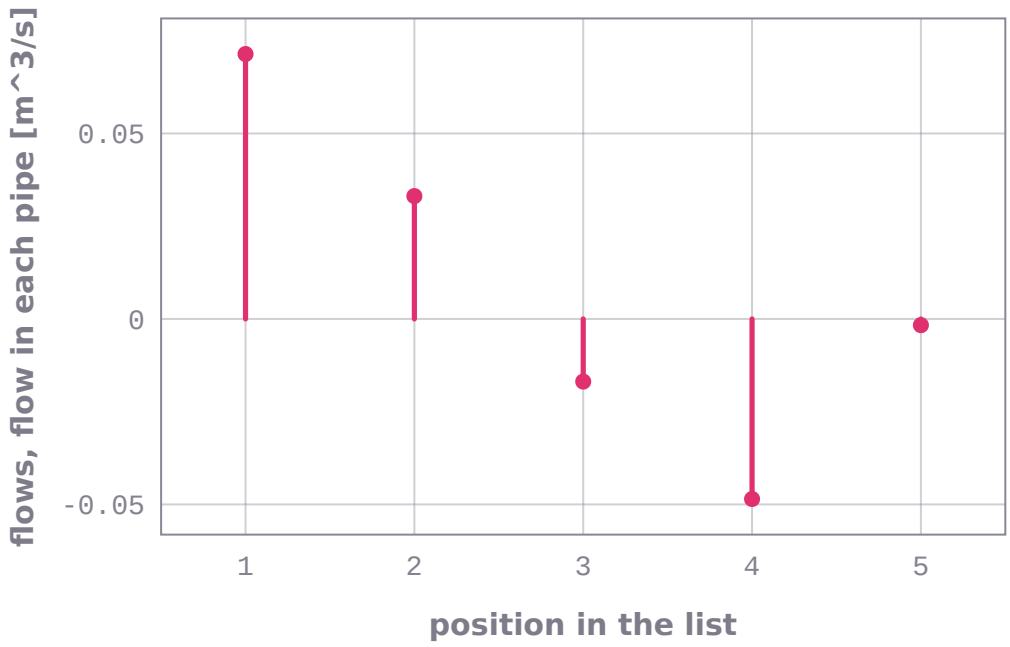
input	meaning	condition
`pipes`	pipes as node pairs $1-2$, $2-3$, ... (the flow sign follows this direction)	
`L`	pipe lengths in m (in pipe order)	
`D`	pipe diameters in mm (in pipe order)	
`C`	Hazen-Williams C of each pipe (in pipe order)	
`demands`	demand at nodes 1..N in m^3/s (node 1, the source, takes 0)	

output	meaning
`flows`	flow in each pipe, positive from its first to its second node (m^3/s)
`head_loss`	head loss from node 1 to each node (m)
`loops`	independent loops
`iterations`	Hardy Cross sweeps before the Newton finish

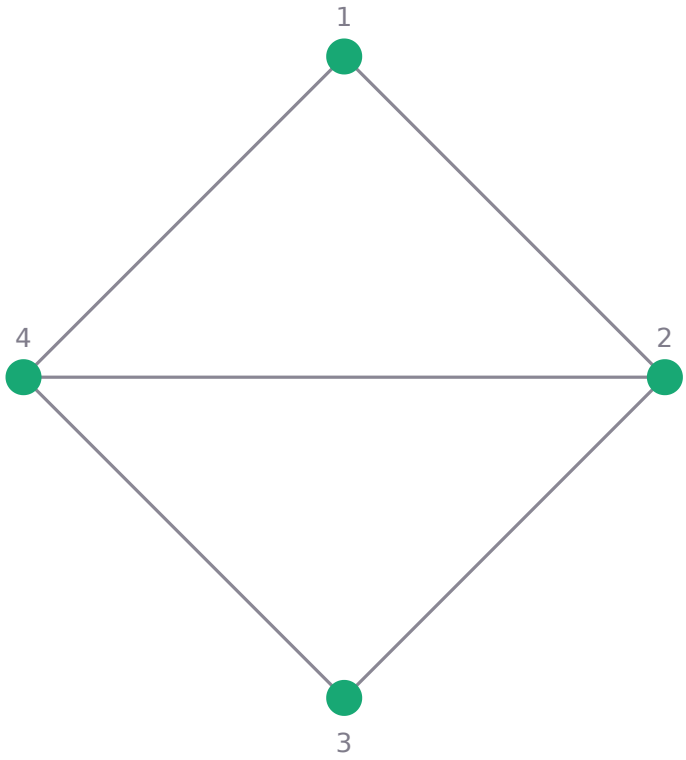
- **Method:** loop corrections dQ = -sum(k Q|Q|^0.852)/(1.852 sum(k|Q|^0.852)) until converged, then Newton on all loops
- **Assumes:** one source (node 1); connected network; $40 \le C \le 160$; demands >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A looped pipe network (5 pipes, 2 loops; lengths [600, 500, 600, 500, 780] m, diameters [300, 250, 200, 250, 200] mm, Hazen–Williams C = 120) balanced by the Hardy Cross method in 14 iterations: arrows show the direction and size of each pipe flow returned by the package (L/s, a negative value flows against the stated pipe direction: 1→2: 71.44, 2→3: 33.1344, 3→4: -16.8656, 4→1: -48.56, 2→4: -1.6944); boxes give each node's demand. The node positions are a layout, not to scale.



flow in each pipe, positive from its first to its second node (m^3/s) (flows) returned by civil-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 0.07144 m^3/s, last -0.0016944 m^3/s, largest 0.07144 m^3/s). Inputs: pipes = 1-2, 2-3, 3-4, 4-1, 2-4, L = [600, 500, 600, 500, 780], D = [300, 250, 200, 250, 200], C = [120, 120, 120, 120, 120], demands = [0, 0.04, 0.05, 0.03].



The network pipes (pipes as node pairs 1-2, 2-3, ... (the flow sign follows this direction)) of the worked example (4 nodes, 5 edges, laid out on a circle). Returned: loops = 2, iterations = 14. Other inputs: L = [600, 500, 600, 500, 780], D = [300, 250, 200, 250, 200], C = [120, 120, 120, 120, 120], demands = [0, 0.04, 0.05, 0.03].

```
civil_engineering_ai999x.solve('hardy_cross_network', pipes='1-2, 2-3, 3-4, 4-1, 2-4', L=
[600, 500, 600, 500, 780], D=[300, 250, 200, 250, 200], C=[120, 120, 120, 120, 120],
demands=[0, 0.04, 0.05, 0.03])
# -> flows = [0.0714399845267, 0.0331343839504, -0.0168656160496, -0.0485600154733,
-0.00169439942369] m^3/s, head_loss = [0.0, 2.39874834362, 3.56973630784, 2.37675423164] m,
loops = 2, iterations = 14   (SOLVED)
```

```
civil_engineering_ai999x.solve('hardy_cross_network', pipes='1-2, 2-3, 3-4, 4-1', L=[1000,
800, 1000, 800], D=[400, 300, 300, 300], C=[130, 110, 110, 120], demands=[0, 0.05, 0.06,
0.04])
# -> flows = [0.090876743882, 0.040876743882, -0.019123256118, -0.059123256118] m^3/s,
head_loss = [0.0, 1.32566122834, 2.66183379844, 2.25278888236] m, loops = 1, iterations = 4
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hardy cross network with pipe list = [1-2, 2-3, 3-4, 4-1, 2-4], lengths = [600, 500, 600, 500, 780], diameters = [300, 250, 200, 250, 200], C factors = [120, 120, 120, 120, 120] and node demands = [0, 0.04, 0.05, 0.03].	SOLVED	flows = [0.0714399845267, 0.0331343839504, -0.0168656160496, -0.0485600154733, -0.00169439942369] m^3/s, head_loss = [0.0, 2.39874834362, 3.56973630784, 2.37675423164] m, loops = 2, iterations = 14

question	outcome	result
Hardy cross network with pipe list = [1-2, 2-3, 3-4, 4-1], lengths = [1000, 800, 1000, 800], diameters = [400, 300, 300, 300], C factors = [130, 110, 110, 120] and node demands = [0, 0.05, 0.06, 0.04].	SOLVED	flows = [0.090876743882, 0.040876743882, -0.019123256118, -0.059123256118] m^3/s, head_loss = [0.0, 1.32566122834, 2.66183379844, 2.25278888236] m, loops = 1, iterations = 4
Hardy cross network with pipe list = [1-2, 2-3, 3-4, 4-1, 2-4], lengths = [600, 500, 600, 500, 780], diameters = [300, 250, 200, 250, 200] and C factors = [120, 120, 120, 120, 120].	NEEDS_INPUT	hardy_cross_network needs demands (demand at nodes 1..N in m^3/s (node 1, the source, takes 0)). Example: solve('hardy_c
Hardy cross network with pipe list = [1-2, 2-3, 3-4, 4-1, 2-4], lengths = [600, 500, 600, 500, 780], diameters = [300, 250, 200, 250, 200], C factors = [120, 120, 120, 120, 120] and node demands = [0, 0.04, 0.05, 0.03] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Three-reservoir problem (Hazen-Williams), `three_reservoir_problem`

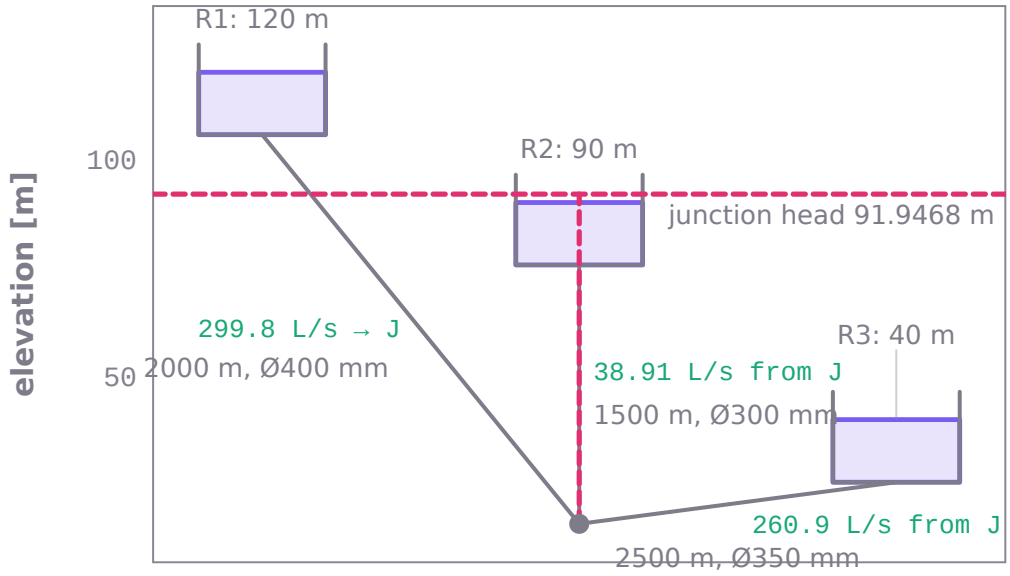
Head at the junction of three pipes from three reservoirs and the flow in each pipe (positive toward the junction), with Hazen-Williams friction and no demand at the junction.

$$\sum_{i=1}^{3} \operatorname{sgn}(z_i - h)\left(\frac{|z_i - h|}{k_i}\right)^{0.54} = 0$$

input	meaning	condition
`z`	reservoir water levels in m (three)	
`L`	pipe lengths in m (three)	
`D`	pipe diameters in mm (three)	
`C`	Hazen-Williams C of each pipe (three)	

output	meaning
`h_junction`	piezometric head at the junction (m)
`flows`	flow in each pipe toward the junction (m^3/s)

- **Method:** bisection on the junction head h: sum of (z_i - h)/|z_i - h| (|z_i - h|/k_i)^(1/1.852) = 0
- **Assumes:** no demand at the junction; $40 \le C \le 160$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Three reservoirs with water levels [120, 90, 40] m joined at a junction J by pipes of lengths [2000, 1500, 2500] m and diameters [400, 300, 350] mm (Hazen–Williams C = [120, 120, 120]); elevations to scale, layout schematic. Marked: the returned piezometric head at the junction 91.9468 m and the pipe flows 299.838, -38.9149, -260.924 L/s (positive towards the junction).

```
civil_engineering_ai999x.solve('three_reservoir_problem', z=[120, 90, 40], L=[2000, 1500, 2500], D=[400, 300, 350], C=[120, 120, 120])
# -> h_junction = 91.9468 m, flows = [0.299838366058, -0.0389148570849, -0.260923508973] m^3/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('three_reservoir_problem', z=[100, 95, 60], L=[1000, 1200, 800], D=[300, 250, 300], C=[130, 110, 100])
# -> h_junction = 87.8349 m, flows = [0.141141557943, 0.0503454547189, -0.191487012662] m^3/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Three reservoir problem with levels = [120, 90, 40], lengths = [2000, 1500, 2500], diameters = [400, 300, 350] and C factors = [120, 120, 120].	SOLVED	h_junction = 91.9468 m, flows = [0.299838366058, -0.0389148570849, -0.260923508973] m^3/s
Three reservoir problem with levels = [100, 95, 60], lengths = [1000, 1200, 800], diameters = [300, 250, 300] and C factors = [130, 110, 100].	SOLVED	h_junction = 87.8349 m, flows = [0.141141557943, 0.0503454547189, -0.191487012662] m^3/s
Three reservoir problem with levels = [120, 90, 40], lengths = [2000, 1500, 2500] and diameters = [400, 300, 350].	NEEDS_INPUT	three_reservoir_problem needs C (Hazen-Williams C of each pipe (three)). Example: solve('three_reservoir_problem', z=[12
Three reservoir problem with levels = [120, 90, 40], lengths = [2000, 1500, 2500], diameters = [400, 300, 350] and C factors = [120, 120, 120] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

SCS curve-number runoff depth, `scs_curve_number_runoff`

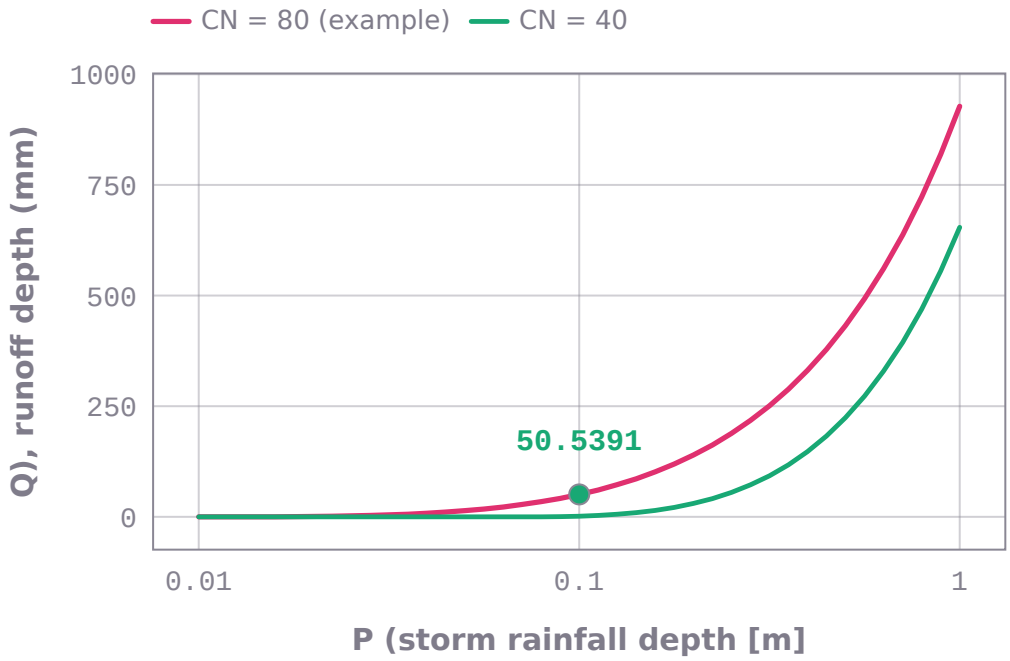
Direct runoff depth from a storm rainfall depth by the SCS (NRCS) curve-number method, with the potential retention S and the initial abstraction $Ia = \lambda$ S (lambda = 0.2 unless stated).

$$Q = \frac{(P - I_a)^2}{P - I_a + S}, \quad S = \frac{25400}{CN} - 254, \quad I_a = \lambda S$$

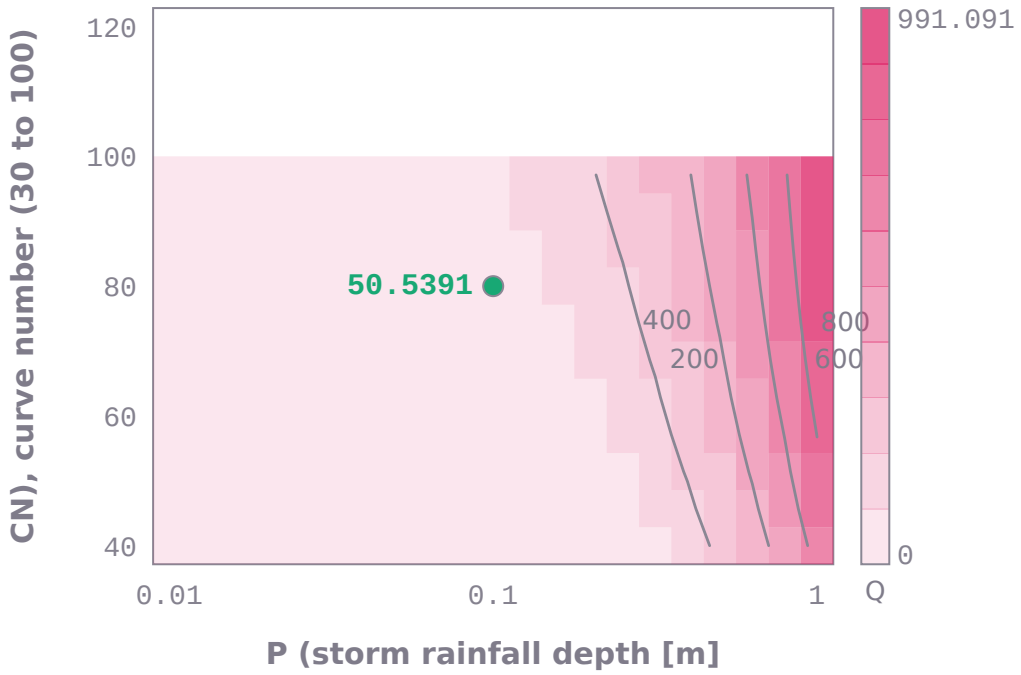
input	meaning	condition
`P`	storm rainfall depth	
`CN`	curve number (30 to 100)	
`lam`	initial-abstraction ratio (default 0.2)	

output	meaning
`Q`	runoff depth (mm)
`S`	potential maximum retention (mm)
`Ia`	initial abstraction (mm)
`C`	runoff ratio Q/P

- **Method:** $S = 25400/CN - 254$ (mm), $Ia = \lambda$ S, $Q = (P - Ia)^2/(P - Ia + S)$ for $P > Ia$, else 0
- **Assumes:** $30 \le CN \le 100$; rainfall depth in mm; $0 \le \lambda \le 0.3$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (runoff depth (mm)) against P (storm rainfall depth) from 0.01 m to 1 m on a logarithmic axis. One curve for each value of CN (curve number (30 to 100)): 80, 40. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, P = 0.1 m, Q = 50.5391.



Q (runoff depth (mm)) over P (storm rainfall depth) from 0.01 m to 1 m and CN (curve number (30 to 100)) from 40 to 120 (logarithmic horizontal axis): each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 0 to dark 991.091); iso-lines at Q = 200, 400, 600, 800; blank cells: inputs the capability refuses. Marked: the worked example, P = 0.1 m, CN = 80, Q = 50.5391.

```
civil_engineering_ai999x.solve('scs_curve_number_runoff', P=0.1, CN=80.0)
# -> Q = 50.5391, S = 63.5, Ia = 12.7, C = 0.505391   (SOLVED)
```

```
civil_engineering_ai999x.solve('scs_curve_number_runoff', P=0.06, CN=70.0, lam=0.05)
# -> Q = 18.2143, S = 108.857, Ia = 5.44286, C = 0.303572   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Scs curve number runoff with rainfall = 100 mm and curve number = 80.	SOLVED	Q = 50.5391, S = 63.5, Ia = 12.7, C = 0.505391
Scs curve number runoff with rainfall = 60 mm, curve number = 70 and initial abstraction ratio = 0.05.	SOLVED	Q = 18.2143, S = 108.857, Ia = 5.44286, C = 0.303572
Scs curve number runoff with rainfall = 100 mm.	NEEDS_INPUT	scs_curve_number_runoff needs CN (curve number (30 to 100)). Example: solve('scs_curve_number_runoff', P='0.1 m', CN=80)
Scs curve number runoff with rainfall = 100 mm and curve number = 80 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Scs curve number runoff with rainfall = 100 mm and curve number = 120.	INVALID_INPUT	SCS curve-number runoff depth: the curve number must lie between 30 and 100.

Direct runoff hydrograph by unit-hydrograph convolution, `unit_hydrograph_convolution`

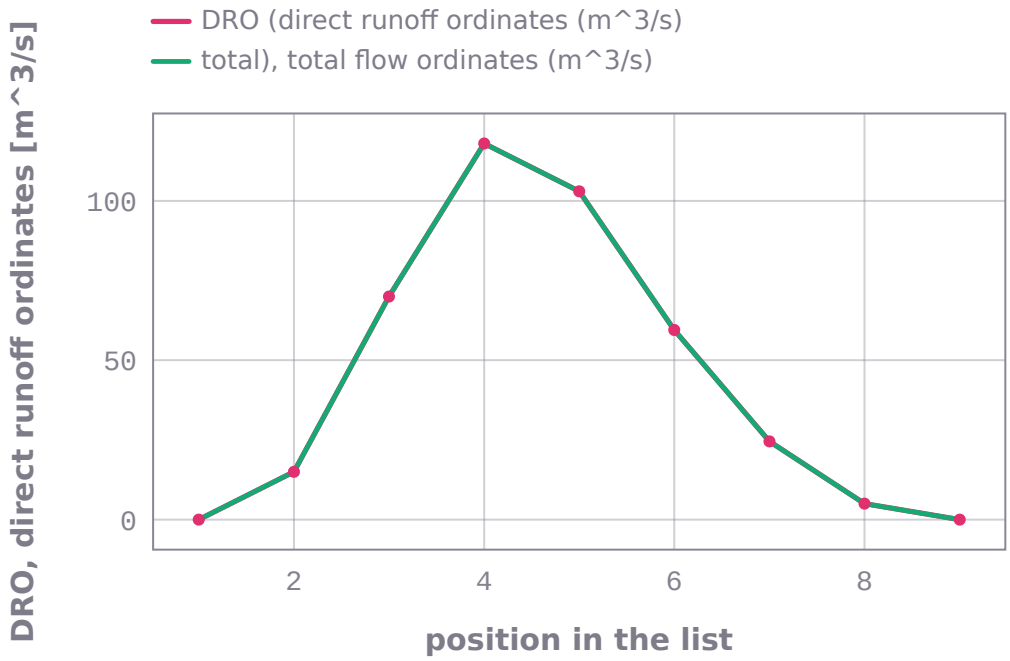
The direct runoff (and total) hydrograph of a storm: the convolution of the excess-rainfall depths of successive periods with the unit hydrograph of the same period, plus a constant base flow.

$$Q_n = \sum_{m=1}^{\min(n,M)} P_m \, U_{n-m+1}$$

input	meaning	condition
`excess`	excess rainfall in cm per period (one per period)	
`uh`	unit-hydrograph ordinates in m^3/s per cm of excess, one per period	
`baseflow`	constant base flow (default 0)	

output	meaning
`DRO`	direct runoff ordinates (m^3/s)
`total`	total flow ordinates (m^3/s)
`peak`	peak total flow (m^3/s)
`peak_period`	period of the peak (1-based)

- **Method:** Q_k = sum_i P_i U_(k-i+1) (discrete convolution)
- **Assumes:** excess depths and the unit hydrograph share one period; linear, time-invariant catchment
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



direct runoff ordinates (m^3/s) (DRO) returned by civil-engineering-ai999x for the worked example, one value for each position in the list (9 values, first 0 m^3/s, last 0 m^3/s, largest 118 m^3/s); with total (total flow ordinates (m^3/s)). Inputs: excess = [1.5, 2.5, 1.0], uh = [0, 10, 30, 22, 12, 5, 0].

```
civil_engineering_ai999x.solve('unit_hydrograph_convolution', excess=[1.5, 2.5, 1.0], uh=
[0, 10, 30, 22, 12, 5, 0])
# -> DRO = [0.0, 15.0, 70.0, 118.0, 103.0, 59.5, 24.5, 5.0, 0.0] m^3/s, total = [0.0, 15.0,
70.0, 118.0, 103.0, 59.5, 24.5, 5.0, 0.0] m^3/s, peak = 118 m^3/s, peak_period = 4
(SOLVED)
```

```
civil_engineering_ai999x.solve('unit_hydrograph_convolution', excess=[2.0, 0.8], uh=[5, 25,
40, 28, 15, 6], baseflow=10.0)
# -> DRO = [10.0, 54.0, 100.0, 88.0, 52.4, 24.0, 4.8] m^3/s, total = [20.0, 64.0, 110.0,
98.0, 62.4, 34.0, 14.8] m^3/s, peak = 110 m^3/s, peak_period = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Unit hydrograph convolution with excess rainfall = [1.5, 2.5, 1] and unit hydrograph = [0, 10, 30, 22, 12, 5, 0].	SOLVED	DRO = [0.0, 15.0, 70.0, 118.0, 103.0, 59.5, 24.5, 5.0, 0.0] m^3/s, total = [0.0, 15.0, 70.0, 118.0, 103.0, 59.5, 24.5, 5.0, 0.0] m^3/s, peak = 118 m^3/s, peak_period = 4
Unit hydrograph convolution with excess rainfall = [2, 0.8], unit hydrograph = [5, 25, 40, 28, 15, 6] and base flow = 10 m^3/s.	SOLVED	DRO = [10.0, 54.0, 100.0, 88.0, 52.4, 24.0, 4.8] m^3/s, total = [20.0, 64.0, 110.0, 98.0, 62.4, 34.0, 14.8] m^3/s, peak = 110 m^3/s, peak_period = 3
Unit hydrograph convolution with excess rainfall = [1.5, 2.5, 1].	NEEDS_INPUT	unit_hydrograph_convolution needs uh (unit-hydrograph ordinates in m^3/s per cm of excess, one per period). Example: sol
Unit hydrograph convolution with excess rainfall = [1.5, 2.5, 1] and unit hydrograph = [0, 10, 30, 22, 12, 5, 0] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Unit hydrograph from a storm by least-squares deconvolution, `unit_hydrograph_derivation`

The unit hydrograph that best reproduces an observed direct-runoff hydrograph from its excess-rainfall depths (least-squares solution of the convolution equations), with the RMS misfit.

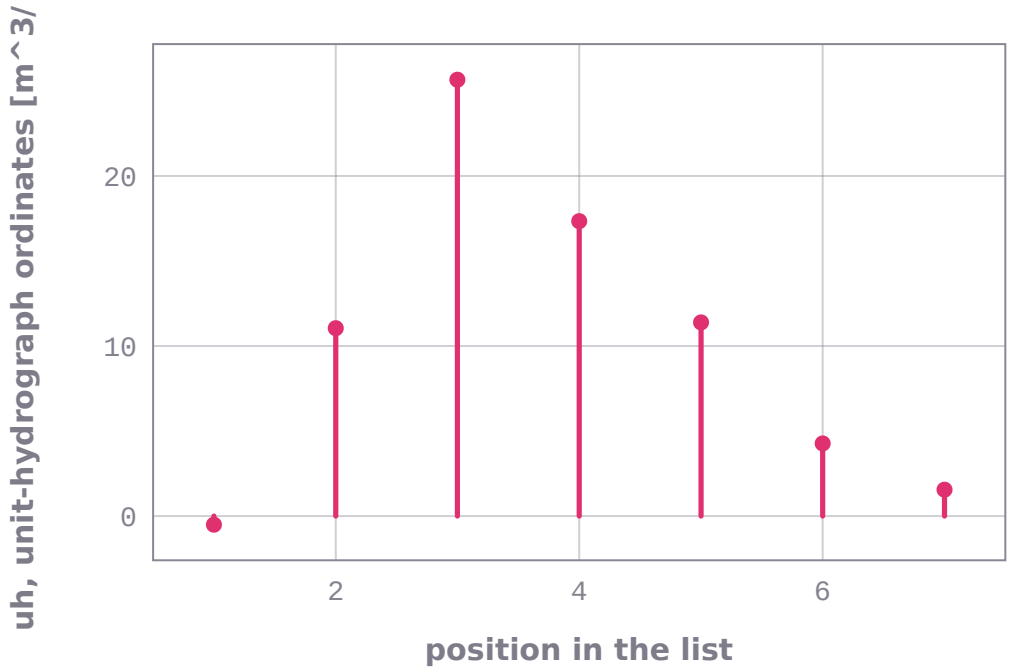
$$\min_{u} \|Au - Q\|_2, \quad A_{kj} = P_{k-j+1}$$

input	meaning	condition
`dro`	observed direct runoff ordinates in m^3/s (base flow removed)	
`excess`	excess rainfall in cm per period	

output	meaning
`uh`	unit-hydrograph ordinates (m^3/s per cm)
`rms_residual`	RMS misfit of the reconstructed runoff (m^3/s)
`peak`	unit-hydrograph peak (m^3/s per cm)

- **Method:** least squares on the convolution system: (A^T A) $u = A^T$ Q
- **Assumes:** len(dro) >= len(excess) + 1; the same period for both

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



unit-hydrograph ordinates (m^3/s per cm) (uh) returned by civil-engineering-ai999x for the worked example, one value for each position in the list (7 values, first -0.502835 m^3/s, last 1.55034 m^3/s, largest 25.6537 m^3/s). Inputs: dro = [0, 15, 61, 77, 52, 29, 11, 3], excess = [1.5, 2.0].

```
civil_engineering_ai999x.solve('unit_hydrograph_derivation', dro=[0, 15, 61, 77, 52, 29,
11, 3], excess=[1.5, 2.0])
# -> uh = [-0.502834815065, 11.0475725314, 25.6537253747, 17.3404996047, 11.3869004489,
4.27012446017, 1.55034025912] m^3/s, rms_residual = 0.401139 m^3/s, peak = 25.6537 m^3/s
(SOLVED)
```

```
civil_engineering_ai999x.solve('unit_hydrograph_derivation', dro=[2, 20, 70, 105, 85, 50,
25, 9, 2], excess=[1.0, 2.5, 0.8])
# -> uh = [1.69772696982, 15.8984865275, 28.8271810427, 20.2485690516, 11.2920123639,
5.60453828918, 1.88161822656] m^3/s, rms_residual = 0.212072 m^3/s, peak = 28.8272 m^3/s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Unit hydrograph derivation with direct runoff = [0, 15, 61, 77, 52, 29, 11, 3] and excess rainfall = [1.5, 2].	SOLVED	uh = [-0.502834815065, 11.0475725314, 25.6537253747, 17.3404996047, 11.3869004489, 4.27012446017, 1.55034025912] m^3/s, rms_residual = 0.401139 m^3/s, peak = 25.6537 m^3/s
Unit hydrograph derivation with direct runoff = [2, 20, 70, 105, 85, 50, 25, 9, 2] and excess rainfall = [1, 2.5, 0.8].	SOLVED	uh = [1.69772696982, 15.8984865275, 28.8271810427, 20.2485690516, 11.2920123639, 5.60453828918, 1.88161822656] m^3/s, rms_residual = 0.212072 m^3/s, peak = 28.8272 m^3/s
Unit hydrograph derivation with direct runoff = [0, 15, 61, 77, 52, 29, 11, 3].	NEEDS_INPUT	unit_hydrograph_derivation needs excess (excess rainfall in cm per period). Example: solve('unit_hydrograph_derivation',

question	outcome	result
Unit hydrograph derivation with direct runoff = [0, 15, 61, 77, 52, 29, 11, 3] and excess rainfall = [1.5, 2] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Areal rainfall by Thiessen polygons, `thiessen_polygon_rainfall`

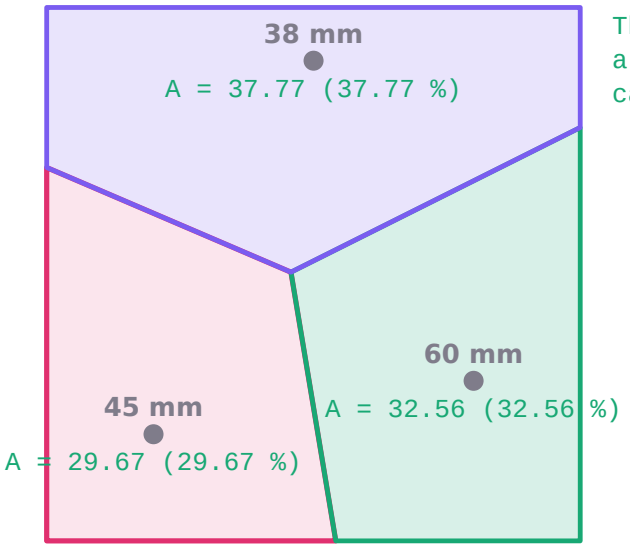
Mean areal rainfall over a catchment by the Thiessen (Voronoi) polygon method: each gauge represents the part of the catchment nearer to it than to any other gauge; the areas, weights and the arithmetic mean are also given. Coordinates in km (areas in km^2), rainfall in mm; the boundary is a simple polygon listed vertex by vertex.

$$\bar{P} = \sum_i \frac{A_i}{A} P_i$$

input	meaning	condition
`gx`	gauge x coordinates in km	
`gy`	gauge y coordinates in km	
`rain`	rainfall at each gauge in mm	
`bx`	catchment boundary x coordinates in km	
`by`	catchment boundary y coordinates in km	

output	meaning
`mean`	Thiessen mean rainfall (mm)
`areas`	area represented by each gauge (km^2)
`weights`	Thiessen weight of each gauge
`arithmetic_mean`	arithmetic mean (mm)
`catchment_area`	catchment area (km^2)

- **Method:** clip the catchment polygon by the perpendicular-bisector half-planes of every pair of gauges; weights = cell area/catchment area
- **Assumes:** distinct gauge locations; simple (non-self-intersecting) boundary polygon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Thiessen polygons of 3 rain gauges (at x = [2, 8, 5], y = [2, 3, 9], rainfall [45, 60, 38] mm) inside the catchment boundary (x = [0, 10, 10, 0], y = [0, 0, 10, 10]); each polygon is the boundary clipped by the perpendicular bisectors to the other gauges, and its area agrees with the area the package returned. Marked: areas 29.6651, 32.5609, 37.774, weights, the returned Thiessen mean 47.24 mm and arithmetic mean 47.6667 mm.

```
civil_engineering_ai999x.solve('thiessen_polygon_rainfall', gx=[2, 8, 5], gy=[2, 3, 9],
rain=[45, 60, 38], bx=[0, 10, 10, 0], by=[0, 0, 10, 10])
# -> mean = 47.24, areas = [29.6650641026, 32.5608974359, 37.7740384615], weights =
[0.296650641026, 0.325608974359, 0.377740384615], arithmetic_mean = 47.6667, catchment_area
= 100   (SOLVED)
```

```
civil_engineering_ai999x.solve('thiessen_polygon_rainfall', gx=[1, 6, 9, 4], gy=[1, 2, 7,
8], rain=[80, 95, 70, 110], bx=[0, 12, 10, 2], by=[0, 0, 10, 9])
# -> mean = 89.6089, areas = [13.2277173913, 34.1398538961, 25.7055444555, 21.926884257],
weights = [0.139239130435, 0.359366883117, 0.270584678479, 0.230809307969], arithmetic_mean
= 88.75, catchment_area = 95   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thiessen polygon rainfall with gauge x = [2, 8, 5], gauge y = [2, 3, 9], rainfall = [45, 60, 38], boundary x = [0, 10, 10, 0] and boundary y = [0, 0, 10, 10].	SOLVED	mean = 47.24, areas = [29.6650641026, 32.5608974359, 37.7740384615], weights = [0.296650641026, 0.325608974359, 0.377740384615], arithmetic_mean = 47.6667, catchment_area = 100
Thiessen polygon rainfall with gauge x = [1, 6, 9, 4], gauge y = [1, 2, 7, 8], rainfall = [80, 95, 70, 110], boundary x = [0, 12, 10, 2] and boundary y = [0, 0, 10, 9].	SOLVED	mean = 89.6089, areas = [13.2277173913, 34.1398538961, 25.7055444555, 21.926884257], weights = [0.139239130435, 0.359366883117, 0.270584678479, 0.230809307969], arithmetic_mean = 88.75, catchment_area = 95
Thiessen polygon rainfall with gauge x = [2, 8, 5], gauge y =	NEEDS_INPUT	thiessen_polygon_rainfall needs by (catchment boundary y coordinates in km). Example:

question	outcome	result
[2, 3, 9], rainfall = [45, 60, 38] and boundary x = [0, 10, 10, 0].		solve('thiessen_polygon_rainfall',
Thiessen polygon rainfall with gauge x = [2, 8, 5], gauge y = [2, 3, 9], rainfall = [45, 60, 38], boundary x = [0, 10, 10, 0] and boundary y = [0, 0, 10, 10] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Risk of exceedance over a design life, `flood_exceedance_risk`

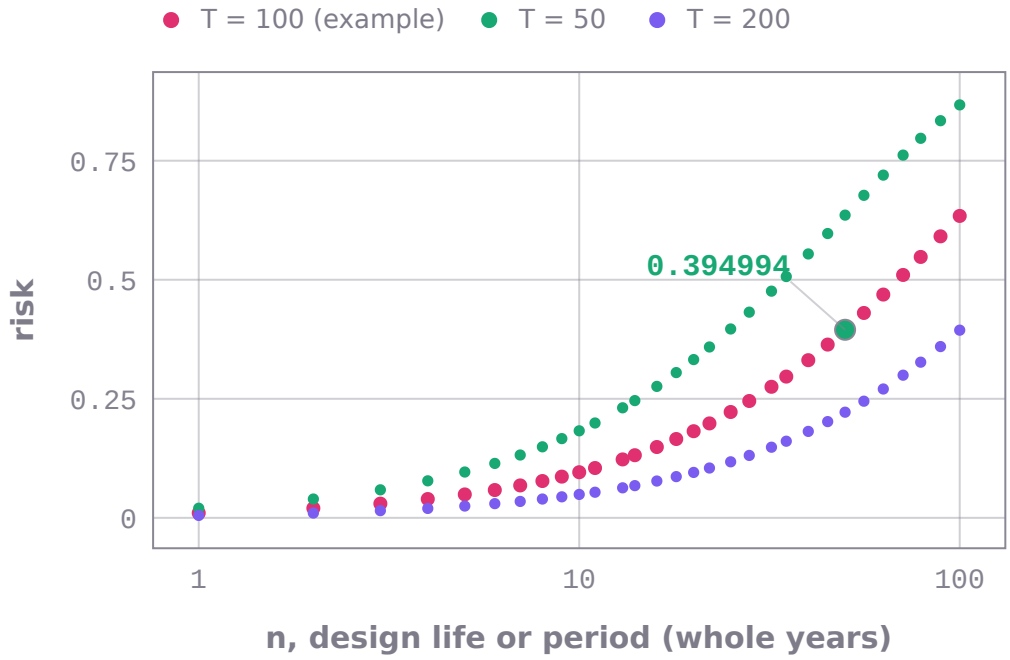
Probability that an event of return period T is equalled or exceeded at least once in n years (independent years), with the annual exceedance probability.

$$R = 1 - \left(1 - \frac{1}{T}\right)^{n}$$

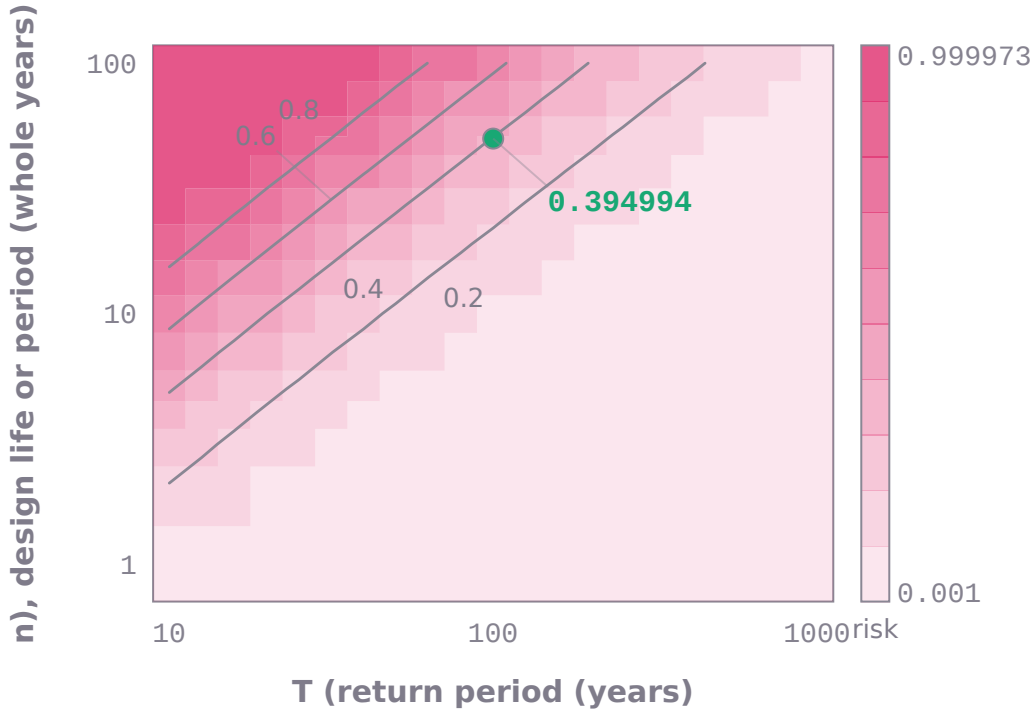
input	meaning	condition
`T`	return period (years)	
`n`	design life or period (whole years)	

output	meaning
`risk`	probability of at least one exceedance in n years
`p_annual`	annual exceedance probability $1/T$
`p_none`	probability of no exceedance

- **Method:** risk = 1 - (1 - 1/T)^n
- **Assumes:** $T > 1$ year; independent years
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



risk (probability of at least one exceedance in n years) against n (design life or period (whole years)) from 1 to 100 on a logarithmic axis. One curve for each value of T (return period (years)): 100, 50, 200. 30 points per curve computed by civil-engineering-ai999x; marked: the worked example, n = 50, risk = 0.394994.



risk (probability of at least one exceedance in n years) over T (return period (years)) from 10 to 1000 and n (design life or period (whole years)) from 1 to 100 (logarithmic axes): each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 0.001 to dark 0.999973); iso-lines at risk = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, T = 100, n = 50, risk = 0.394994.

```
civil_engineering_ai999x.solve('flood_exceedance_risk', T=100.0, n=50)
# -> risk = 0.394994, p_annual = 0.01, p_none = 0.605006   (SOLVED)
```

```
civil_engineering_ai999x.solve('flood_exceedance_risk', T=25.0, n=10)
# -> risk = 0.335167, p_annual = 0.04, p_none = 0.664833   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flood exceedance risk with return period = 100 years and design life = 50 years.	SOLVED	risk = 0.394994, p_annual = 0.01, p_none = 0.605006
Flood exceedance risk with return period = 25 years and design life = 10 years.	SOLVED	risk = 0.335167, p_annual = 0.04, p_none = 0.664833
Flood exceedance risk with return period = 100 years.	NEEDS_INPUT	flood_exceedance_risk needs n (design life or period (whole years)). Example: solve('flood_exceedance_risk', T=100, n=50
Flood exceedance risk with return period = 100 years and design life = 50 years and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Flood exceedance risk with return period = 0.5 years and design life = 10 years.	INVALID_INPUT	Risk of exceedance over a design life: the return period must exceed 1 year.

Return period for an acceptable risk over a design life, `design_return_period_for_risk`

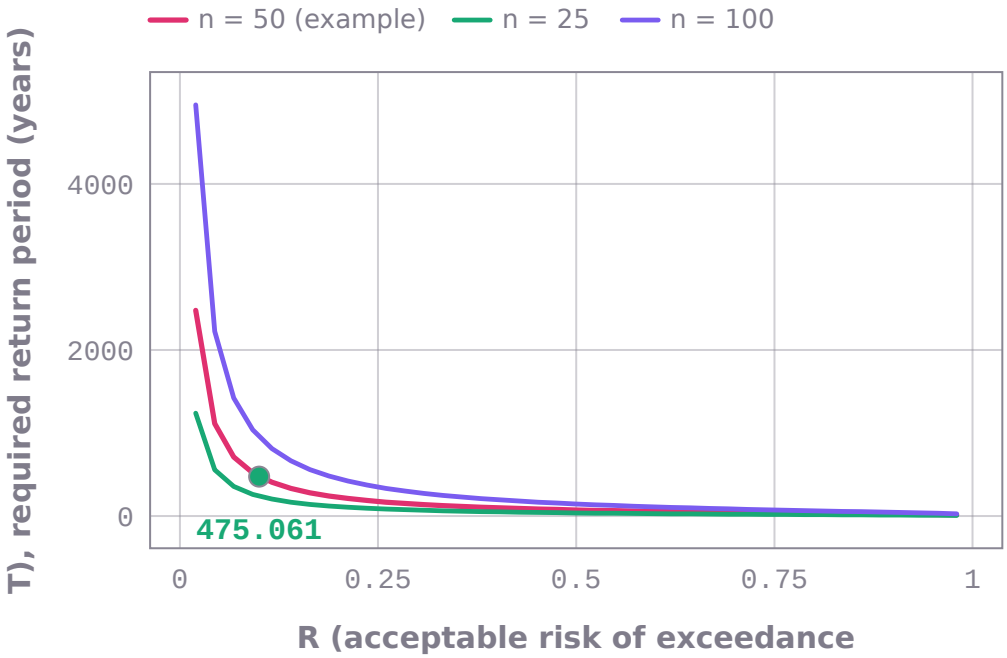
The design return period whose event has only the stated probability R of being exceeded during an n-year life.

$$T = \frac{1}{1-(1-R)^{1/n}}$$

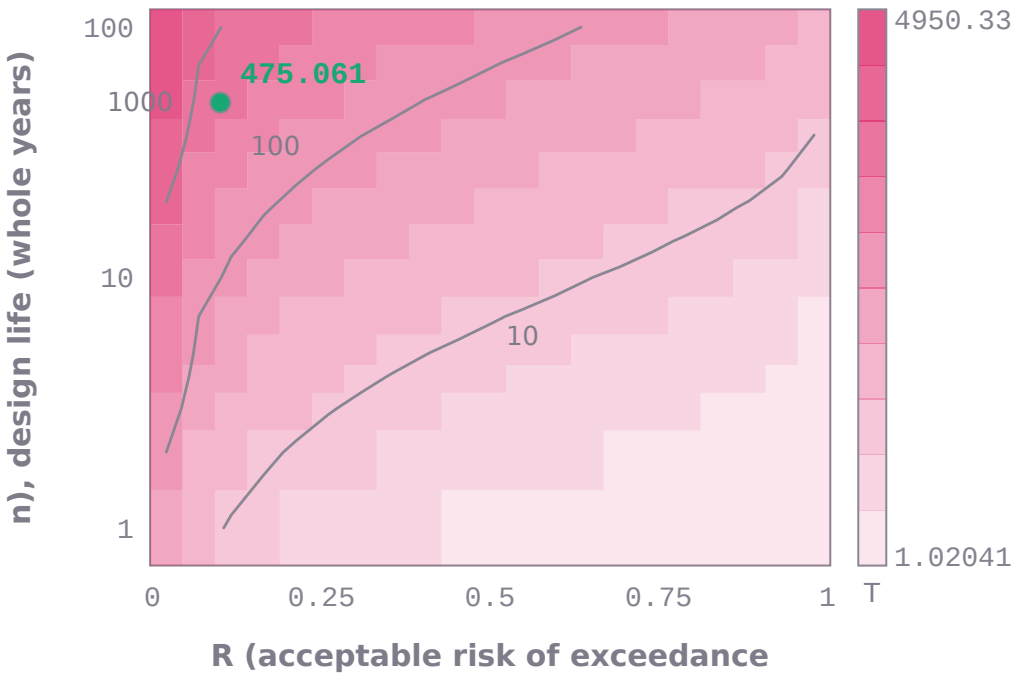
input	meaning	condition
`R`	acceptable risk of exceedance (fraction, or % in a question)	
`n`	design life (whole years)	

output	meaning
`T`	required return period (years)
`p_annual`	annual exceedance probability

- **Method:** T = 1/(1 - (1 - R)^(1/n))
- **Assumes:** $0 < R < 1$; independent years
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



T (required return period (years)) against R (acceptable risk of exceedance (fraction, or % in a question)) from 0.02 to 0.98. One curve for each value of n (design life (whole years)): 50, 25, 100. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, R = 0.1, T = 475.061.



T (required return period (years)) over R (acceptable risk of exceedance (fraction, or % in a question)) from 0.02 to 0.98 and n (design life (whole years)) from 1 to 100 (logarithmic vertical axis): each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 1.02041 to dark 4950.33, logarithmic shading); iso-lines at T = 10, 100, 1000. Marked: the worked example, R = 0.1, n = 50, T = 475.061.

```
civil_engineering_ai999x.solve('design_return_period_for_risk', R=0.1, n=50)
# -> T = 475.061, p_annual = 0.00210499   (SOLVED)
```

```
civil_engineering_ai999x.solve('design_return_period_for_risk', R=0.05, n=25)
# -> T = 487.893, p_annual = 0.00204963   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Design return period for a risk with risk = 0.1 and design life = 50 years.	SOLVED	T = 475.061, p_annual = 0.00210499
Design return period for a risk with risk = 0.05 and design life = 25 years.	SOLVED	T = 487.893, p_annual = 0.00204963
Design return period for a risk with risk = 0.1.	NEEDS_INPUT	design_return_period_for_risk needs n (design life (whole years)). Example: solve('design_return_period_for_risk', R=0.1
Design return period for a risk with risk = 0.1 and design life = 50 years and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Design return period for a risk with risk = 1.5 and design life = 10 years.	INVALID_INPUT	Return period for an acceptable risk over a design life: the acceptable risk must be a fraction between 0 and 1.

Gumbel (EV1) design flood, `gumbel_design_flood`

The T-year flood from a series of annual maximum discharges by the Gumbel (extreme value type I) distribution fitted by the method of moments (Chow's frequency factor for a long record; sample standard deviation with n - 1).

$$Q_T = \bar{Q} + K_T s, \quad K_T = -\frac{\sqrt{6}}{\pi}\left[0.5772 + \ln\ln\frac{T}{T-1}\right]$$

input	meaning	condition
`peaks`	annual maximum discharges in m^3/s	
`T`	return period (years)	

output	meaning
`Q_T`	T-year flood (m^3/s)
`K_T`	frequency factor
`mean`	mean annual maximum (m^3/s)
`std`	sample standard deviation (m^3/s)

- **Method:** K_T = -(sqrt 6/pi)(0.5772 + ln ln($T/(T-1)$)), Q_T = mean + K_T s
- **Assumes:** at least five annual maxima; $T > 1$ year; method of moments (no sample-size correction table)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q_T (T-year flood (m^3/s)) against T (return period (years)) from 10 to 1000 on a logarithmic axis. Other inputs as in the worked example: peaks = [420, 380, 510, 290, 640, 455, 370, 520, 600…. 41 points computed by civil-engineering-ai999x; marked: the worked example, T = 100, Q_T = 812.637 m^3/s.

```
civil_engineering_ai999x.solve('gumbel_design_flood', peaks=[420, 380, 510, 290, 640, 455,
370, 520, 600, 330], T=100.0)
# -> Q_T = 812.637 m^3/s, K_T = 3.13667, mean = 451.5 m^3/s, std = 115.134 m^3/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('gumbel_design_flood', peaks=[1200, 980, 1450, 1100, 860,
1600, 1320, 1010, 940, 1250, 1380, 1150], T=50.0)
# -> Q_T = 1766.25 m^3/s, K_T = 2.59228, mean = 1186.67 m^3/s, std = 223.58 m^3/s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gumbel design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330] and return period = 100 years.	SOLVED	Q_T = 812.637 m^3/s, K_T = 3.13667, mean = 451.5 m^3/s, std = 115.134 m^3/s
Gumbel design flood with annual maxima = [1200, 980, 1450, 1100, 860, 1600, 1320, 1010, 940, 1250, 1380, 1150] and return period = 50 years.	SOLVED	Q_T = 1766.25 m^3/s, K_T = 2.59228, mean = 1186.67 m^3/s, std = 223.58 m^3/s
Gumbel design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330].	NEEDS_INPUT	gumbel_design_flood needs T (return period (years)). Example: solve('gumbel_design_flood', peaks=[420, 380, 510, 290, 64
Gumbel design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330] and return period = 100 years and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Log-Pearson type III design flood, `log_pearson_iii_design_flood`

The T-year flood by the log-Pearson type III distribution fitted by moments of the base-10 logarithms (station skew only, no regional skew weighting or outlier tests), frequency factor from the exact Pearson III quantile.

$$\log Q_T = \bar{y} + K_T(G, T)\, s_y$$

input	meaning	condition
`peaks`	annual maximum discharges in m^3/s	
`T`	return period (years)	

output	meaning
`Q_T`	T-year flood (m^3/s)
`K_T`	Pearson III frequency factor
`mean_log`	mean of log10 Q
`std_log`	standard deviation of log10 Q
`skew`	skew coefficient of log10 Q (unbiased)

- **Method:** moments of log10 Q; K_T = Pearson III quantile (gamma inverse with shape 4/G^2); Q_T = 10^(m + K s)
- **Assumes:** at least ten annual maxima; station skew only; no outlier screening
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q_T (T-year flood (m^3/s)) against T (return period (years)) from 10 to 1000 on a logarithmic axis. Other inputs as in the worked example: peaks = [420, 380, 510, 290, 640, 455, 370, 520, 600…. 41 points computed by civil-engineering-ai999x; marked: the worked example, T = 100, Q_T = 1083.77 m^3/s.

```
civil_engineering_ai999x.solve('log_pearson_iii_design_flood', peaks=[420, 380, 510, 290, 640, 455, 370, 520, 600, 330, 880, 410], T=100.0)
# -> Q_T = 1083.77 m^3/s, K_T = 2.75391, mean_log = 2.66461, std_log = 0.134473, skew = 0.598211   (SOLVED)
```

```
civil_engineering_ai999x.solve('log_pearson_iii_design_flood', peaks=[1200, 980, 1450, 1100, 860, 1600, 1320, 1010, 940, 1250, 1380, 1150, 2400, 1050], T=50.0)
# -> Q_T = 2446.89 m^3/s, K_T = 2.65405, mean_log = 3.08634, std_log = 0.113893, skew = 1.26856   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Log pearson iii design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330, 880, 410] and return period = 100 years.	SOLVED	Q_T = 1083.77 m^3/s, K_T = 2.75391, mean_log = 2.66461, std_log = 0.134473, skew = 0.598211
Log pearson iii design flood with annual maxima = [1200, 980, 1450, 1100, 860, 1600, 1320, 1010, 940, 1250, 1380, 1150, 2400, 1050] and return period = 50 years.	SOLVED	Q_T = 2446.89 m^3/s, K_T = 2.65405, mean_log = 3.08634, std_log = 0.113893, skew = 1.26856
Log pearson iii design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330, 880, 410].	NEEDS_INPUT	log_pearson_iii_design_flood needs T (return period (years)). Example: solve('log_pearson_iii_design_flood', peaks=[420,
Log pearson iii design flood with annual maxima = [420, 380, 510, 290, 640, 455, 370, 520, 600, 330, 880, 410] and return period = 100 years and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Return periods by the Weibull plotting position, `weibull_plotting_positions`

Ranks annual maxima from largest to smallest and gives each its exceedance probability $m/(n+1)$ and return period $(n+1)/m$ (Weibull plotting position).

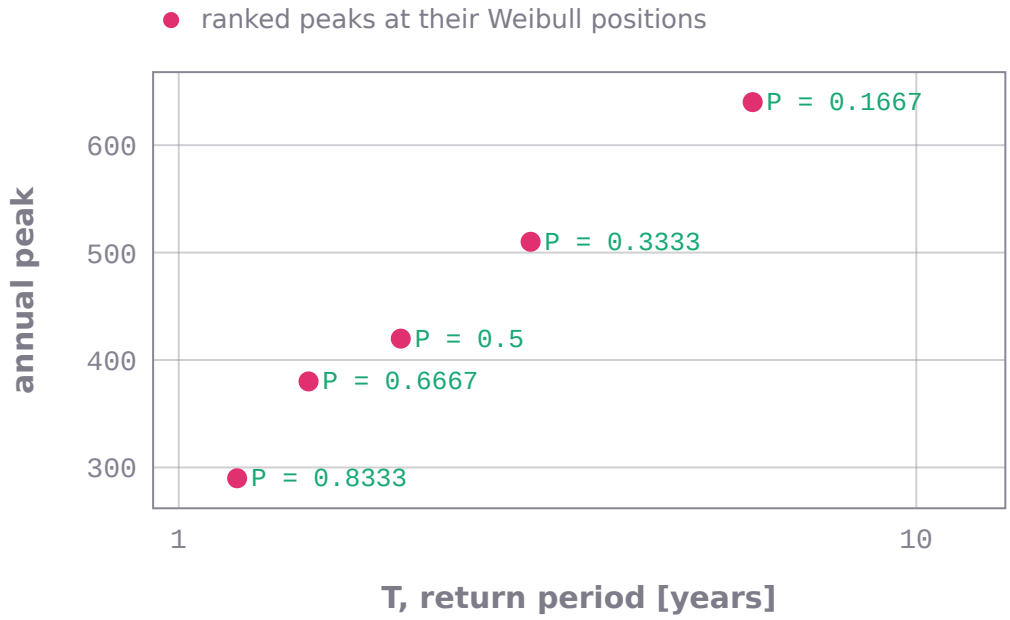
$$P = \frac{m}{n+1}, \quad T = \frac{n+1}{m}$$

input	meaning	condition
`peaks`	annual maxima (any unit; the ranking keeps it)	

output	meaning
`ranked`	values ranked from largest to smallest
`T`	return period of each (years)
`P`	exceedance probability of each

- **Method:** rank m from the largest; $P = m/(n+1)$, $T = 1/P$
- **Assumes:** annual series; ties keep distinct ranks

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Annual peaks [420, 380, 510, 290, 640] ranked from the largest and plotted at the return periods the package returned (Weibull, T = (n + 1)/m): T = 6, 3, 2, 1.5, 1.2 years, exceedance probabilities 0.166667, 0.333333, 0.5, 0.666667, 0.833333.

```
civil_engineering_ai999x.solve('weibull_plotting_positions', peaks=[420, 380, 510, 290, 640])
# -> ranked = [640.0, 510.0, 420.0, 380.0, 290.0], T = [6.0, 3.0, 2.0, 1.5, 1.2], P = [0.166666666667, 0.333333333333, 0.5, 0.666666666667, 0.833333333333]   (SOLVED)
```

```
civil_engineering_ai999x.solve('weibull_plotting_positions', peaks=[12.5, 18.2, 9.7, 22.1, 15.3, 11.0, 19.8])
# -> ranked = [22.1, 19.8, 18.2, 15.3, 12.5, 11.0, 9.7], T = [8.0, 4.0, 2.66666666667, 2.0, 1.6, 1.33333333333, 1.14285714286], P = [0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull plotting position with annual maxima = [420, 380, 510, 290, 640].	SOLVED	ranked = [640.0, 510.0, 420.0, 380.0, 290.0], T = [6.0, 3.0, 2.0, 1.5, 1.2], P = [0.166666666667, 0.333333333333, 0.5, 0.666666666667, 0.833333333333]
Weibull plotting position with annual maxima = [12.5, 18.2, 9.7, 22.1, 15.3, 11, 19.8].	SOLVED	ranked = [22.1, 19.8, 18.2, 15.3, 12.5, 11.0, 9.7], T = [8.0, 4.0, 2.66666666667, 2.0, 1.6, 1.33333333333, 1.14285714286], P = [0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875]
Weibull plotting position with annual maxima = [420, 380, 510, 290, 640] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Nash-Sutcliffe efficiency, RMSE and percent bias, `nash_sutcliffe_efficiency`

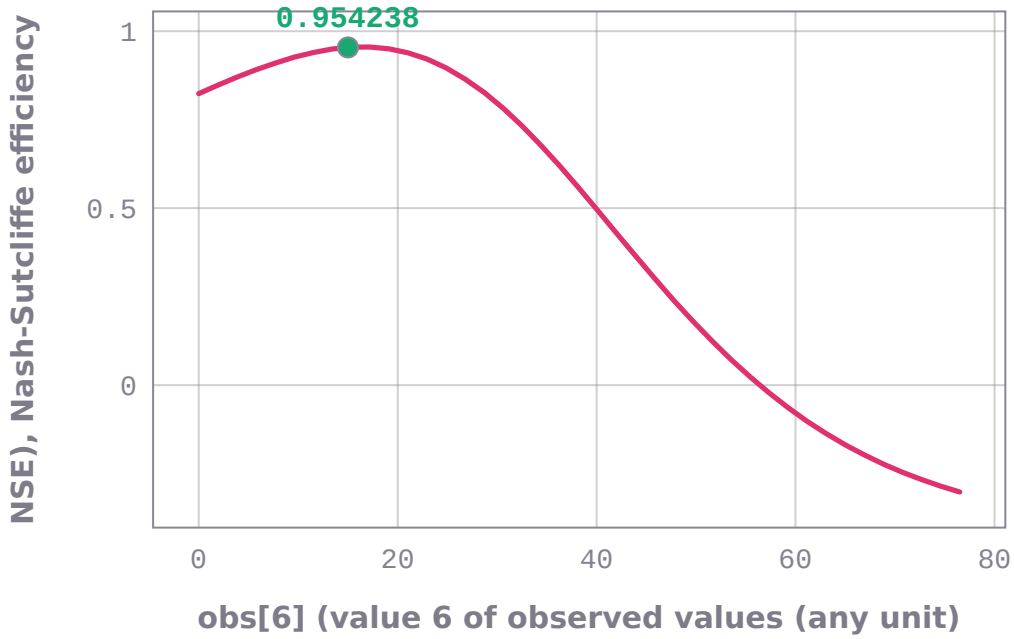
Goodness of fit of simulated to observed flows: Nash-Sutcliffe efficiency, root-mean-square error and percent bias (positive when the model underestimates, as hydroeval defines it).

$$NSE = 1 - \frac{\sum (O_i - S_i)^2}{\sum (O_i - \bar{O})^2}$$

input	meaning	condition
`obs`	observed values (any unit)	
`sim`	simulated values, same unit and order	

output	meaning
`NSE`	Nash-Sutcliffe efficiency
`RMSE`	root-mean-square error (unit of the data)
`PBIAS`	percent bias 100 sum(obs - sim)/sum(obs)

- **Method:** NSE = 1 - sum(obs - sim)^2/sum(obs - mean obs)^2
- **Assumes:** paired series of equal length; observed values not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



NSE (Nash-Sutcliffe efficiency) as value 6 of obs (observed values (any unit)) moves from 0 to 76.5, the other values as given ([12, 30, 55, 41, 22, 15]); 41 runs of civil-engineering-ai999x. Other inputs: sim = [10, 34, 50, 44, 20, 17]. Marked: the worked example, obs[6] = 15, NSE = 0.954238.

```
civil_engineering_ai999x.solve('nash_sutcliffe_efficiency', obs=[12, 30, 55, 41, 22, 15],
sim=[10, 34, 50, 44, 20, 17])
# -> NSE = 0.954238, RMSE = 3.21455, PBIAS = 0   (SOLVED)
```

```
civil_engineering_ai999x.solve('nash_sutcliffe_efficiency', obs=[3.2, 4.1, 8.7, 15.2, 9.4,
5.5, 4.0], sim=[3.0, 4.5, 7.9, 13.8, 10.1, 6.0, 3.6])
# -> NSE = 0.966308, RMSE = 0.727029, PBIAS = 2.39521   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nash sutcliffe efficiency with observed = [12, 30, 55, 41, 22, 15] and simulated = [10, 34, 50, 44, 20, 17].	SOLVED	NSE = 0.954238, RMSE = 3.21455, PBIAS = 0
Nash sutcliffe efficiency with observed = [3.2, 4.1, 8.7, 15.2, 9.4, 5.5, 4] and simulated = [3, 4.5, 7.9, 13.8, 10.1, 6, 3.6].	SOLVED	NSE = 0.966308, RMSE = 0.727029, PBIAS = 2.39521
Nash sutcliffe efficiency with observed = [12, 30, 55, 41, 22, 15].	NEEDS_INPUT	nash_sutcliffe_efficiency needs sim (simulated values, same unit and order). Example: solve('nash_sutcliffe_efficiency',
Nash sutcliffe efficiency with observed = [12, 30, 55, 41, 22, 15] and simulated = [10, 34, 50, 44, 20, 17] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Kling-Gupta efficiency, `kling_gupta_efficiency`

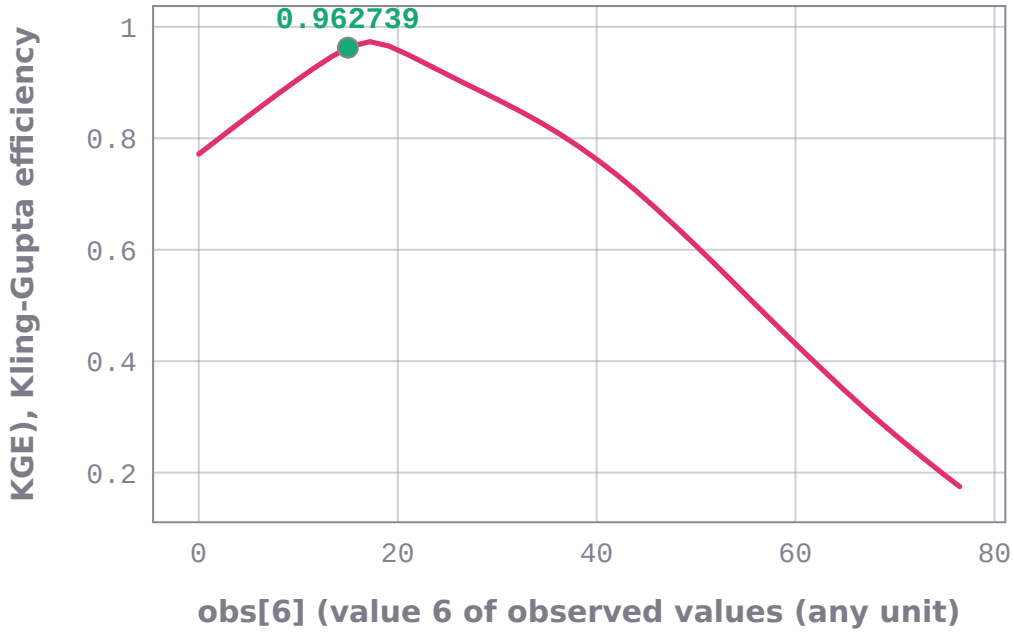
Kling-Gupta efficiency of simulated against observed values (Gupta et al. 2009) with its components: the correlation r, the variability ratio alpha (population standard deviations) and the bias ratio beta.

$$KGE = 1 - \sqrt{(r-1)^2 + (\alpha-1)^2 + (\beta-1)^2}$$

input	meaning	condition
`obs`	observed values (any unit)	
`sim`	simulated values, same unit and order	

output	meaning
`KGE`	Kling-Gupta efficiency
`r`	Pearson correlation
`alpha`	ratio of standard deviations sim/obs
`beta`	ratio of means sim/obs

- **Method:** $KGE = 1 - \sqrt{(\$r-1\$)^2 + (\alpha-1)^2 + (\beta-1)^2}$
- **Assumes:** paired series of equal length; both series vary
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



KGE (Kling-Gupta efficiency) as value 6 of obs (observed values (any unit)) moves from 0 to 76.5, the other values as given ([12, 30, 55, 41, 22, 15]); 41 runs of civil-engineering-ai999x. Other inputs: sim = [10, 34, 50, 44, 20, 17]. Marked: the worked example, obs[6] = 15, KGE = 0.962739.

```
civil_engineering_ai999x.solve('kling_gupta_efficiency', obs=[12, 30, 55, 41, 22, 15], sim=
[10, 34, 50, 44, 20, 17])
# -> KGE = 0.962739, r = 0.97687, alpha = 0.970787, beta = 1   (SOLVED)
```

```
civil_engineering_ai999x.solve('kling_gupta_efficiency', obs=[3.2, 4.1, 8.7, 15.2, 9.4,
5.5, 4.0], sim=[3.0, 4.5, 7.9, 13.8, 10.1, 6.0, 3.6])
# -> KGE = 0.908455, r = 0.986748, alpha = 0.912643, beta = 0.976048   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kling gupta efficiency with observed = [12, 30, 55, 41, 22, 15] and simulated = [10, 34, 50, 44, 20, 17].	SOLVED	KGE = 0.962739, r = 0.97687, alpha = 0.970787, beta = 1
Kling gupta efficiency with observed = [3.2, 4.1, 8.7, 15.2, 9.4, 5.5, 4] and simulated = [3, 4.5, 7.9, 13.8, 10.1, 6, 3.6].	SOLVED	KGE = 0.908455, r = 0.986748, alpha = 0.912643, beta = 0.976048
Kling gupta efficiency with observed = [12, 30, 55, 41, 22, 15].	NEEDS_INPUT	kling_gupta_efficiency needs sim (simulated values, same unit and order). Example: solve('kling_gupta_efficiency', obs=[
Kling gupta efficiency with observed = [12, 30, 55, 41, 22, 15] and simulated = [10, 34, 50, 44, 20, 17] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Horton infiltration rate and cumulative infiltration, `horton_infiltration`

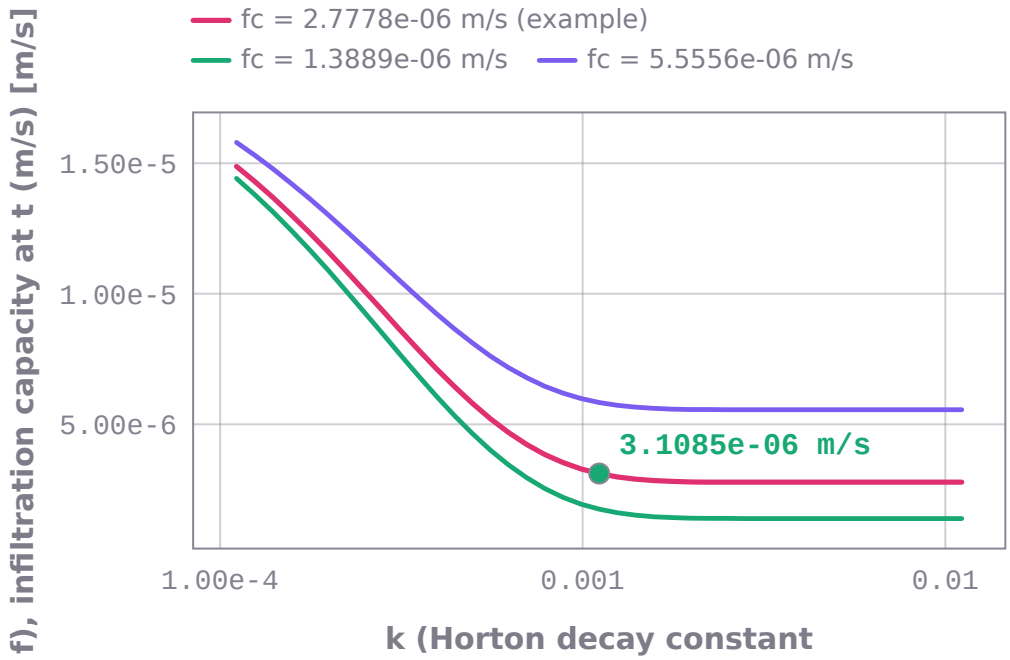
Infiltration capacity at time t and cumulative infiltration from the start of ponding by Horton's equation.

$$f = f_c + (f_0 - f_c)e^{-kt}, \quad F = f_c t + \frac{f_0 - f_c}{k}\left(1 - e^{-kt}\right)$$

input	meaning	condition
`f0`	initial infiltration capacity	
`fc`	final (constant) infiltration capacity	
`k`	Horton decay constant	
`t`	time since ponding began	

output	meaning
`f`	infiltration capacity at t (m/s)
`F`	cumulative infiltration (m)
`f_mm_h`	infiltration capacity (mm/h)
`F_mm`	cumulative infiltration (mm)

- **Method:** $f = fc + (f_0 - fc)$ e^(-k t), $F = fc\,t + (f_0 - fc)$(1 - e^(-k t))/k
- **Assumes:** $0 \le fc < f_0$; ponded surface from $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f (infiltration capacity at t (m/s)) against k (Horton decay constant) from 0.000111111 to 0.0111111 on a logarithmic axis. One curve for each value of fc (final (constant) infiltration capacity): 2.7778e-06 m/s, 1.3889e-06 m/s, 5.5556e-06 m/s. Other inputs as in the worked example: f0 = 2.0833333333333333e-05 m/s, t = 3600.0 s. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, k = 0.00111111, f = 3.1085e-06 m/s.

```
civil_engineering_ai999x.solve('horton_infiltration', f0=2.0833333333333333e-05,
fc=2.7777777777777775e-06, k=0.0011111111111111111, t=3600.0)
# -> f = 3.10848e-06 m/s, F = 0.0259524 m, f_mm_h = 11.1905, F_mm = 25.9524   (SOLVED)
```

```
civil_engineering_ai999x.solve('horton_infiltration', f0=1.3888888888888888e-05,
fc=1.6666666666666667e-06, k=0.0006944444444444445, t=1800.0)
# -> f = 5.16839e-06 m/s, F = 0.0155575 m, f_mm_h = 18.6062, F_mm = 15.5575   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Horton infiltration with initial rate = 75 mm/h, final rate = 10 mm/h, decay constant = 4 1/h and time = 1 h.	SOLVED	f = 3.10848e-06 m/s, F = 0.0259524 m, f_mm_h = 11.1905, F_mm = 25.9524
Horton infiltration with initial rate = 50 mm/h, final rate = 6 mm/h, decay constant = 2.5 1/h and time = 0.5 h.	SOLVED	f = 5.16839e-06 m/s, F = 0.0155575 m, f_mm_h = 18.6062, F_mm = 15.5575
Horton infiltration with initial rate = 75 mm/h, final rate = 10 mm/h and decay constant = 4 1/h.	NEEDS_INPUT	horton_infiltration needs t (time since ponding began). Example: solve('horton_infiltration', f0='2.08333e-05 m/s', fc='
Horton infiltration with initial rate = 75 mm/h, final rate = 10 mm/h, decay constant = 4 1/h and time = 1 h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Horton infiltration with initial rate = 10 mm/h, final rate = 75 mm/h, decay constant = 4 1/h and time = 1 h.	INVALID_INPUT	Horton infiltration rate and cumulative infiltration: need 0 <= fc < f0.

Green-Ampt cumulative infiltration under ponding, `green_ampt_infiltration`

Cumulative infiltration and infiltration rate at time t for a soil ponded from $t = 0$, by the Green-Ampt model (implicit equation solved to full precision).

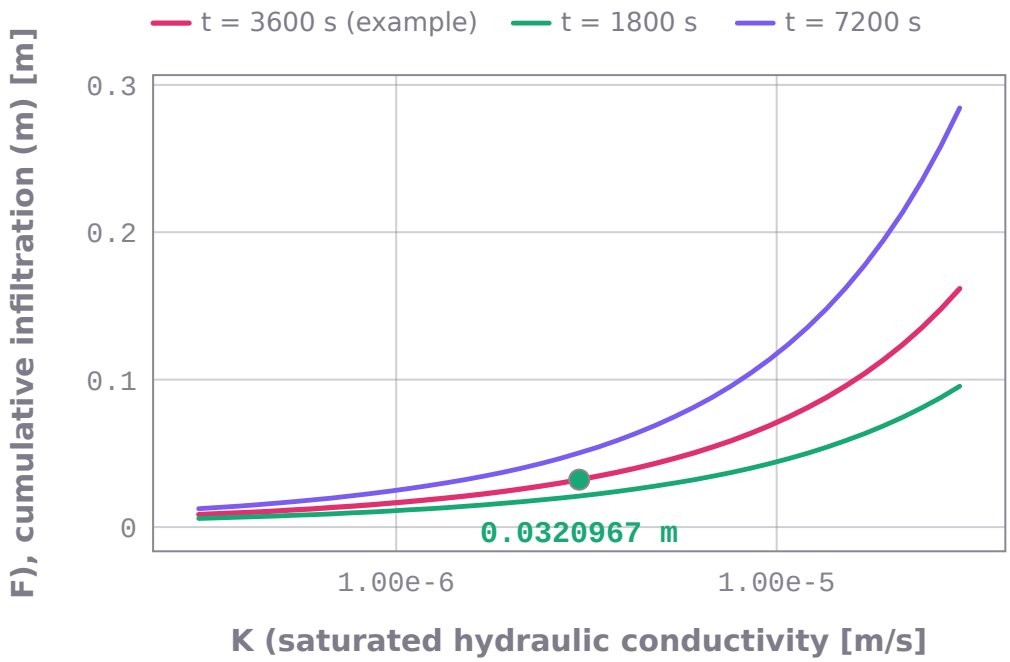
$$F - \psi\Delta\theta \ln\left(1 + \frac{F}{\psi\Delta\theta}\right) = Kt, \quad f = K\left(1 + \frac{\psi\Delta\theta}{F}\right)$$

input	meaning	condition
`K`	saturated hydraulic conductivity	
`psi`	wetting-front suction head	
`dtheta`	moisture deficit (porosity minus initial water content)	
`t`	time since ponding began	

output	meaning
`F`	cumulative infiltration (m)
`f`	infiltration rate (m/s)
`F_mm`	cumulative infiltration (mm)
`f_mm_h`	infiltration rate (mm/h)

- **Method:** F - psi dtheta ln(1 + F/(psi dtheta)) = K t (bisection); $f = K(1 + \psi$ dtheta/F)
- **Assumes:** ponded surface from $t = 0$; homogeneous soil, sharp wetting front

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F (cumulative infiltration (m)) against K (saturated hydraulic conductivity) from 3.0278e-07 m/s to 3.0278e-05 m/s on a logarithmic axis. One curve for each value of t (time since ponding began): 3600 s, 1800 s, 7200 s. Other inputs as in the worked example: psi = 0.1101 m, dtheta = 0.247. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, K = 3.0278e-06 m/s, F = 0.0320967 m.

```
civil_engineering_ai999x.solve('green_ampt_infiltration', K=3.0277777777777776e-06,
psi=0.1101, dtheta=0.247, t=3600.0)
# -> F = 0.0320967 m, f = 5.59314e-06 m/s, F_mm = 32.0967, f_mm_h = 20.1353   (SOLVED)
```

```
civil_engineering_ai999x.solve('green_ampt_infiltration', K=9.444444444444443e-07,
psi=0.1668, dtheta=0.34, t=7200.0)
# -> F = 0.0324786 m, f = 2.59357e-06 m/s, F_mm = 32.4786, f_mm_h = 9.33685   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Green ampt infiltration with hydraulic conductivity = 10.9 mm/h, suction head = 110.1 mm, moisture deficit = 0.247 and time = 1 h.	SOLVED	F = 0.0320967 m, f = 5.59314e-06 m/s, F_mm = 32.0967, f_mm_h = 20.1353
Green ampt infiltration with hydraulic conductivity = 3.4 mm/h, suction head = 166.8 mm, moisture deficit = 0.34 and time = 2 h.	SOLVED	F = 0.0324786 m, f = 2.59357e-06 m/s, F_mm = 32.4786, f_mm_h = 9.33685
Green ampt infiltration with hydraulic conductivity = 10.9 mm/h, suction head = 110.1 mm and moisture deficit = 0.247.	NEEDS_INPUT	green_ampt_infiltration needs t (time since ponding began). Example: solve('green_ampt_infiltration', K='3.02778e-06 m/s
Green ampt infiltration with hydraulic conductivity = 10.9 mm/h, suction head	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
= 110.1 mm, moisture deficit = 0.247 and time = 1 h and colour = 3.		

Phi-index of a storm, `phi_index`

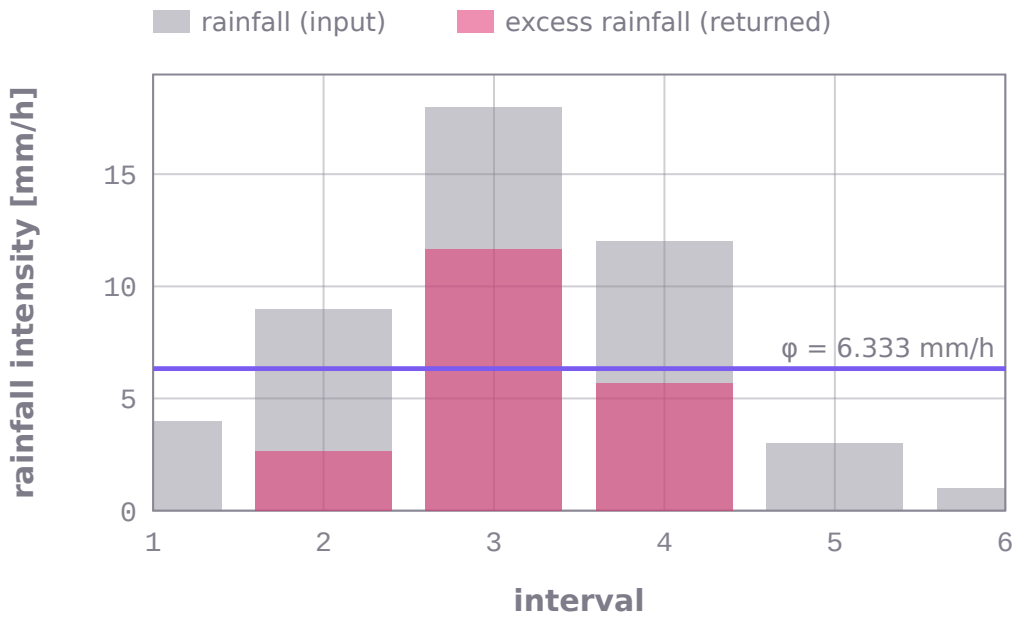
The constant loss rate (phi-index) that, subtracted from every interval of a rainfall hyetograph, leaves exactly the observed runoff depth; with the excess rainfall of each interval.

$$\sum_i \max(P_i - \phi\,\Delta t,\, 0) = R$$

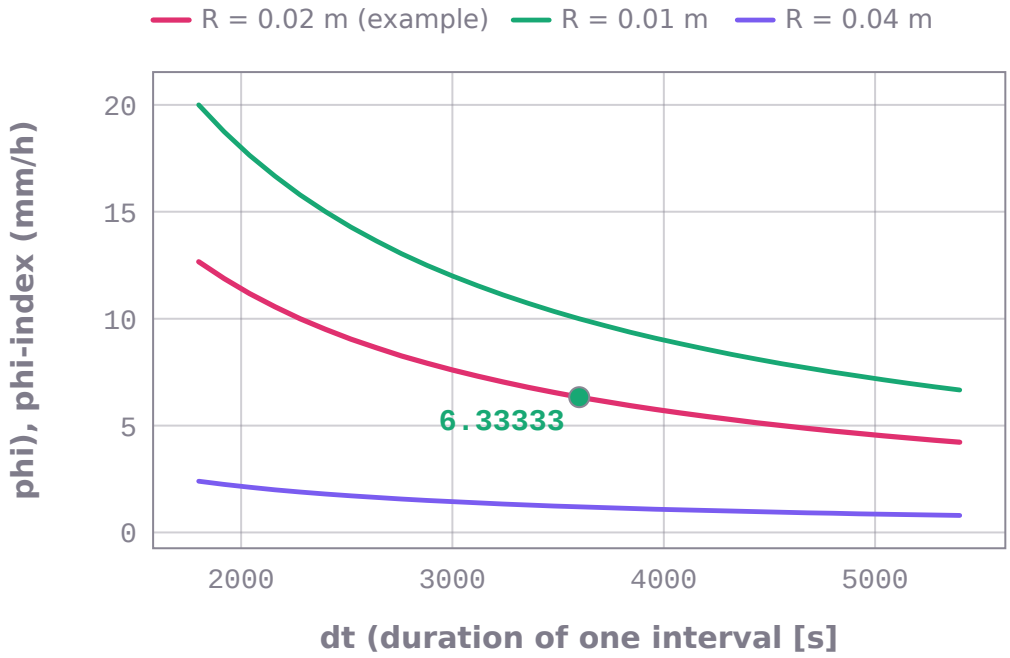
input	meaning	condition
`rain`	rainfall depth in each interval in mm	
`dt`	duration of one interval	
`R`	observed direct runoff depth	

output	meaning
`phi`	phi-index (mm/h)
`excess`	excess rainfall in each interval (mm)
`contributing`	number of intervals producing runoff

- **Method:** the k largest intervals contribute: phi dt = (sum of the k largest - R)/k, with k found by consistency
- **Assumes:** 0 < runoff < total rainfall; equal intervals
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hyetograph of a storm (rain [4, 9, 18, 12, 3, 1] mm per interval of 1 h) and the returned φ-index 6.33333 mm/h: the constant loss that leaves the observed runoff of 0.02 mm. Marked: the returned excess rainfall of each interval (0, 2.66667, 11.6667, 5.66667, 0, 0 mm), from 3 contributing intervals.



phi (phi-index (mm/h)) against dt (duration of one interval) from 1800 s to 5400 s. One curve for each value of R (observed direct runoff depth): 0.02 m, 0.01 m, 0.04 m. Other inputs as in the worked example: rain = [4, 9, 18, 12, 3, 1]. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, dt = 3600 s, phi = 6.33333.

```
civil_engineering_ai999x.solve('phi_index', rain=[4, 9, 18, 12, 3, 1], dt=3600.0, R=0.02)
# -> phi = 6.33333, excess = [0.0, 2.66666666667, 11.6666666667, 5.66666666667, 0.0, 0.0],
contributing = 3   (SOLVED)
```

```
civil_engineering_ai999x.solve('phi_index', rain=[2.5, 7.5, 12.0, 5.0, 1.5], dt=1800.0,
R=0.009000000000000001)
# -> phi = 10.5, excess = [0.0, 2.25, 6.75, 0.0, 0.0], contributing = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Phi index with rainfall = [4, 9, 18, 12, 3, 1], interval = 1 h and runoff = 20 mm.	SOLVED	phi = 6.33333, excess = [0.0, 2.66666666667, 11.6666666667, 5.66666666667, 0.0, 0.0], contributing = 3
Phi index with rainfall = [2.5, 7.5, 12, 5, 1.5], interval = 0.5 h and runoff = 9 mm.	SOLVED	phi = 10.5, excess = [0.0, 2.25, 6.75, 0.0, 0.0], contributing = 2
Phi index with rainfall = [4, 9, 18, 12, 3, 1] and interval = 1 h.	NEEDS_INPUT	phi_index needs R (observed direct runoff depth). Example: solve('phi_index', rain=[4, 9, 18, 12, 3, 1], dt='3600 s', R=
Phi index with rainfall = [4, 9, 18, 12, 3, 1], interval = 1 h and runoff = 20 mm and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Muskingum flood routing, `muskingum_routing`

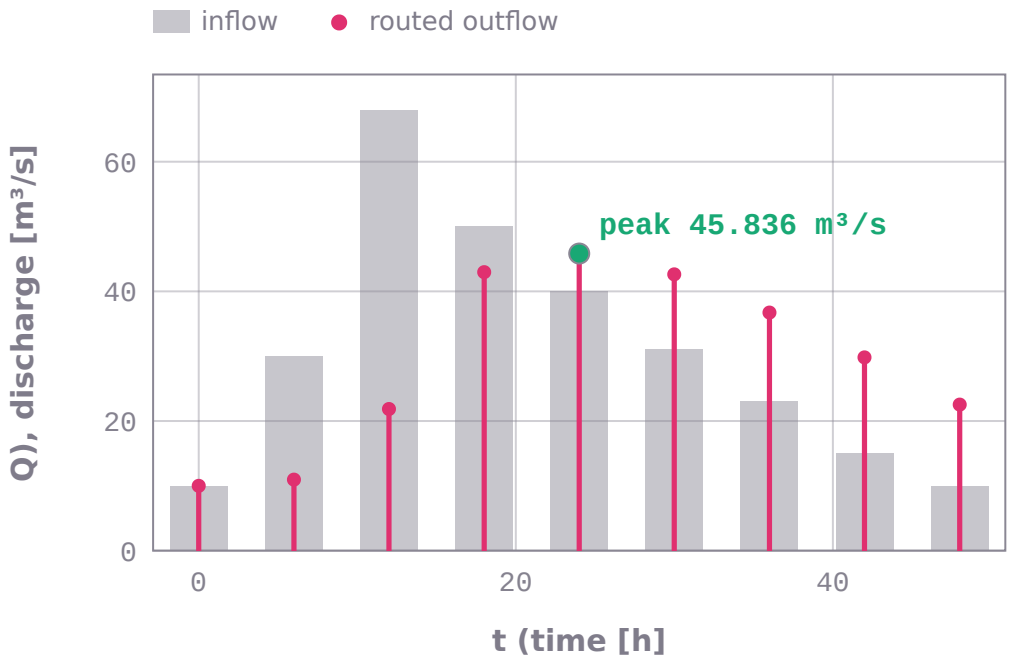
Routes an inflow hydrograph through a river reach by the Muskingum method (storage $S = K[X I + (1 - X) O]$), giving the outflow ordinates, routing coefficients, peak outflow and attenuation. The first outflow equals the first inflow unless stated.

$$O_{j+1} = C_0 I_{j+1} + C_1 I_j + C_2 O_j, \quad C_0 = \frac{\Delta t - 2KX}{2K(1-X) + \Delta t}$$

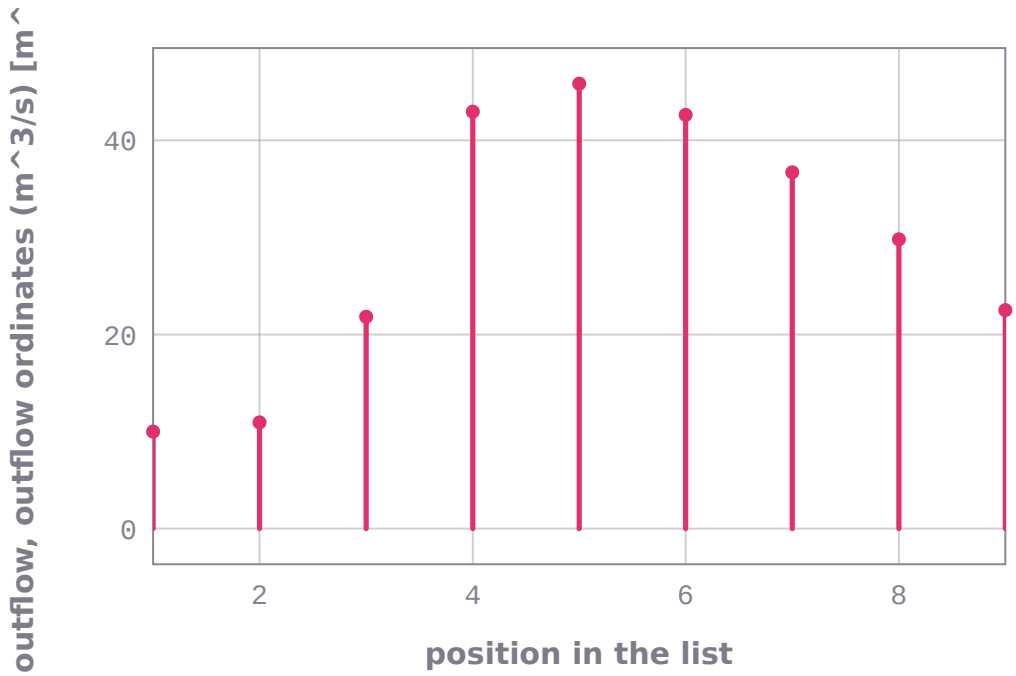
input	meaning	condition
`inflow`	inflow ordinates in m^3/s at equal intervals	
`K`	storage (travel-time) constant	
`X`	weighting factor (0 to 0.5)	
`dt`	routing interval	
`O1`	initial outflow (default: the first inflow)	

output	meaning
`outflow`	outflow ordinates (m^3/s)
`C0`	coefficient C0
`C1`	coefficient C1
`C2`	coefficient C2
`peak_outflow`	peak outflow (m^3/s)
`attenuation`	peak attenuation (m^3/s)

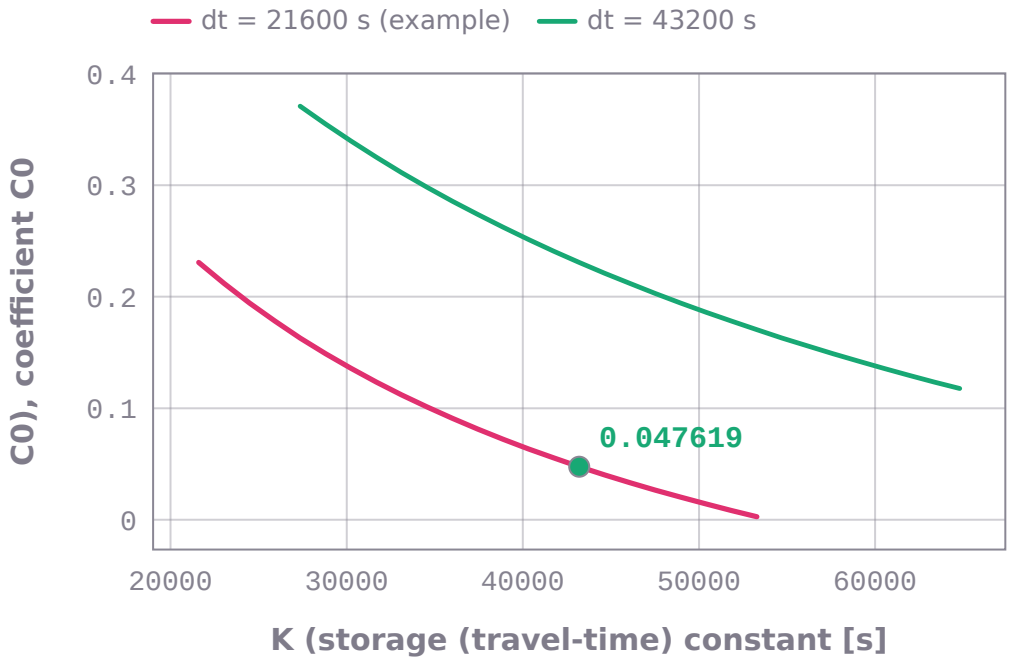
- **Method:** $O_j + 1 = C_0\ I_j + 1 + C_1\ I_j + C_2$ O_j, $C_0 = (dt - 2KX)/D$, $C_1 = (dt + 2KX)/D$, C2 = (2K(1 - X) - dt)/D, $D = 2K$(1 - X) + dt
- **Assumes:** $0 \le X \le 0.5$; 2 K $X \le dt \le 2$ K $(1 - X)$ (no negative coefficient)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A flood wave routed through a river reach by the Muskingum method (K = 12 h, X = 0.2, Δt = 6 h; C0 = 0.047619, C1 = 0.428571, C2 = 0.52381): inflow per interval as bars, the outflow returned by the package as stems, lower and later; peak inflow 68 m³/s, marked: peak outflow t = 24 h, Q = 45.836 m³/s (attenuation 22.164 m³/s).



outflow ordinates (m^3/s) (outflow) returned by civil-engineering-ai999x for the worked example, one value for each position in the list (9 values, first 10 m^3/s, last 22.5146 m^3/s, largest 45.836 m^3/s). Inputs: inflow = [10, 30, 68, 50, 40, 31, 23, 15, 10], K = 43200.0 s, X = 0.2, dt = 21600.0 s.



C0 (coefficient C0) against K (storage (travel-time) constant) from 21600 s to 64800 s. One curve for each value of dt (routing interval): 21600 s, 43200 s. Other inputs as in the worked example: inflow = [10, 30, 68, 50, 40, 31, 23, 15, 10], X = 0.2. 23 points per curve computed by civil-engineering-ai999x; marked: the worked example, K = 43200 s, C0 = 0.047619.

```
civil_engineering_ai999x.solve('muskingum_routing', inflow=[10, 30, 68, 50, 40, 31, 23, 15, 10], K=43200.0, X=0.2, dt=21600.0)
# -> outflow = [10.0, 10.9523809524, 21.8321995465, 42.959723572, 45.8360456806, 42.6284048803, 36.7101168421, 29.8005373935, 22.5145672061] m^3/s, C0 = 0.047619, C1 = 0.428571, C2 = 0.52381, peak_outflow = 45.836 m^3/s, attenuation = 22.164 m^3/s   (SOLVED)
```

```
civil_engineering_ai999x.solve('muskingum_routing', inflow=[20, 80, 210, 240, 190, 130, 80, 45, 30, 22], K=28800.0, X=0.25, dt=14400.0)
# -> outflow = [20.0, 20.0, 50.0, 130.0, 185.0, 187.5, 158.75, 119.375, 82.1875, 56.09375] m^3/s, C0 = 0, C1 = 0.5, C2 = 0.5, peak_outflow = 187.5 m^3/s, attenuation = 52.5 m^3/s (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Muskingum routing with inflow = [10, 30, 68, 50, 40, 31, 23, 15, 10], storage constant = 12 h, weighting factor = 0.2 and interval = 6 h.	SOLVED	outflow = [10.0, 10.9523809524, 21.8321995465, 42.959723572, 45.8360456806, 42.6284048803, 36.7101168421, 29.8005373935, 22.5145672061] m^3/s, C0 = 0.047619, C1 = 0.428571, C2 = 0.52381, peak_outflow = 45.836 m^3/s, attenuation = 22.164 m^3/s
Muskingum routing with inflow = [20, 80, 210, 240, 190, 130, 80, 45, 30, 22], storage constant = 8 h, weighting factor = 0.25 and interval = 4 h.	SOLVED	outflow = [20.0, 20.0, 50.0, 130.0, 185.0, 187.5, 158.75, 119.375, 82.1875, 56.09375] m^3/s, C0 = 0, C1 = 0.5, C2 = 0.5, peak_outflow = 187.5 m^3/s, attenuation = 52.5 m^3/s
Muskingum routing with inflow = [10, 30, 68, 50, 40, 31, 23, 15, 10], storage constant = 12 h and weighting factor = 0.2.	NEEDS_INPUT	muskingum_routing needs dt (routing interval). Example: solve('muskingum_routing', inflow=[10, 30, 68, 50, 40, 31, 23, 1
Muskingum routing with inflow = [10, 30, 68, 50, 40, 31, 23, 15, 10], storage constant = 12 h, weighting factor = 0.2 and interval = 6 h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Muskingum routing with inflow = [10, 30, 68, 50], storage constant = 12 h, weighting factor = 0.2 and interval = 30 h.	INVALID_INPUT	Muskingum flood routing: a routing coefficient is negative: choose 2 K X <= dt <= 2 K (1 - X).

Darcy flow across soil layers in series; equivalent conductivities, `darcy_flow_layered_soil`

Flow rate, specific discharge and the heads at the layer interfaces for one-dimensional Darcy flow across one or more soil layers in series (flow perpendicular to the layering), with the equivalent vertical (series) and horizontal (parallel) conductivities. Heads are measured from the downstream face.

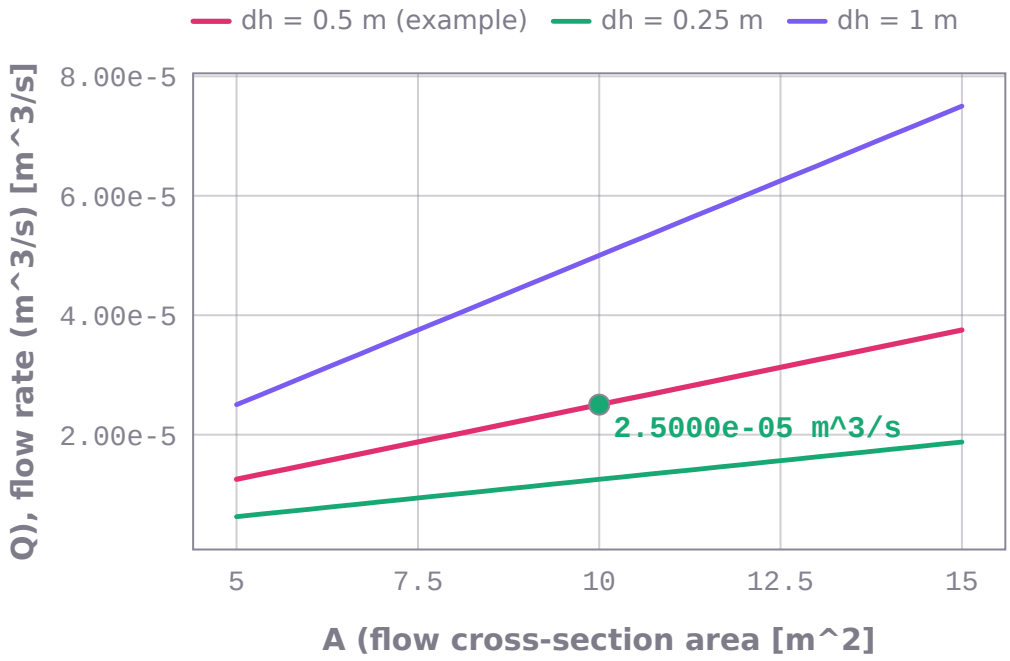
$$K_v = \frac{\sum H_i}{\sum H_i/K_i}, \quad K_h = \frac{\sum K_i H_i}{\sum H_i}, \quad Q = K_v \frac{\Delta h}{\sum H_i} A$$

input	meaning	condition
`H`	layer thicknesses in m, in flow order	
`K`	hydraulic conductivity of each layer in m/s	

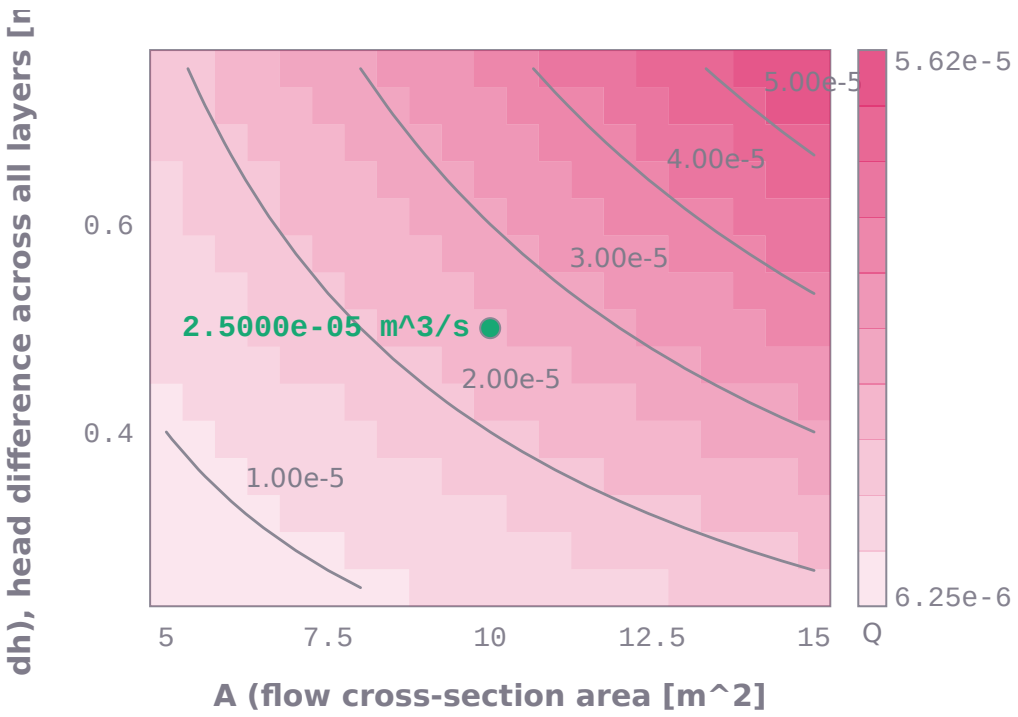
input	meaning	condition
`A`	flow cross-section area	
`dh`	head difference across all layers	

output	meaning
`Q`	flow rate (m^3/s)
`q`	specific discharge (Darcy velocity, m/s)
`Kv`	equivalent conductivity across the layers (m/s)
`Kh`	equivalent conductivity along the layers (m/s)
`heads`	head at each interior interface above the outlet (m)

- **Method:** Kv = sum H / sum(H_i/K_i), Kh = sum(K_i H_i)/sum H, $q = Kv$ dh/sum H, $Q = q$ A
- **Assumes:** steady, saturated, laminar flow; flow perpendicular to the layers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (flow rate (m^3/s)) against A (flow cross-section area) from 5 m^2 to 15 m^2. One curve for each value of dh (head difference across all layers): 0.5 m, 0.25 m, 1 m. Other inputs as in the worked example: H = [2.0], K = [1e-05]. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, A = 10 m^2, Q = 2.5000e-05 m^3/s.



Q (flow rate (m^3/s)) over A (flow cross-section area) from 5 m^2 to 15 m^2 and dh (head difference across all layers) from 0.25 m to 0.75 m: each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 6.2500e-06 m^3/s to dark 5.6250e-05 m^3/s); iso-lines at Q = 1.00e-5, 2.00e-5, 3.00e-5, 4.00e-5, 5.00e-5 m^3/s. Other inputs as in the worked example: H = [2.0], K = [1e-05]. Marked: the worked example, A = 10 m^2, dh = 0.5 m, Q = 2.5000e-05 m^3/s.

```
civil_engineering_ai999x.solve('darcy_flow_layered_soil', H=[2.0], K=[1e-05], A=10.0,
dh=0.5)
# -> Q = 2.5e-05 m^3/s, q = 2.5e-06 m/s, Kv = 1e-05 m/s, Kh = 1e-05 m/s, heads = [0.0] m
(SOLVED)
```

```
civil_engineering_ai999x.solve('darcy_flow_layered_soil', H=[1.5, 2.0, 1.0], K=[2e-05, 5e-
07, 0.0001], A=50.0, dh=3.0)
# -> Q = 3.67197e-05 m^3/s, q = 7.34394e-07 m/s, Kv = 1.10159e-06 m/s, Kh = 2.91111e-05
m/s, heads = [2.94492044064, 0.00734394124847] m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Darcy flow through layered soil with thicknesses = [2], conductivities = [1e-05], area = 10 m^2 and head difference = 0.5 m.	SOLVED	Q = 2.5e-05 m^3/s, q = 2.5e-06 m/s, Kv = 1e-05 m/s, Kh = 1e-05 m/s, heads = [0.0] m
Darcy flow through layered soil with thicknesses = [1.5, 2, 1], conductivities = [2e-05, 5e-07, 0.0001], area = 50 m^2 and head difference = 3 m.	SOLVED	Q = 3.67197e-05 m^3/s, q = 7.34394e-07 m/s, Kv = 1.10159e-06 m/s, Kh = 2.91111e-05 m/s, heads = [2.94492044064, 0.00734394124847] m
Darcy flow through layered soil with thicknesses = [2], conductivities = [1e-05] and area = 10 m^2.	NEEDS_INPUT	darcy_flow_layered_soil needs dh (head difference across all layers). Example: solve('darcy_flow_layered_soil', H=[2.0],
Darcy flow through layered soil with thicknesses = [2], conductivities =	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
[1e-05], area = 10 m^2 and head difference = 0.5 m and colour = 3.		

Hydraulic conductivity from a falling-head test, `falling_head_permeability`

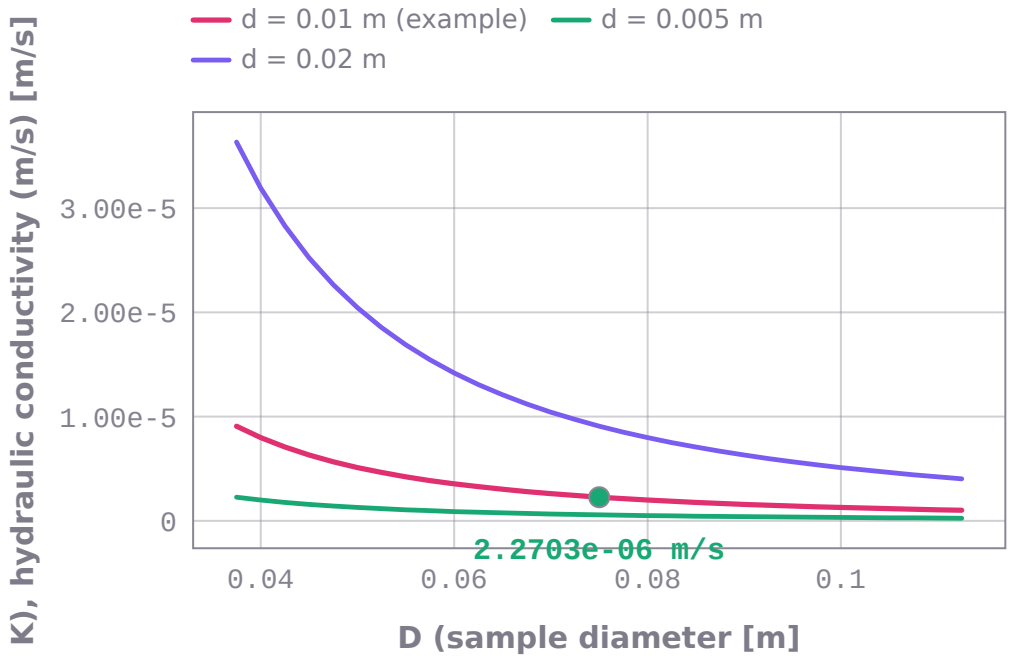
Hydraulic conductivity of a soil sample from a falling-head permeameter: standpipe diameter d, sample diameter D and length L, the standpipe head falling from h1 to h2 in time t.

$$K = \frac{aL}{At} \ln \frac{h_1}{h_2}$$

input	meaning	condition
`d`	standpipe diameter	
`D`	sample diameter	
`L`	sample length	
`h1`	initial head	
`h2`	final head	
`t`	elapsed time	

output	meaning
`K`	hydraulic conductivity (m/s)
`K_cm_s`	hydraulic conductivity (cm/s)

- **Method:** K = (a L/(A t)) ln(h1/h2), a and A the standpipe and sample areas
- **Assumes:** $h_2 < h_1$; saturated sample, laminar flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



K (hydraulic conductivity (m/s)) against D (sample diameter) from 0.0375 m to 0.1125 m. One curve for each value of d (standpipe diameter): 0.01 m, 0.005 m, 0.02 m. Other inputs as in the worked example: L = 0.15 m, h1 = 1.0 m, h2 = 0.6 m, t = 600.0 s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, D = 0.075 m, K = 2.2703e-06 m/s.

```
civil_engineering_ai999x.solve('falling_head_permeability', d=0.01, D=0.075, L=0.15,
h1=1.0, h2=0.6, t=600.0)
# -> K = 2.27034e-06 m/s, K_cm_s = 0.000227034   (SOLVED)
```

```
civil_engineering_ai999x.solve('falling_head_permeability', d=0.006, D=0.1, L=0.12, h1=0.8,
h2=0.4, t=1800.0)
# -> K = 1.66355e-07 m/s, K_cm_s = 1.66355e-05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Falling head permeability with standpipe diameter = 10 mm, sample diameter = 75 mm, sample length = 150 mm, initial head = 100 cm, final head = 60 cm and time = 600 s.	SOLVED	K = 2.27034e-06 m/s, K_cm_s = 0.000227034
Falling head permeability with standpipe diameter = 6 mm, sample diameter = 100 mm, sample length = 120 mm, initial head = 80 cm, final head = 40 cm and time = 1800 s.	SOLVED	K = 1.66355e-07 m/s, K_cm_s = 1.66355e-05
Falling head permeability with standpipe diameter = 10 mm, sample diameter = 75 mm, sample length = 150 mm, initial head = 100 cm and final head = 60 cm.	NEEDS_INPUT	falling_head_permeability needs t (elapsed time). Example: solve('falling_head_permeability', d='0.01 m', D='0.075 m', L
Falling head permeability with standpipe diameter = 10 mm, sample diameter = 75 mm, sample length = 150 mm, initial head	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
= 100 cm, final head = 60 cm and time = 600 s and colour = 3.		
Falling head permeability with standpipe diameter = 10 mm, sample diameter = 75 mm, sample length = 150 mm, initial head = 60 cm, final head = 100 cm and time = 600 s.	INVALID_INPUT	Hydraulic conductivity from a falling-head test: the head must fall: h2 < h1.

Transmissivity from steady pumping (Thiem, confined aquifer), `thiem_confined_transmissivity`

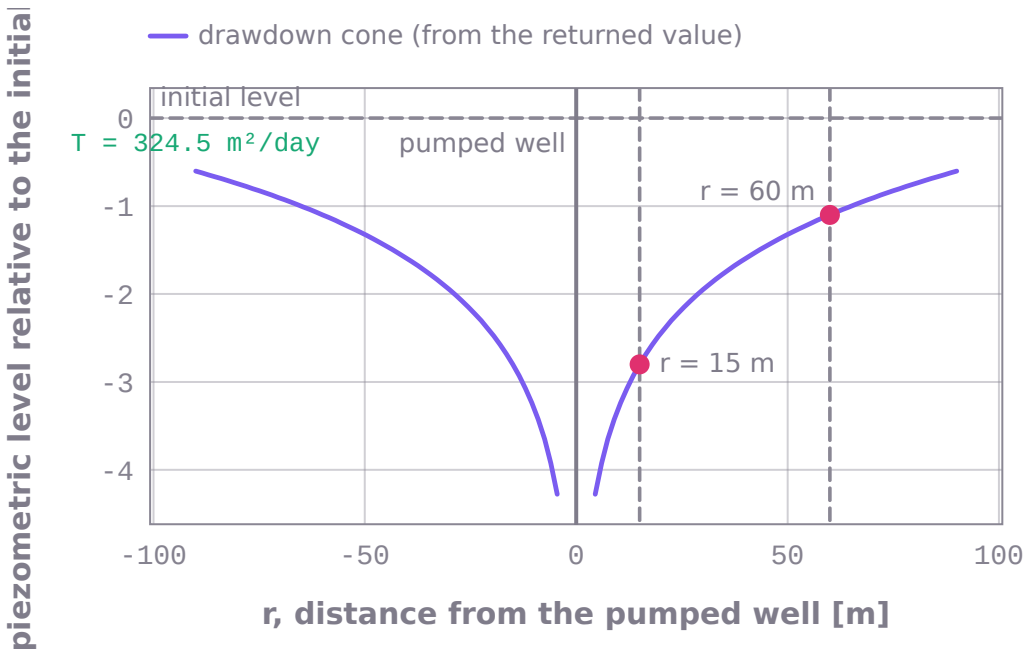
Transmissivity (and conductivity, if the aquifer thickness is given) of a confined aquifer from the steady drawdowns in two observation wells at distances $r_1 < r_2$ from a well pumped at Q (Thiem equation).

$$T = \frac{Q \ln(r_2/r_1)}{2\pi(s_1 - s_2)}$$

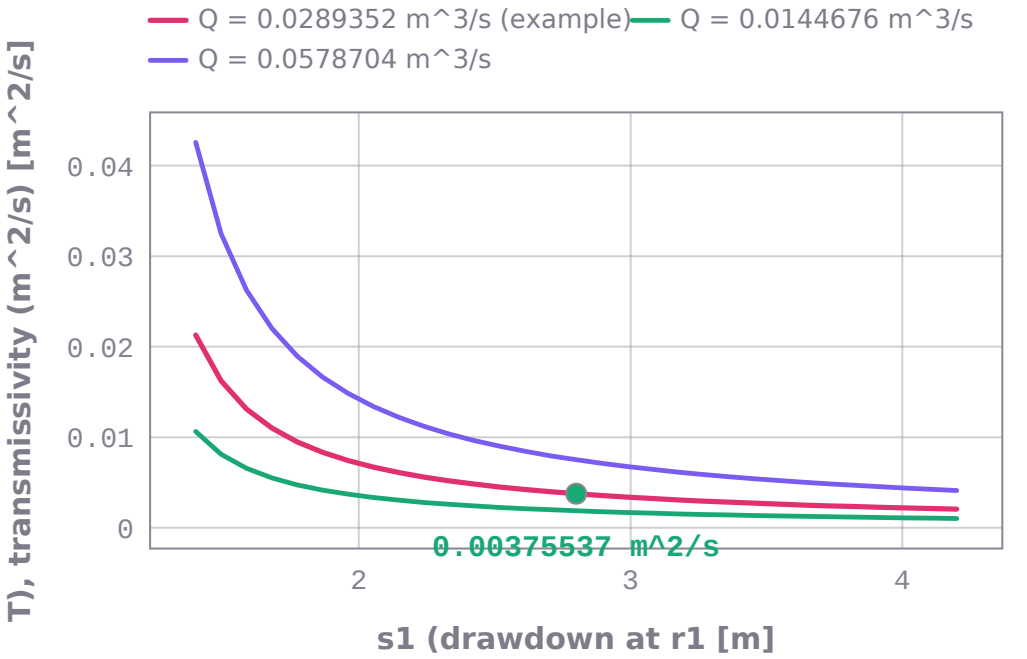
input	meaning	condition
`Q`	pumping rate	
`r1`	distance of the nearer observation well	
`r2`	distance of the farther observation well	
`s1`	drawdown at r1	
`s2`	drawdown at r2	
`b`	aquifer thickness (optional)	

output	meaning
`T`	transmissivity (m^2/s)
`T_m2_d`	transmissivity (m^2/day)
`K`	hydraulic conductivity T/b (m/s)

- **Method:** $T = Q$ ln$(r_2/r_1)/(2$ pi $(s_1 - s_2))$
- **Assumes:** steady state; confined aquifer, fully penetrating well; $r_1 < r_2$, $s_1 > s_2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Steady radial flow to a well pumping Q = 0.0289352 m³/s, observed at r1 = 15 m and r2 = 60 m (drawdowns 2.8 m and 1.1 m, the observation points marked). The cone through them is the Thiem relation with the returned transmissivity: T = 324.5 m²/day (T = 0.00375537 m²/s); drawn on both sides of the well.



T (transmissivity (m^2/s)) against s1 (drawdown at r1) from 1.4 m to 4.2 m. One curve for each value of Q (pumping rate): 0.0289352 m^3/s, 0.0144676 m^3/s, 0.0578704 m^3/s. Other inputs as in the worked example: r1 = 15.0 m, r2 = 60.0 m, s2 = 1.1 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, s1 = 2.8 m, T = 0.00375537 m^2/s.

```
civil_engineering_ai999x.solve('thiem_confined_transmissivity', Q=0.028935185185185185,
r1=15.0, r2=60.0, s1=2.8, s2=1.1)
# -> T = 0.00375537 m^2/s, T_m2_d = 324.464   (SOLVED)
```

```
civil_engineering_ai999x.solve('thiem_confined_transmissivity', Q=0.013888888888888888,
r1=10.0, r2=100.0, s1=4.2, s2=0.9, b=20.0)
# -> T = 0.00154237 m^2/s, T_m2_d = 133.261, K = 7.71186e-05 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thiem confined transmissivity with pumping rate = 2500 m^3/d, first distance = 15 m, second distance = 60 m, first drawdown = 2.8 m and second drawdown = 1.1 m.	SOLVED	T = 0.00375537 m^2/s, T_m2_d = 324.464
Thiem confined transmissivity with pumping rate = 1200 m^3/d, first distance = 10 m, second distance = 100 m, first drawdown = 4.2 m, second drawdown = 0.9 m and aquifer thickness = 20 m.	SOLVED	T = 0.00154237 m^2/s, T_m2_d = 133.261, K = 7.71186e-05 m/s
Thiem confined transmissivity with pumping rate = 2500 m^3/d, first distance = 15 m, second distance = 60 m and first drawdown = 2.8 m.	NEEDS_INPUT	thiem_confined_transmissivity needs s2 (drawdown at r2). Example: solve('thiem_confined_transmissivity', Q='0.0289352 m^
Thiem confined transmissivity with pumping rate = 2500 m^3/d, first distance = 15 m, second distance = 60 m, first drawdown = 2.8 m and second drawdown = 1.1 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Thiem confined transmissivity with pumping rate = 2500 m^3/d, first distance = 60 m, second distance = 15 m, first drawdown = 2.8 m and second drawdown = 1.1 m.	INVALID_INPUT	Transmissivity from steady pumping (Thiem, confined aquifer): the second observation well must be farther from the pumpe

Hydraulic conductivity from steady pumping (unconfined aquifer),

`thiem_unconfined_conductivity`

Hydraulic conductivity of an unconfined aquifer from the steady saturated thicknesses $h_1 < h_2$ in two observation wells at distances $r_1 < r_2$ from a well pumped at Q (Thiem-Dupuit equation).

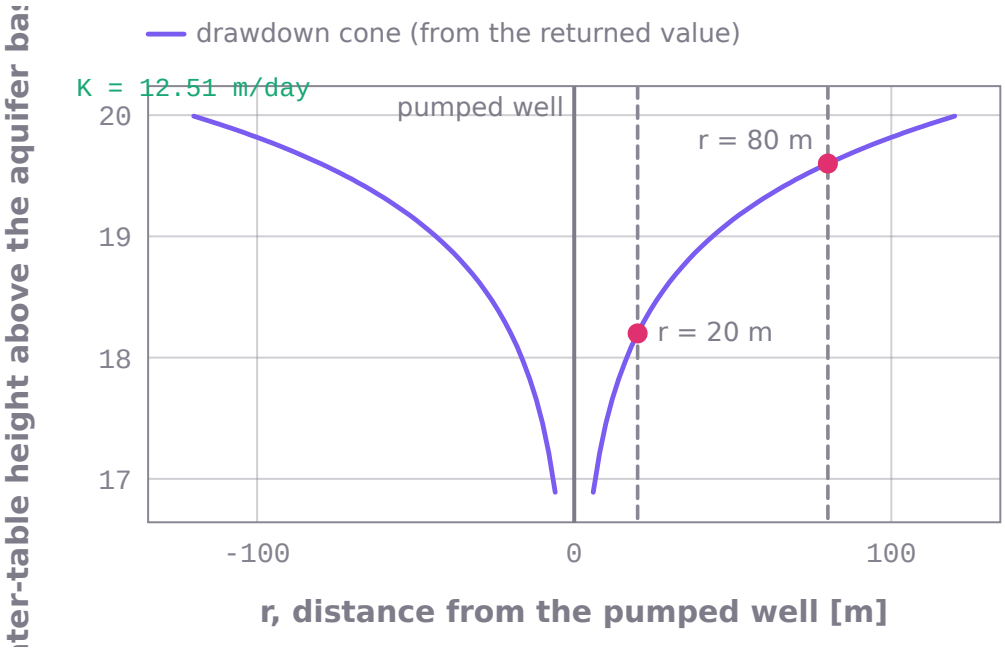
$$K = \frac{Q \ln(r_2/r_1)}{\pi (h_2^2 - h_1^2)}$$

input	meaning	condition
`Q`	pumping rate	
`r1`	distance of the nearer observation well	
`r2`	distance of the farther observation well	
`h1`	saturated thickness at r1	
`h2`	saturated thickness at r2	

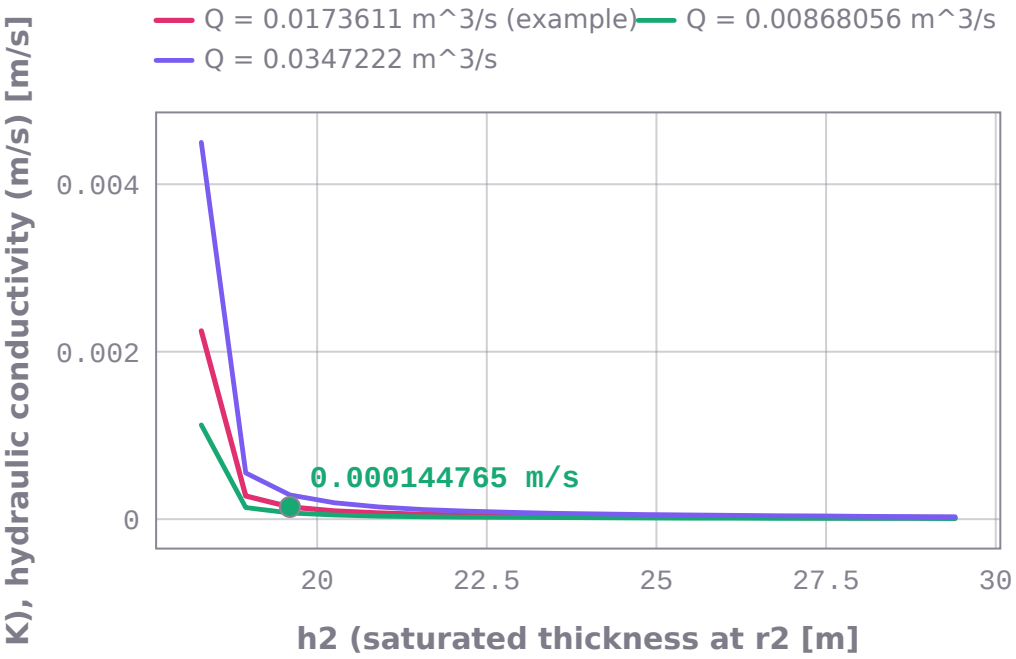
output	meaning
`K`	hydraulic conductivity (m/s)
`K_m_d`	hydraulic conductivity (m/day)

- **Method:** $K = Q$ ln$(r_2/r_1)/(pi\ (h_2^2 - h_1^2))$
- **Assumes:** steady state; unconfined aquifer, Dupuit assumptions; $r_1 < r_2$, $h_1 < h_2$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Steady radial flow to a well pumping Q = 0.0173611 m³/s, observed at r1 = 20 m and r2 = 80 m (water-table heights 18.2 m and 19.6 m, the observation points marked). The cone through them is the Thiem relation with the returned hydraulic conductivity: K = 12.51 m/day (K = 0.000144765 m/s); drawn on both sides of the well.



K (hydraulic conductivity (m/s)) against h2 (saturated thickness at r2) from 9.8 m to 29.4 m. One curve for each value of Q (pumping rate): 0.0173611 m^3/s, 0.00868056 m^3/s, 0.0347222 m^3/s. Other inputs as in the worked example: r1 = 20.0 m, r2 = 80.0 m, h1 = 18.2 m. 18 points per curve computed by civil-engineering-ai999x; marked: the worked example, h2 = 19.6 m, K = 0.000144765 m/s.

```
civil_engineering_ai999x.solve('thiem_unconfined_conductivity', Q=0.017361111111111112,
r1=20.0, r2=80.0, h1=18.2, h2=19.6)
# -> K = 0.000144765 m/s, K_m_d = 12.5077   (SOLVED)
```

```
civil_engineering_ai999x.solve('thiem_unconfined_conductivity', Q=0.009259259259259259,
r1=12.0, r2=45.0, h1=9.5, h2=11.0)
# -> K = 0.000126687 m/s, K_m_d = 10.9458   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thiem unconfined conductivity with pumping rate = 1500 m^3/d, first distance = 20 m, second distance = 80 m, first head = 18.2 m and second head = 19.6 m.	SOLVED	K = 0.000144765 m/s, K_m_d = 12.5077
Thiem unconfined conductivity with pumping rate = 800 m^3/d, first distance = 12 m, second distance = 45 m, first head = 9.5 m and second head = 11 m.	SOLVED	K = 0.000126687 m/s, K_m_d = 10.9458
Thiem unconfined conductivity with pumping rate = 1500 m^3/d, first distance = 20 m, second distance = 80 m and first head = 18.2 m.	NEEDS_INPUT	thiem_unconfined_conductivity needs h2 (saturated thickness at r2). Example: solve('thiem_unconfined_conductivity', Q='0
Thiem unconfined conductivity with pumping rate = 1500 m^3/d, first distance = 20 m, second distance = 80 m, first head = 18.2 m and second head = 19.6 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Theis drawdown around a pumped well, `theis_drawdown`

Transient drawdown at distance r and time t from a well pumped at a constant rate in a confined aquifer (Theis), with u and the well function W(u) = E1(u).

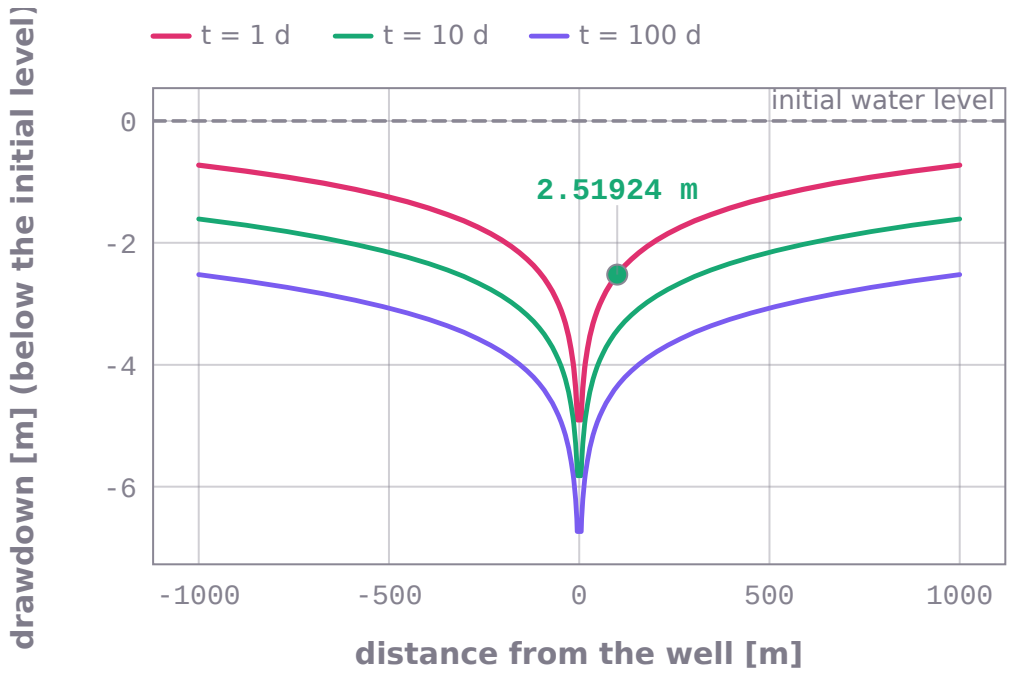
$$s = \frac{Q}{4\pi T} W(u), \quad u = \frac{r^2 S}{4Tt}, \quad W(u) = \int_u^\infty \frac{e^{-x}}{x}\, dx$$

input	meaning	condition
`Q`	pumping rate	
`T`	transmissivity	
`S`	storativity (storage coefficient)	
`r`	distance from the pumped well	
`t`	time since pumping began	

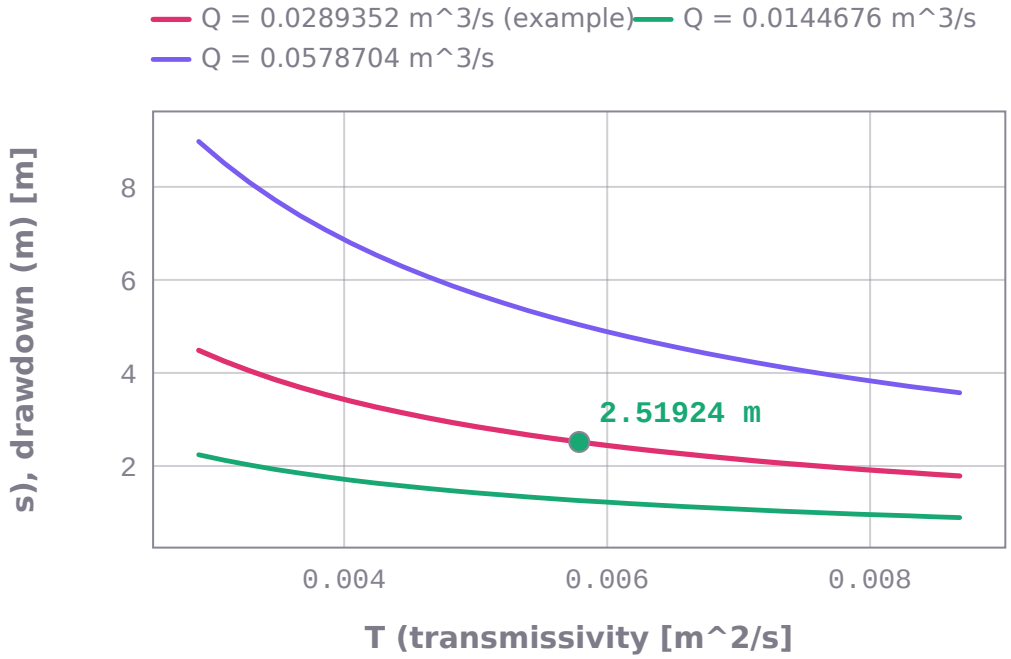
output	meaning
`s`	drawdown (m)
`u`	$u = r^2$ S/(4 T t)
`W`	well function W(u)

- **Method:** $u = r^2$ S/(4 T t), W(u) = E1(u) (series for $u \le 1$, continued fraction above), $s = Q$ W/(4 pi T)

- **Assumes:** confined, homogeneous, infinite aquifer; fully penetrating well, constant rate; $u \leq 50$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Drawdown cone around a well pumping Q = 2500 m³/d from a confined aquifer (T = 500 m²/d, S = 0.0002) after 1, 10 and 100 days, drawdown computed by the package at 41 distances from 5 m to 1 km (shown on both sides of the well): 4.90277 m at r = 5 m after 1 d; 5.81894 m at r = 5 m after 10 d; 6.73511 m at r = 5 m after 100 d. Marked: r = 100 m, t = 1 d, s = 2.51924 m.



s (drawdown (m)) against T (transmissivity) from 0.00289352 m^2/s to 0.00868056 m^2/s. One curve for each value of Q (pumping rate): 0.0289352 m^3/s, 0.0144676 m^3/s, 0.0578704 m^3/s. Other inputs as in the worked example: S = 0.0002, r = 100.0 m, t = 86400.0 s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, T = 0.00578704 m^2/s, s = 2.51924 m.

```
civil_engineering_ai999x.solve('theis_drawdown', Q=0.028935185185185185,
T=0.005787037037037037, S=0.0002, r=100.0, t=86400.0)
# -> s = 2.51924 m, u = 0.001, W = 6.33154   (SOLVED)
```

```
civil_engineering_ai999x.solve('theis_drawdown', Q=0.011574074074074073,
T=0.001736111111111111, S=0.001, r=60.0, t=10800.0)
# -> s = 1.32988 m, u = 0.048, W = 2.50677   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Theis drawdown with pumping rate = 2500 m^3/d, transmissivity = 500 m^2/d, storativity = 0.0002, distance = 100 m and time = 24 h.	SOLVED	s = 2.51924 m, u = 0.001, W = 6.33154
Theis drawdown with pumping rate = 1000 m^3/d, transmissivity = 150 m^2/d, storativity = 0.001, distance = 60 m and time = 3 h.	SOLVED	s = 1.32988 m, u = 0.048, W = 2.50677
Theis drawdown with pumping rate = 2500 m^3/d, transmissivity = 500 m^2/d, storativity = 0.0002 and distance = 100 m.	NEEDS_INPUT	theis_drawdown needs t (time since pumping began). Example: solve('theis_drawdown', Q='0.0289352 m^3/s', T='0.00578704 m
Theis drawdown with pumping rate = 2500 m^3/d, transmissivity = 500 m^2/d, storativity = 0.0002, distance = 100 m and time = 24 h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Transmissivity and storativity by the Cooper-Jacob straight line, `cooper_jacob_analysis`

Transmissivity and storativity of a confined aquifer from a pumping test by the Cooper-Jacob method: a least-squares straight line through drawdown against log10(time) (late-time data, $u < 0.05$, chosen by the user). Times are in minutes, drawdowns in m.

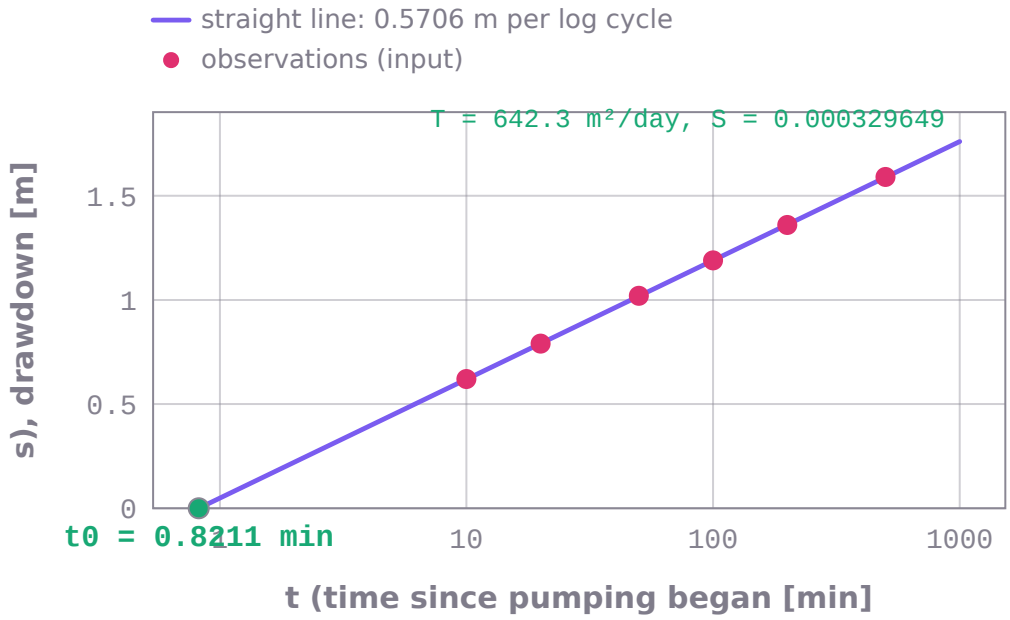
$$T = \frac{2.303\,Q}{4\pi\,\Delta s}, \quad S = \frac{2.25\,T\,t_0}{r^2}$$

input	meaning	condition
`times`	times since pumping began, in minutes	
`drawdowns`	drawdowns at those times, in m	
`Q`	pumping rate	
`r`	distance of the observation well	

output	meaning
`T`	transmissivity (m^2/s)
`S`	storativity
`ds_per_cycle`	drawdown per log cycle of time (m)
`t0_min`	zero-drawdown intercept time (min)
`T_m2_d`	transmissivity (m^2/day)

- **Method:** fit $s = a + b$ log10 t; $T = 2.303$ Q/(4 pi b), $S = 2.25$ T t_0/r^2 with $t_0 = 10^{-a/b}$
- **Assumes:** late-time data on the straight line $(u < 0.05)$; confined aquifer

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cooper–Jacob analysis of a pumping test (Q = 0.0231481 m³/s, observation well at r = 50.0 m): drawdowns [0.62, 0.79, 1.02, 1.19, 1.36, 1.59] m at [10, 20, 50, 100, 200, 500] min on a semi-log plot, and the straight line with the returned slope 0.570634 m per log cycle and zero-drawdown intercept t0 = 0.821137 min. Marked: the returned transmissivity 642.327 m²/day (0.00743434 m²/s) and storativity 0.000329649.



T (transmissivity (m^2/s)) against Q (pumping rate) from 0.0115741 m^3/s to 0.0347222 m^3/s. Other inputs as in the worked example: times = [10, 20, 50, 100, 200, 500], drawdowns = [0.62, 0.79, 1.02, 1.19, 1.36, 1.59], r = 50.0 m. 31 points computed by civil-engineering-ai999x; marked: the worked example, Q = 0.0231481 m^3/s, T = 0.00743434 m^2/s.

```
civil_engineering_ai999x.solve('cooper_jacob_analysis', times=[10, 20, 50, 100, 200, 500],
drawdowns=[0.62, 0.79, 1.02, 1.19, 1.36, 1.59], Q=0.023148148148148147, r=50.0)
# -> T = 0.00743434 m^2/s, S = 0.000329649, ds_per_cycle = 0.570634 m, t0_min = 0.821137,
T_m2_d = 642.327   (SOLVED)
```

```
civil_engineering_ai999x.solve('cooper_jacob_analysis', times=[30, 60, 120, 240, 480],
drawdowns=[1.1, 1.32, 1.55, 1.77, 2.0], Q=0.017361111111111112, r=30.0)
# -> T = 0.00425685 m^2/s, S = 0.000650541, ds_per_cycle = 0.747434 m, t0_min = 1.01881,
T_m2_d = 367.792   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cooper jacob analysis with times = [10, 20, 50, 100, 200, 500], drawdowns = [0.62, 0.79, 1.02, 1.19, 1.36, 1.59], pumping rate = 2000 m^3/d and distance = 50 m.	SOLVED	T = 0.00743434 m^2/s, S = 0.000329649, ds_per_cycle = 0.570634 m, t0_min = 0.821137, T_m2_d = 642.327
Cooper jacob analysis with times = [30, 60, 120, 240, 480], drawdowns = [1.1, 1.32, 1.55, 1.77, 2], pumping rate = 1500 m^3/d and distance = 30 m.	SOLVED	T = 0.00425685 m^2/s, S = 0.000650541, ds_per_cycle = 0.747434 m, t0_min = 1.01881, T_m2_d = 367.792
Cooper jacob analysis with times = [10, 20, 50, 100, 200, 500], drawdowns = [0.62, 0.79, 1.02, 1.19, 1.36, 1.59] and pumping rate = 2000 m^3/d.	NEEDS_INPUT	cooper_jacob_analysis needs r (distance of the observation well). Example: solve('cooper_jacob_analysis', times=[10, 20,
Cooper jacob analysis with times = [10, 20, 50, 100, 200, 500], drawdowns = [0.62, 0.79, 1.02, 1.19, 1.36, 1.59], pumping rate = 2000 m^3/d and distance = 50 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Drawdown from several wells by superposition (Theis), `well_field_drawdown`

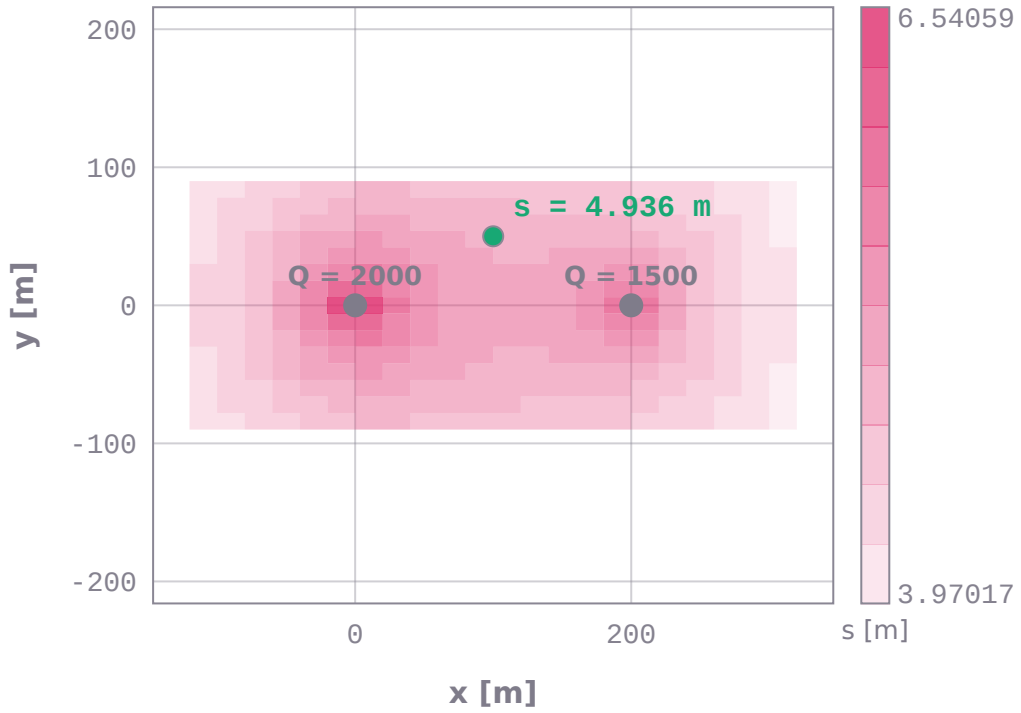
Total drawdown at a point from several wells pumping (positive rate) or injecting (negative rate) at constant rates in a confined aquifer, by superposing Theis solutions; with each well's contribution. Coordinates in m, rates in m^3/day.

$$s = \sum_i \frac{Q_i}{4\pi T} W\left(\frac{r_i^2 S}{4Tt}\right)$$

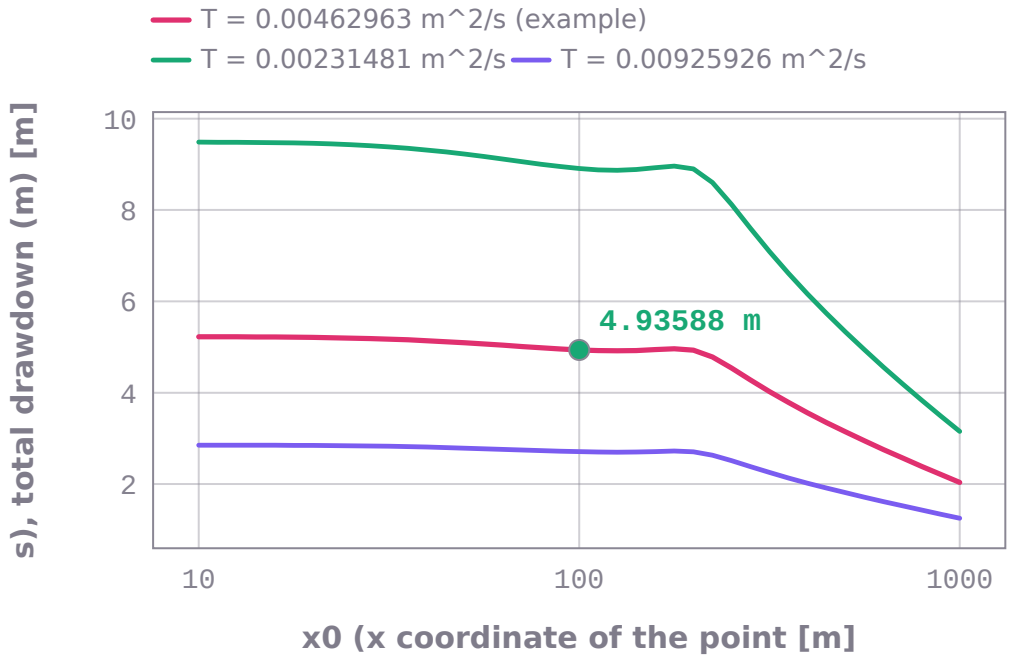
input	meaning	condition
`wx`	well x coordinates in m	
`wy`	well y coordinates in m	
`Qs`	pumping rates in m^3/day (negative for injection)	
`T`	transmissivity	
`S`	storativity	
`t`	time since pumping began (all wells together)	
`x0`	x coordinate of the point	
`y0`	y coordinate of the point	

output	meaning
`s`	total drawdown (m)
`parts`	drawdown from each well (m)

- **Method:** s = sum Q_i W(u_i)/(4 pi T), $u_i = r_i^2$ S/(4 T t)
- **Assumes:** confined, homogeneous, infinite aquifer; all wells started together; point at least 5 cm from any well
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Drawdown around 2 pumped wells (at x = [0, 200], y = [0, 0] m, pumping [2000, 1500] m³/day; T = 0.00462963 m²/s, S = 0.0003, after t = 432000 s) by superposition: the drawdown in 330 cells, each computed by the package, shaded from 3.97017 m to 6.54059 m. Marked: the returned drawdown s = 4.93588 m at (100, 50) m, the sum of the wells' parts 2.8205, 2.11538 m.



s (total drawdown (m)) against x0 (x coordinate of the point) from 10 m to 1000 m on a logarithmic axis. One curve for each value of T (transmissivity): 0.00462963 m^2/s, 0.00231481 m^2/s, 0.00925926 m^2/s. Other inputs as in the worked example: wx = [0, 200], wy = [0, 0], Qs = [2000, 1500], S = 0.0003, t = 432000.0 s, y0 = 50.0 m. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, x0 = 100 m, s = 4.93588 m.

```
civil_engineering_ai999x.solve('well_field_drawdown', wx=[0, 200], wy=[0, 0], Qs=[2000,
1500], T=0.004629629629629629, S=0.0003, t=432000.0, x0=100.0, y0=50.0)
# -> s = 4.93588 m, parts = [2.82050172973, 2.1153762973] m   (SOLVED)
```

```
civil_engineering_ai999x.solve('well_field_drawdown', wx=[-150, 150, 0], wy=[0, 0, 200],
Qs=[1800, -1000, 1200], T=0.009259259259259259, S=0.0001, t=172800.0, x0=0.0, y0=0.0)
# -> s = 1.39881 m, parts = [1.32071407719, -0.733730042886, 0.811829646889] m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Well field drawdown with well x = [0, 200], well y = [0, 0], pumping rates = [2000, 1500], transmissivity = 400 m^2/d, storativity = 0.0003, time = 5 d, point x = 100 m and point y = 50 m.	SOLVED	s = 4.93588 m, parts = [2.82050172973, 2.1153762973] m
Well field drawdown with well x = [-150, 150, 0], well y = [0, 0, 200], pumping rates = [1800, -1000, 1200], transmissivity = 800 m^2/d, storativity = 0.0001, time = 2 d, point x = 0 m and point y = 0 m.	SOLVED	s = 1.39881 m, parts = [1.32071407719, -0.733730042886, 0.811829646889] m
Well field drawdown with well x = [0, 200], well y = [0, 0], pumping rates = [2000, 1500], transmissivity = 400 m^2/d, storativity = 0.0003, time = 5 d and point x = 100 m.	NEEDS_INPUT	well_field_drawdown needs y0 (y coordinate of the point). Example: solve('well_field_drawdown', wx=[0, 200], wy=[0, 0],
Well field drawdown with well x = [0, 200], well y = [0, 0], pumping rates = [2000, 1500], transmissivity = 400 m^2/d,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
storativity = 0.0003, time = 5 d, point x = 100 m and point y = 50 m and colour = 3.		

Unconfined flow between two water bodies with recharge (Dupuit),

`dupuit_flow_with_recharge`

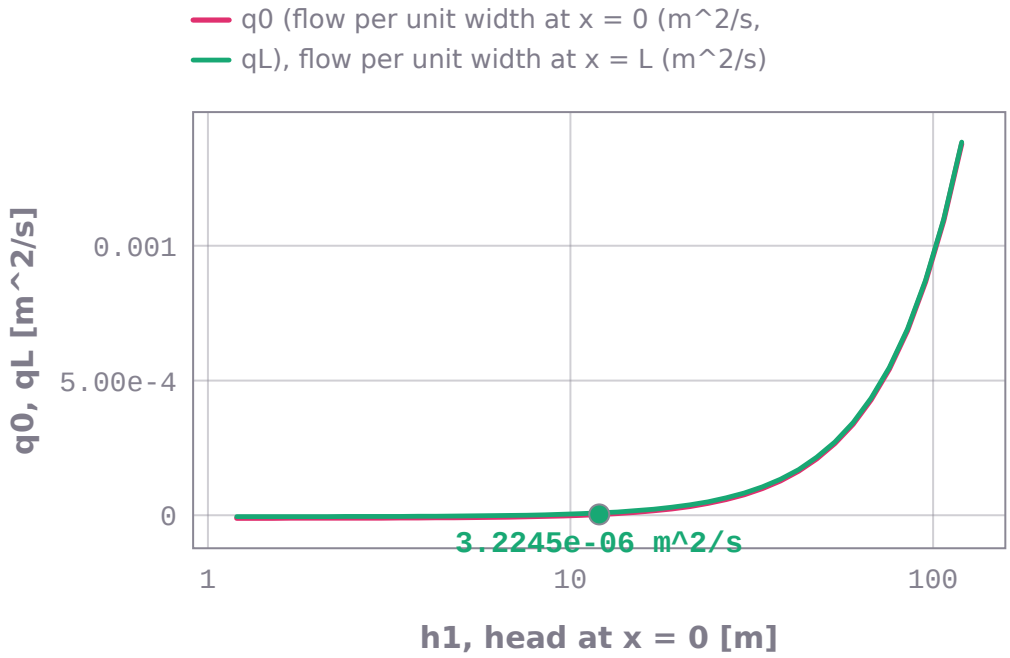
Steady unconfined flow per unit width between two water bodies (heads h1 at $x = 0$ and h2 at $x = L$ above the base) with uniform recharge W: the flows at both ends (positive toward x = L), the groundwater divide and the highest water table (Dupuit-Forchheimer).

$$h^2 = h_1^2 - \frac{(h_1^2 - h_2^2)x}{L} + \frac{W}{K}(L - x)x, \quad q(x) = \frac{K(h_1^2 - h_2^2)}{2L} - W\left(\frac{L}{2} - x\right)$$

input	meaning	condition
`K`	hydraulic conductivity	
`h1`	head at $x = 0$ (above the impermeable base)	
`h2`	head at $x = L$	
`L`	distance between the water bodies	
`W`	recharge rate	

output	meaning
`q0`	flow per unit width at $x = 0$ (m^2/s, positive toward x = L)
`qL`	flow per unit width at $x = L$ (m^2/s)
`x_divide`	position of the groundwater divide (m); outside 0..L means no divide
`h_max`	highest water table between the water bodies (m)

- **Method:** $h^2 = h_1^2 - (h_1^2 - h_2^2)$ x/L + (W/K)(L - x) x; $q = K$(h1^2 - h2^2)/(2L) - W(L/2 - x)
- **Assumes:** steady, unconfined, homogeneous aquifer on a horizontal base; Dupuit assumptions; $W > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



q0 (flow per unit width at x = 0 (m^2/s, positive toward x = L)) against h1 (head at x = 0 (above the impermeable base)) from 1.2 m to 120 m on a logarithmic axis. Outputs with the same unit drawn together: qL (flow per unit width at x = L (m^2/s)). Other inputs as in the worked example: K = 0.00011574074074074073 m/s, h2 = 9.0 m, L = 600.0 m, W = 9.506426344208684e-09 m/s. 41 points computed by civil-engineering-ai999x; marked: the worked example, h1 = 12 m, q0 = 3.2245e-06 m^2/s.

```
civil_engineering_ai999x.solve('dupuit_flow_with_recharge', K=0.00011574074074074073,
h1=12.0, h2=9.0, L=600.0, W=9.506426344208684e-09)
# -> q0 = 3.22446e-06 m^2/s, qL = 8.92832e-06 m^2/s, x_divide = -339.188 m, h_max = 12 m
(SOLVED)
```

```
civil_engineering_ai999x.solve('dupuit_flow_with_recharge', K=2.3148148148148147e-05,
h1=8.0, h2=8.0, L=400.0, W=1.5844043907014474e-08)
# -> q0 = -3.16881e-06 m^2/s, qL = 3.16881e-06 m^2/s, x_divide = 200 m, h_max = 9.55921 m
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dupuit recharge flow with hydraulic conductivity = 10 m/d, first head = 12 m, second head = 9 m, distance = 600 m and recharge = 300 mm/yr.	SOLVED	q0 = 3.22446e-06 m^2/s, qL = 8.92832e-06 m^2/s, x_divide = -339.188 m, h_max = 12 m
Dupuit recharge flow with hydraulic conductivity = 2 m/d, first head = 8 m, second head = 8 m, distance = 400 m and recharge = 500 mm/yr.	SOLVED	q0 = -3.16881e-06 m^2/s, qL = 3.16881e-06 m^2/s, x_divide = 200 m, h_max = 9.55921 m
Dupuit recharge flow with hydraulic conductivity = 10 m/d, first head = 12 m, second head = 9 m and distance = 600 m.	NEEDS_INPUT	dupuit_flow_with_recharge needs W (recharge rate). Example: solve('dupuit_flow_with_recharge', K='0.000115741 m/s', h1='
Dupuit recharge flow with hydraulic conductivity = 10 m/d, first head = 12 m, second head = 9 m, distance = 600	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
m and recharge = 300 mm/yr and colour = 3.		

Soil phase relations from void ratio and water content, `soil_phase_relations`

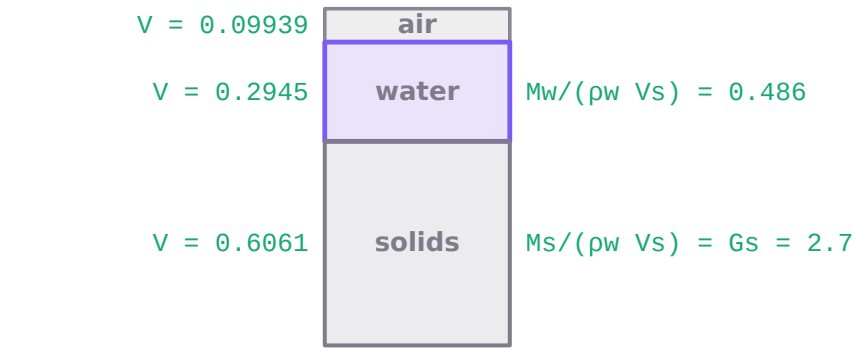
Porosity, degree of saturation and the dry, bulk, saturated and submerged unit weights of a soil from its specific gravity of solids Gs, void ratio e and water content w (unit weight of water 9.81 kN/m^3).

$$Se = wG_s, \quad \gamma = \frac{G_s(1+w)}{1+e}\gamma_w, \quad \gamma_{sat} = \frac{G_s+e}{1+e}\gamma_w$$

input	meaning	condition
`Gs`	specific gravity of the solids	
`e`	void ratio	
`w`	water content (fraction, or % in a question)	

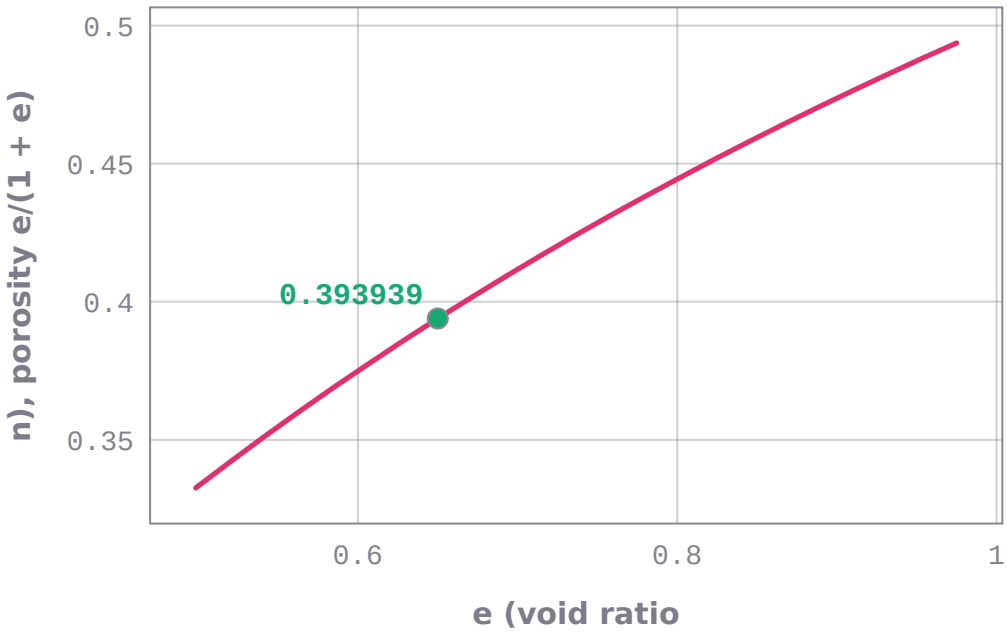
output	meaning
`n`	porosity $e/(1+e)$
`S`	degree of saturation w Gs/e
`gamma_d`	dry unit weight (N/m^3)
`gamma`	bulk (moist) unit weight (N/m^3)
`gamma_sat`	saturated unit weight (N/m^3)
`gamma_sub`	submerged (buoyant) unit weight (N/m^3)

- **Method:** $n = e/(1+e)$, $S = w$ Gs/e, $gamma_d = Gs\ gamma_w/(1+e)$, $\gamma = Gs$ gamma_w $(1+w)/(1+e)$, gamma_sat = (Gs + e) $gamma_w/(1+e)$
- **Assumes:** w $Gs \le e$ (not oversaturated); $2.0 \le Gs \le 3.0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n = 0.3939, S = 0.7477
γ = 18.94 kN/m³, γd = 16.05 kN/m³
γsat = 19.92, γ' = 10.11 kN/m³

Phase diagram of a soil with Gs = 2.7, void ratio e = 0.65 and water content w = 0.18: volumes as fractions of the total, solids 1/(1 + e), water S·e/(1 + e) with the returned degree of saturation S = 0.747692, the rest air; masses relative to ρw·Vs. Marked: the returned porosity n = 0.393939 (= the void fraction), unit weights γ = 18.9422, γd = 16.0527, γsat = 19.9173, submerged 10.1073 kN/m³.



n (porosity e/(1 + e)) against e (void ratio) from 0.325 to 0.975. Other inputs as in the worked example: Gs = 2.7, w = 0.18. 23 points computed by civil-engineering-ai999x; marked: the worked example, e = 0.65, n = 0.393939.

```
civil_engineering_ai999x.solve('soil_phase_relations', Gs=2.7, e=0.65, w=0.18)
# -> n = 0.393939, S = 0.747692, gamma_d = 16052.7 N/m^3, gamma = 18942.2 N/m^3, gamma_sat
= 19917.3 N/m^3, gamma_sub = 10107.3 N/m^3   (SOLVED)
```

```
civil_engineering_ai999x.solve('soil_phase_relations', Gs=2.65, e=0.9, w=0.3)
# -> n = 0.473684, S = 0.883333, gamma_d = 13682.4 N/m^3, gamma = 17787.1 N/m^3, gamma_sat
= 18329.2 N/m^3, gamma_sub = 8519.21 N/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Soil phase relations with specific gravity = 2.7, void ratio = 0.65 and water content = 18 %.	SOLVED	n = 0.393939, S = 0.747692, gamma_d = 16052.7 N/m^3, gamma = 18942.2 N/m^3, gamma_sat = 19917.3 N/m^3, gamma_sub = 10107.3 N/m^3
Soil phase relations with specific gravity = 2.65, void ratio = 0.9 and water content = 30 %.	SOLVED	n = 0.473684, S = 0.883333, gamma_d = 13682.4 N/m^3, gamma = 17787.1 N/m^3, gamma_sat = 18329.2 N/m^3, gamma_sub = 8519.21 N/m^3
Soil phase relations with specific gravity = 2.7 and void ratio = 0.65.	NEEDS_INPUT	soil_phase_relations needs w (water content (fraction, or % in a question)). Example: solve('soil_phase_relations', Gs=2
Soil phase relations with specific gravity = 2.7, void ratio = 0.65 and water content = 18 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Soil phase relations with specific gravity = 2.7, void ratio = 0.5 and water content = 40 %.	INVALID_INPUT	Soil phase relations from void ratio and water content: w Gs exceeds e: the soil would be more than saturated.

Soil properties from a weighed sample, `soil_phase_from_lab_masses`

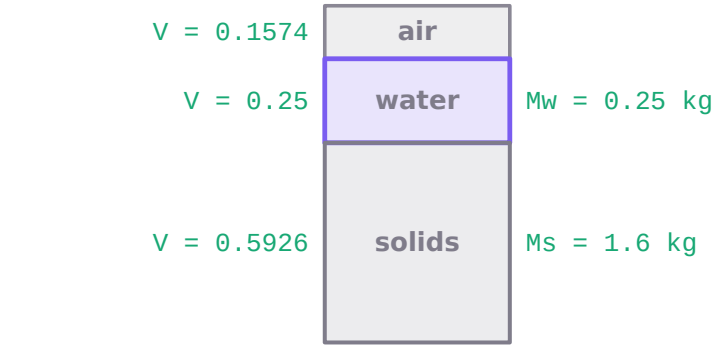
Water content, bulk and dry densities and unit weights, void ratio, porosity and degree of saturation of a soil sample from its wet mass, oven-dry mass, volume and the specific gravity of the solids.

$$w = \frac{M - M_d}{M_d}, \quad e = \frac{G_s \rho_w}{\rho_d} - 1, \quad S = \frac{w G_s}{e}$$

input	meaning	condition
`M`	wet (moist) mass of the sample	
`Md`	oven-dry mass of the sample	
`V`	volume of the sample	
`Gs`	specific gravity of the solids	

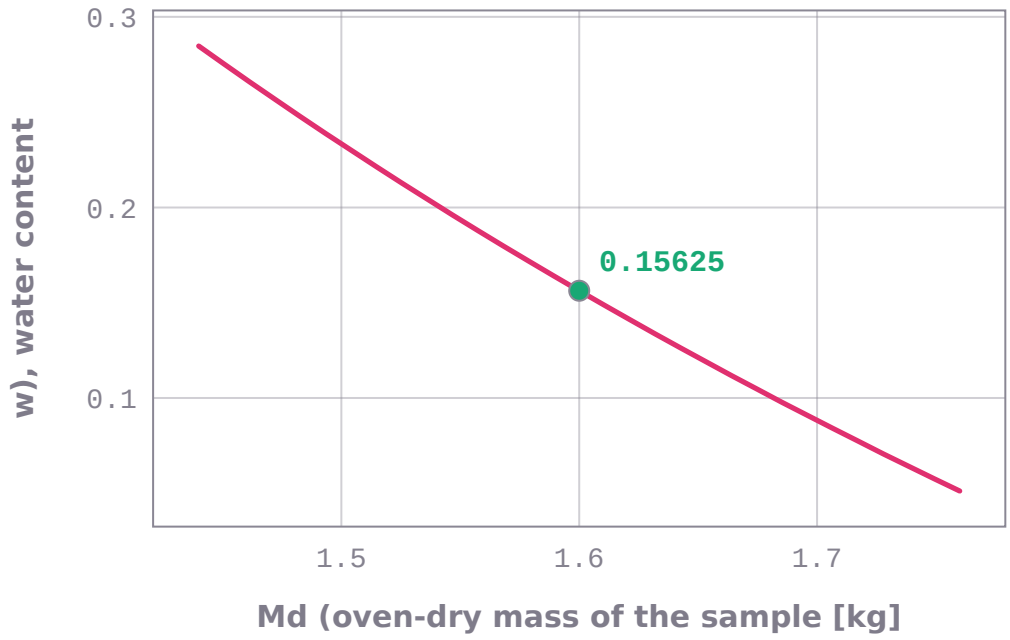
output	meaning
`w`	water content
`rho`	bulk density (kg/m^3)
`rho_d`	dry density (kg/m^3)
`e`	void ratio
`n`	porosity
`S`	degree of saturation
`gamma`	bulk unit weight (N/m^3)
`gamma_d`	dry unit weight (N/m^3)

- **Method:** $w = (M - Md)/Md$, $rho_d = Md/V$, $e = Gs\, rho_w/rho_d - 1$, $S = w$ Gs/e
- **Assumes:** $M \geq Md$; consistent data (0 < e, S <= 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



w = 0.1562, e = 0.6875, n = 0.4074, S = 0.6136
ρ = 1850 kg/m³, ρd = 1600 kg/m³
γ = 18.15 kN/m³, γd = 15.7 kN/m³

Phase diagram of a lab specimen (mass M = 1.85 kg, dry mass 1.6 kg, volume 0.001 m³, Gs = 2.7): volumes as fractions of the specimen volume, solids V/(1 + e) and water S·(V − Vs) from the returned void ratio e = 0.6875 and saturation S = 0.613636; masses as stated (water = M − Md). Marked: the returned w = 0.15625, n = 0.407407, ρ = 1850 kg/m³, ρd = 1600 kg/m³, γ = 18.1485 kN/m³, γd = 15.696 kN/m³.



w (water content) against Md (oven-dry mass of the sample) from 1.44 kg to 1.76 kg. Other inputs as in the worked example: M = 1.85 kg, V = 0.001 m^3, Gs = 2.7. 31 points computed by civil-engineering-ai999x; marked: the worked example, Md = 1.6 kg, w = 0.15625.

```
civil_engineering_ai999x.solve('soil_phase_from_lab_masses', M=1.85, Md=1.6, V=0.001,
Gs=2.7)
# -> w = 0.15625, rho = 1850 kg/m^3, rho_d = 1600 kg/m^3, e = 0.6875, n = 0.407407, S =
0.613636, gamma = 18148.5 N/m^3, gamma_d = 15696 N/m^3   (SOLVED)
```

```
civil_engineering_ai999x.solve('soil_phase_from_lab_masses', M=0.20500000000000002,
Md=0.178, V=0.00010499999999999999, Gs=2.68)
# -> w = 0.151685, rho = 1952.38 kg/m^3, rho_d = 1695.24 kg/m^3, e = 0.580899, n =
0.367448, S = 0.699807, gamma = 19152.9 N/m^3, gamma_d = 16630.3 N/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Soil properties from a weighed sample with wet mass = 1850 g, dry mass = 1600 g, volume = 1000 cm^3 and specific gravity = 2.7.	SOLVED	w = 0.15625, rho = 1850 kg/m^3, rho_d = 1600 kg/m^3, e = 0.6875, n = 0.407407, S = 0.613636, gamma = 18148.5 N/m^3, gamma_d = 15696 N/m^3
Soil properties from a weighed sample with wet mass = 205 g, dry mass = 178 g, volume = 105 cm^3 and specific gravity = 2.68.	SOLVED	w = 0.151685, rho = 1952.38 kg/m^3, rho_d = 1695.24 kg/m^3, e = 0.580899, n = 0.367448, S = 0.699807, gamma = 19152.9 N/m^3, gamma_d = 16630.3 N/m^3
Soil properties from a weighed sample with wet mass = 1850 g, dry mass = 1600 g and volume = 1000 cm^3.	NEEDS_INPUT	soil_phase_from_lab_masses needs Gs (specific gravity of the solids). Example: solve('soil_phase_from_lab_masses', M='1.
Soil properties from a weighed sample with wet mass = 1850 g, dry mass = 1600 g, volume = 1000 cm^3 and specific gravity = 2.7 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Soil properties from a weighed sample with wet mass = 1500 g, dry mass = 1600 g, volume = 1000 cm^3 and specific gravity = 2.7.	INVALID_INPUT	Soil properties from a weighed sample: the wet mass cannot be less than the dry mass.

Relative density of a granular soil, `relative_density_from_unit_weights`

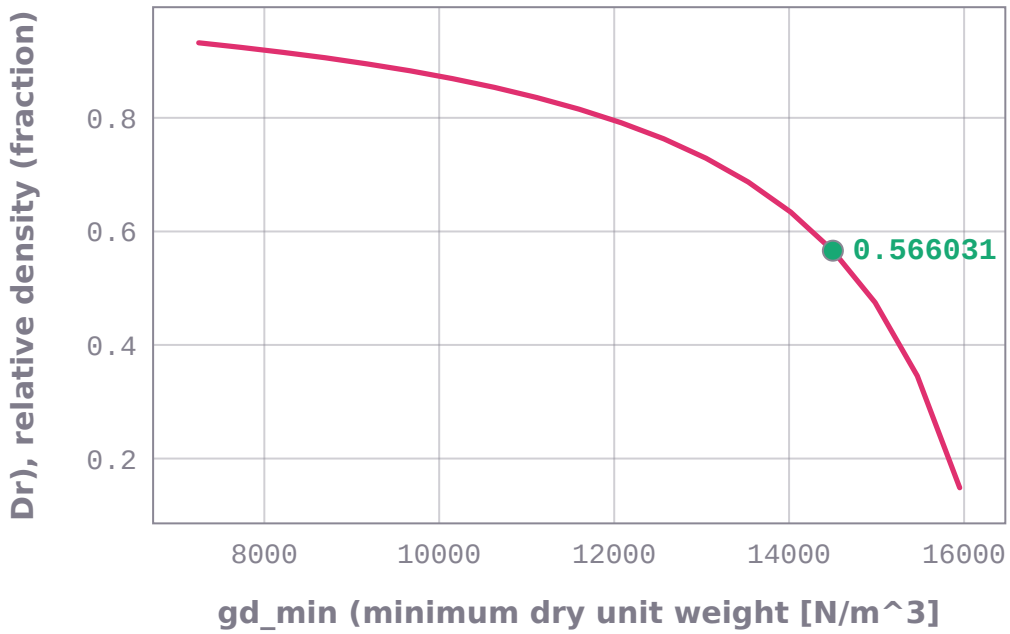
Relative density (density index) of a sand from its in-place dry unit weight and the minimum and maximum dry unit weights of the laboratory tests.

$$D_r = \frac{\gamma_{d,max}}{\gamma_d}\,\frac{\gamma_d - \gamma_{d,min}}{\gamma_{d,max} - \gamma_{d,min}}$$

input	meaning	condition
`gd`	in-place dry unit weight	
`gd_min`	minimum dry unit weight	
`gd_max`	maximum dry unit weight	

output	meaning
`Dr`	relative density (fraction)
`Dr_percent`	relative density (%)

- **Method:** Dr = (gd_max/gd)(gd - gd_min)/(gd_max - gd_min) = (e_max - e)/(e_max - e_min)
- **Assumes:** gd_min <= gd <= gd_max
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Dr (relative density (fraction)) against gd_min (minimum dry unit weight) from 7250 N/m^3 to 21750 N/m^3. Other inputs as in the worked example: gd = 16200.0 N/m^3, gd_max = 17800.0 N/m^3. 19 points computed by civil-engineering-ai999x; marked: the worked example, gd_min = 14500 N/m^3, Dr = 0.566031.

```
civil_engineering_ai999x.solve('relative_density_from_unit_weights', gd=16200.0,
gd_min=14500.0, gd_max=17800.0)
# -> Dr = 0.566031, Dr_percent = 56.6031   (SOLVED)
```

```
civil_engineering_ai999x.solve('relative_density_from_unit_weights', gd=15000.0,
gd_min=13800.0, gd_max=18100.0)
# -> Dr = 0.336744, Dr_percent = 33.6744   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relative density from unit weights with dry unit weight = 16.2 kN/m^3, minimum dry unit weight = 14.5 kN/m^3 and maximum dry unit weight = 17.8 kN/m^3.	SOLVED	Dr = 0.566031, Dr_percent = 56.6031
Relative density from unit weights with dry unit weight = 15 kN/m^3, minimum dry unit weight = 13.8 kN/m^3 and	SOLVED	Dr = 0.336744, Dr_percent = 33.6744

question	outcome	result
maximum dry unit weight = 18.1 kN/m^3.		
Relative density from unit weights with dry unit weight = 16.2 kN/m^3 and minimum dry unit weight = 14.5 kN/m^3.	NEEDS_INPUT	relative_density_from_unit_weights needs gd_max (maximum dry unit weight). Example: solve('relative_density_from_unit_we
Relative density from unit weights with dry unit weight = 16.2 kN/m^3, minimum dry unit weight = 14.5 kN/m^3 and maximum dry unit weight = 17.8 kN/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Relative density from unit weights with dry unit weight = 19 kN/m^3, minimum dry unit weight = 14.5 kN/m^3 and maximum dry unit weight = 17.8 kN/m^3.	INVALID_INPUT	Relative density of a granular soil: the dry unit weight must lie between the minimum and the maximum.

Liquid limit from Casagrande cup tests, `liquid_limit_flow_curve`

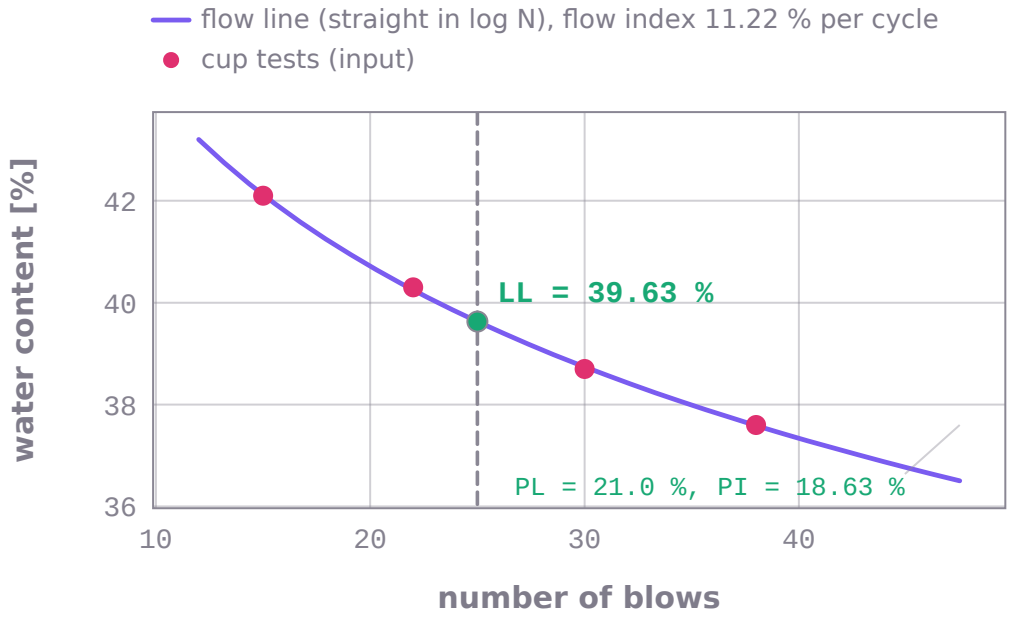
Liquid limit (water content at 25 blows) and flow index from Casagrande cup tests by a least-squares line of water content against log10(blows), with the plasticity index when the plastic limit is given. Water contents in percent.

$$LL = w(N = 25), \quad PI = LL - PL$$

input	meaning	condition
`blows`	number of blows of each test	
`w`	water content of each test in %	
`PL`	plastic limit in % (optional)	

output	meaning
`LL`	liquid limit (%)
`flow_index`	flow index (% per log cycle of blows)
`PI`	plasticity index LL - PL (%)

- **Method:** least squares $w = a + b$ log10 N; $LL = a + b$ log10 25; flow index = -b
- **Assumes:** water content falls with blows; water contents in %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Casagrande cup tests ([15, 22, 30, 38] blows at water contents [42.1, 40.3, 38.7, 37.6] %) and the flow line with the returned flow index 11.2179 % per log cycle, through the returned liquid limit LL = 39.6292 % at 25 blows; with PL = 21.0 %, the plasticity index PI = 18.6292 %.

```
civil_engineering_ai999x.solve('liquid_limit_flow_curve', blows=[15, 22, 30, 38], w=[42.1,
40.3, 38.7, 37.6], PL=21.0)
# -> LL = 39.6292, flow_index = 11.2179, PI = 18.6292   (SOLVED)
```

```
civil_engineering_ai999x.solve('liquid_limit_flow_curve', blows=[12, 20, 28, 35], w=[58.0,
54.6, 52.1, 50.5])
# -> LL = 52.9097, flow_index = 16.163   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Liquid limit from cup tests with number of blows = [15, 22, 30, 38], water contents = [42.1, 40.3, 38.7, 37.6] and plastic limit = 21.	SOLVED	LL = 39.6292, flow_index = 11.2179, PI = 18.6292
Liquid limit from cup tests with number of blows = [12, 20, 28, 35] and water contents = [58, 54.6, 52.1, 50.5].	SOLVED	LL = 52.9097, flow_index = 16.163
Liquid limit from cup tests with number of blows = [15, 22, 30, 38] and plastic limit = 21.	NEEDS_INPUT	liquid_limit_flow_curve needs w (water content of each test in %). Example: solve('liquid_limit_flow_curve', blows=[15,
Liquid limit from cup tests with number of blows = [15, 22, 30, 38], water contents = [42.1, 40.3, 38.7, 37.6] and plastic limit = 21 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Effective sizes and gradation coefficients from a sieve analysis, `grain_size_coefficients`

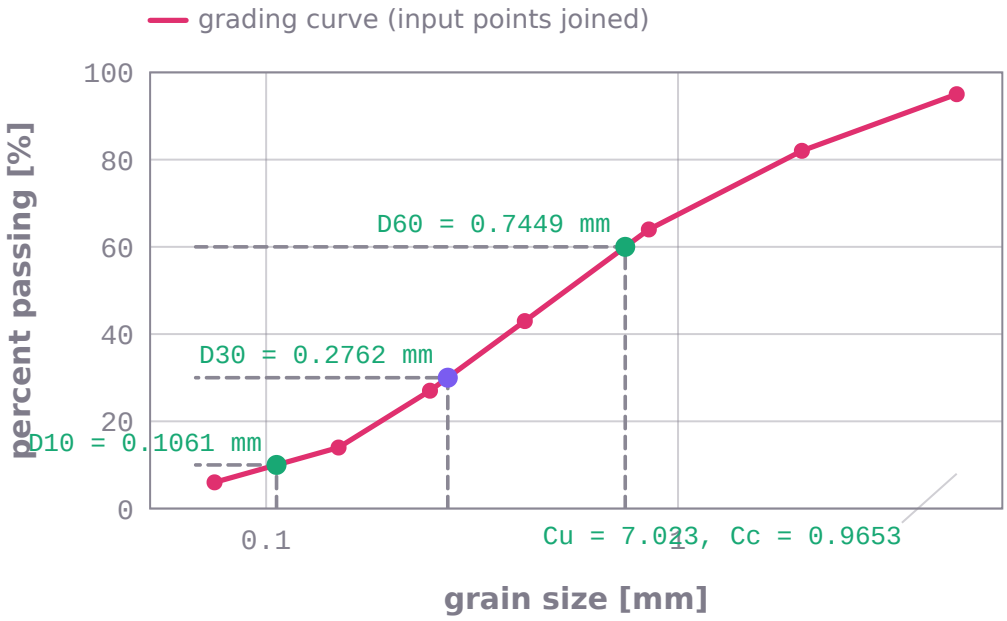
D10, D30 and D60 by log-linear interpolation of the grading curve, and the coefficients of uniformity Cu and curvature Cc. Sieve sizes in mm, percent passing in %.

$$C_u = \frac{D_{60}}{D_{10}}, \quad C_c = \frac{D_{30}^2}{D_{10} D_{60}}$$

input	meaning	condition
`sizes`	sieve sizes in mm	
`passing`	percent passing each sieve, in %	

output	meaning
`D10`	D10 (mm)
`D30`	D30 (mm)
`D60`	D60 (mm)
`Cu`	coefficient of uniformity $D60/D10$
`Cc`	coefficient of curvature D30^2/(D10 D60)

- **Method:** log-linear interpolation of size against percent passing
- **Assumes:** percent passing rises strictly with size; the curve spans 10% to 60%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Grain-size distribution from a sieve analysis (sizes [4.75, 2.0, 0.85, 0.425, 0.25, 0.15, 0.075] mm, percent passing [95, 82, 64, 43, 27, 14, 6]), on a logarithmic size axis. Marked: the returned effective sizes D10 = 0.106066 mm, D30 = 0.276153 mm, D60 = 0.744869 mm, uniformity coefficient Cu = 7.02269 and curvature coefficient Cc = 0.965255.

```
civil_engineering_ai999x.solve('grain_size_coefficients', sizes=[4.75, 2.0, 0.85, 0.425,
0.25, 0.15, 0.075], passing=[95, 82, 64, 43, 27, 14, 6])
# -> D10 = 0.106066, D30 = 0.276153, D60 = 0.744869, Cu = 7.02269, Cc = 0.965255   (SOLVED)
```

```
civil_engineering_ai999x.solve('grain_size_coefficients', sizes=[19, 9.5, 4.75, 2.36, 1.18,
0.6, 0.3, 0.15], passing=[100, 88, 70, 52, 36, 22, 11, 4])
# -> D10 = 0.271717, D30 = 0.883075, D60 = 3.22052, Cu = 11.8525, Cc = 0.891152   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Grain size coefficients with sieve sizes = [4.75, 2, 0.85, 0.425, 0.25, 0.15, 0.075] and percent passing = [95, 82, 64, 43, 27, 14, 6].	SOLVED	D10 = 0.106066, D30 = 0.276153, D60 = 0.744869, Cu = 7.02269, Cc = 0.965255
Grain size coefficients with sieve sizes = [19, 9.5, 4.75, 2.36, 1.18, 0.6, 0.3, 0.15] and percent passing = [100, 88, 70, 52, 36, 22, 11, 4].	SOLVED	D10 = 0.271717, D30 = 0.883075, D60 = 3.22052, Cu = 11.8525, Cc = 0.891152
Grain size coefficients with sieve sizes = [4.75, 2, 0.85, 0.425, 0.25, 0.15, 0.075].	NEEDS_INPUT	grain_size_coefficients needs passing (percent passing each sieve, in %). Example: solve('grain_size_coefficients', size
Grain size coefficients with sieve sizes = [4.75, 2, 0.85, 0.425, 0.25, 0.15, 0.075] and percent passing = [95, 82, 64, 43, 27, 14, 6] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Total, pore-water and effective vertical stress at a depth, `effective_stress_at_depth`

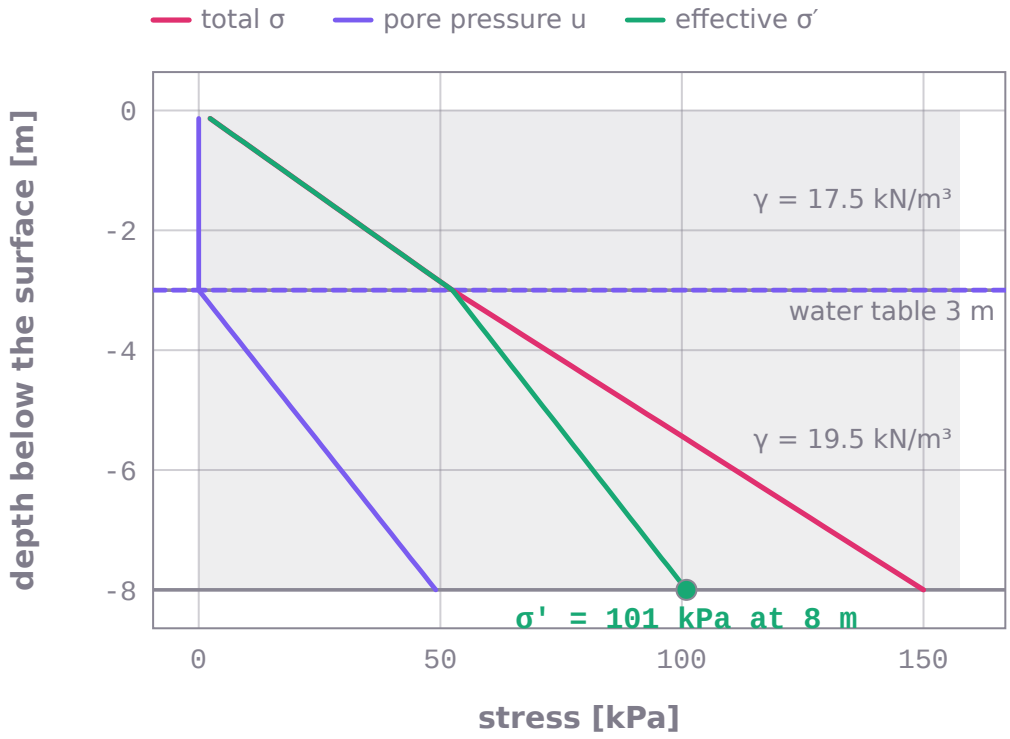
Total vertical stress, hydrostatic pore-water pressure and effective vertical stress at depth z in a layered soil with a water table at depth zw (no capillary rise, hydrostatic water). Each layer's unit weight is the one that applies there (moist above the water table, saturated below), in kN/m^3.

$$\sigma'_v = \sum \gamma_i h_i - \gamma_w (z - z_w)$$

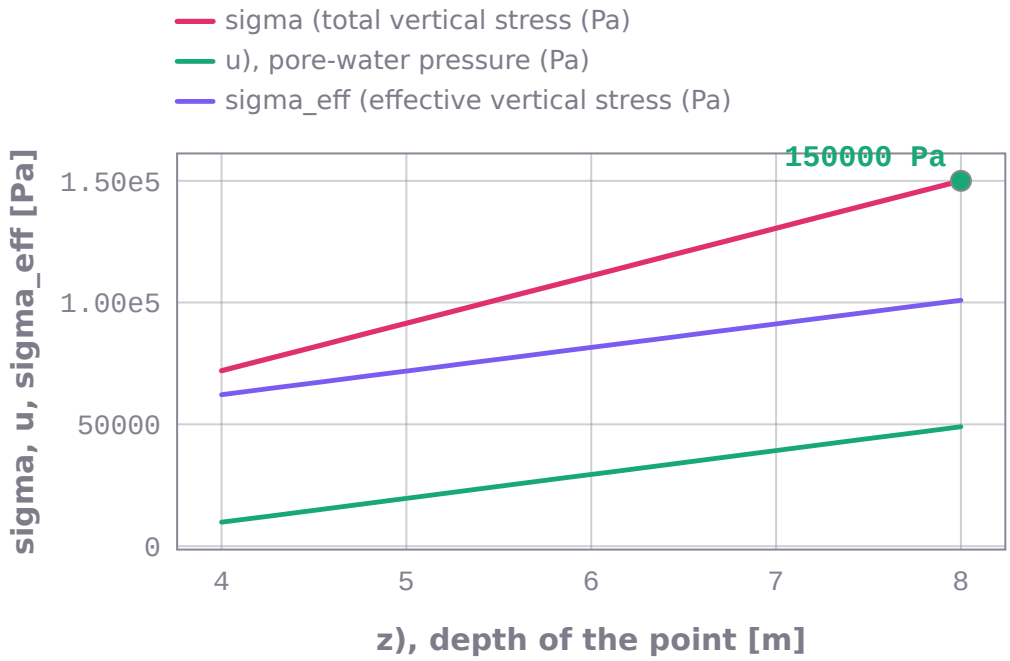
input	meaning	condition
`H`	layer thicknesses in m, from the surface down	
`gam`	unit weight of each layer in kN/m^3	
`zw`	depth of the water table	
`z`	depth of the point	

output	meaning
`sigma`	total vertical stress (Pa)
`u`	pore-water pressure (Pa)
`sigma_eff`	effective vertical stress (Pa)
`sigma_eff_kPa`	effective vertical stress (kPa)

- **Method:** sigma = sum gamma_i h_i above z, $u = gamma_w$ (z - zw) below the water table, sigma' = sigma - u
- **Assumes:** hydrostatic pore water; no capillary rise; 0 < z <= total thickness
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vertical stresses with depth in layered soil (layers [3, 5] m thick, unit weights [17.5, 19.5] kN/m³, water table at 3 m): total stress, pore-water pressure and effective stress at 61 depths, each computed by the package. Marked: at z = 8 m the returned σ = 150 kPa, u = 49.05 kPa, σ′ = 100.95 kPa.



sigma (total vertical stress (Pa)) against z (depth of the point) from 4 m to 12 m. Outputs with the same unit drawn together: u (pore-water pressure (Pa)), sigma_eff (effective vertical stress (Pa)). Other inputs as in the worked example: H = [3, 5], gam = [17.5, 19.5], zw = 3.0 m. 16 points computed by civil-engineering-ai999x; marked: the worked example, z = 8 m, sigma = 150000 Pa.

```
civil_engineering_ai999x.solve('effective_stress_at_depth', H=[3, 5], gam=[17.5, 19.5],
zw=3.0, z=8.0)
# -> sigma = 150000 Pa, u = 49050 Pa, sigma_eff = 100950 Pa, sigma_eff_kPa = 100.95
(SOLVED)
```

```
civil_engineering_ai999x.solve('effective_stress_at_depth', H=[2, 4, 6], gam=[16.8, 18.9, 20.1], zw=1.5, z=9.0)
# -> sigma = 169500 Pa, u = 73575 Pa, sigma_eff = 95925 Pa, sigma_eff_kPa = 95.925 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective stress at depth with thicknesses = [3, 5], unit weights = [17.5, 19.5], water table depth = 3 m and depth = 8 m.	SOLVED	sigma = 150000 Pa, u = 49050 Pa, sigma_eff = 100950 Pa, sigma_eff_kPa = 100.95
Effective stress at depth with thicknesses = [2, 4, 6], unit weights = [16.8, 18.9, 20.1], water table depth = 1.5 m and depth = 9 m.	SOLVED	sigma = 169500 Pa, u = 73575 Pa, sigma_eff = 95925 Pa, sigma_eff_kPa = 95.925
Effective stress at depth with thicknesses = [3, 5], unit weights = [17.5, 19.5] and water table depth = 3 m.	NEEDS_INPUT	effective_stress_at_depth needs z (depth of the point). Example: solve('effective_stress_at_depth', H=[3, 5], gam=[17.5,
Effective stress at depth with thicknesses = [3, 5], unit weights = [17.5, 19.5], water table depth = 3 m and depth = 8 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Effective stress at depth with thicknesses = [3, 5], unit weights = [17.5, 19.5], water table depth = 3 m and depth = 12 m.	INVALID_INPUT	Total, pore-water and effective vertical stress at a depth: the depth must lie within the layers.

Primary consolidation settlement of a normally consolidated clay,

`consolidation_settlement_nc`

Primary consolidation settlement of a normally consolidated clay layer of thickness H from the compression index Cc, the initial void ratio and the initial and added effective stresses at mid-depth.

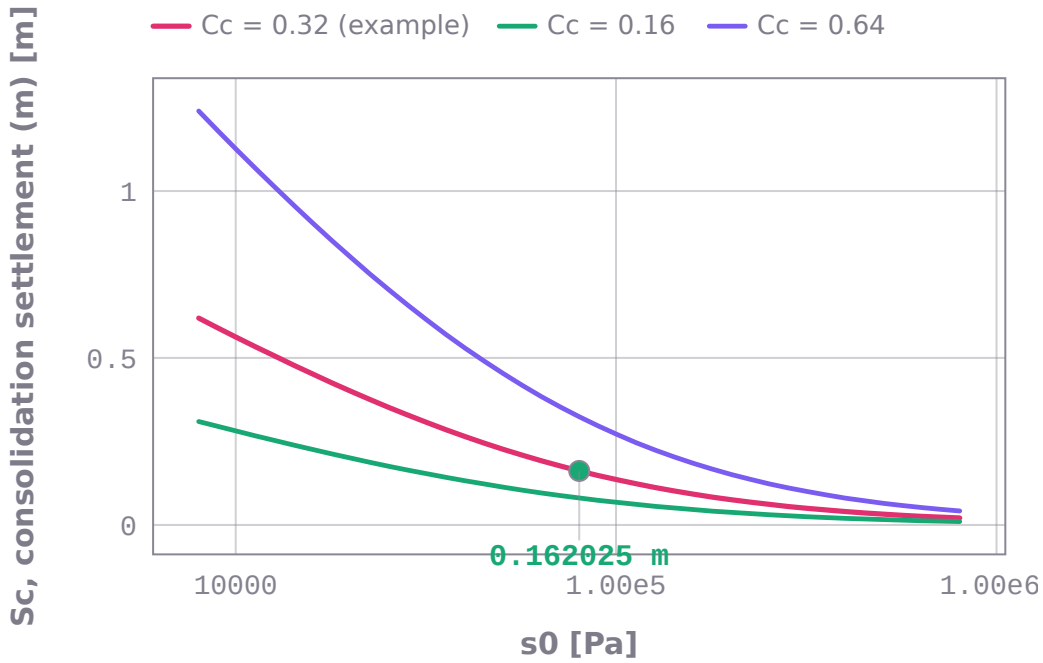
$$S_c = \frac{C_c H}{1 + e_0} \log_{10} \frac{\sigma'_0 + \Delta\sigma}{\sigma'_0}$$

input	meaning	condition
`Cc`	compression index	
`e0`	initial void ratio	
`H`	clay layer thickness	
`s0`	initial effective vertical stress at mid-depth	
`ds`	added vertical stress at mid-depth	

output	meaning
`Sc`	consolidation settlement (m)
`de`	change in void ratio

output	meaning
`e_final`	final void ratio

- **Method:** $Sc = Cc\,H/(1+e_0)$ log10((s0 + ds)/s0)
- **Assumes:** normally consolidated clay; stresses at mid-depth of the layer
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sc (consolidation settlement (m)) against s0 (initial effective vertical stress at mid-depth) from 8000 Pa to 800000 Pa on a logarithmic axis. One curve for each value of Cc (compression index): 0.32, 0.16, 0.64. Other inputs as in the worked example: e0 = 0.92, H = 4.0 m, ds = 60000.0 Pa. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, s0 = 80000 Pa, Sc = 0.162025 m.

```
civil_engineering_ai999x.solve('consolidation_settlement_nc', Cc=0.32, e0=0.92, H=4.0,
s0=80000.0, ds=60000.0)
# -> Sc = 0.162025 m, de = 0.0777722, e_final = 0.842228   (SOLVED)
```

```
civil_engineering_ai999x.solve('consolidation_settlement_nc', Cc=0.5, e0=1.4, H=6.0,
s0=55000.0, ds=100000.0)
# -> Sc = 0.562461 m, de = 0.224985, e_final = 1.17502   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Consolidation settlement of a normally consolidated clay with compression index = 0.32, initial void ratio = 0.92, thickness = 4 m, initial effective stress = 80 kPa and stress increase = 60 kPa.	SOLVED	Sc = 0.162025 m, de = 0.0777722, e_final = 0.842228
Consolidation settlement of a normally consolidated clay with compression index = 0.5, initial void ratio = 1.4, thickness = 6 m, initial effective stress = 55 kPa and stress increase = 100 kPa.	SOLVED	Sc = 0.562461 m, de = 0.224985, e_final = 1.17502

question	outcome	result
Consolidation settlement of a normally consolidated clay with compression index = 0.32, initial void ratio = 0.92, thickness = 4 m and initial effective stress = 80 kPa.	NEEDS_INPUT	consolidation_settlement_nc needs ds (added vertical stress at mid-depth). Example: solve('consolidation_settlement_nc',
Consolidation settlement of a normally consolidated clay with compression index = 0.32, initial void ratio = 0.92, thickness = 4 m, initial effective stress = 80 kPa and stress increase = 60 kPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Primary consolidation settlement of an overconsolidated clay, `consolidation_settlement_oc`

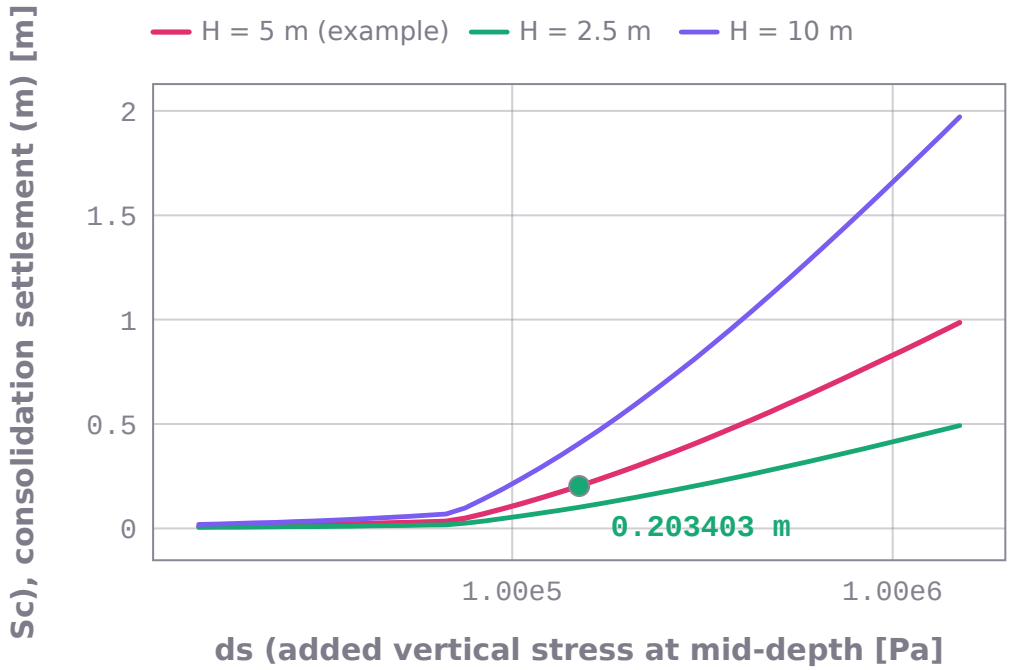
Primary consolidation settlement of an overconsolidated clay: recompression with the swelling index Cs up to the preconsolidation stress, then virgin compression with Cc.

$$S_c = \frac{H}{1+e_0}\left[C_s \log\frac{\sigma'_p}{\sigma'_0} + C_c \log\frac{\sigma'_0+\Delta\sigma}{\sigma'_p}\right]$$

input	meaning	condition
`Cc`	compression index	
`Cs`	swelling (recompression) index	
`e0`	initial void ratio	
`H`	clay layer thickness	
`s0`	initial effective vertical stress at mid-depth	
`ds`	added vertical stress at mid-depth	
`sp`	preconsolidation stress	

output	meaning
`Sc`	consolidation settlement (m)
`de`	change in void ratio
`OCR`	overconsolidation ratio sp/s_0
`case`	whether the load passes the preconsolidation stress

- **Method:** $sigma1 \le sp$: $Sc = Cs\ H/(1+e_0)$ log(s1/s0); otherwise $Sc = H/(1+e_0)$[Cs log(sp/s0) + Cc log(s1/sp)]
- **Assumes:** $Cs < Cc$; $sp \ge s_0$; stresses at mid-depth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sc (consolidation settlement (m)) against ds (added vertical stress at mid-depth) from 15000 Pa to 1.5e+06 Pa on a logarithmic axis. One curve for each value of H (clay layer thickness): 5 m, 2.5 m, 10 m. Other inputs as in the worked example: Cc = 0.4, Cs = 0.06, e0 = 1.1, s0 = 90000.0 Pa, sp = 160000.0 Pa. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, ds = 150000 Pa, Sc = 0.203403 m.

```
civil_engineering_ai999x.solve('consolidation_settlement_oc', Cc=0.4, Cs=0.06, e0=1.1,
H=5.0, s0=90000.0, ds=150000.0, sp=160000.0)
# -> Sc = 0.203403 m, de = 0.0854292, OCR = 1.77778, case = passes the preconsolidation
stress   (SOLVED)
```

```
civil_engineering_ai999x.solve('consolidation_settlement_oc', Cc=0.35, Cs=0.05, e0=0.95,
H=3.0, s0=70000.0, ds=40000.0, sp=130000.0)
# -> Sc = 0.0150996 m, de = 0.00981473, OCR = 1.85714, case = stays overconsolidated
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Consolidation settlement of an overconsolidated clay with compression index = 0.4, swelling index = 0.06, initial void ratio = 1.1, thickness = 5 m, initial effective stress = 90 kPa, stress increase = 150 kPa and preconsolidation stress = 160 kPa.	SOLVED	Sc = 0.203403 m, de = 0.0854292, OCR = 1.77778, case = passes the preconsolidation stress
Consolidation settlement of an overconsolidated clay with compression index = 0.35, swelling index = 0.05, initial void ratio = 0.95, thickness = 3 m, initial effective stress = 70 kPa, stress increase = 40 kPa and preconsolidation stress = 130 kPa.	SOLVED	Sc = 0.0150996 m, de = 0.00981473, OCR = 1.85714, case = stays overconsolidated

question	outcome	result
Consolidation settlement of an overconsolidated clay with compression index = 0.4, swelling index = 0.06, initial void ratio = 1.1, thickness = 5 m, initial effective stress = 90 kPa and stress increase = 150 kPa.	NEEDS_INPUT	consolidation_settlement_oc needs sp (preconsolidation stress). Example: solve('consolidation_settlement_oc', Cc=0.4, Cs
Consolidation settlement of an overconsolidated clay with compression index = 0.4, swelling index = 0.06, initial void ratio = 1.1, thickness = 5 m, initial effective stress = 90 kPa, stress increase = 150 kPa and preconsolidation stress = 160 kPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Consolidation settlement of an overconsolidated clay with compression index = 0.4, swelling index = 0.06, initial void ratio = 1.1, thickness = 5 m, initial effective stress = 200 kPa, stress increase = 150 kPa and preconsolidation stress = 160 kPa.	INVALID_INPUT	Primary consolidation settlement of an overconsolidated clay: the preconsolidation stress cannot be below the present ef

Consolidation settlement of a thick clay layer integrated over depth,

`thick_clay_layer_settlement`

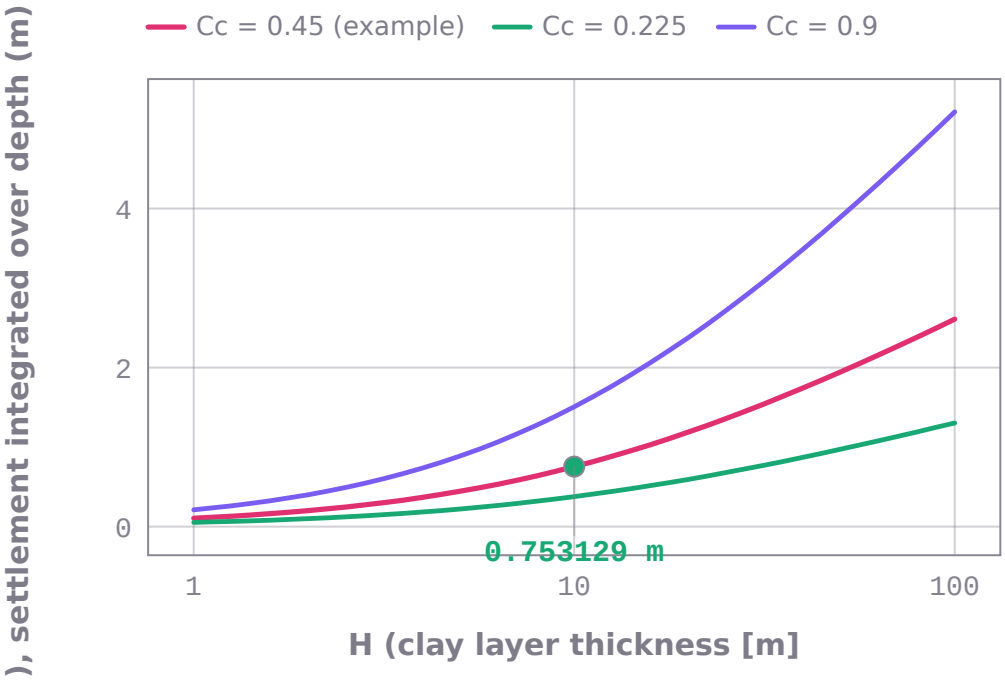
Primary consolidation settlement of a thick normally consolidated clay layer under a uniform added stress, with the initial effective stress rising linearly with depth (integrated exactly over the thickness), and the usual single-layer estimate at mid-depth for comparison.

$$S = \frac{C_c}{1+e_0}\int_0^H \log_{10}\frac{\sigma_0'(z)+\Delta\sigma}{\sigma_0'(z)}\,dz$$

input	meaning	condition
`H`	clay layer thickness	
`s_top`	initial effective vertical stress at the top of the clay	
`gsub`	submerged unit weight of the clay	
`ds`	uniform added vertical stress	
`Cc`	compression index	
`e0`	initial void ratio	

output	meaning
`S`	settlement integrated over depth (m)
`S_one_layer`	single-layer estimate at mid-depth (m)

- **Method:** $S = Cc/(1+e_0)$ integral_0^H log10((s0'(z) + ds)/s0'(z)) dz in closed form
- **Assumes:** normally consolidated clay; uniform added stress with depth; s0' = s_top + gsub z
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (settlement integrated over depth (m)) against H (clay layer thickness) from 1 m to 100 m on a logarithmic axis. One curve for each value of Cc (compression index): 0.45, 0.225, 0.9. Other inputs as in the worked example: s_top = 40000.0 Pa, gsub = 8000.0 N/m^3, ds = 100000.0 Pa, e0 = 1.2. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, H = 10 m, S = 0.753129 m.

```
civil_engineering_ai999x.solve('thick_clay_layer_settlement', H=10.0, s_top=40000.0,
gsub=8000.0, ds=100000.0, Cc=0.45, e0=1.2)
# -> S = 0.753129 m, S_one_layer = 0.720373 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('thick_clay_layer_settlement', H=16.0, s_top=25000.0,
gsub=7000.0, ds=60000.0, Cc=0.6, e0=1.6)
# -> S = 0.996495 m, S_one_layer = 0.888864 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thick clay layer settlement with thickness = 10 m, top effective stress = 40 kPa, submerged unit weight = 8 kN/m^3, stress increase = 100 kPa, compression index = 0.45 and initial void ratio = 1.2.	SOLVED	S = 0.753129 m, S_one_layer = 0.720373 m
Thick clay layer settlement with thickness = 16 m, top effective stress = 25 kPa, submerged unit weight = 7 kN/m^3, stress increase = 60 kPa, compression index = 0.6 and initial void ratio = 1.6.	SOLVED	S = 0.996495 m, S_one_layer = 0.888864 m
Thick clay layer settlement with thickness = 10 m, top effective stress = 40 kPa, submerged unit weight = 8 kN/m^3, stress increase = 100 kPa and compression index = 0.45.	NEEDS_INPUT	thick_clay_layer_settlement needs e0 (initial void ratio). Example: solve('thick_clay_layer_settlement', H='10 m', s_top
Thick clay layer settlement with thickness = 10 m, top effective stress = 40 kPa, submerged unit weight = 8 kN/m^3, stress	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
increase = 100 kPa, compression index = 0.45 and initial void ratio = 1.2 and colour = 3.		

Average degree of consolidation after a time (Terzaghi), `degree_of_consolidation`

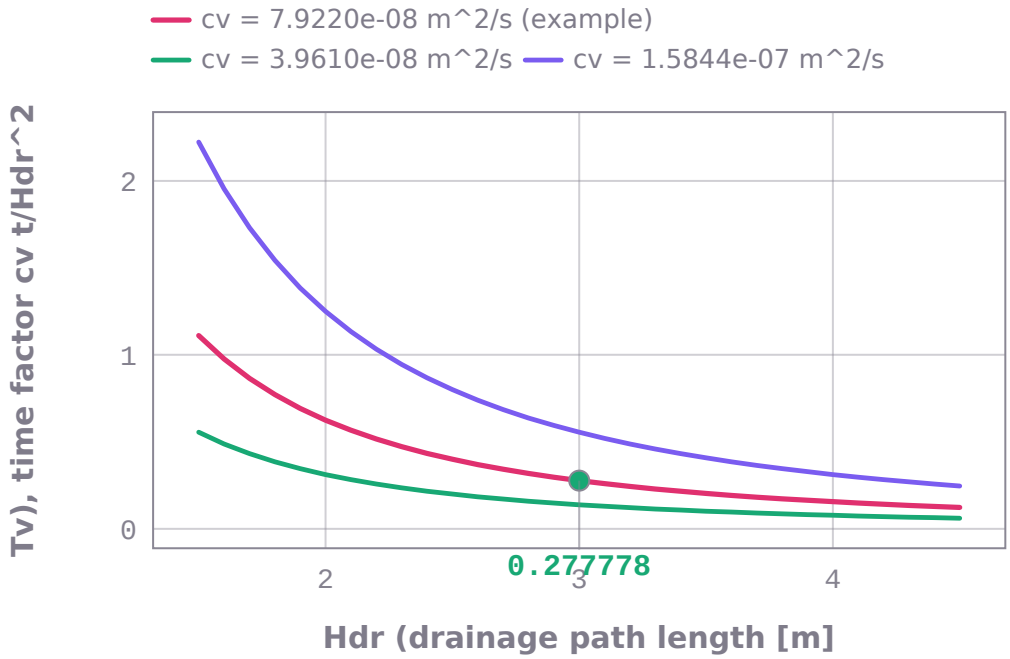
Time factor and average degree of one-dimensional consolidation of a clay layer after time t (Terzaghi, uniform initial excess pore pressure), and the settlement reached if the final consolidation settlement is given. Hdr is the drainage path: half the thickness for double drainage, the full thickness for single drainage.

$$U = 1 - \sum_{m=0}^{\infty} \frac{2}{M^2} e^{-M^2 T_v}, \quad M = \frac{\pi}{2}(2m+1), \quad T_v = \frac{c_v t}{H_{dr}^2}$$

input	meaning	condition
`cv`	coefficient of consolidation	
`t`	time since loading	
`Hdr`	drainage path length	
`Sc`	final consolidation settlement (optional)	

output	meaning
`Tv`	time factor cv t/Hdr^2
`U`	average degree of consolidation
`U_percent`	average degree of consolidation (%)
`St`	settlement at time t (m)

- **Method:** $Tv = cv$ t/Hdr^2; U = 1 - sum $2/M^2$ exp($-M^2$ Tv), $M = pi$(2m + 1)/2
- **Assumes:** $1e-4 \le Tv \le 5$; uniform initial excess pore pressure; Terzaghi $1-D$ theory
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tv (time factor cv t/Hdr^2) against Hdr (drainage path length) from 1.5 m to 4.5 m. One curve for each value of cv (coefficient of consolidation): 7.9220e-08 m^2/s, 3.9610e-08 m^2/s, 1.5844e-07 m^2/s. Other inputs as in the worked example: t = 31557600.0 s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, Hdr = 3 m, Tv = 0.277778.

```
civil_engineering_ai999x.solve('degree_of_consolidation', cv=7.922021953507237e-08,
t=31557600.0, Hdr=3.0)
# -> Tv = 0.277778, U = 0.59137, U_percent = 59.137   (SOLVED)
```

```
civil_engineering_ai999x.solve('degree_of_consolidation', cv=1.2675235125611158e-07,
t=15778800.0, Hdr=2.5, Sc=0.18)
# -> Tv = 0.32, U = 0.631895, U_percent = 63.1895, St = 0.113741 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Degree of consolidation with coefficient of consolidation = 2.5 m^2/yr, time = 1 yr and drainage path = 3 m.	SOLVED	Tv = 0.277778, U = 0.59137, U_percent = 59.137
Degree of consolidation with coefficient of consolidation = 4 m^2/yr, time = 0.5 yr, drainage path = 2.5 m and final settlement = 180 mm.	SOLVED	Tv = 0.32, U = 0.631895, U_percent = 63.1895, St = 0.113741 m
Degree of consolidation with coefficient of consolidation = 2.5 m^2/yr and time = 1 yr.	NEEDS_INPUT	degree_of_consolidation needs Hdr (drainage path length). Example: solve('degree_of_consolidation', cv='7.92202e-08 m^2/
Degree of consolidation with coefficient of consolidation = 2.5 m^2/yr, time = 1 yr and drainage path = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Time to reach a degree of consolidation, `consolidation_time`

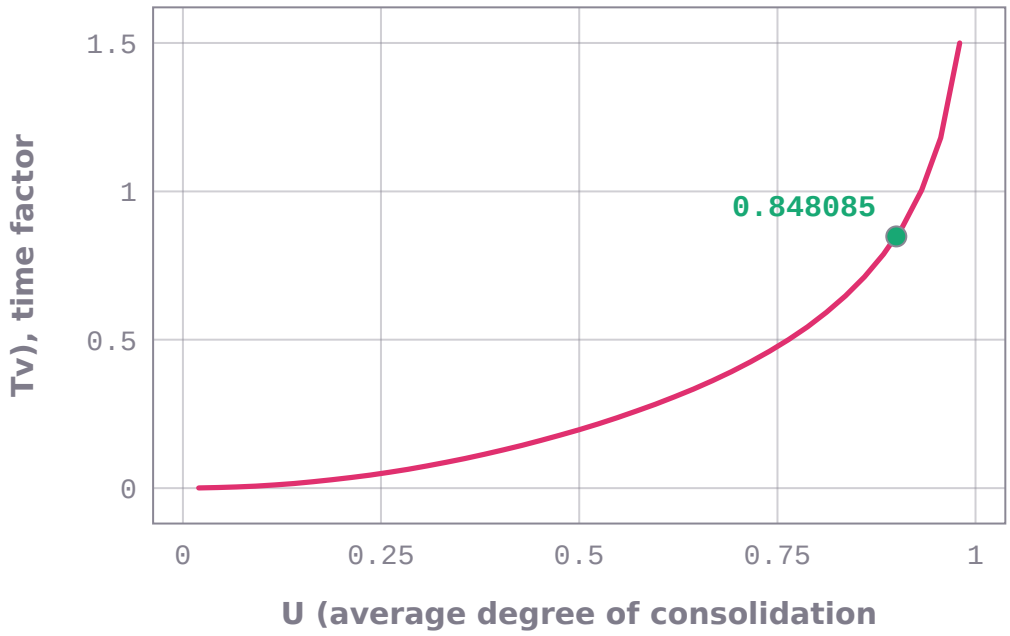
Time needed for a clay layer to reach an average degree of consolidation U (Terzaghi), with the time factor.

$$t = \frac{T_v(U)\, H_{dr}^2}{c_v}$$

input	meaning	condition
`cv`	coefficient of consolidation	
`Hdr`	drainage path length	
`U`	average degree of consolidation (fraction, or % in a question)	

output	meaning
`Tv`	time factor
`t`	time (s)
`t_days`	time (days)

- **Method:** Tv from U(Tv) = U (bisection on the Fourier series), $t = Tv$ Hdr^2/cv
- **Assumes:** 1% <= $U \leq 99$%; Terzaghi $1 - D$ theory
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tv (time factor) against U (average degree of consolidation (fraction, or % in a question)) from 0.02 to 0.98. Other inputs as in the worked example: cv = 9.506426344208684e-08 m^2/s, Hdr = 2.5 m. 41 points computed by civil-engineering-ai999x; marked: the worked example, U = 0.9, Tv = 0.848085.

```
civil_engineering_ai999x.solve('consolidation_time', cv=9.506426344208684e-08, Hdr=2.5,
U=0.9)
# -> Tv = 0.848085, t = 5.57574e+07 s, t_days = 645.34   (SOLVED)
```

```
civil_engineering_ai999x.solve('consolidation_time', cv=4.753213172104342e-08, Hdr=4.0,
U=0.5)
# -> Tv = 0.196731, t = 6.62224e+07 s, t_days = 766.463   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Consolidation time with coefficient of consolidation = 3 m^2/yr, drainage path = 2.5 m and degree of consolidation = 90 %.	SOLVED	Tv = 0.848085, t = 5.57574e+07 s, t_days = 645.34
Consolidation time with coefficient of consolidation = 1.5 m^2/yr, drainage path = 4 m and degree of consolidation = 50 %.	SOLVED	Tv = 0.196731, t = 6.62224e+07 s, t_days = 766.463
Consolidation time with coefficient of consolidation = 3 m^2/yr and drainage path = 2.5 m.	NEEDS_INPUT	consolidation_time needs U (average degree of consolidation (fraction, or % in a question)). Example: solve('consolidati
Consolidation time with coefficient of consolidation = 3 m^2/yr, drainage path = 2.5 m and degree of consolidation = 90 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Consolidation time with coefficient of consolidation = 3 m^2/yr, drainage path = 2.5 m and degree of consolidation = 100 %.	INVALID_INPUT	Time to reach a degree of consolidation: the degree of consolidation must lie between 1% and 99%.

Coefficient of consolidation from a consolidation time, `coefficient_of_consolidation`

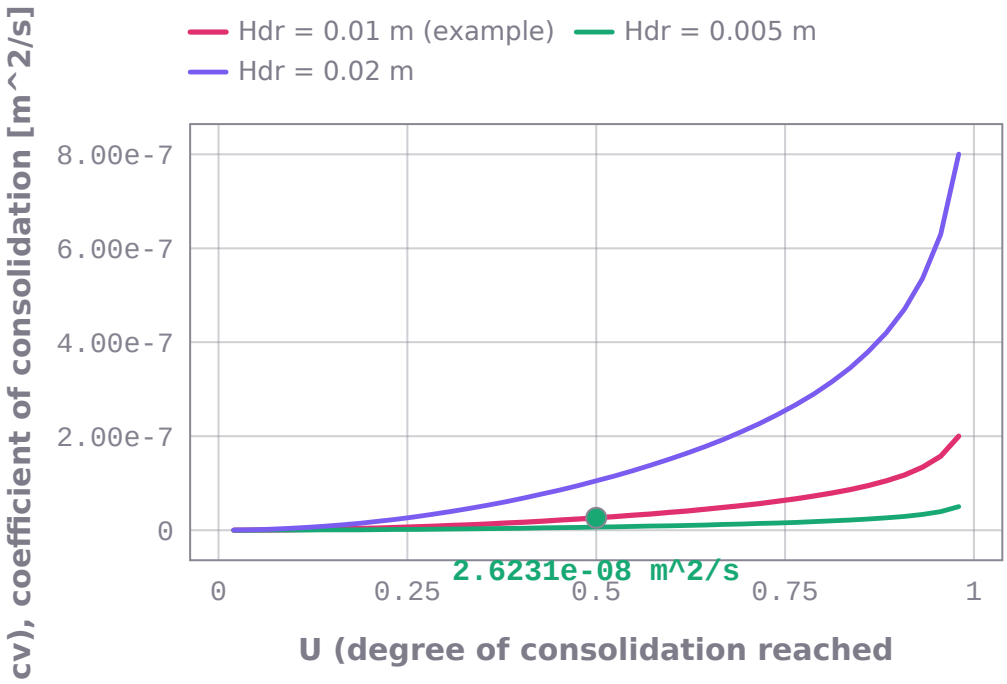
Coefficient of consolidation from the time at which a degree of consolidation is reached (for example t50 by Casagrande's method or t90 by Taylor's), using the exact time factor rather than the rounded 0.197 or 0.848.

$$c_v = \frac{T_v(U)\, H_{dr}^2}{t}$$

input	meaning	condition
`Hdr`	drainage path length (half the specimen height for double drainage)	
`t`	time to reach U	
`U`	degree of consolidation reached (fraction, or % in a question)	

output	meaning
`cv`	coefficient of consolidation (m^2/s)
`Tv`	time factor for U
`cv_m2_yr`	coefficient of consolidation (m^2/yr)

- **Method:** Tv(U) exactly (bisection), $cv = Tv$ Hdr^2/t
- **Assumes:** 1% <= $U \leq 99$%; Terzaghi $1-D$ theory
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cv (coefficient of consolidation (m^2/s)) against U (degree of consolidation reached (fraction, or % in a question)) from 0.02 to 0.98. One curve for each value of Hdr (drainage path length (half the specimen height for double drainage)): 0.01 m, 0.005 m, 0.02 m. Other inputs as in the worked example: t = 750.0 s. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, U = 0.5, cv = 2.6231e-08 m^2/s.

```
civil_engineering_ai999x.solve('coefficient_of_consolidation', Hdr=0.01, t=750.0, U=0.5)
# -> cv = 2.62308e-08 m^2/s, Tv = 0.196731, cv_m2_yr = 0.82778   (SOLVED)
```

```
civil_engineering_ai999x.solve('coefficient_of_consolidation', Hdr=0.0095, t=2880.0, U=0.9)
# -> cv = 2.65763e-08 m^2/s, Tv = 0.848085, cv_m2_yr = 0.838684   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coefficient of consolidation with drainage path = 10 mm, time = 12.5 min and degree of consolidation = 50 %.	SOLVED	cv = 2.62308e-08 m^2/s, Tv = 0.196731, cv_m2_yr = 0.82778
Coefficient of consolidation with drainage path = 9.5 mm, time = 48 min and degree of consolidation = 90 %.	SOLVED	cv = 2.65763e-08 m^2/s, Tv = 0.848085, cv_m2_yr = 0.838684
Coefficient of consolidation with drainage path = 10 mm and time = 12.5 min.	NEEDS_INPUT	coefficient_of_consolidation needs U (degree of consolidation reached (fraction, or % in a question)). Example: solve('c
Coefficient of consolidation with drainage path = 10 mm, time = 12.5 min and degree of consolidation = 50 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Excess pore pressure at a depth during consolidation, `excess_pore_pressure_isochrone`

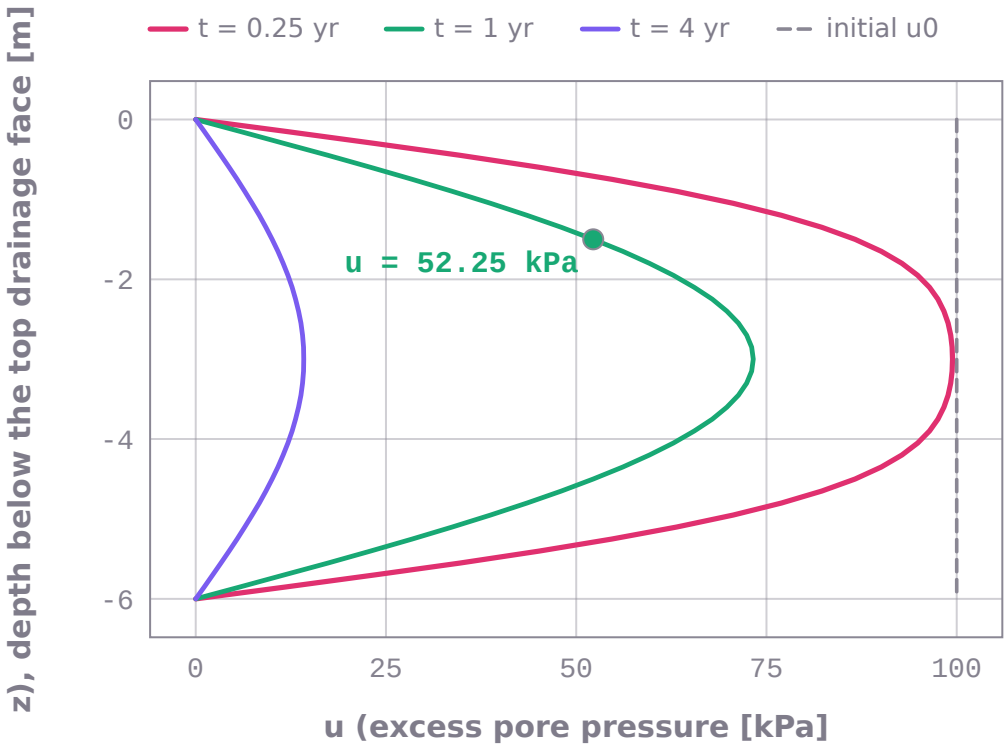
Excess pore-water pressure and local degree of consolidation at depth z and time t in a doubly drained clay layer of thickness 2 Hdr under a uniform initial excess pressure u0 (Terzaghi); z is measured from the top drainage face (for a singly drained layer of thickness Hdr, use 0 <= z <= Hdr).

$$u(z,t) = \sum_{m=0}^{\infty} \frac{2u_0}{M} \sin \frac{Mz}{H_{dr}}\, e^{-M^2 T_v}$$

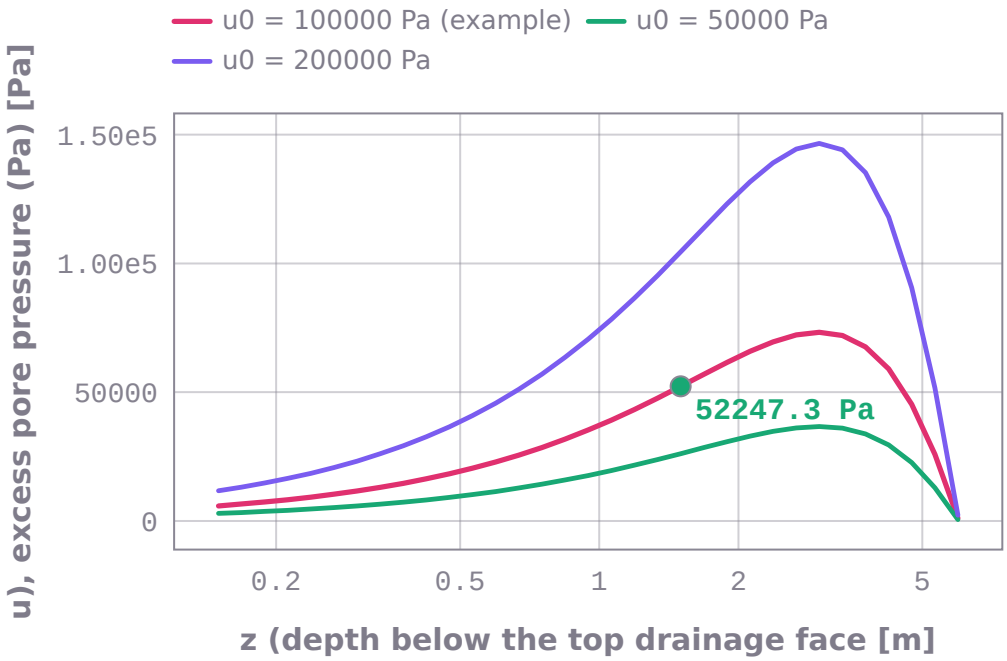
input	meaning	condition
`u0`	initial excess pore pressure	
`cv`	coefficient of consolidation	
`t`	time since loading	
`Hdr`	drainage path length	
`z`	depth below the top drainage face	

output	meaning
`u`	excess pore pressure (Pa)
`Uz`	local degree of consolidation $1 - u/u_0$
`Tv`	time factor

- **Method:** u = sum $2u0/M$ sin(M z/Hdr) exp($-M^2$ Tv), $M = pi$(2m + 1)/2
- **Assumes:** $0.001 \le Tv \le 3$; uniform initial excess pore pressure; double drainage (or symmetric half)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Isochrones of excess pore pressure in a doubly drained clay layer of thickness 6 m (cv = 6.3376e-08 m²/s, u0 = 100 kPa) at a quarter of, at, and at four times the worked example's time t = 3.1558e+07 s; 41 depths each, computed by the package. Marked: at z = 1.5 m the returned u = 52.2473 kPa, local consolidation 0.477527, time factor Tv = 0.222222.



u (excess pore pressure (Pa)) against z (depth below the top drainage face) from 0.15 m to 15 m on a logarithmic axis. One curve for each value of u0 (initial excess pore pressure): 100000 Pa, 50000 Pa, 200000 Pa. Other inputs as in the worked example: cv = 6.33761756280579e-08 m^2/s, t = 31557600.0 s, Hdr = 3.0 m. 33 points per curve computed by civil-engineering-ai999x; marked: the worked example, z = 1.5 m, u = 52247.3 Pa.

```
civil_engineering_ai999x.solve('excess_pore_pressure_isochrone', u0=100000.0,
cv=6.33761756280579e-08, t=31557600.0, Hdr=3.0, z=1.5)
# -> u = 52247.3 Pa, Uz = 0.477527, Tv = 0.222222   (SOLVED)
```

```
civil_engineering_ai999x.solve('excess_pore_pressure_isochrone', u0=80000.0,
cv=1.5844043907014474e-07, t=7889400.0, Hdr=2.0, z=2.0)
# -> u = 47079.1 Pa, Uz = 0.411511, Tv = 0.3125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Excess pore pressure during consolidation with initial excess pore pressure = 100 kPa, coefficient of consolidation = 2 m^2/yr, time = 1 yr, drainage path = 3 m and depth = 1.5 m.	SOLVED	u = 52247.3 Pa, Uz = 0.477527, Tv = 0.222222
Excess pore pressure during consolidation with initial excess pore pressure = 80 kPa, coefficient of consolidation = 5 m^2/yr, time = 0.25 yr, drainage path = 2 m and depth = 2 m.	SOLVED	u = 47079.1 Pa, Uz = 0.411511, Tv = 0.3125
Excess pore pressure during consolidation with initial excess pore pressure = 100 kPa, coefficient of consolidation = 2 m^2/yr, time = 1 yr and drainage path = 3 m.	NEEDS_INPUT	excess_pore_pressure_isochrone needs z (depth below the top drainage face). Example: solve('excess_pore_pressure_isochro
Excess pore pressure during consolidation with initial excess pore pressure = 100 kPa, coefficient of consolidation = 2 m^2/yr, time = 1 yr, drainage path = 3 m and depth = 1.5 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Vertical stress below the centre of a uniformly loaded circle, `stress_below_circular_load`

Increase in vertical stress at depth z below the centre of a flexible circular area of radius a carrying a uniform pressure q (Boussinesq, elastic half-space).

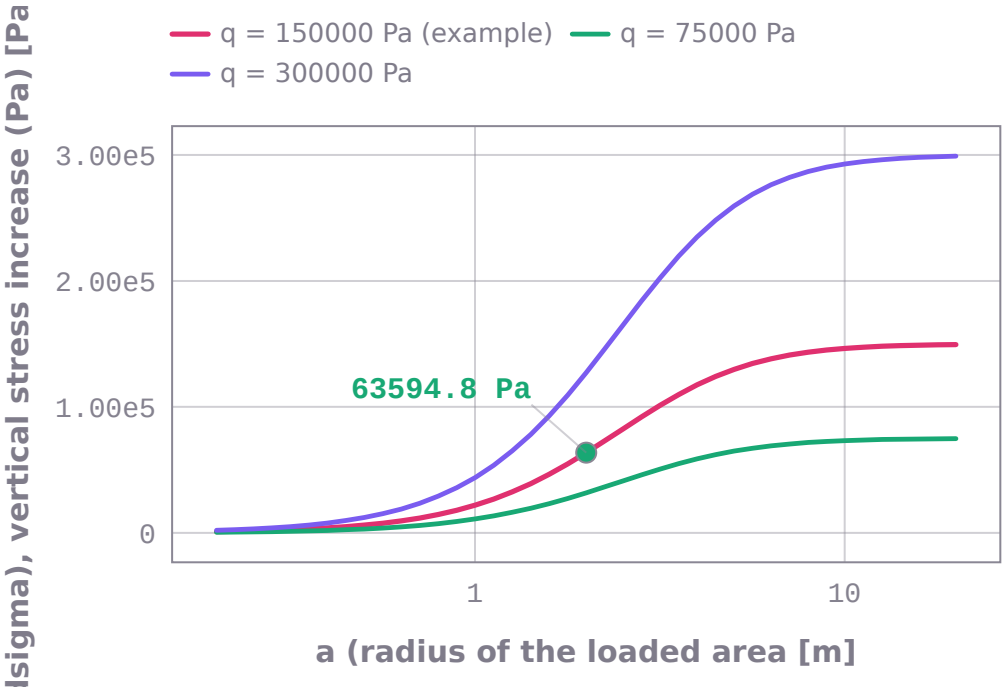
$$\Delta\sigma_z = q\left[1 - \left(\frac{1}{1+(a/z)^2}\right)^{3/2}\right]$$

input	meaning	condition
`q`	uniform applied pressure	
`a`	radius of the loaded area	
`z`	depth below the surface	

output	meaning
`dsigma`	vertical stress increase (Pa)
`I`	influence factor dsigma/q

- **Method:** dsigma = q [1 - (1/(1 + (a/z)^2))^$(3/2)$]
- **Assumes:** elastic, homogeneous, isotropic half-space; flexible uniform load; below the centre

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dsigma (vertical stress increase (Pa)) against a (radius of the loaded area) from 0.2 m to 20 m on a logarithmic axis. One curve for each value of q (uniform applied pressure): 150000 Pa, 75000 Pa, 300000 Pa. Other inputs as in the worked example: z = 3.0 m. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, a = 2 m, dsigma = 63594.8 Pa.

```
civil_engineering_ai999x.solve('stress_below_circular_load', q=150000.0, a=2.0, z=3.0)
# -> dsigma = 63594.8 Pa, I = 0.423965   (SOLVED)
```

```
civil_engineering_ai999x.solve('stress_below_circular_load', q=100000.0, a=5.0, z=10.0)
# -> dsigma = 28445.8 Pa, I = 0.284458   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stress below a circular load with pressure = 150 kPa, radius = 2 m and depth = 3 m.	SOLVED	dsigma = 63594.8 Pa, I = 0.423965
Stress below a circular load with pressure = 100 kPa, radius = 5 m and depth = 10 m.	SOLVED	dsigma = 28445.8 Pa, I = 0.284458
Stress below a circular load with pressure = 150 kPa and radius = 2 m.	NEEDS_INPUT	stress_below_circular_load needs z (depth below the surface). Example: solve('stress_below_circular_load', q='150000 Pa'
Stress below a circular load with pressure = 150 kPa, radius = 2 m and depth = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Vertical stress below a uniformly loaded rectangle (Newmark), `stress_below_rectangular_load`

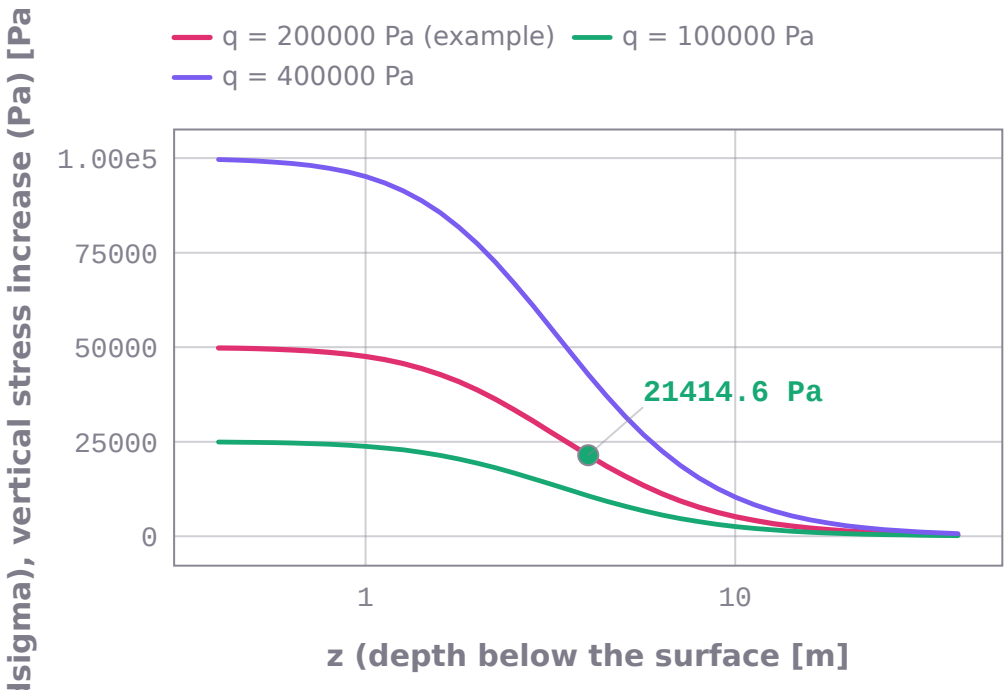
Increase in vertical stress at depth z below the corner (or the centre, by superposition of four corners) of a flexible rectangular area B x L under a uniform pressure q (Boussinesq integrated by Newmark).

$$\Delta\sigma_z = q\,I(m,n), \quad m = \frac{B}{z},\ n = \frac{L}{z}$$

input	meaning	condition
`q`	uniform applied pressure	
`B`	width of the loaded area	
`L`	length of the loaded area	
`z`	depth below the surface	
`position`	point below the 'corner' (default) or the 'centre'	

output	meaning
`dsigma`	vertical stress increase (Pa)
`I`	influence factor dsigma/q

- **Method:** I = (1/4 pi)[2mn sqrt(s)/($s + m^2$ n^2) (s + 1)/s + atan2(2mn sqrt(s), $s - m^2$ n^2)], $s = m^2 + n^2 + 1$, $m = B/z$, $n = L/z$
- **Assumes:** elastic, homogeneous, isotropic half-space; flexible uniform load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dsigma (vertical stress increase (Pa)) against z (depth below the surface) from 0.4 m to 40 m on a logarithmic axis. One curve for each value of q (uniform applied pressure): 200000 Pa, 100000 Pa, 400000 Pa. Other inputs as in the worked example: B = 2.0 m, L = 3.0 m. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, z = 4 m, dsigma = 21414.6 Pa.

```
civil_engineering_ai999x.solve('stress_below_rectangular_load', q=200000.0, B=2.0, L=3.0,
z=4.0)
# -> dsigma = 21414.6 Pa, I = 0.107073   (SOLVED)
```

```
civil_engineering_ai999x.solve('stress_below_rectangular_load', q=120000.0, B=4.0, L=6.0,
z=3.0, position='centre')
# -> dsigma = 69630.3 Pa, I = 0.580253   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stress below a rectangular load with pressure = 200 kPa, width = 2 m, length = 3 m and depth = 4 m.	SOLVED	dsigma = 21414.6 Pa, I = 0.107073
Stress below a rectangular load with pressure = 120 kPa, width = 4 m, length = 6 m, depth = 3 m and position = centre.	SOLVED	dsigma = 69630.3 Pa, I = 0.580253
Stress below a rectangular load with pressure = 200 kPa, width = 2 m and length = 3 m.	NEEDS_INPUT	stress_below_rectangular_load needs z (depth below the surface). Example: solve('stress_below_rectangular_load', q='2000
Stress below a rectangular load with pressure = 200 kPa, width = 2 m, length = 3 m and depth = 4 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Vertical stress below a uniformly loaded strip, `stress_below_strip_load`

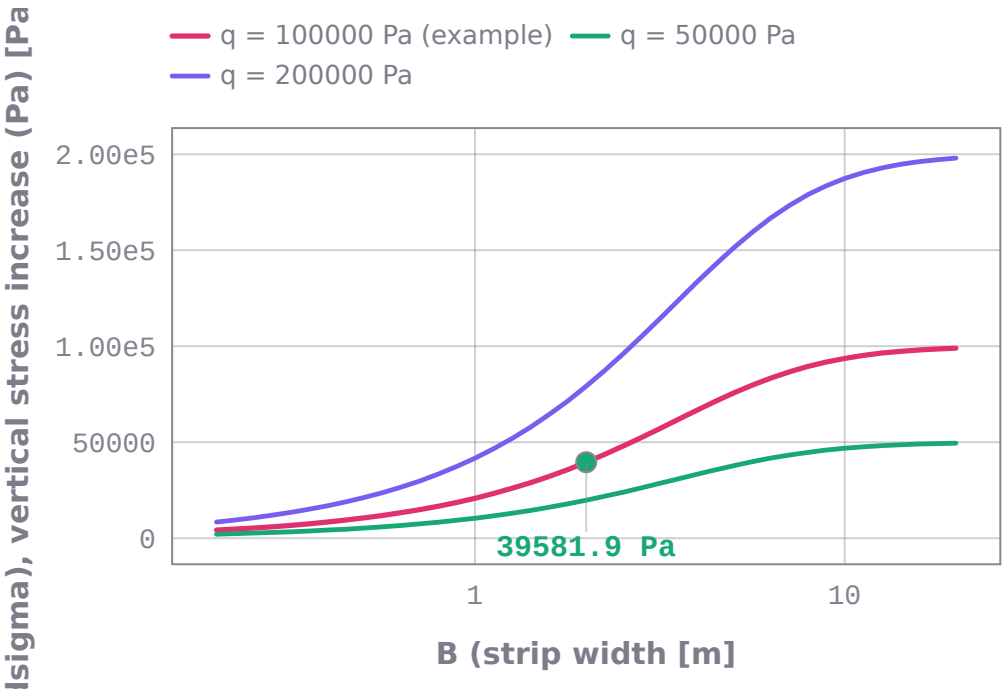
Increase in vertical stress at depth z and horizontal offset x from the centreline of an infinitely long strip of width B under a uniform pressure q (elastic half-space).

$$\Delta\sigma_z = \frac{q}{\pi}[\alpha + \sin\alpha\cos(\alpha + 2\delta)]$$

input	meaning	condition
`q`	uniform applied pressure	
`B`	strip width	
`z`	depth below the surface	
`x`	horizontal offset from the strip centreline (default 0)	

output	meaning
`dsigma`	vertical stress increase (Pa)
`I`	influence factor dsigma/q

- **Method:** dsigma = (q/pi)[atan(u/z) + u $z/(u^2+z^2)$] from $u = x - B/2$ to $x + B/2$
- **Assumes:** elastic, homogeneous, isotropic half-space; flexible uniform load, plane strain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dsigma (vertical stress increase (Pa)) against B (strip width) from 0.2 m to 20 m on a logarithmic axis. One curve for each value of q (uniform applied pressure): 100000 Pa, 50000 Pa, 200000 Pa. Other inputs as in the worked example: z = 3.0 m. 41 points per curve computed by civil-engineering-ai999x; marked: the worked example, B = 2 m, dsigma = 39581.9 Pa.

```
civil_engineering_ai999x.solve('stress_below_strip_load', q=100000.0, B=2.0, z=3.0)
# -> dsigma = 39581.9 Pa, I = 0.395819   (SOLVED)
```

```
civil_engineering_ai999x.solve('stress_below_strip_load', q=150000.0, B=3.0, z=2.0, x=2.5)
# -> dsigma = 30724.9 Pa, I = 0.204833   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stress below a strip load with pressure = 100 kPa, width = 2 m and depth = 3 m.	SOLVED	dsigma = 39581.9 Pa, I = 0.395819
Stress below a strip load with pressure = 150 kPa, width = 3 m, depth = 2 m and offset = 2.5 m.	SOLVED	dsigma = 30724.9 Pa, I = 0.204833
Stress below a strip load with pressure = 100 kPa and width = 2 m.	NEEDS_INPUT	stress_below_strip_load needs z (depth below the surface). Example: solve('stress_below_strip_load', q='100000 Pa', B='2
Stress below a strip load with pressure = 100 kPa, width = 2 m and depth = 3 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Ultimate bearing capacity of a strip footing, `strip_footing_bearing_capacity`

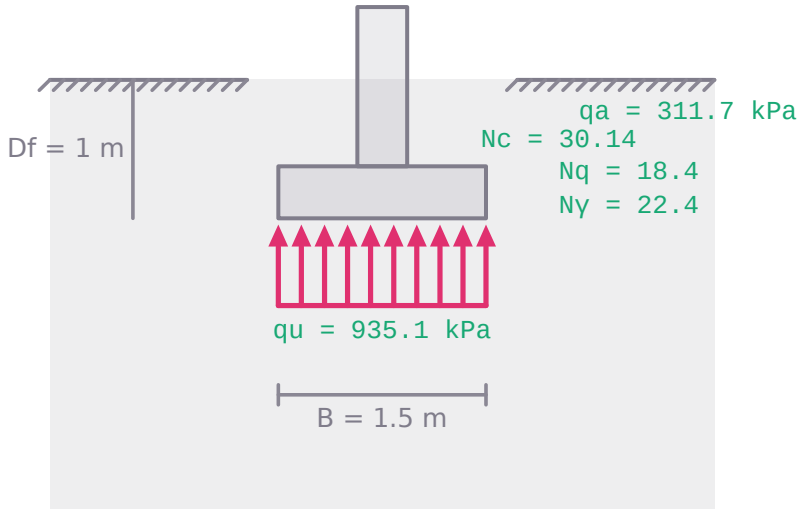
Ultimate and allowable bearing capacity of a strip footing from the general bearing capacity equation with Prandtl-Reissner factors Nc, Nq and Vesic's (1973) Ngamma (no shape, depth or inclination factors; drained parameters, water table well below the footing).

$$q_u = cN_c + \gamma D_f N_q + \tfrac{1}{2}\gamma B N_\gamma$$

input	meaning	condition
`c`	cohesion	
`phi`	friction angle (degrees)	
`gam`	unit weight of the soil	
`B`	footing width	
`Df`	embedment depth	
`FS`	factor of safety for the allowable pressure (default 3)	

output	meaning
`qu`	ultimate bearing capacity (Pa)
`qa`	allowable bearing pressure qu/FS (Pa)
`Nc`	bearing capacity factor Nc
`Nq`	bearing capacity factor Nq
`Ngamma`	bearing capacity factor Ngamma (Vesic)

- **Method:** Nq = e^(pi tan phi) tan^2(45 + phi/2), $Nc = (Nq - 1)$ cot phi, Ngamma = 2(Nq + 1) tan phi; $qu = c$ Nc + gam Df Nq + B gam Ngamma/2
- **Assumes:** $20 \leq \phi \leq 45$ degrees; general shear failure; strip footing, vertical central load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A strip footing B = 1.5 m wide founded at Df = 1.0 m in soil with c = 10 kPa, φ = 30.0°, γ = 18 kN/m³. Marked: the returned ultimate bearing capacity qu = 935.05 kPa, allowable qa = 311.683 kPa and factors Nc = 30.1396, Nq = 18.4011, Nγ = 22.4025. Footing width and depth to scale.



qu (ultimate bearing capacity (Pa)) against phi (friction angle (degrees)) from 15 to 45. One curve for each value of gam (unit weight of the soil): 18000 N/m^3, 9000 N/m^3, 36000 N/m^3. Other inputs as in the worked example: c = 10000.0 Pa, B = 1.5 m, Df = 1.0 m. 26 points per curve computed by civil-engineering-ai999x; marked: the worked example, phi = 30, qu = 935050 Pa.

```
civil_engineering_ai999x.solve('strip_footing_bearing_capacity', c=10000.0, phi=30.0,
gam=18000.0, B=1.5, Df=1.0)
# -> qu = 935050 Pa, qa = 311683 Pa, Nc = 30.1396, Nq = 18.4011, Ngamma = 22.4025
(SOLVED)
```

```
civil_engineering_ai999x.solve('strip_footing_bearing_capacity', c=0.0, phi=35.0,
gam=19000.0, B=2.0, Df=1.5, FS=2.5)
# -> qu = 1.86149e+06 Pa, qa = 744594 Pa, Nc = 46.1236, Nq = 33.2961, Ngamma = 48.0288
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Strip footing bearing capacity with cohesion = 10 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3, width = 1.5 m and depth = 1 m.	SOLVED	qu = 935050 Pa, qa = 311683 Pa, Nc = 30.1396, Nq = 18.4011, Ngamma = 22.4025
Strip footing bearing capacity with cohesion = 0 kPa, friction angle = 35 degrees, unit weight = 19 kN/m^3, width = 2 m, depth = 1.5 m and factor of safety = 2.5.	SOLVED	qu = 1.86149e+06 Pa, qa = 744594 Pa, Nc = 46.1236, Nq = 33.2961, Ngamma = 48.0288
Strip footing bearing capacity with cohesion = 10 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3 and width = 1.5 m.	NEEDS_INPUT	strip_footing_bearing_capacity needs Df (embedment depth). Example: solve('strip_footing_bearing_capacity', c='10000 Pa'
Strip footing bearing capacity with cohesion = 10 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
width = 1.5 m and depth = 1 m and colour = 3.		
Strip footing bearing capacity with cohesion = 10 kPa, friction angle = 60 degrees, unit weight = 18 kN/m^3, width = 1.5 m and depth = 1 m.	INVALID_INPUT	Ultimate bearing capacity of a strip footing: the friction angle must lie between 20 and 45 degrees.

Rankine active earth pressure and thrust on a wall, `rankine_active_thrust`

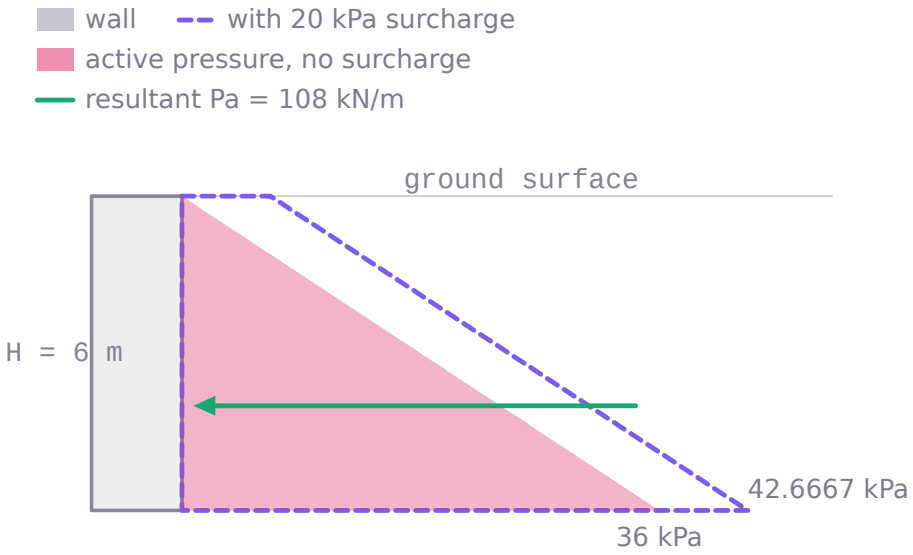
Rankine active pressure coefficient, active pressure at the base, total active thrust per metre of a smooth vertical wall retaining a horizontal backfill (with optional cohesion and surcharge; the tension zone is ignored), the tension-crack depth and the height of the resultant above the base. Dry backfill.

$$K_a = \frac{1-\sin\phi}{1+\sin\phi}, \quad \sigma_a = K_a(\gamma z + q) - 2c\sqrt{K_a}$$

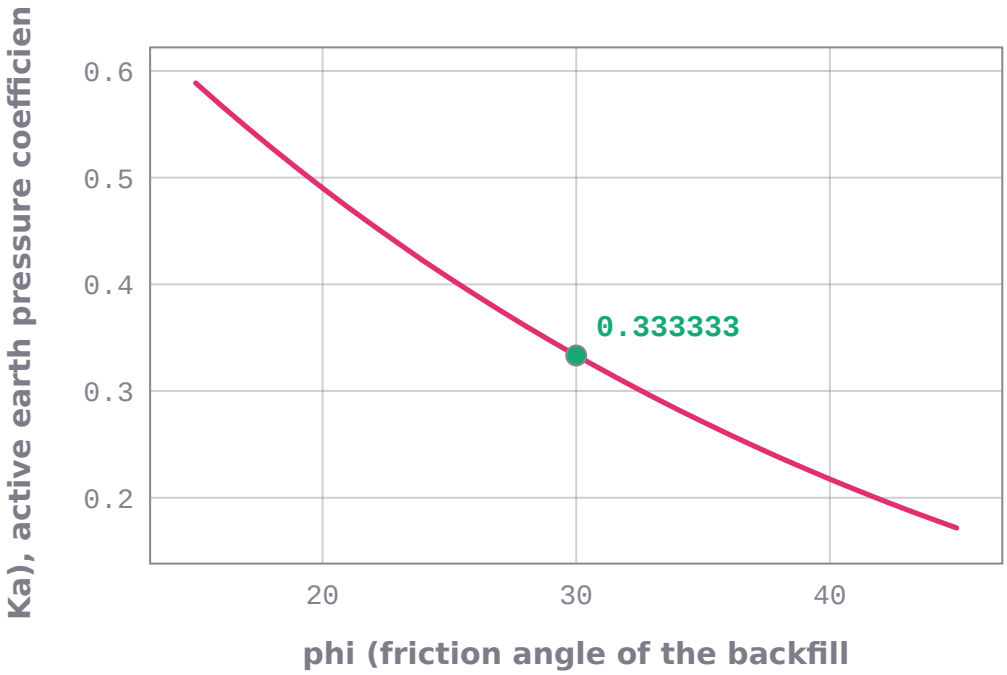
input	meaning	condition
`phi`	friction angle of the backfill (degrees)	
`gam`	unit weight of the backfill	
`H`	wall height	
`c`	cohesion (default 0)	
`q`	uniform surcharge (default 0)	

output	meaning
`Ka`	active earth pressure coefficient
`sigma_base`	active pressure at the base (Pa)
`Pa`	active thrust per metre of wall (N/m)
`z_crack`	tension-crack depth (m)
`ybar`	height of the resultant above the base (m)

- **Method:** Ka = (1 - sin ϕ)/(1 + sin phi); $sigma_a = Ka$(gam z + q) - 2c sqrt(Ka); Pa = area of the positive pressure diagram
- **Assumes:** smooth vertical wall, horizontal backfill; dry backfill; tension zone ignored
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rankine active earth pressure on a 6 m wall retaining soil with φ = 30° and γ = 18 kN/m³ (Ka = 0.333333), drawn as the pressure diagram against the wall: the pressure at each of 40 depths is the base pressure the package returns for a wall of that height. Without surcharge it grows to 36 kPa at the base, resultant Pa = 108 kN/m acting 2 m above the base (arrow); with a 20 kPa surcharge (dashed) the base pressure is 42.6667 kPa and Pa = 148 kN/m.



Ka (active earth pressure coefficient) against phi (friction angle of the backfill (degrees)) from 15 to 45. Other inputs as in the worked example: gam = 18000.0 N/m^3, H = 6.0 m. 31 points computed by civil-engineering-ai999x; marked: the worked example, phi = 30, Ka = 0.333333.

```
civil_engineering_ai999x.solve('rankine_active_thrust', phi=30.0, gam=18000.0, H=6.0)
# -> Ka = 0.333333, sigma_base = 36000 Pa, Pa = 108000 N/m, z_crack = 0 m, ybar = 2 m
(SOLVED)
```

```
civil_engineering_ai999x.solve('rankine_active_thrust', phi=25.0, gam=19000.0, H=5.0,
c=10000.0, q=15000.0)
# -> Ka = 0.405859, sigma_base = 31903 Pa, Pa = 65994.2 N/m, z_crack = 0.862827 m, ybar =
1.37906 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rankine active thrust with friction angle = 30 degrees, unit weight = 18 kN/m^3 and height = 6 m.	SOLVED	Ka = 0.333333, sigma_base = 36000 Pa, Pa = 108000 N/m, z_crack = 0 m, ybar = 2 m
Rankine active thrust with friction angle = 25 degrees, unit weight = 19 kN/m^3, height = 5 m, cohesion = 10 kPa and surcharge = 15 kPa.	SOLVED	Ka = 0.405859, sigma_base = 31903 Pa, Pa = 65994.2 N/m, z_crack = 0.862827 m, ybar = 1.37906 m
Rankine active thrust with friction angle = 30 degrees and unit weight = 18 kN/m^3.	NEEDS_INPUT	rankine_active_thrust needs H (wall height). Example: solve('rankine_active_thrust', phi=30, gam='18000 N/m^3', H='6 m')
Rankine active thrust with friction angle = 30 degrees, unit weight = 18 kN/m^3 and height = 6 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Rankine active thrust with friction angle = 70 degrees, unit weight = 18 kN/m^3 and height = 6 m.	INVALID_INPUT	Rankine active earth pressure and thrust on a wall: the friction angle must lie between 10 and 50 degrees.

Rankine passive earth pressure and thrust, `rankine_passive_thrust`

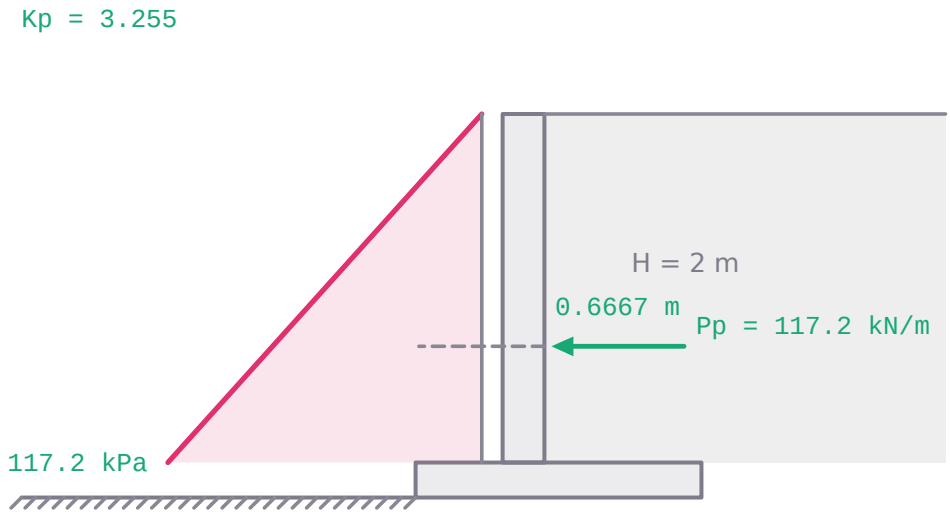
Rankine passive pressure coefficient, passive pressure at the base, total passive thrust per metre on a smooth vertical wall with a horizontal soil surface (optional cohesion and surcharge) and the height of the resultant.

$$K_p = \frac{1+\sin\phi}{1-\sin\phi}, \quad \sigma_p = K_p(\gamma z + q) + 2c\sqrt{K_p}$$

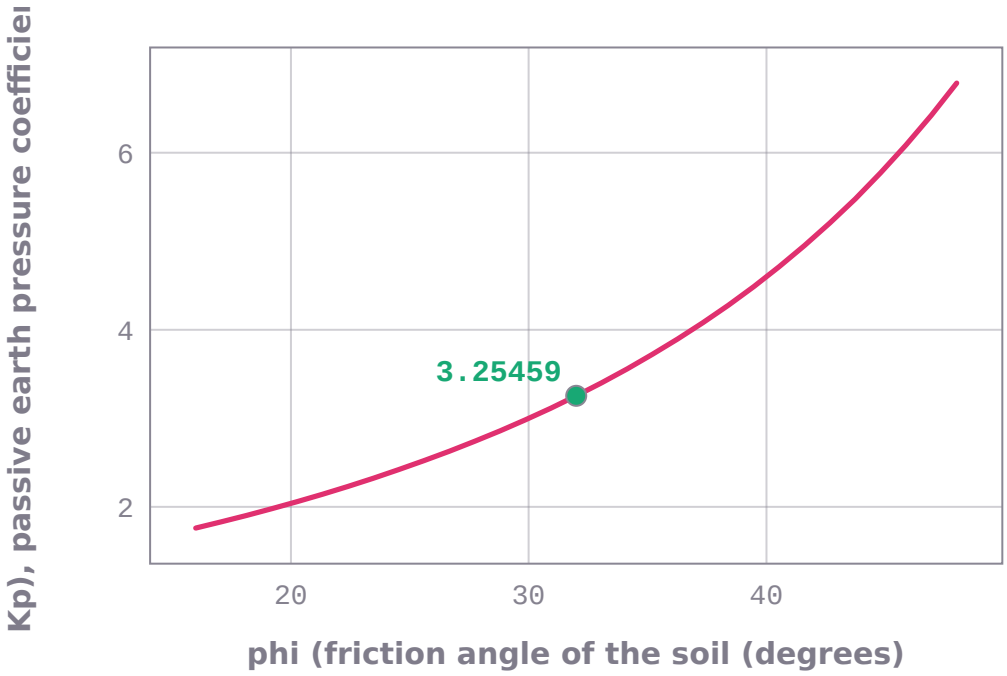
input	meaning	condition
`phi`	friction angle of the soil (degrees)	
`gam`	unit weight of the soil	
`H`	height of the wall in contact	
`c`	cohesion (default 0)	
`q`	uniform surcharge (default 0)	

output	meaning
`Kp`	passive earth pressure coefficient
`sigma_base`	passive pressure at the base (Pa)
`Pp`	passive thrust per metre of wall (N/m)
`ybar`	height of the resultant above the base (m)

- **Method:** Kp = (1 + sin ϕ)/(1 − sin phi); $sigma_p = Kp$(gam z + q) + 2c sqrt(Kp); Pp = area of the pressure diagram
- **Assumes:** smooth vertical wall, horizontal soil surface; dry soil
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A wall of height H = 2 m pushed into a level cohesionless soil (φ = 32.0°, γ = 18 kN/m³): the Rankine passive pressure diagram drawn from the returned base pressure 117.165 kPa, and the thrust Pp = 117.165 kN/m at the returned height 0.666667 m above the base. Marked: Kp = 3.25459.



Kp (passive earth pressure coefficient) against phi (friction angle of the soil (degrees)) from 16 to 48. Other inputs as in the worked example: gam = 18000.0 N/m^3, H = 2.0 m. 31 points computed by civil-engineering-ai999x; marked: the worked example, phi = 32, Kp = 3.25459.

```
civil_engineering_ai999x.solve('rankine_passive_thrust', phi=32.0, gam=18000.0, H=2.0)
# -> Kp = 3.25459, sigma_base = 117165 Pa, Pp = 117165 N/m, ybar = 0.666667 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('rankine_passive_thrust', phi=25.0, gam=19000.0, H=3.0,
c=12000.0)
# -> Kp = 2.46391, sigma_base = 178115 Pa, Pp = 323682 N/m, ybar = 1.17458 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rankine passive thrust with friction angle = 32 degrees, unit weight = 18 kN/m^3 and height = 2 m.	SOLVED	Kp = 3.25459, sigma_base = 117165 Pa, Pp = 117165 N/m, ybar = 0.666667 m
Rankine passive thrust with friction angle = 25 degrees, unit weight = 19 kN/m^3, height = 3 m and cohesion = 12 kPa.	SOLVED	Kp = 2.46391, sigma_base = 178115 Pa, Pp = 323682 N/m, ybar = 1.17458 m
Rankine passive thrust with friction angle = 32 degrees and unit weight = 18 kN/m^3.	NEEDS_INPUT	rankine_passive_thrust needs H (height of the wall in contact). Example: solve('rankine_passive_thrust', phi=32, gam='18
Rankine passive thrust with friction angle = 32 degrees, unit weight = 18 kN/m^3 and height = 2 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Rankine passive thrust with friction angle = 70 degrees, unit weight = 18 kN/m^3 and height = 2 m.	INVALID_INPUT	Rankine passive earth pressure and thrust: the friction angle must lie between 10 and 50 degrees.

Coulomb active thrust on a rough wall with sloping backfill, `coulomb_active_thrust`

Coulomb active earth pressure coefficient and thrust per metre on a rough vertical wall (wall friction delta) retaining a cohesionless backfill sloping at beta, with the horizontal and vertical components of the thrust.

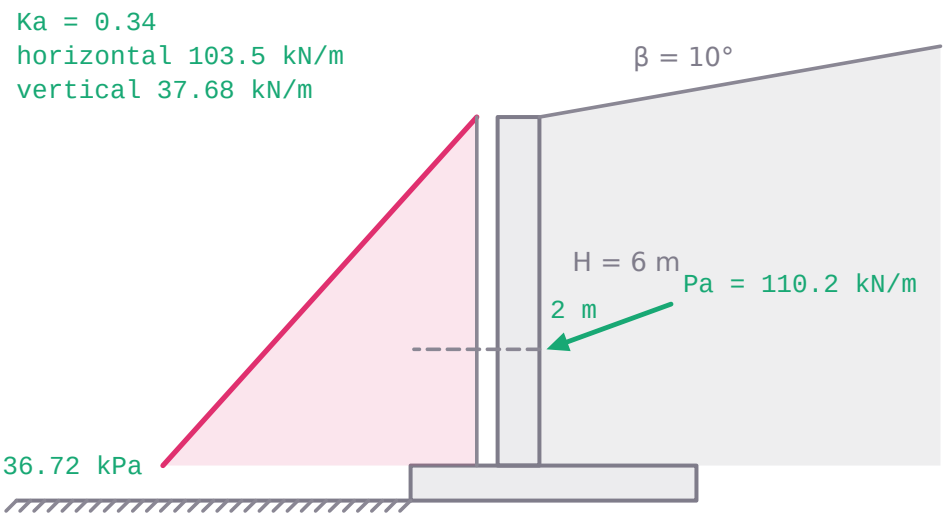
$$K_a = \frac{\cos^2\phi}{\cos\delta\left[1+\sqrt{\frac{\sin(\phi+\delta)\sin(\phi-\beta)}{\cos\delta\cos\beta}}\right]^2}$$

input	meaning	condition
`phi`	friction angle of the backfill (degrees)	
`delta`	wall friction angle (degrees)	
`beta`	backfill slope angle above horizontal (degrees)	
`gam`	unit weight of the backfill	
`H`	wall height	

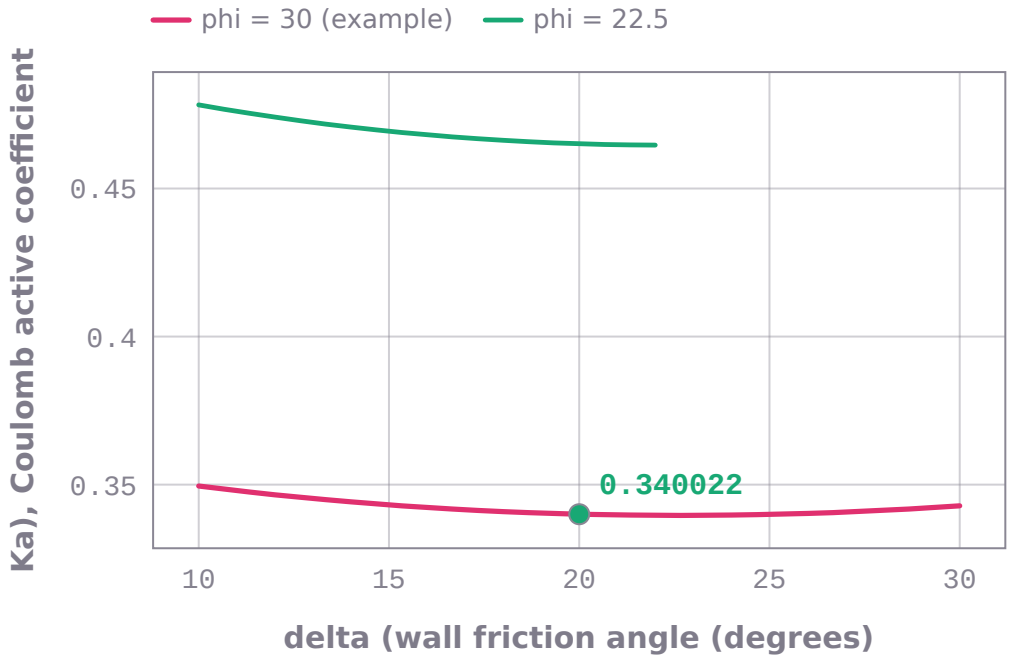
output	meaning
`Ka`	Coulomb active coefficient
`Pa`	active thrust per metre, inclined at delta (N/m)

output	meaning
`Pa_h`	horizontal component (N/m)
`Pa_v`	vertical component (N/m)

- **Method:** $Ka = cos^2$ phi/(cos delta [$1 + sqrt$(sin(phi + delta) sin(phi - beta)/(cos delta cos beta))]^2), $Pa = Ka$ gam $H^2/2$
- **Assumes:** vertical wall back; cohesionless dry backfill; $0 \le \delta \le \phi$; $0 \le \beta < \phi$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Coulomb active thrust on a rough vertical wall of height H = 6 m (φ = 30.0°, wall friction δ = 20.0°, backfill slope β = 10.0°, γ = 18 kN/m³). The triangular pressure distribution is drawn with the resultant the package returned (base value 2P/H = 36.7224 kPa, derived), the thrust at H/3 inclined at δ. Marked: K = 0.340022, P = 110.167 kN/m, horizontal 103.523 kN/m, vertical 37.6794 kN/m.



Ka (Coulomb active coefficient) against delta (wall friction angle (degrees)) from 10 to 30. One curve for each value of phi (friction angle of the backfill (degrees)): 30, 22.5. Other inputs as in the worked example: beta = 10.0, gam = 18000.0 N/m^3, H = 6.0 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, delta = 20, Ka = 0.340022.

```
civil_engineering_ai999x.solve('coulomb_active_thrust', phi=30.0, delta=20.0, beta=10.0,
gam=18000.0, H=6.0)
# -> Ka = 0.340022, Pa = 110167 N/m, Pa_h = 103523 N/m, Pa_v = 37679.4 N/m   (SOLVED)
```

```
civil_engineering_ai999x.solve('coulomb_active_thrust', phi=35.0, delta=23.0, beta=0.0,
gam=19000.0, H=4.0)
# -> Ka = 0.244429, Pa = 37153.3 N/m, Pa_h = 34199.8 N/m, Pa_v = 14516.9 N/m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coulomb active thrust with friction angle = 30 degrees, wall friction angle = 20 degrees, backfill slope = 10 degrees, unit weight = 18 kN/m^3 and height = 6 m.	SOLVED	Ka = 0.340022, Pa = 110167 N/m, Pa_h = 103523 N/m, Pa_v = 37679.4 N/m
Coulomb active thrust with friction angle = 35 degrees, wall friction angle = 23 degrees, backfill slope = 0 degrees, unit weight = 19 kN/m^3 and height = 4 m.	SOLVED	Ka = 0.244429, Pa = 37153.3 N/m, Pa_h = 34199.8 N/m, Pa_v = 14516.9 N/m
Coulomb active thrust with friction angle = 30 degrees, wall friction angle = 20 degrees, backfill slope = 10 degrees and unit weight = 18 kN/m^3.	NEEDS_INPUT	coulomb_active_thrust needs H (wall height). Example: solve('coulomb_active_thrust', phi=30, delta=20, beta=10, gam='180
Coulomb active thrust with friction angle = 30 degrees, wall friction angle = 20 degrees, backfill slope = 10 degrees, unit weight = 18 kN/m^3 and height = 6 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
Coulomb active thrust with friction angle = 30 degrees, wall friction angle = 20 degrees, backfill slope = 35 degrees, unit weight = 18 kN/m^3 and height = 6 m.	INVALID_INPUT	Coulomb active thrust on a rough wall with sloping backfill: the backfill slope must lie between 0 and phi.

Coulomb passive thrust on a rough wall, `coulomb_passive_thrust`

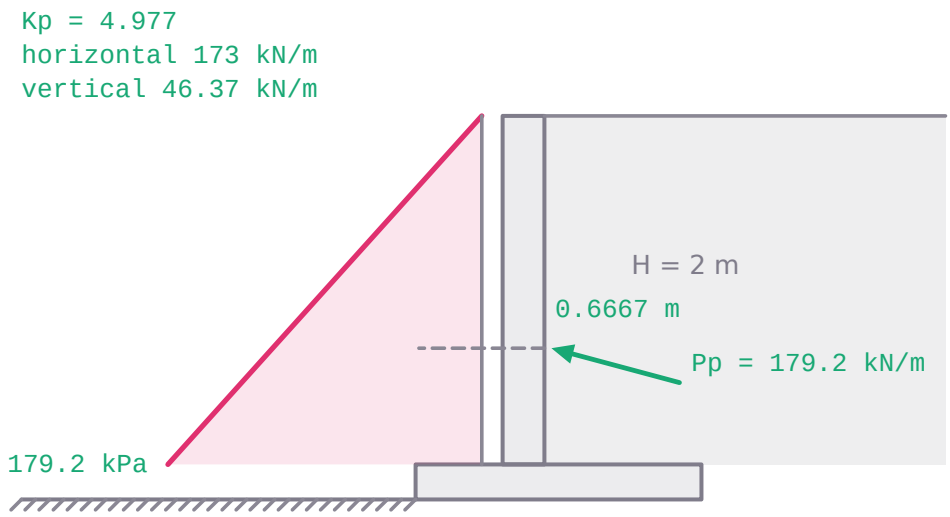
Coulomb (plane-wedge) passive earth pressure coefficient and thrust per metre on a rough vertical wall with wall friction delta and a soil surface rising at beta (plane wedges overestimate the passive resistance when delta is large; keep delta below about $\phi/2$ in design).

$$K_p = \frac{\cos^2\phi}{\cos\delta\left[1-\sqrt{\frac{\sin(\phi+\delta)\sin(\phi+\beta)}{\cos\delta\cos\beta}}\right]^2}$$

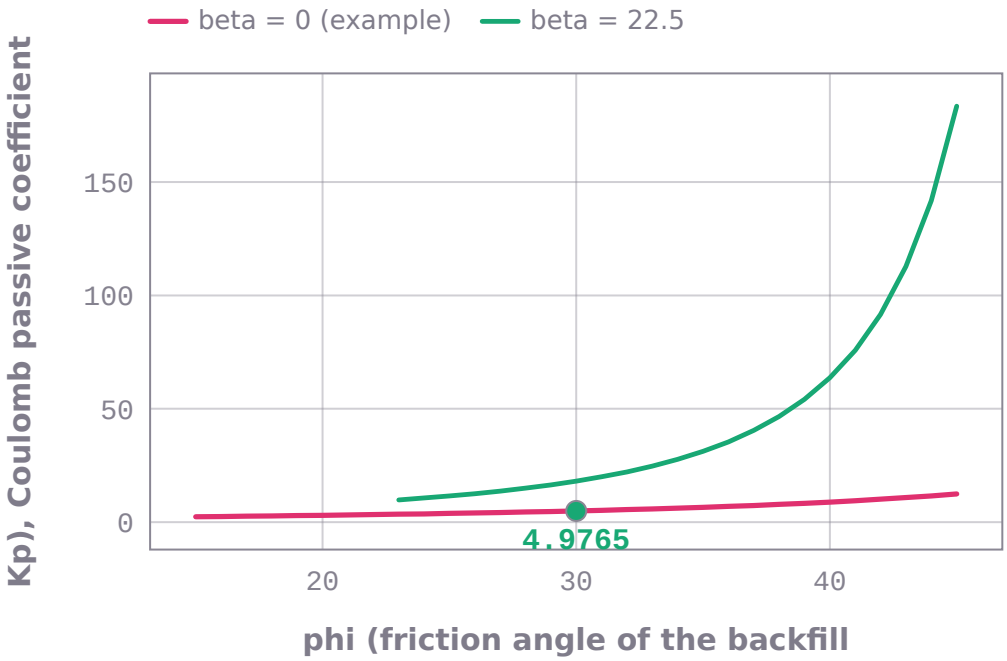
input	meaning	condition
`phi`	friction angle of the backfill (degrees)	
`delta`	wall friction angle (degrees)	
`beta`	backfill slope angle above horizontal (degrees)	
`gam`	unit weight of the backfill	
`H`	wall height	

output	meaning
`Kp`	Coulomb passive coefficient
`Pp`	passive thrust per metre, inclined at delta (N/m)
`Pp_h`	horizontal component (N/m)
`Pp_v`	vertical component (N/m)

- **Method:** $Kp = cos^2$ phi/(cos delta [$1 - sqrt$(sin(phi + delta) sin(phi + beta)/(cos delta cos beta))]^2), $Pp = Kp$ gam $H^2/2$
- **Assumes:** vertical wall back; cohesionless dry soil; plane wedge $(\phi + \delta + \beta < 90)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Coulomb passive thrust on a rough vertical wall of height H = 2 m (φ = 30.0°, wall friction δ = 15.0°, backfill slope β = 0.0°, γ = 18 kN/m³). The triangular pressure distribution is drawn with the resultant the package returned (base value 2P/H = 179.154 kPa, derived), the thrust at H/3 inclined at δ. Marked: K = 4.9765, P = 179.154 kN/m, horizontal 173.049 kN/m, vertical 46.3685 kN/m.



Kp (Coulomb passive coefficient) against phi (friction angle of the backfill (degrees)) from 15 to 45. One curve for each value of beta (backfill slope angle above horizontal (degrees)): 0, 22.5. Other inputs as in the worked example: delta = 15.0, gam = 18000.0 N/m^3, H = 2.0 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, phi = 30, Kp = 4.9765.

```
civil_engineering_ai999x.solve('coulomb_passive_thrust', phi=30.0, delta=15.0, beta=0.0,
gam=18000.0, H=2.0)
# -> Kp = 4.9765, Pp = 179154 N/m, Pp_h = 173049 N/m, Pp_v = 46368.5 N/m   (SOLVED)
```

```
civil_engineering_ai999x.solve('coulomb_passive_thrust', phi=34.0, delta=12.0, beta=5.0,
gam=19000.0, H=3.0)
# -> Kp = 6.93097, Pp = 592598 N/m, Pp_h = 579648 N/m, Pp_v = 123208 N/m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coulomb passive thrust with friction angle = 30 degrees, wall friction angle = 15 degrees, backfill slope = 0 degrees, unit weight = 18 kN/m^3 and height = 2 m.	SOLVED	Kp = 4.9765, Pp = 179154 N/m, Pp_h = 173049 N/m, Pp_v = 46368.5 N/m
Coulomb passive thrust with friction angle = 34 degrees, wall friction angle = 12 degrees, backfill slope = 5 degrees, unit weight = 19 kN/m^3 and height = 3 m.	SOLVED	Kp = 6.93097, Pp = 592598 N/m, Pp_h = 579648 N/m, Pp_v = 123208 N/m
Coulomb passive thrust with friction angle = 30 degrees, wall friction angle = 15 degrees, backfill slope = 0 degrees and unit weight = 18 kN/m^3.	NEEDS_INPUT	coulomb_passive_thrust needs H (wall height). Example: solve('coulomb_passive_thrust', phi=30, delta=15, beta=0, gam='18
Coulomb passive thrust with friction angle = 30 degrees, wall friction angle = 15 degrees, backfill slope = 0 degrees, unit weight = 18 kN/m^3 and height = 2 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Coulomb passive thrust with friction angle = 30 degrees, wall friction angle = 40 degrees, backfill slope = 0 degrees, unit weight = 18 kN/m^3 and height = 2 m.	INVALID_INPUT	Coulomb passive thrust on a rough wall: the wall friction angle must lie between 0 and phi.

Major principal stress at failure (Mohr-Coulomb), `mohr_coulomb_failure_stress`

Major principal stress at failure for a given minor principal stress (triaxial test), with the normal and shear stresses on the failure plane and its angle to the major principal plane $(45 + \phi/2)$.

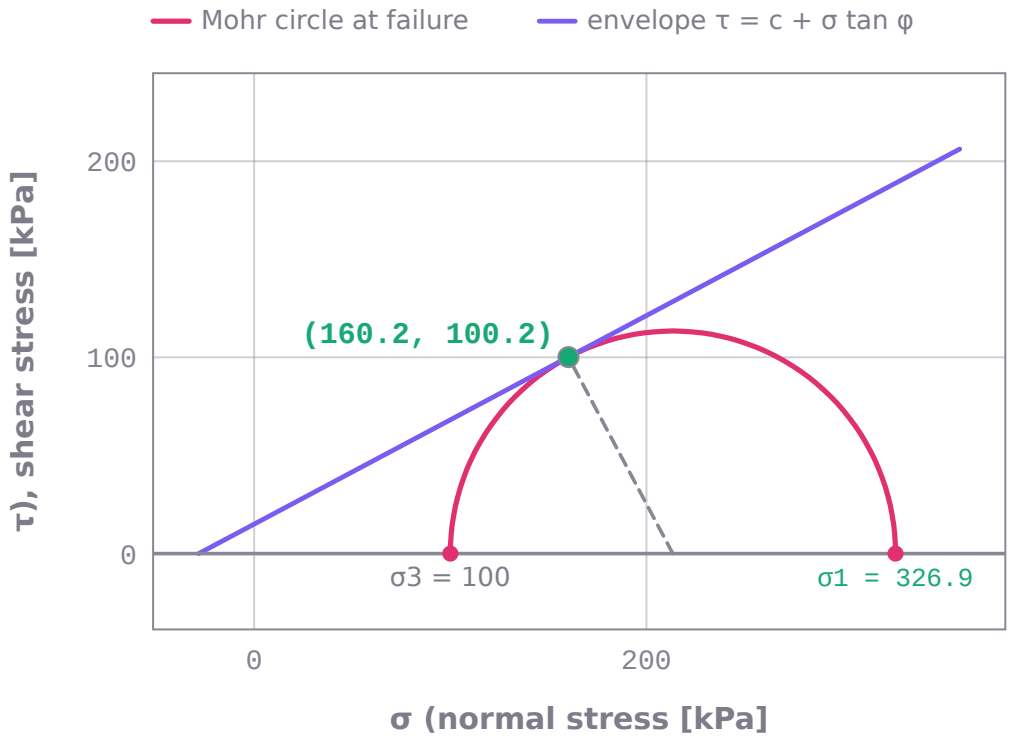
$$\sigma_1 = \sigma_3 \tan^2\left(45^\circ + \frac{\phi}{2}\right) + 2c \tan\left(45^\circ + \frac{\phi}{2}\right)$$

input	meaning	condition
`s3`	minor principal (confining) stress	
`c`	cohesion	
`phi`	friction angle (degrees)	

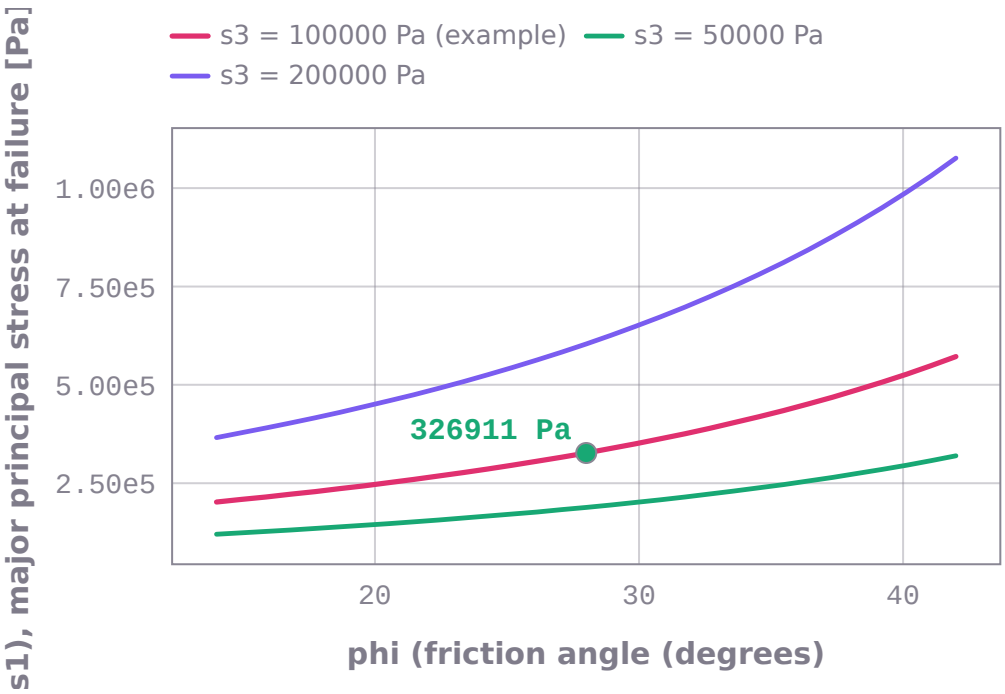
output	meaning
`s1`	major principal stress at failure (Pa)
`sigma_f`	normal stress on the failure plane (Pa)
`tau_f`	shear stress on the failure plane (Pa)

output	meaning
`theta`	failure plane angle from the major principal plane (degrees)

- **Method:** $s_1 = s_3\ tan^2$(45 + phi/2) + 2c tan(45 + phi/2); tangent point at $\sigma = p - r$ sin phi, $\tau = r$ cos phi
- **Assumes:** $0 < \phi < 60$ degrees; s3, $c \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mohr–Coulomb failure for σ3 = 100 kPa, c = 15 kPa, φ = 28.0°: the circle through σ3 and the returned major principal stress at failure σ1 = 326.911 kPa touches the envelope at the returned stresses on the failure plane σf = 160.191 kPa, τf = 100.175 kPa (plane at θ = 59° to the major principal plane).



s1 (major principal stress at failure (Pa)) against phi (friction angle (degrees)) from 14 to 42. One curve for each value of s3 (minor principal (confining) stress): 100000 Pa, 50000 Pa, 200000 Pa. Other inputs as in the worked example: c = 15000.0 Pa. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, phi = 28, s1 = 326911 Pa.

```
civil_engineering_ai999x.solve('mohr_coulomb_failure_stress', s3=100000.0, c=15000.0,
phi=28.0)
# -> s1 = 326911 Pa, sigma_f = 160191 Pa, tau_f = 100175 Pa, theta = 59   (SOLVED)
```

```
civil_engineering_ai999x.solve('mohr_coulomb_failure_stress', s3=200000.0, c=0.0, phi=35.0)
# -> s1 = 738034 Pa, sigma_f = 314715 Pa, tau_f = 220366 Pa, theta = 62.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mohr coulomb failure stress with confining stress = 100 kPa, cohesion = 15 kPa and friction angle = 28 degrees.	SOLVED	s1 = 326911 Pa, sigma_f = 160191 Pa, tau_f = 100175 Pa, theta = 59
Mohr coulomb failure stress with confining stress = 200 kPa, cohesion = 0 kPa and friction angle = 35 degrees.	SOLVED	s1 = 738034 Pa, sigma_f = 314715 Pa, tau_f = 220366 Pa, theta = 62.5
Mohr coulomb failure stress with confining stress = 100 kPa and cohesion = 15 kPa.	NEEDS_INPUT	mohr_coulomb_failure_stress needs phi (friction angle (degrees)). Example: solve('mohr_coulomb_failure_stress', s3='1000
Mohr coulomb failure stress with confining stress = 100 kPa, cohesion = 15 kPa and friction angle = 28 degrees and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Mohr coulomb failure stress with confining stress = 100 kPa, cohesion = 15 kPa and friction angle = 75 degrees.	INVALID_INPUT	Major principal stress at failure (Mohr-Coulomb): the friction angle must lie between 0 and 60 degrees.

Cohesion and friction angle from triaxial tests, `triaxial_strength_parameters`

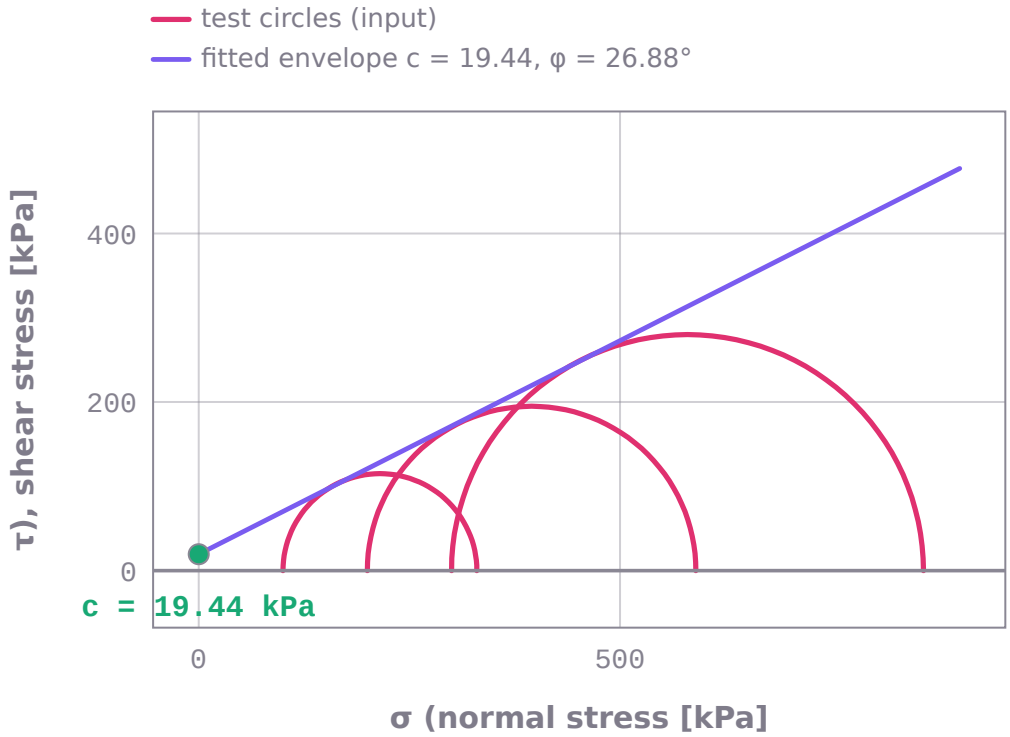
Mohr-Coulomb cohesion and friction angle from two or more triaxial tests at failure, by a least-squares line through the stress points (p, q) = ($(s_1 + s_3)/2$, (s1 - s3)/2): sin phi = slope, c = intercept/cos phi. Stresses in kPa (effective or total, consistently).

$$\sin\phi = \tan\alpha, \quad c = \frac{a}{\cos\phi}$$

input	meaning	condition
`s3`	confining (minor principal) stresses at failure in kPa	
`s1`	major principal stresses at failure in kPa	

output	meaning
`phi`	friction angle (degrees)
`c`	cohesion (kPa)
`alpha`	slope angle of the Kf line (degrees)

- **Method:** least squares $q = a + p$ tan(alpha); phi = asin(tan alpha), $c = a/cos$ phi
- **Assumes:** at least two tests; $s_1 > s_3$; a non-negative intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Triaxial tests at confining stresses σ3 = [100, 200, 300] kPa failing at σ1 = [330, 590, 860] kPa (circles from these inputs) and the strength envelope from the returned parameters c = 19.4378 kPa and φ = 26.8778° (failure plane angle α = 24.3272°).

```
civil_engineering_ai999x.solve('triaxial_strength_parameters', s3=[100, 200, 300], s1=[330,
590, 860])
# -> phi = 26.8778, c = 19.4378, alpha = 24.3272   (SOLVED)
```

```
civil_engineering_ai999x.solve('triaxial_strength_parameters', s3=[50, 150], s1=[210, 480])
# -> phi = 27.3522, c = 22.8218, alpha = 24.6769   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Triaxial strength parameters with confining stresses = [100, 200, 300] and major principal stresses = [330, 590, 860].	SOLVED	phi = 26.8778, c = 19.4378, alpha = 24.3272
Triaxial strength parameters with confining stresses = [50, 150] and major principal stresses = [210, 480].	SOLVED	phi = 27.3522, c = 22.8218, alpha = 24.6769
Triaxial strength parameters with confining stresses = [100, 200, 300].	NEEDS_INPUT	triaxial_strength_parameters needs s1 (major principal stresses at failure in kPa). Example: solve('triaxial_strength_pa
Triaxial strength parameters with confining stresses = [100, 200, 300] and major principal stresses = [330, 590, 860] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Cohesion and friction angle from direct shear tests, `direct_shear_strength_parameters`

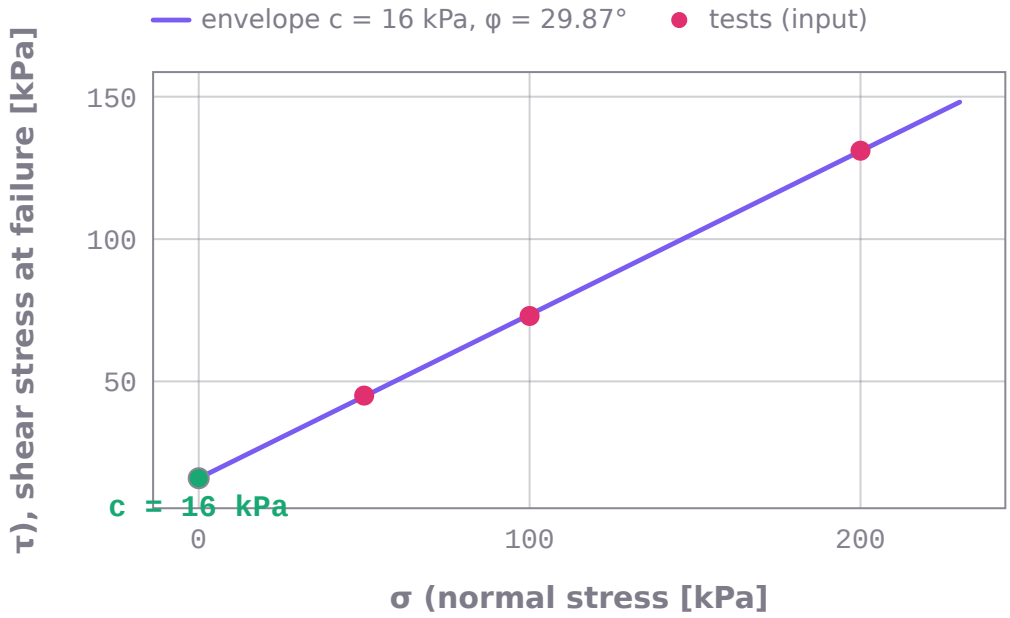
Mohr-Coulomb cohesion and friction angle from direct shear tests by a least-squares line of peak shear stress against normal stress. Stresses in kPa.

$$\tau_f = c + \sigma \tan \phi$$

input	meaning	condition
`sigma`	normal stresses in kPa	
`tau`	shear stresses at failure in kPa	

output	meaning
`phi`	friction angle (degrees)
`c`	cohesion intercept (kPa)
`tan_phi`	slope tan(phi)

- **Method:** least squares $\tau = c + \sigma$ tan phi
- **Assumes:** at least two tests; shear strength rises with normal stress
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Direct shear tests (σ = [50, 100, 200] kPa, τ at failure = [45, 73, 131] kPa) and the straight strength envelope from the returned c = 16 kPa and tan φ = 0.574286 (φ = 29.8681°).

```
civil_engineering_ai999x.solve('direct_shear_strength_parameters', sigma=[50, 100, 200],
tau=[45, 73, 131])
# -> phi = 29.8681, c = 16, tan_phi = 0.574286   (SOLVED)
```

```
civil_engineering_ai999x.solve('direct_shear_strength_parameters', sigma=[100, 200, 300,
400], tau=[68, 132, 195, 262])
# -> phi = 32.822, c = 3, tan_phi = 0.645   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Direct shear strength parameters with normal stresses = [50, 100, 200] and shear stresses = [45, 73, 131].	SOLVED	phi = 29.8681, c = 16, tan_phi = 0.574286
Direct shear strength parameters with normal stresses = [100, 200, 300, 400] and shear stresses = [68, 132, 195, 262].	SOLVED	phi = 32.822, c = 3, tan_phi = 0.645
Direct shear strength parameters with normal stresses = [50, 100, 200].	NEEDS_INPUT	direct_shear_strength_parameters needs tau (shear stresses at failure in kPa). Example: solve('direct_shear_strength_par
Direct shear strength parameters with normal stresses = [50, 100, 200] and shear stresses = [45, 73, 131] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Optimum water content and maximum dry density from a compaction test,

`proctor_compaction_optimum`

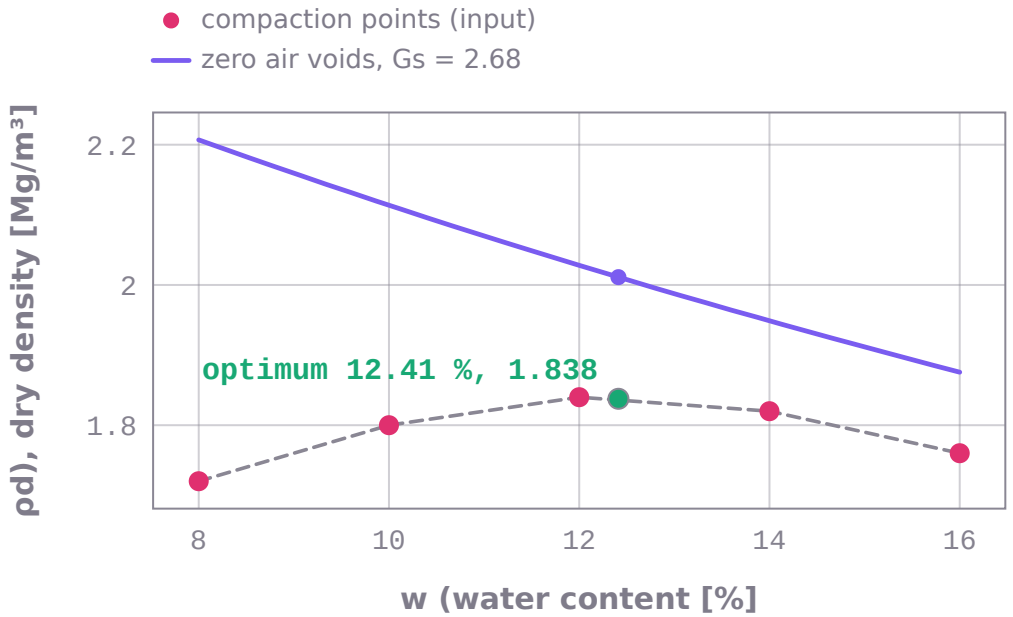
Optimum water content and maximum dry density from Proctor compaction points by a least-squares parabola, with the zero-air-voids dry density at the optimum when Gs is given. Water contents in %, dry densities in Mg/m^3.

$$w_{opt} = -\frac{c_1}{2c_2}, \quad \rho_{zav} = \frac{G_s \rho_w}{1 + w G_s}$$

input	meaning	condition
`w`	water contents in %	
`rho_d`	dry densities in Mg/m^3	
`Gs`	specific gravity of the solids (optional)	

output	meaning
`w_opt`	optimum water content (%)
`rho_dmax`	maximum dry density (Mg/m^3)
`rho_zav`	zero-air-voids dry density at the optimum (Mg/m^3)

- **Method:** least-squares parabola $rho_d = c_0 + c_1\, w + c_2\, w^2$; w_opt = -c1/(2 c2); zero air voids rho_w Gs/(1 + w Gs)
- **Assumes:** at least three points around the peak; a parabola opening downward
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Proctor compaction points (water contents [8, 10, 12, 14, 16] %, dry densities [1.72, 1.8, 1.84, 1.82, 1.76] Mg/m³) and the zero-air-voids line for Gs = 2.68 (from Gs ρw/(1 + w Gs)). Marked: the returned optimum water content 12.4118 % and maximum dry density 1.8376 Mg/m³ (from the package's least-squares parabola), and the returned zero-air-voids density 2.01105 Mg/m³ at the optimum.

```
civil_engineering_ai999x.solve('proctor_compaction_optimum', w=[8, 10, 12, 14, 16], rho_d=
[1.72, 1.8, 1.84, 1.82, 1.76], Gs=2.68)
# -> w_opt = 12.4118, rho_dmax = 1.8376, rho_zav = 2.01105   (SOLVED)
```

```
civil_engineering_ai999x.solve('proctor_compaction_optimum', w=[12, 14, 16, 18, 20, 22],
rho_d=[1.58, 1.64, 1.68, 1.69, 1.66, 1.61])
# -> w_opt = 17.4241, rho_dmax = 1.68723   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Proctor compaction optimum with water contents = [8, 10, 12, 14, 16], dry densities = [1.72, 1.8, 1.84, 1.82, 1.76] and specific gravity = 2.68.	SOLVED	w_opt = 12.4118, rho_dmax = 1.8376, rho_zav = 2.01105
Proctor compaction optimum with water contents = [12, 14, 16, 18, 20, 22] and dry densities = [1.58, 1.64, 1.68, 1.69, 1.66, 1.61].	SOLVED	w_opt = 17.4241, rho_dmax = 1.68723
Proctor compaction optimum with water contents = [8, 10, 12, 14, 16] and specific gravity = 2.68.	NEEDS_INPUT	proctor_compaction_optimum needs rho_d (dry densities in Mg/m^3). Example: solve('proctor_compaction_optimum', w=[8, 10,
Proctor compaction optimum with water contents = [8, 10, 12, 14, 16], dry densities = [1.72, 1.8, 1.84, 1.82, 1.76] and specific gravity = 2.68 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Closed traverse: misclosure and Bowditch (compass rule) adjustment,

`traverse_bowditch_adjustment`

Latitudes and departures of a closed loop traverse from whole-circle bearings (azimuths clockwise from north, degrees) and distances (m), the misclosure and relative precision (1 in X), the coordinates adjusted by the Bowditch (compass) rule, and the enclosed area. The last leg returns to the starting station.

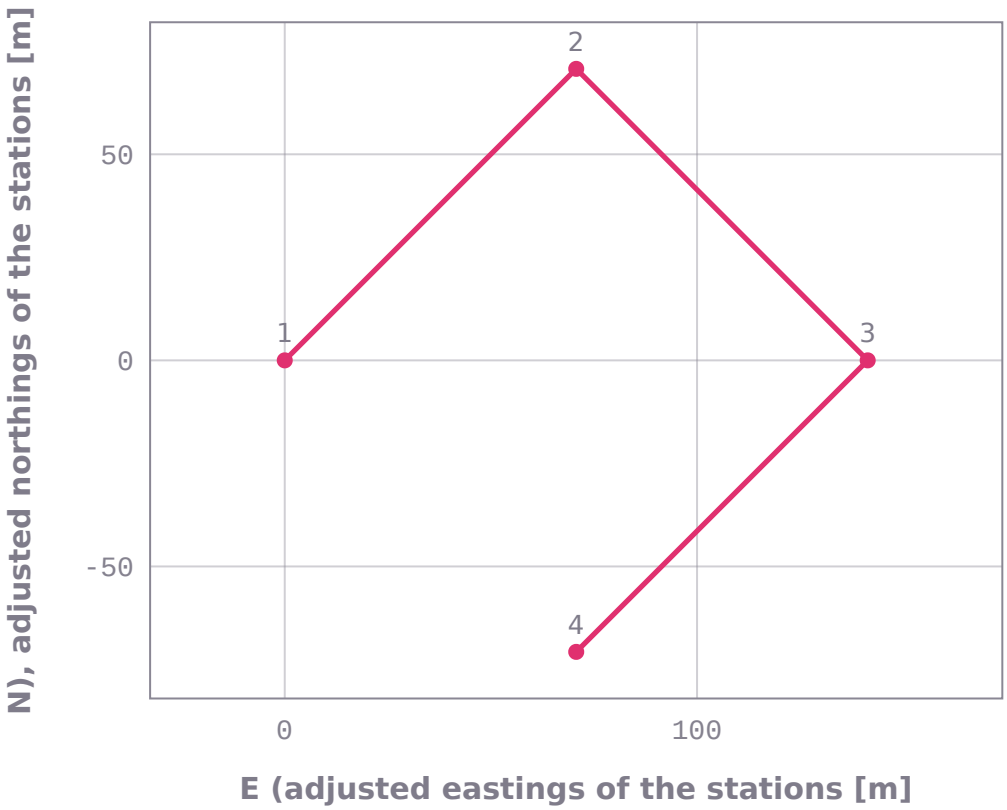
$$\delta E_i = -e_E \frac{d_i}{\sum d}, \quad \delta N_i = -e_N \frac{d_i}{\sum d}$$

input	meaning	condition
`az`	azimuths (whole-circle bearings) of the legs in degrees	
`d`	leg lengths in m	
`E0`	easting of the starting station (default 0)	
`N0`	northing of the starting station (default 0)	

output	meaning
`misclosure_E`	misclosure in departure (m)
`misclosure_N`	misclosure in latitude (m)
`linear_misclosure`	linear misclosure (m)

output	meaning
`precision`	relative precision X in 1:X (0 if it closes exactly)
`E`	adjusted eastings of the stations (m)
`N`	adjusted northings of the stations (m)
`area`	enclosed area (m^2)

- **Method:** dep = d sin(az), lat = d cos(az); corrections -e d_i/sum d (compass rule); area by coordinates
- **Assumes:** closed loop traverse; azimuths 0 to 360 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The points (E, N) returned by civil-engineering-ai999x for the worked example (az = [45, 135.01, 225, 315], d = [100, 100.02, 99.98, 100]), joined in order: 1: (0, 0); 2: (70.7067, 70.7138); 3: (141.415, -0.0203129); 4: (70.7147, -70.7138).

```
civil_engineering_ai999x.solve('traverse_bowditch_adjustment', az=[45, 135.01, 225, 315],
d=[100, 100.02, 99.98, 100])
# -> misclosure_E = 0.0159394 m, misclosure_N = -0.0123427 m, linear_misclosure = 0.0201595
m, precision = 19841.7, E = [0.0, 70.7066932726, 141.415182997, 70.7146629647] m, N = [0.0,
70.7137638018, -0.020312884737, -70.7137638018] m, area = 10000 m^2   (SOLVED)
```

```
civil_engineering_ai999x.solve('traverse_bowditch_adjustment', az=[30.5, 112.25, 201.75,
290.1], d=[250.4, 310.25, 280.9, 240.55], E0=1000.0, N0=2000.0)
# -> misclosure_E = 84.2478 m, misclosure_N = -79.9591 m, linear_misclosure = 116.151 m,
precision = 9.31629, E = [1000.0, 1107.59249531, 1370.58665187, 1244.62735276] m, N =
[2000.0, 2234.25461947, 2139.70402872, 1899.55782373] m, area = 71589.2 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Traverse bowditch adjustment with azimuths = [45, 135.01, 225, 315] and distances = [100, 100.02, 99.98, 100].	SOLVED	misclosure_E = 0.0159394 m, misclosure_N = -0.0123427 m, linear_misclosure = 0.0201595 m, precision = 19841.7, E = [0.0, 70.7066932726, 141.415182997, 70.7146629647] m, N = [0.0, 70.7137638018, -0.020312884737, -70.7137638018] m, area = 10000 m^2
Traverse bowditch adjustment with azimuths = [30.5, 112.25, 201.75, 290.1], distances = [250.4, 310.25, 280.9, 240.55], start easting = 1000 m and start northing = 2000 m.	SOLVED	misclosure_E = 84.2478 m, misclosure_N = -79.9591 m, linear_misclosure = 116.151 m, precision = 9.31629, E = [1000.0, 1107.59249531, 1370.58665187, 1244.62735276] m, N = [2000.0, 2234.25461947, 2139.70402872, 1899.55782373] m, area = 71589.2 m^2
Traverse bowditch adjustment with azimuths = [45, 135.01, 225, 315].	NEEDS_INPUT	traverse_bowditch_adjustment needs d (leg lengths in m). Example: solve('traverse_bowditch_adjustment', az=[45, 135.01,
Traverse bowditch adjustment with azimuths = [45, 135.01, 225, 315] and distances = [100, 100.02, 99.98, 100] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Reduced levels by the rise-and-fall method, `levelling_rise_and_fall`

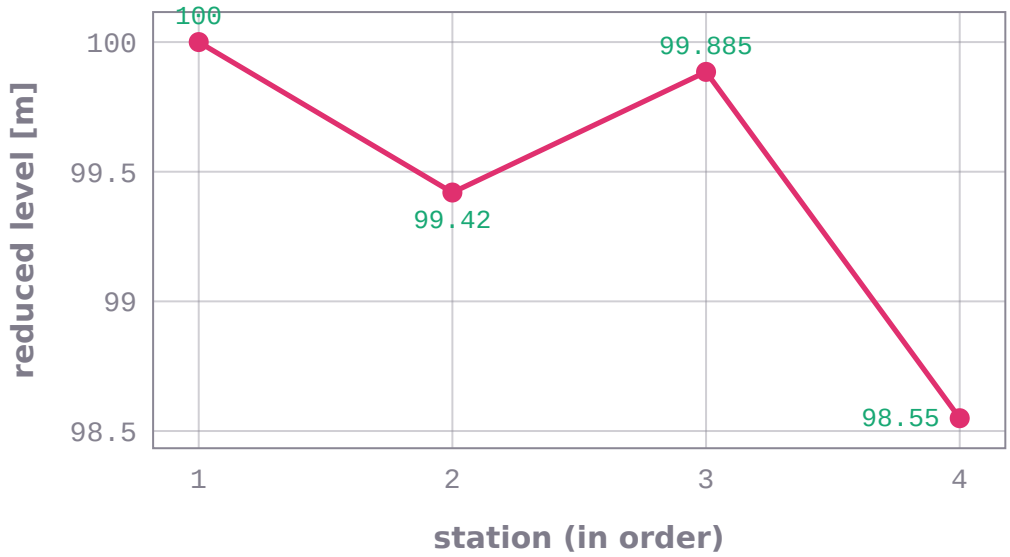
Reduced levels of every station of a levelling run from staff readings (m) and their sight types (BS, IS, FS in reading order; a BS after an FS is the new set-up on the same change point), with the total rise and fall and the arithmetic check sum BS - sum FS (= last RL - first RL).

$$\sum \mathrm{BS} - \sum \mathrm{FS} = \sum \mathrm{rise} - \sum \mathrm{fall} = RL_{last} - RL_{first}$$

input	meaning	condition
`readings`	staff readings in m, in reading order	
`sights`	sight type of each reading: BS, IS or FS	
`RL0`	reduced level of the first station (benchmark)	

output	meaning
`RL`	reduced level of each station (m)
`rise`	sum of rises (m)
`fall`	sum of falls (m)
`last_RL`	reduced level of the last station (m)
`check`	sum BS - sum FS (m)

- **Method:** rise or fall = previous reading - current reading within a set-up; RL = previous RL + rise - fall
- **Assumes:** first reading BS, last FS; a BS after the first follows an FS (change point)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Reduced levels of a levelling run (staff readings [1.525, 2.105, 1.64, 0.985, 2.32] m, sights BS, IS, FS, BS, FS, benchmark RL 100.0 m) by rise and fall: the reduced level of every station as returned (100, 99.42, 99.885, 98.55 m). Marked: total rise 0.465 m, total fall 1.915 m, ΣBS − ΣFS = -1.45 m = last RL − first RL.

```
civil_engineering_ai999x.solve('levelling_rise_and_fall', readings=[1.525, 2.105, 1.64, 0.985, 2.32], sights='BS, IS, FS, BS, FS', RL0=100.0)
# -> RL = [100.0, 99.42, 99.885, 98.55] m, rise = 0.465 m, fall = 1.915 m, last_RL = 98.55 m, check = -1.45 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('levelling_rise_and_fall', readings=[2.15, 1.725, 0.98, 1.315, 2.64, 1.105], sights='BS, IS, IS, FS, BS, FS', RL0=52.75)
# -> RL = [52.75, 53.175, 53.92, 53.585, 55.12] m, rise = 2.705 m, fall = 0.335 m, last_RL = 55.12 m, check = 2.37 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Levelling rise and fall with readings = [1.525, 2.105, 1.64, 0.985, 2.32], sight types = [BS, IS, FS, BS, FS] and benchmark level = 100 m.	SOLVED	RL = [100.0, 99.42, 99.885, 98.55] m, rise = 0.465 m, fall = 1.915 m, last_RL = 98.55 m, check = -1.45 m
Levelling rise and fall with readings = [2.15, 1.725, 0.98, 1.315, 2.64, 1.105], sight types = [BS, IS, IS, FS, BS, FS] and benchmark level = 52.75 m.	SOLVED	RL = [52.75, 53.175, 53.92, 53.585, 55.12] m, rise = 2.705 m, fall = 0.335 m, last_RL = 55.12 m, check = 2.37 m
Levelling rise and fall with readings = [1.525, 2.105, 1.64, 0.985, 2.32] and sight types = [BS, IS, FS, BS, FS].	NEEDS_INPUT	levelling_rise_and_fall needs RL0 (reduced level of the first station (benchmark)). Example: solve('levelling_rise_and_f
Levelling rise and fall with readings = [1.525, 2.105, 1.64, 0.985, 2.32], sight types = [BS, IS, FS, BS, FS] and benchmark level = 100 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Elements of a simple horizontal circular curve, `horizontal_circular_curve`

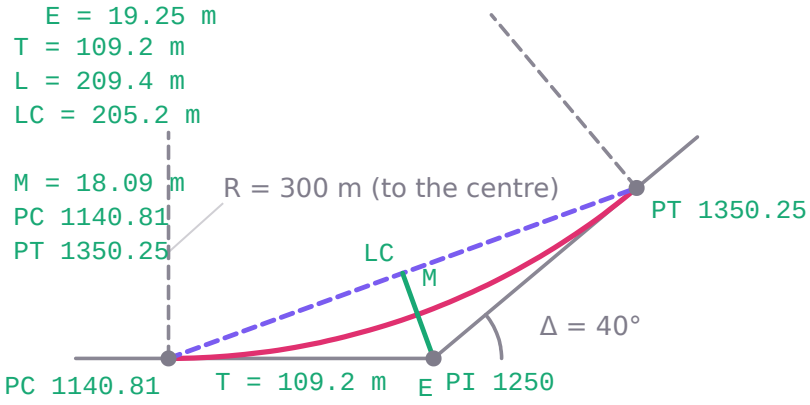
Tangent length, curve length, long chord, external distance, middle ordinate and the chainages of the tangent points of a simple circular curve of radius R joining tangents that deflect by Delta at the intersection point PI.

$$T = R\tan\frac{\Delta}{2}, \quad L = R\Delta, \quad LC = 2R\sin\frac{\Delta}{2}$$

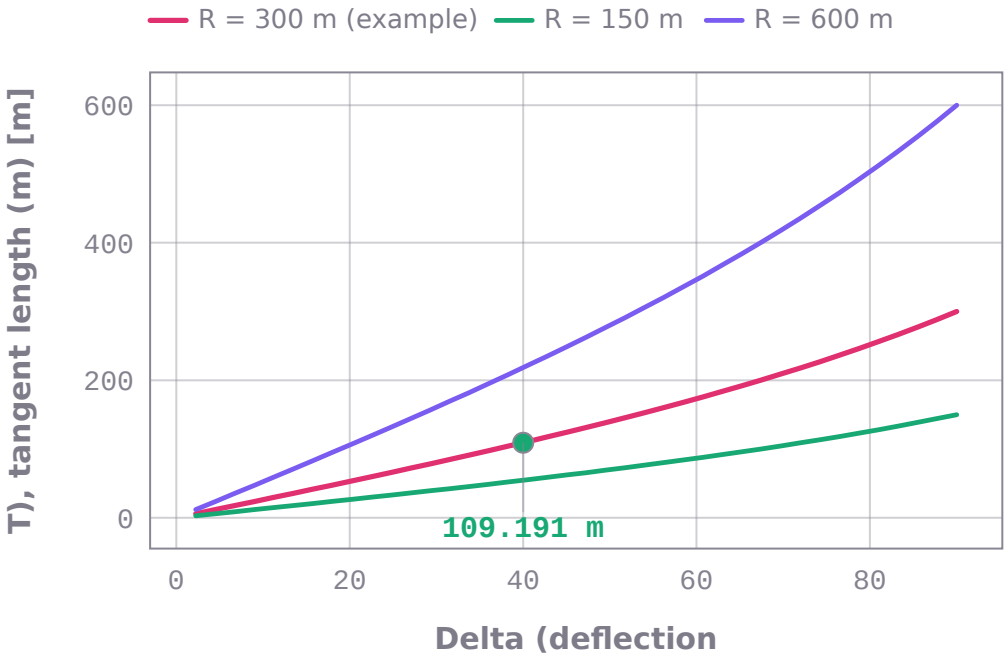
input	meaning	condition
`R`	curve radius	
`Delta`	deflection (intersection) angle (degrees)	
`PI`	chainage of the intersection point (default 0)	

output	meaning
`T`	tangent length (m)
`L`	curve length (m)
`LC`	long chord (m)
`E`	external distance (m)
`M`	middle ordinate (m)
`PC`	chainage of the start of the curve (m)
`PT`	chainage of the end of the curve (m)

- **Method:** $T = R$ tan(D/2), $L = R$ D, $LC = 2R$ sin(D/2), $E = R$(sec(D/2) - 1), $M = R(1 - cos$ (D/2)), $PC = PI - T$, $PT = PC + L$
- **Assumes:** $0 < Delta < 180$ degrees; chainage measured along the tangents then the curve
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Plan of a simple circular curve, radius R = 300 m, deflection angle Δ = 40°, PI at chainage 1250.0 m, drawn to scale with the returned elements: tangent length T = 109.191 m, curve length L = 209.44 m, long chord LC = 205.212 m (dashed), external distance E = 19.2533 m and middle ordinate M = 18.0922 m (green), and the chainages PC = 1140.81 m, PT = 1350.25 m.



T (tangent length (m)) against Delta (deflection (intersection) angle (degrees)) from 0 to 90. One curve for each value of R (curve radius): 300 m, 150 m, 600 m. Other inputs as in the worked example: PI = 1250.0 m. 40 points per curve computed by civil-engineering-ai999x; marked: the worked example, Delta = 40, T = 109.191 m.

```
civil_engineering_ai999x.solve('horizontal_circular_curve', R=300.0, Delta=40.0, PI=1250.0)
# -> T = 109.191 m, L = 209.44 m, LC = 205.212 m, E = 19.2533 m, M = 18.0922 m, PC =
1140.81 m, PT = 1350.25 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('horizontal_circular_curve', R=600.0, Delta=25.5, PI=3420.5)
# -> T = 135.766 m, L = 267.035 m, LC = 264.837 m, E = 15.1686 m, M = 14.7946 m, PC =
3284.73 m, PT = 3551.77 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Horizontal circular curve with radius = 300 m, deflection angle = 40 degrees and PI chainage = 1250 m.	SOLVED	T = 109.191 m, L = 209.44 m, LC = 205.212 m, E = 19.2533 m, M = 18.0922 m, PC = 1140.81 m, PT = 1350.25 m
Horizontal circular curve with radius = 600 m, deflection angle = 25.5 degrees and PI chainage = 3420.5 m.	SOLVED	T = 135.766 m, L = 267.035 m, LC = 264.837 m, E = 15.1686 m, M = 14.7946 m, PC = 3284.73 m, PT = 3551.77 m
Horizontal circular curve with radius = 300 m and PI chainage = 1250 m.	NEEDS_INPUT	horizontal_circular_curve needs Delta (deflection (intersection) angle (degrees)). Example: solve('horizontal_circular_c
Horizontal circular curve with radius = 300 m, deflection angle = 40 degrees and PI chainage = 1250 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Horizontal circular curve with radius = 300 m, deflection angle = 190 degrees and PI chainage = 1250 m.	INVALID_INPUT	Elements of a simple horizontal circular curve: the deflection angle must lie between 0 and 180 degrees.

Elevations on a vertical parabolic curve, `vertical_parabolic_curve`

Elevation at a station on an equal-tangent parabolic vertical curve, with the highest (crest) or lowest (sag) point on the curve and the rate of change of grade. Grades are fractions or percentages; stations and lengths in m.

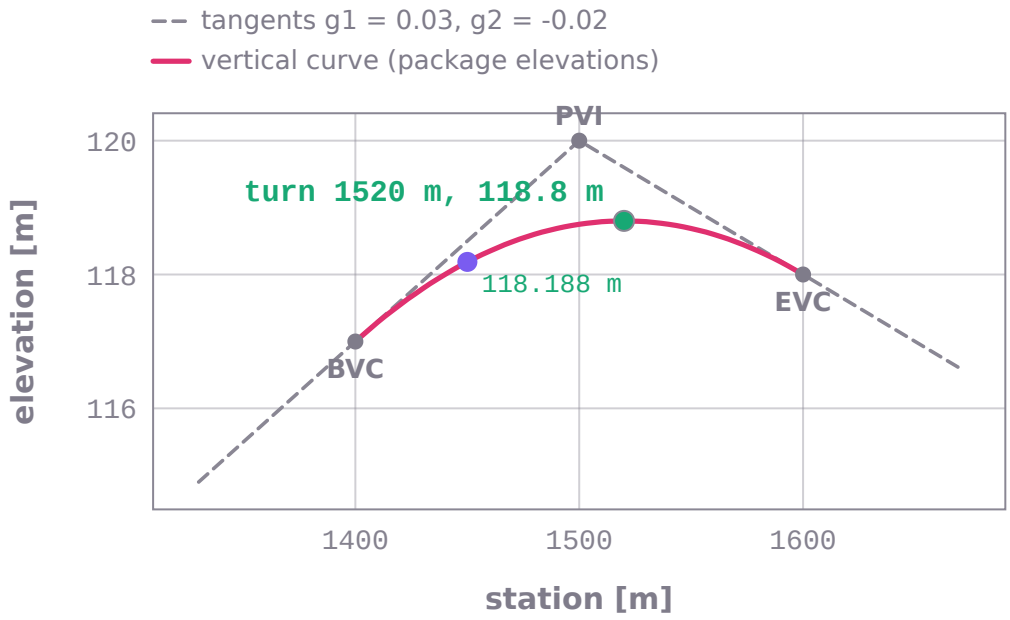
$$y = y_{BVC} + g_1 x + \frac{g_2 - g_1}{2L} x^2, \quad x_t = -\frac{g_1 L}{g_2 - g_1}$$

input	meaning	condition
`g1`	entering grade (fraction, or %)	
`g2`	leaving grade (fraction, or %)	
`L`	curve length (horizontal)	
`PVI`	station of the intersection point	
`elev_PVI`	elevation of the intersection point	
`x`	station where the elevation is wanted	

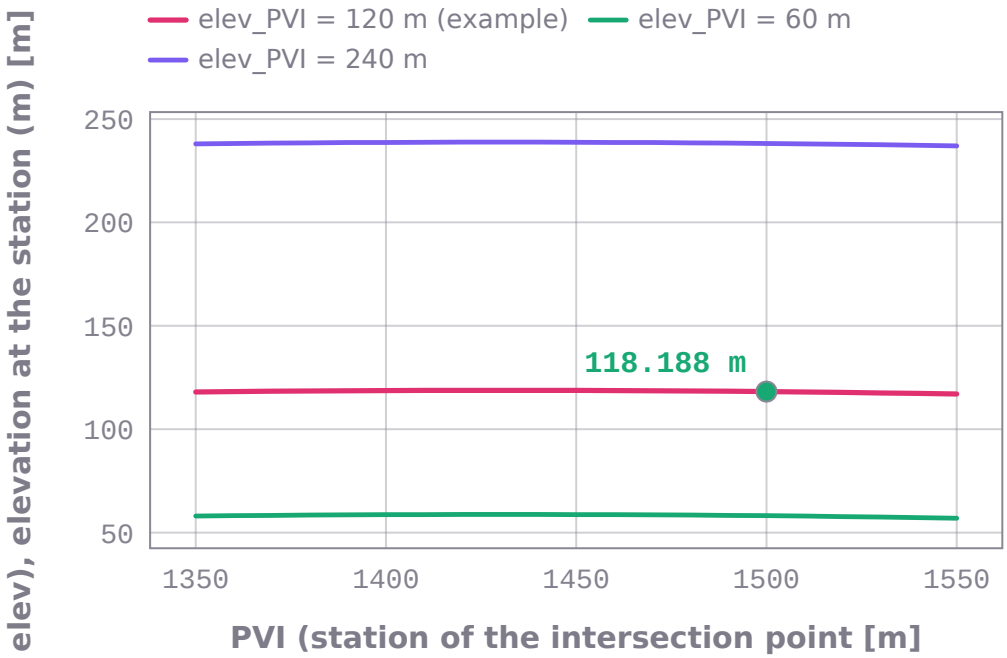
output	meaning
`elev`	elevation at the station (m)
`x_turn`	station of the highest/lowest point on the curve (m)
`elev_turn`	elevation of that point (m)
`r`	rate of change of grade (% per m)

output	meaning
`BVC`	station of the beginning of the curve (m)
`elev_BVC`	elevation of the BVC (m)

- **Method:** y = y_BVC + g1 $x + (g_2 - g_1)\, x^2/(2L)$, x from the BVC; turning point at $x = -g_1\, L/(g_2 - g_1)$
- **Assumes:** equal-tangent parabola; BVC <= station <= EVC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equal-tangent parabolic vertical curve, L = 200 m, PVI at station 1500 m and elevation 120 m, grades g1 = 0.03, g2 = -0.02: 61 elevations along the curve computed by the package, over the two tangents. Marked: the returned BVC 1400 m (elevation 117 m), the turning point 1520 m at 118.8 m, and the elevation 118.188 m at the worked station 1450.0 m (rate of change of grade -0.025 %/m).



elev (elevation at the station (m)) against PVI (station of the intersection point) from 1350 m to 1650 m. One curve for each value of elev_PVI (elevation of the intersection point): 120 m, 60 m, 240 m. Other inputs as in the worked example: g1 = 0.03, g2 = -0.02, L = 200.0 m, x = 1450.0 m. 21 points per curve computed by civil-engineering-ai999x; marked: the worked example, PVI = 1500 m, elev = 118.188 m.

```
civil_engineering_ai999x.solve('vertical_parabolic_curve', g1=0.03, g2=-0.02, L=200.0,
PVI=1500.0, elev_PVI=120.0, x=1450.0)
# -> elev = 118.188 m, x_turn = 1520 m, elev_turn = 118.8 m, r = -0.025, BVC = 1400 m,
elev_BVC = 117 m   (SOLVED)
```

```
civil_engineering_ai999x.solve('vertical_parabolic_curve', g1=-0.025, g2=0.015, L=160.0,
PVI=820.0, elev_PVI=64.2, x=840.0)
# -> elev = 64.95 m, x_turn = 840 m, elev_turn = 64.95 m, r = 0.025, BVC = 740 m, elev_BVC
= 66.2 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vertical parabolic curve with entering grade = 3 %, leaving grade = -2 %, curve length = 200 m, PVI station = 1500 m, PVI elevation = 120 m and station = 1450 m.	SOLVED	elev = 118.188 m, x_turn = 1520 m, elev_turn = 118.8 m, r = -0.025, BVC = 1400 m, elev_BVC = 117 m
Vertical parabolic curve with entering grade = -2.5 %, leaving grade = 1.5 %, curve length = 160 m, PVI station = 820 m, PVI elevation = 64.2 m and station = 840 m.	SOLVED	elev = 64.95 m, x_turn = 840 m, elev_turn = 64.95 m, r = 0.025, BVC = 740 m, elev_BVC = 66.2 m
Vertical parabolic curve with entering grade = 3 %, leaving grade = -2 %, curve length = 200 m, PVI station = 1500 m and PVI elevation = 120 m.	NEEDS_INPUT	vertical_parabolic_curve needs x (station where the elevation is wanted). Example: solve('vertical_parabolic_curve', g1=
Vertical parabolic curve with entering grade = 3 %, leaving grade = -2 %, curve	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
length = 200 m, PVI station = 1500 m, PVI elevation = 120 m and station = 1450 m and colour = 3.		civil_engineering_ai999x.template
Vertical parabolic curve with entering grade = 3 %, leaving grade = -2 %, curve length = 200 m, PVI station = 1500 m, PVI elevation = 120 m and station = 1700 m.	INVALID_INPUT	Elevations on a vertical parabolic curve: the station must lie on the curve (between BVC and EVC).

Earthwork volume from cross-section areas, `earthwork_volume_end_areas`

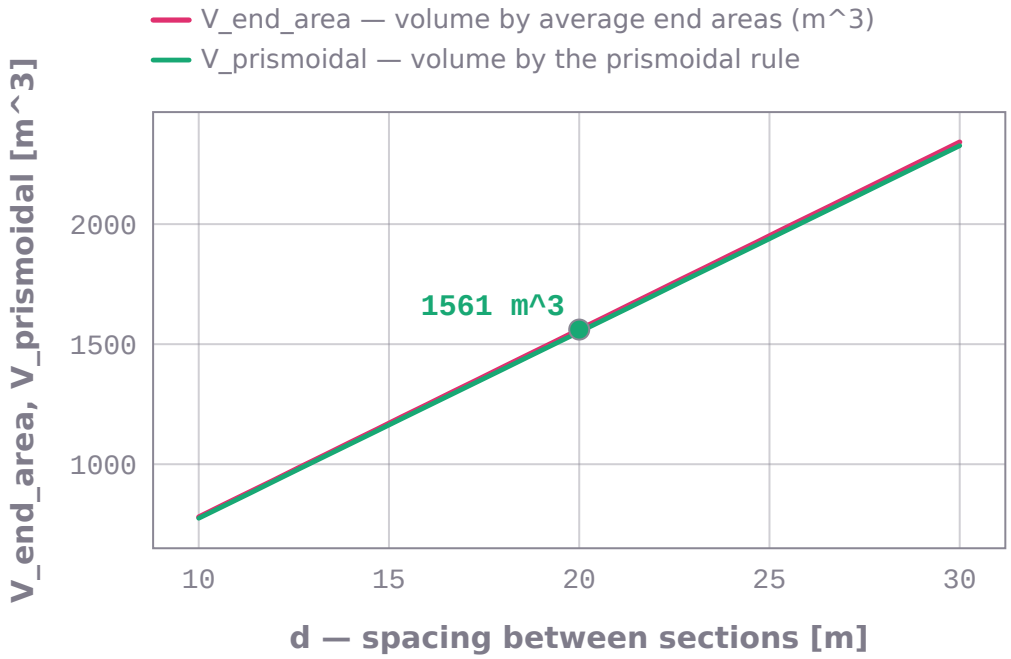
Volume of cut or fill between equally spaced cross-sections by the average-end-area method, and by the prismoidal (Simpson) rule when the number of sections is odd. Areas in m^2.

$$V = d\left(\frac{A_1 + A_n}{2} + \sum_{i=2}^{n-1} A_i\right)$$

input	meaning	condition
`areas`	cross-section areas in m^2, in order	
`d`	spacing between sections	

output	meaning
`V_end_area`	volume by average end areas (m^3)
`V_prismoidal`	volume by the prismoidal rule (m^3)

- **Method:** $V = d\,(A_1/2 + A_2 + \ldots + An/2)$; prismoidal $V = (d/3)(A_1 + 4A2 + 2A3 + \ldots + An)$
- **Assumes:** equally spaced sections; prismoidal rule only for an odd number of sections
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V_end_area (volume by average end areas (m^3)) against d (spacing between sections) from 10 m to 30 m. Outputs with the same unit drawn together: V_prismoidal (volume by the prismoidal rule (m^3)). Other inputs as in the worked example: areas = [12.5, 18.2, 25.6, 20.1, 15.8]. 31 points computed by civil-engineering-ai999x; marked: the worked example, d = 20 m, V_end_area = 1561 m^3.

```
civil_engineering_ai999x.solve('earthwork_volume_end_areas', areas=[12.5, 18.2, 25.6, 20.1, 15.8], d=20.0)
# -> V_end_area = 1561 m^3, V_prismoidal = 1551.33 m^3   (SOLVED)
```

```
civil_engineering_ai999x.solve('earthwork_volume_end_areas', areas=[30, 42, 51, 47], d=25.0)
# -> V_end_area = 3287.5 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Earthwork volume from cross sections with areas = [12.5, 18.2, 25.6, 20.1, 15.8] and spacing = 20 m.	SOLVED	V_end_area = 1561 m^3, V_prismoidal = 1551.33 m^3
Earthwork volume from cross sections with areas = [30, 42, 51, 47] and spacing = 25 m.	SOLVED	V_end_area = 3287.5 m^3
Earthwork volume from cross sections with areas = [12.5, 18.2, 25.6, 20.1, 15.8].	NEEDS_INPUT	earthwork_volume_end_areas needs d (spacing between sections). Example: solve('earthwork_volume_end_areas', areas=[12.5,
Earthwork volume from cross sections with areas = [12.5, 18.2, 25.6, 20.1, 15.8] and spacing = 20 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Area between a survey line and a boundary from offsets — `area_from_offsets`

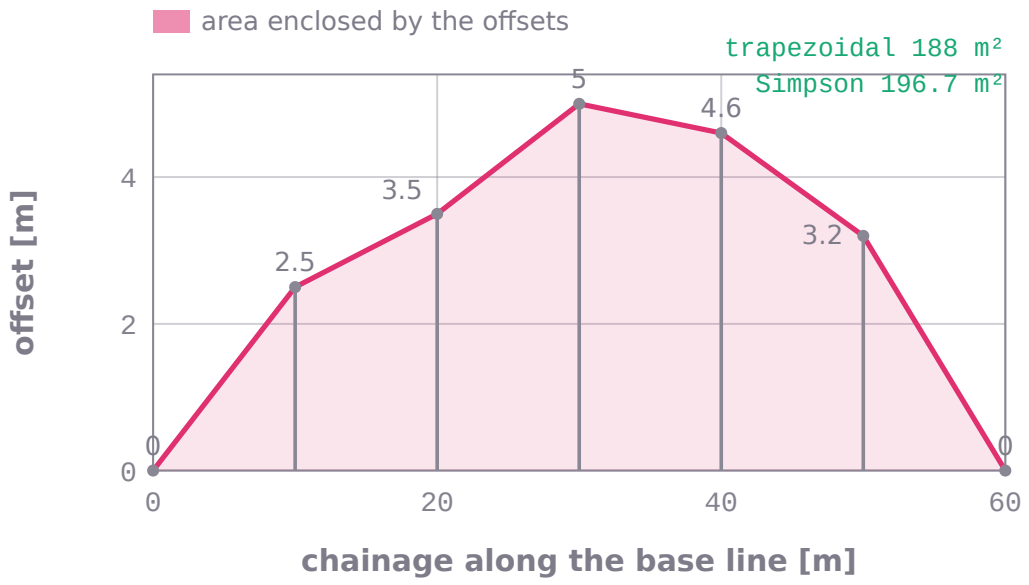
Area between a chain line and an irregular boundary from offsets at a regular interval, by the trapezoidal rule and, for an odd number of offsets, Simpson's one-third rule. Offsets in m.

$$A = \frac{d}{3}\left(y_1 + y_n + 4\sum y_{even} + 2\sum y_{odd}\right)$$

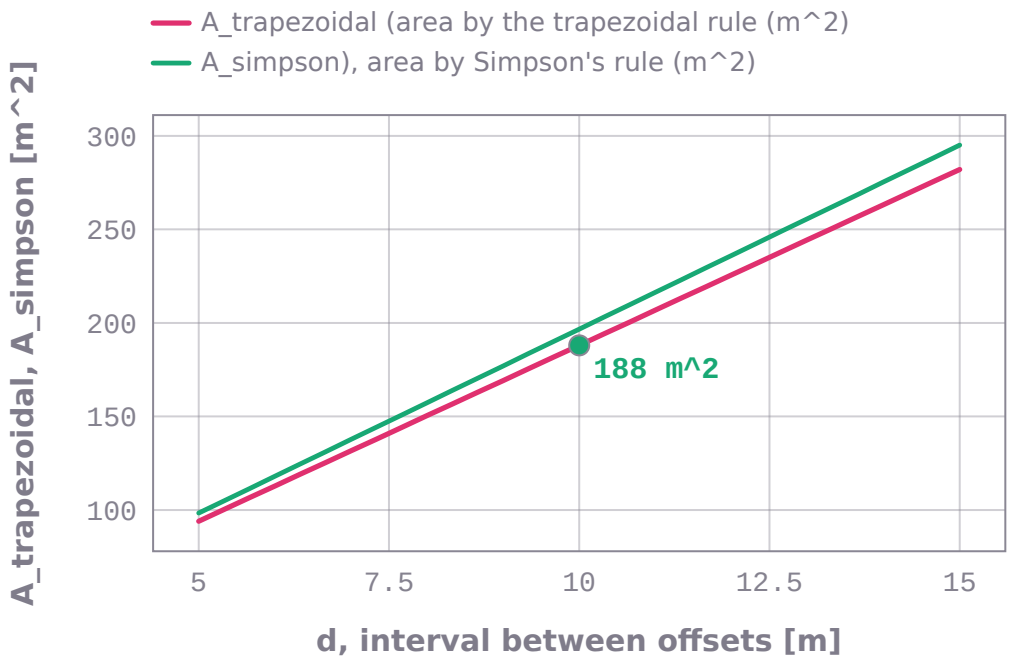
input	meaning	condition
`offsets`	offsets in m at the regular interval	
`d`	interval between offsets	

output	meaning
`A_trapezoidal`	area by the trapezoidal rule (m^2)
`A_simpson`	area by Simpson's rule (m^2)

- **Method:** trapezoidal: d $(y_1/2 + y_2 + \ldots + yn/2)$; Simpson: $(d/3)(y_1 + 4y2 + 2y3 + \ldots + yn)$
- **Assumes:** regular interval; Simpson's rule only for an odd number of offsets
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Offsets [0, 2.5, 3.5, 5.0, 4.6, 3.2, 0] m taken every 10 m from a base line, joined by straight lines (the trapezoidal reading). Marked: the returned areas by the trapezoidal rule 188 m² and Simpson's rule 196.667 m².



A_trapezoidal (area by the trapezoidal rule (m^2)) against d (interval between offsets) from 5 m to 15 m. Outputs with the same unit drawn together: A_simpson (area by Simpson's rule (m^2)). Other inputs as in the worked example: offsets = [0, 2.5, 3.5, 5.0, 4.6, 3.2, 0]. 31 points computed by civil-engineering-ai999x; marked: the worked example, d = 10 m, A_trapezoidal = 188 m^2.

```
civil_engineering_ai999x.solve('area_from_offsets', offsets=[0, 2.5, 3.5, 5.0, 4.6, 3.2,
0], d=10.0)
# -> A_trapezoidal = 188 m^2, A_simpson = 196.667 m^2   (SOLVED)
```

```
civil_engineering_ai999x.solve('area_from_offsets', offsets=[3.2, 5.1, 6.8, 7.5, 6.2, 4.9],
d=15.0)
# -> A_trapezoidal = 444.75 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Area from offsets with offsets = [0, 2.5, 3.5, 5, 4.6, 3.2, 0] and interval = 10 m.	SOLVED	A_trapezoidal = 188 m^2, A_simpson = 196.667 m^2
Area from offsets with offsets = [3.2, 5.1, 6.8, 7.5, 6.2, 4.9] and interval = 15 m.	SOLVED	A_trapezoidal = 444.75 m^2
Area from offsets with offsets = [0, 2.5, 3.5, 5, 4.6, 3.2, 0].	NEEDS_INPUT	area_from_offsets needs d (interval between offsets). Example: solve('area_from_offsets', offsets=[0, 2.5, 3.5, 5.0, 4.6
Area from offsets with offsets = [0, 2.5, 3.5, 5, 4.6, 3.2, 0] and interval = 10 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

Peak runoff by the rational method, `rational_method_peak_runoff`

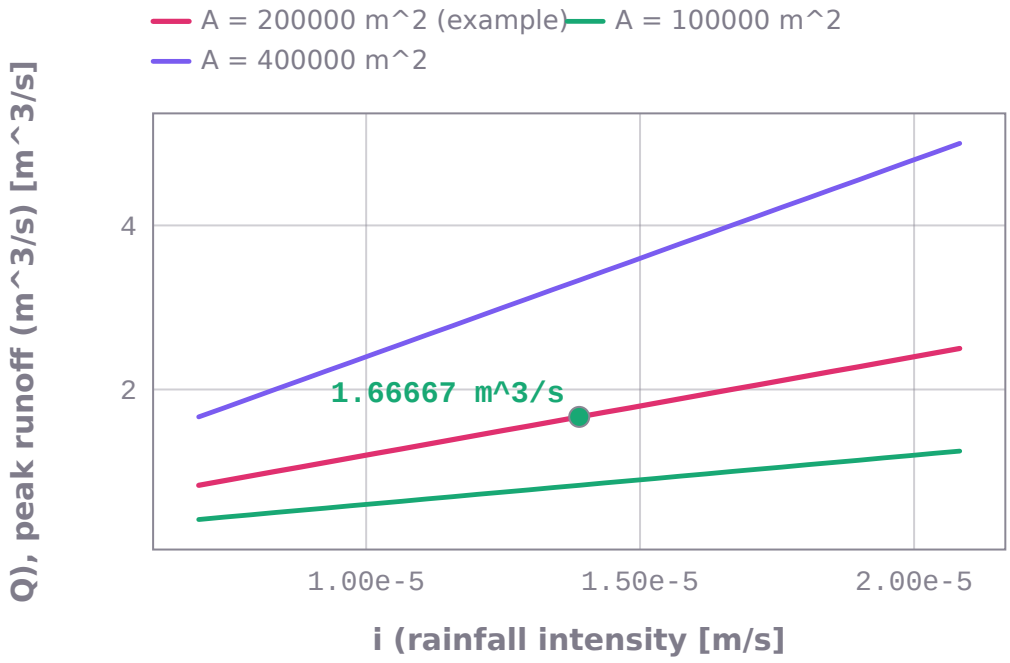
The peak runoff $Q = C$ i A of a small catchment from the runoff coefficient, the rainfall intensity for a duration equal to the time of concentration, and the catchment area.

$$Q = C\,i\,A$$

input	meaning	condition
`C`	runoff coefficient (0 to 1)	
`i`	rainfall intensity	
`A`	catchment area	

output	meaning
`Q`	peak runoff (m^3/s)
`Q_Ls`	peak runoff (L/s)

- **Method:** $Q = C$ i A (SI); = 0.278 C i[mm/h] A[km^2]
- **Assumes:** $0 < C \leq 1$; $A \leq 800$ ha (small catchments); intensity for a duration equal to the time of concentration
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (peak runoff (m^3/s)) against i (rainfall intensity) from 6.9444e-06 m/s to 2.0833e-05 m/s. One curve for each value of A (catchment area): 200000 m^2, 100000 m^2, 400000 m^2. Other inputs as in the worked example: C = 0.6. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, i = 1.3889e-05 m/s, Q = 1.66667 m^3/s.

```
civil_engineering_ai999x.solve('rational_method_peak_runoff', C=0.6, i=1.3888888888888888e-05, A=200000.0)
# -> Q = 1.66667 m^3/s, Q_Ls = 1666.67   (SOLVED)
```

```
civil_engineering_ai999x.solve('rational_method_peak_runoff', C=0.85, i=3.333333333333333e-05, A=25000.0)
# -> Q = 0.708333 m^3/s, Q_Ls = 708.333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rational method peak runoff with runoff coefficient = 0.6, rainfall intensity = 50 mm/h and catchment area = 20 ha.	SOLVED	Q = 1.66667 m^3/s, Q_Ls = 1666.67
Rational method peak runoff with runoff coefficient = 0.85, rainfall intensity = 120 mm/h and catchment area = 2.5 ha.	SOLVED	Q = 0.708333 m^3/s, Q_Ls = 708.333
Rational method peak runoff with runoff coefficient = 0.6 and rainfall intensity = 50 mm/h.	NEEDS_INPUT	rational_method_peak_runoff needs A (catchment area). Example: solve('rational_method_peak_runoff', C=0.6, i='1.38889e-0
Rational method peak runoff with runoff coefficient = 0.6, rainfall intensity = 50 mm/h and catchment area = 20 ha and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Rational method peak runoff with runoff coefficient = 1.5, rainfall intensity = 50 mm/h and catchment area = 20 ha.	INVALID_INPUT	Peak runoff by the rational method: the runoff coefficient must lie in (0, 1].

Time of concentration by the Kirpich formula, `kirpich_time_of_concentration`

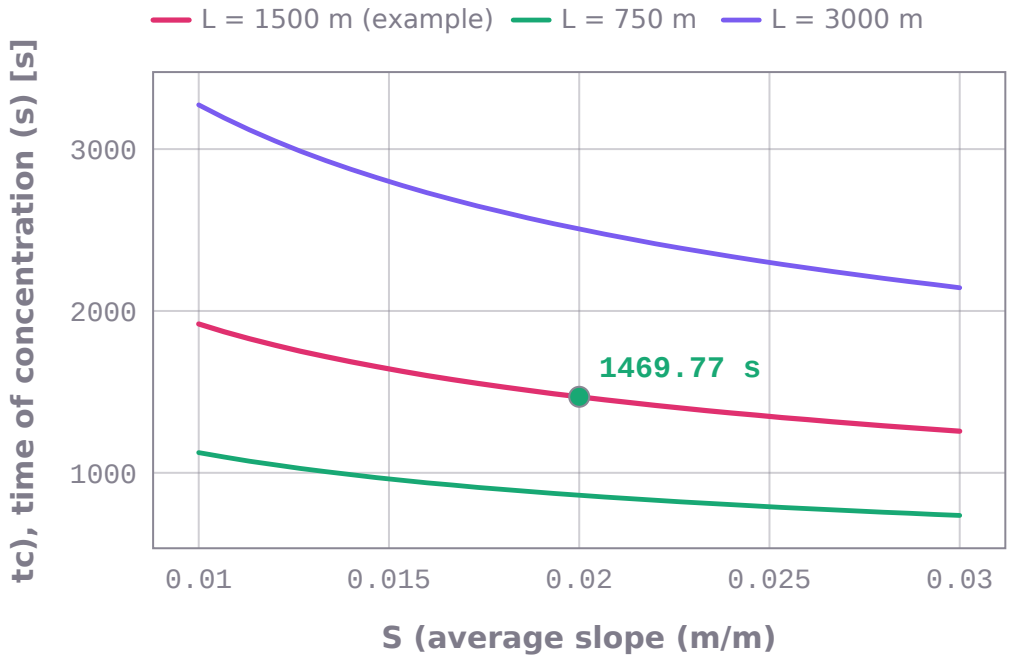
The time of concentration $tc = 0.0078\ L^{0.77}\ S^{-0.385}$ minutes (L in feet; equivalent to 0.0195 $L^{0.77}\ S^{-0.385}$ with L in metres) of a small rural catchment from the flow length and the average slope.

$$t_c = 0.0078\, L^{0.77} S^{-0.385} \ (\text{min, L in ft})$$

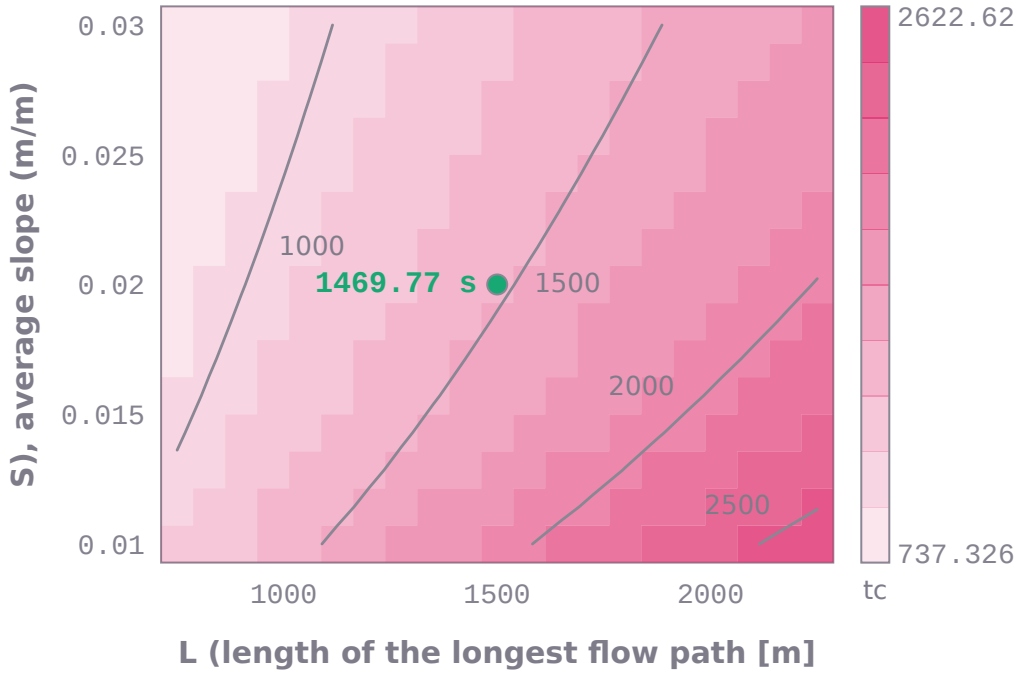
input	meaning	condition
`L`	length of the longest flow path	
`S`	average slope (m/m)	

output	meaning
`tc`	time of concentration (s)
`tc_min`	time of concentration (min)
`tc_h`	time of concentration (h)

- **Method:** $tc = 0.0078\ L_ft^{0.77}\ S^{-0.385}$ min, $L_ft = L/0.3048$
- **Assumes:** $0 < S \leq 1$; $L \leq 100$ km; small rural catchments (Kirpich's data: 0.4 to 45 ha)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tc (time of concentration (s)) against S (average slope (m/m)) from 0.01 to 0.03. One curve for each value of L (length of the longest flow path): 1500 m, 750 m, 3000 m. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, S = 0.02, tc = 1469.77 s.



tc (time of concentration (s)) over L (length of the longest flow path) from 750 m to 2250 m and S (average slope (m/m)) from 0.01 to 0.03: each of the 21 × 15 cells is one run of civil-engineering-ai999x, shaded by its value (light 737.326 s to dark 2622.62 s); iso-lines at tc = 1000, 1500, 2000, 2500 s. Marked: the worked example, L = 1500 m, S = 0.02, tc = 1469.77 s.

```
civil_engineering_ai999x.solve('kirpich_time_of_concentration', L=1500.0, S=0.02)
# -> tc = 1469.77 s, tc_min = 24.4961, tc_h = 0.408268   (SOLVED)
```

```
civil_engineering_ai999x.solve('kirpich_time_of_concentration', L=300.0, S=0.05)
# -> tc = 299.112 s, tc_min = 4.9852, tc_h = 0.0830867   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kirpich time of concentration with flow length = 1500 m and slope = 0.02.	SOLVED	tc = 1469.77 s, tc_min = 24.4961, tc_h = 0.408268
Kirpich time of concentration with flow length = 300 m and slope = 0.05.	SOLVED	tc = 299.112 s, tc_min = 4.9852, tc_h = 0.0830867
Kirpich time of concentration with flow length = 1500 m.	NEEDS_INPUT	kirpich_time_of_concentration needs S (average slope (m/m)). Example: solve('kirpich_time_of_concentration', L='1500 m',
Kirpich time of concentration with flow length = 1500 m and slope = 0.02 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Kirpich time of concentration with flow length = 1500 m and slope = -0.02.	INVALID_INPUT	Time of concentration by the Kirpich formula: S must be positive.

Factor of safety of an infinite slope, `infinite_slope_factor_of_safety`

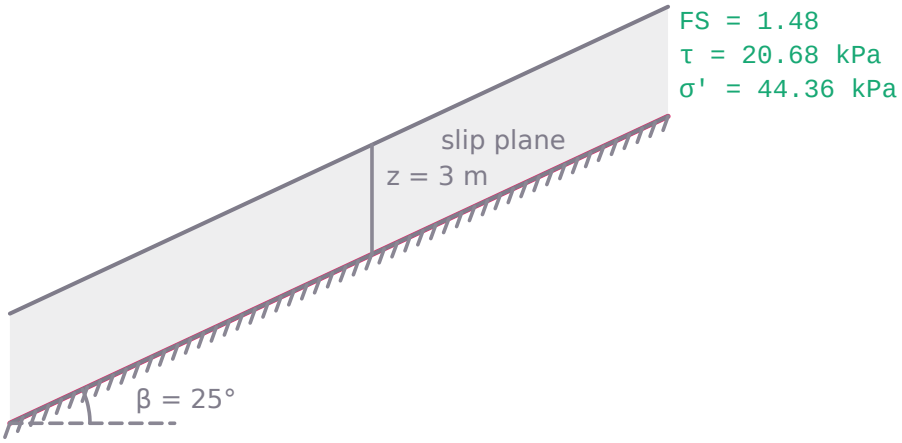
The factor of safety against translational sliding of an infinite slope on a plane at depth z, with seepage parallel to the slope and the water table at a height m z above the slip plane (m = 0 dry, $m = 1$ at the surface).

$$FS = \frac{c' + (\gamma - m\gamma_w)\, z \cos^2\beta \, \tan\phi'}{\gamma z \sin\beta \cos\beta}$$

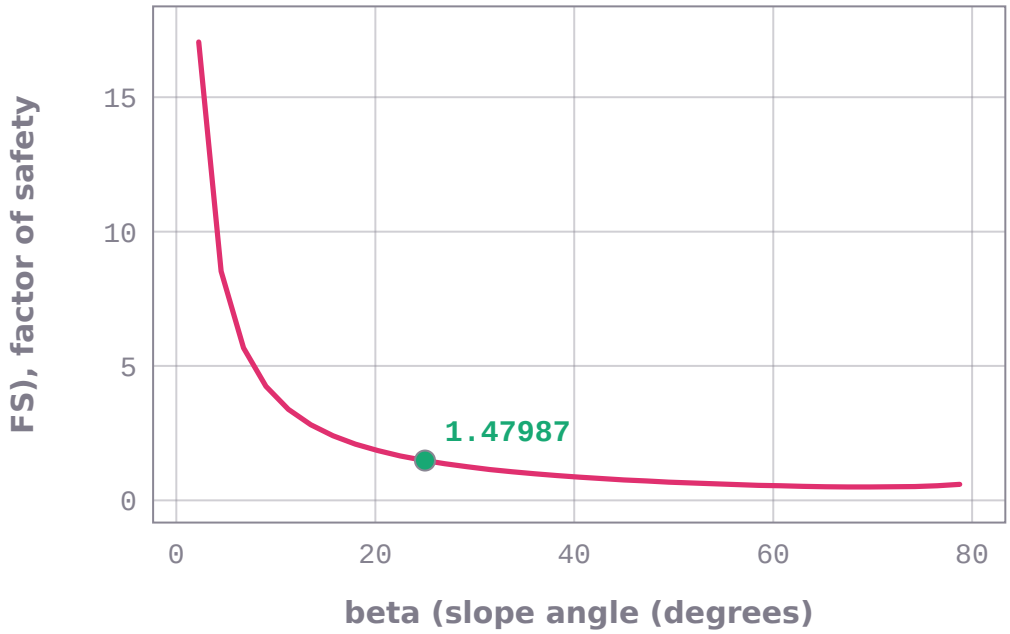
input	meaning	condition
`c`	effective cohesion	
`phi`	effective friction angle (degrees)	
`gam`	unit weight of the soil (saturated below the water table)	
`z`	depth of the slip plane (vertical)	
`beta`	slope angle (degrees)	
`m`	water-table height above the slip plane as a fraction of z (default 0, dry)	

output	meaning
`FS`	factor of safety
`tau`	driving shear stress on the slip plane (Pa)
`sigma_eff`	effective normal stress on the slip plane (Pa)

- **Method:** FS = [c' + (gamma - m gamma_w) z cos^2(beta) tan(phi')]/(gamma z sin(beta) cos(beta))
- **Assumes:** $c \geq 0$; $0 \leq \phi < 60$ degrees; $1 \leq \beta \leq 80$ degrees; $0 \leq m \leq 1$; seepage parallel to the slope
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



An infinite slope at β = 25.0° with a slip plane at vertical depth z = 3 m (c′ = 5 kPa, φ′ = 30.0°, γ = 18 kN/m³, water table ratio m = 0), drawn to scale. Marked: the returned factor of safety FS = 1.47987, driving shear stress 20.6832 kPa and effective normal stress 44.3553 kPa on the slip plane.



FS (factor of safety) against beta (slope angle (degrees)) from 0 to 90. Other inputs as in the worked example: c = 5000.0 Pa, phi = 30.0, gam = 18000.0 N/m^3, z = 3.0 m. 35 points computed by civil-engineering-ai999x; marked: the worked example, beta = 25, FS = 1.47987.

```
civil_engineering_ai999x.solve('infinite_slope_factor_of_safety', c=5000.0, phi=30.0,
gam=18000.0, z=3.0, beta=25.0)
# -> FS = 1.47987, tau = 20683.2 Pa, sigma_eff = 44355.3 Pa   (SOLVED)
```

```
civil_engineering_ai999x.solve('infinite_slope_factor_of_safety', c=0.0, phi=32.0,
gam=20000.0, z=2.0, beta=20.0, m=1.0)
# -> FS = 0.874717, tau = 12855.8 Pa, sigma_eff = 17996 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Infinite slope factor of safety with cohesion = 5 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3, depth = 3 m and slope angle = 25 degrees.	SOLVED	FS = 1.47987, tau = 20683.2 Pa, sigma_eff = 44355.3 Pa
Infinite slope factor of safety with cohesion = 0 kPa, friction angle = 32 degrees, unit weight = 20 kN/m^3, depth = 2 m, slope angle = 20 degrees and water table ratio = 1.	SOLVED	FS = 0.874717, tau = 12855.8 Pa, sigma_eff = 17996 Pa
Infinite slope factor of safety with cohesion = 5 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3 and depth = 3 m.	NEEDS_INPUT	infinite_slope_factor_of_safety needs beta (slope angle (degrees)). Example: solve('infinite_slope_factor_of_safety', c=
Infinite slope factor of safety with cohesion = 5 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3, depth = 3 m and slope angle = 25 degrees and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Infinite slope factor of safety with cohesion = 5 kPa, friction angle = 30 degrees, unit weight = 18 kN/m^3, depth = 3 m and slope angle = 85 degrees.	INVALID_INPUT	Factor of safety of an infinite slope: the slope angle must lie between 1 and 80 degrees.

Terzaghi bearing capacity with stated factors, `terzaghi_bearing_capacity`

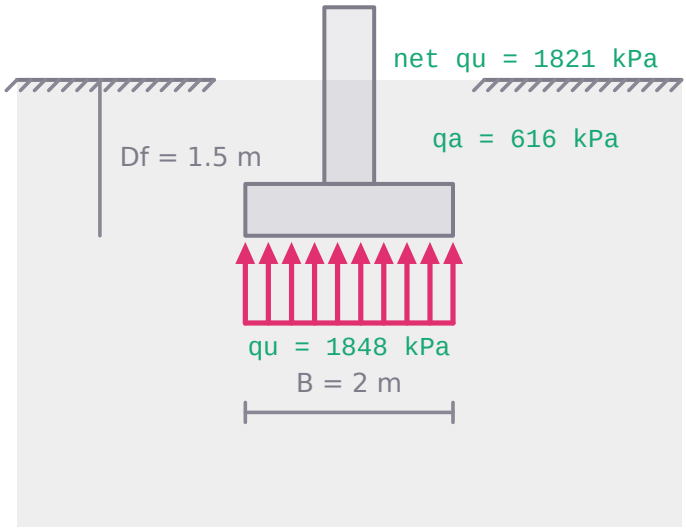
Ultimate, allowable and net bearing capacity of a strip, square or circular footing by Terzaghi's equation with the bearing capacity factors Nc, Nq and Ngamma stated (from a table): $qu = s_c$ c Nc + q Nq + s_g gamma B Ngamma, $s_c = 1.3$ and $s_g = 0.4$ (square) or 0.3 (circular, B = diameter).

$$q_u = s_c\, c N_c + \gamma D_f N_q + s_\gamma\, \gamma B N_\gamma$$

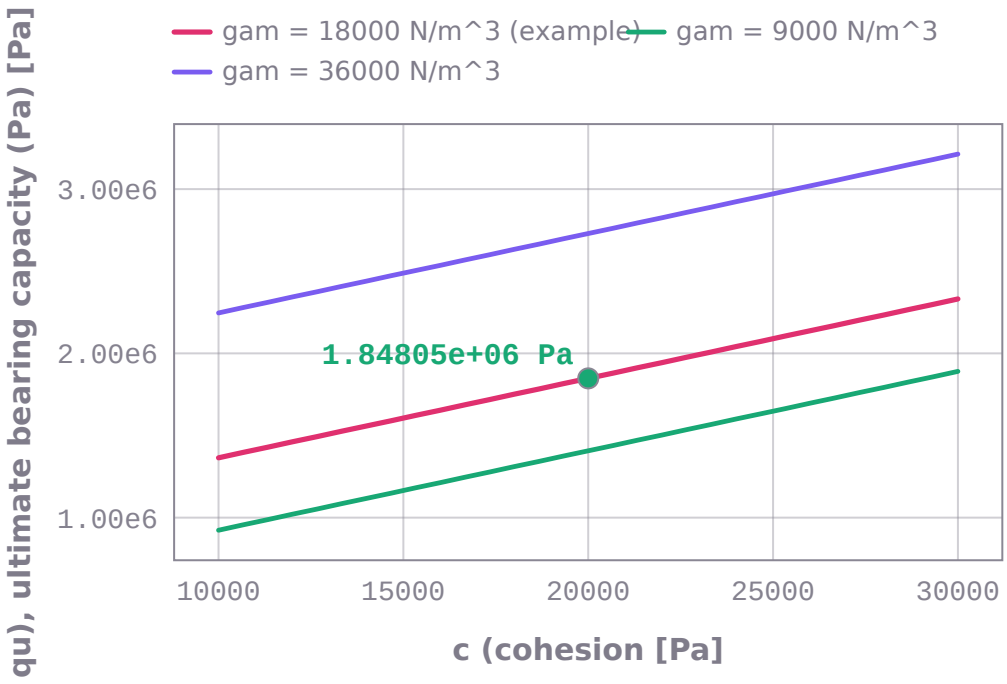
input	meaning	condition
`c`	cohesion	
`gam`	unit weight of the soil	
`B`	footing width (diameter of a circular footing)	
`Df`	embedment depth	
`Nc`	bearing capacity factor Nc (stated)	
`Nq`	bearing capacity factor Nq (stated)	
`Ngamma`	bearing capacity factor Ngamma (stated)	
`shape`	footing shape: strip, square or circular (default strip)	
`FS`	factor of safety (default 3)	

output	meaning
`qu`	ultimate bearing capacity (Pa)
`qa`	allowable bearing capacity qu/FS (Pa)
`q_net`	net ultimate bearing capacity qu - gamma Df (Pa)

- **Method:** $qu = s_c$ c Nc + gamma Df Nq + s_g gamma B Ngamma (strip 1, 0.5; square 1.3, 0.4; circular 1.3, 0.3)
- **Assumes:** c, $Df \geq 0$; factors as stated (Terzaghi's table for the friction angle); general shear failure, vertical central load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A square footing B = 2.0 m founded at Df = 1.5 m (c = 20 kPa, γ = 18 kN/m³, Nc = 37.16, Nq = 22.46, Nγ = 19.13). Marked: the returned ultimate bearing capacity qu = 1848.05 kPa, allowable qa = 616.017 kPa and net 1821.05 kPa. Footing width and depth to scale.



qu (ultimate bearing capacity (Pa)) against c (cohesion) from 10000 Pa to 30000 Pa. One curve for each value of gam (unit weight of the soil): 18000 N/m^3, 9000 N/m^3, 36000 N/m^3. Other inputs as in the worked example: B = 2.0 m, Df = 1.5 m, Nc = 37.16, Nq = 22.46, Ngamma = 19.13, shape = square. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, c = 20000 Pa, qu = 1.84805e+06 Pa.

```
civil_engineering_ai999x.solve('terzaghi_bearing_capacity', c=20000.0, gam=18000.0, B=2.0,
Df=1.5, Nc=37.16, Nq=22.46, Ngamma=19.13, shape='square')
# -> qu = 1.84805e+06 Pa, qa = 616017 Pa, q_net = 1.82105e+06 Pa   (SOLVED)
```

```
civil_engineering_ai999x.solve('terzaghi_bearing_capacity', c=0.0, gam=19000.0, B=1.5,
Df=1.0, Nc=57.75, Nq=41.44, Ngamma=45.41)
# -> qu = 1.43445e+06 Pa, qa = 478151 Pa, q_net = 1.41545e+06 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Terzaghi bearing capacity with cohesion = 20 kPa, unit weight = 18 kN/m^3, width = 2 m, depth = 1.5 m, Nc = 37.16, Nq = 22.46, Ngamma = 19.13 and footing shape = square.	SOLVED	qu = 1.84805e+06 Pa, qa = 616017 Pa, q_net = 1.82105e+06 Pa
Terzaghi bearing capacity with cohesion = 0 kPa, unit weight = 19 kN/m^3, width = 1.5 m, depth = 1 m, Nc = 57.75, Nq = 41.44 and Ngamma = 45.41.	SOLVED	qu = 1.43445e+06 Pa, qa = 478151 Pa, q_net = 1.41545e+06 Pa
Terzaghi bearing capacity with cohesion = 20 kPa, unit weight = 18 kN/m^3, width = 2 m, depth = 1.5 m, Nc = 37.16, Nq = 22.46 and footing shape = square.	NEEDS_INPUT	terzaghi_bearing_capacity needs Ngamma (bearing capacity factor Ngamma (stated)). Example: solve('terzaghi_bearing_capac
Terzaghi bearing capacity with cohesion = 20 kPa, unit weight = 18 kN/m^3, width = 2 m, depth = 1.5 m, Nc = 37.16, Nq = 22.46, Ngamma = 19.13 and footing shape = square and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template

question	outcome	result
Terzaghi bearing capacity with cohesion = 20 kPa, unit weight = 18 kN/m^3, width = 2 m, depth = 1.5 m, Nc = 3, Nq = 22.46 and Ngamma = 19.13.	INVALID_INPUT	Terzaghi bearing capacity with stated factors: the bearing capacity factors must satisfy Nc >= 5.14, Nq >= 1, Ngamma >=

Plasticity, liquidity and consistency indices, `plasticity_liquidity_index`

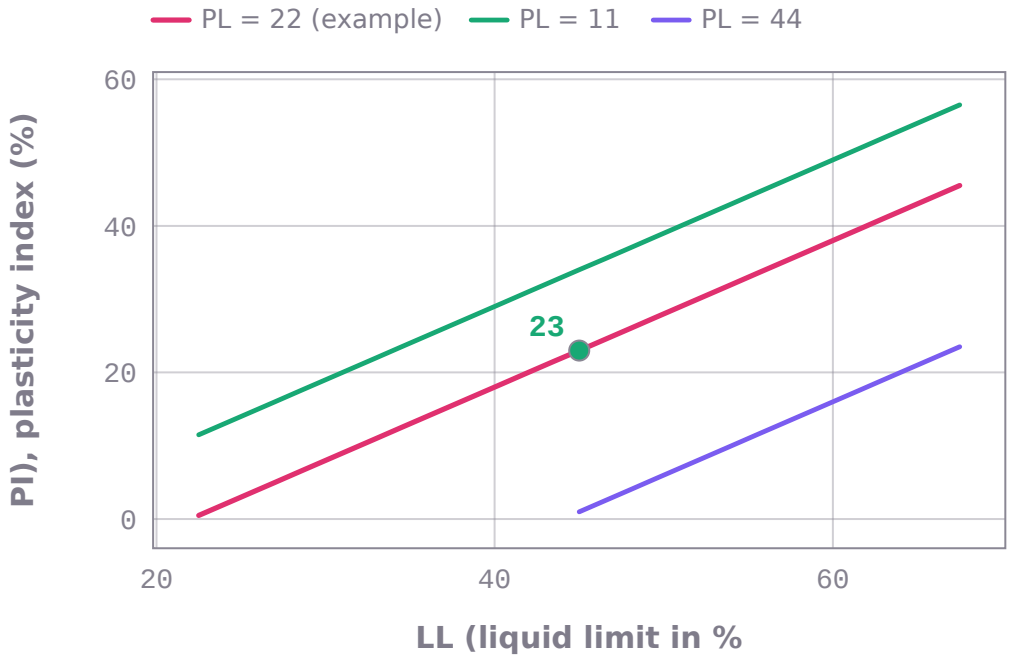
The plasticity index $PI = LL - PL$, the Casagrande A-line value 0.73 $(LL - 20)$ and, with the natural water content, the liquidity index (w - PL)/PI and consistency index (LL - w)/PI. Limits and water content in percent.

$$PI = LL - PL, \quad LI = \frac{w - PL}{PI}, \quad CI = \frac{LL - w}{PI}$$

input	meaning	condition
`LL`	liquid limit in %	
`PL`	plastic limit in %	
`w`	natural water content in % (optional)	

output	meaning
`PI`	plasticity index (%)
`A_line_PI`	plasticity index on the A-line at this LL (%)
`LI`	liquidity index
`CI`	consistency index

- **Method:** $PI = LL - PL$; $LI = (w - PL)/PI$; $CI = (LL - w)/PI$; A-line $PI = 0.73\,(LL - 20)$
- **Assumes:** $5 \le LL \le 1000$ %; $0 \le PL < LL$; inputs in percent
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



PI (plasticity index (%)) against LL (liquid limit in %) from 22.5 to 67.5. One curve for each value of PL (plastic limit in %): 22, 11, 44. Other inputs as in the worked example: w = 30.0. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, LL = 45, PI = 23.

```
civil_engineering_ai999x.solve('plasticity_liquidity_index', LL=45.0, PL=22.0, w=30.0)
# -> PI = 23, A_line_PI = 18.25, LI = 0.347826, CI = 0.652174   (SOLVED)
```

```
civil_engineering_ai999x.solve('plasticity_liquidity_index', LL=62.0, PL=28.0)
# -> PI = 34, A_line_PI = 30.66   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plasticity and liquidity index with liquid limit = 45 %, plastic limit = 22 % and water content = 30 %.	SOLVED	PI = 23, A_line_PI = 18.25, LI = 0.347826, CI = 0.652174
Plasticity and liquidity index with liquid limit = 62 % and plastic limit = 28 %.	SOLVED	PI = 34, A_line_PI = 30.66
Plasticity and liquidity index with liquid limit = 45 % and water content = 30 %.	NEEDS_INPUT	plasticity_liquidity_index needs PL (plastic limit in %). Example: solve('plasticity_liquidity_index', LL=45, PL=22, w=3
Plasticity and liquidity index with liquid limit = 45 %, plastic limit = 22 % and water content = 30 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Plasticity and liquidity index with liquid limit = 30 % and plastic limit = 35 %.	INVALID_INPUT	Plasticity, liquidity and consistency indices: the plastic limit must lie below the liquid limit.

Secondary compression (creep) settlement of a clay layer, `secondary_compression_settlement`

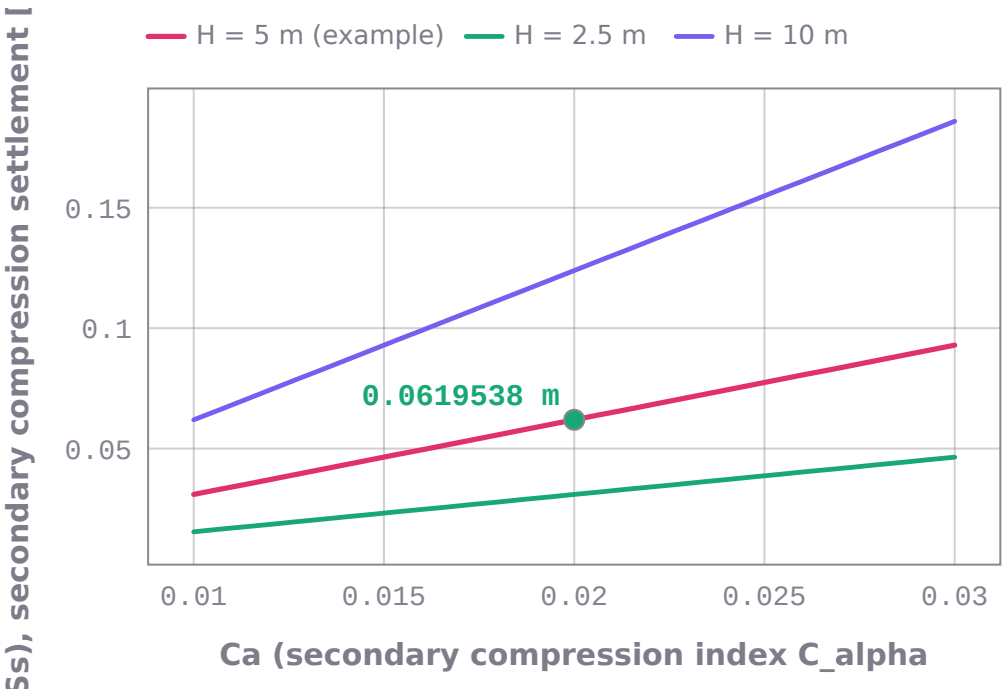
The secondary compression settlement $S_s = C_alpha\ H/(1+e_p)$ log10(t2/t1) of a clay layer of thickness H after the end of primary consolidation t1, up to a later time t2.

$$S_s = \frac{C_\alpha H}{1+e_p}\log_{10}\frac{t_2}{t_1}$$

input	meaning	condition
`Ca`	secondary compression index C_alpha	
`H`	layer thickness (at the end of primary consolidation)	
`ep`	void ratio at the end of primary consolidation	
`t1`	time at the end of primary consolidation	
`t2`	later time	

output	meaning
`Ss`	secondary compression settlement (m)
`Ss_mm`	settlement (mm)
`C_alpha_mod`	modified secondary compression index $C_alpha/(1+e_p)$

- **Method:** $S_s = C_alpha\ H/(1+e_p)$ log10(t2/t1)
- **Assumes:** $0 < C_alpha \le 0.2$; $t_2 > t_1$ (t1 = end of primary consolidation)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ss (secondary compression settlement (m)) against Ca (secondary compression index C_alpha) from 0.01 to 0.03. One curve for each value of H (layer thickness (at the end of primary consolidation)): 5 m, 2.5 m, 10 m. Other inputs as in the worked example: ep = 1.1, t1 = 31557600.0 s, t2 = 631152000.0 s. 31 points per curve computed by civil-engineering-ai999x; marked: the worked example, Ca = 0.02, Ss = 0.0619538 m.

```
civil_engineering_ai999x.solve('secondary_compression_settlement', Ca=0.02, H=5.0, ep=1.1,
t1=31557600.0, t2=631152000.0)
# -> Ss = 0.0619538 m, Ss_mm = 61.9538, C_alpha_mod = 0.00952381   (SOLVED)
```

```
civil_engineering_ai999x.solve('secondary_compression_settlement', Ca=0.008, H=3.0, ep=0.8,
t1=63115200.0, t2=946728000.0)
# -> Ss = 0.0156812 m, Ss_mm = 15.6812, C_alpha_mod = 0.00444444   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Secondary compression settlement with secondary compression index = 0.02, thickness = 5 m, void ratio = 1.1, end of primary = 1 yr and later time = 20 yr.	SOLVED	Ss = 0.0619538 m, Ss_mm = 61.9538, C_alpha_mod = 0.00952381
Secondary compression settlement with secondary compression index = 0.008, thickness = 3 m, void ratio = 0.8, end of primary = 2 yr and later time = 30 yr.	SOLVED	Ss = 0.0156812 m, Ss_mm = 15.6812, C_alpha_mod = 0.00444444
Secondary compression settlement with secondary compression index = 0.02, thickness = 5 m, void ratio = 1.1 and end of primary = 1 yr.	NEEDS_INPUT	secondary_compression_settlement needs t2 (later time). Example: solve('secondary_compression_settlement', Ca=0.02, H='5
Secondary compression settlement with secondary compression index = 0.02, thickness = 5 m, void ratio = 1.1, end of primary = 1 yr and later time = 20 yr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: civil_engineering_ai999x.template
Secondary compression settlement with secondary compression index = 0.02, thickness = 5 m, void ratio = 1.1, end of primary = 20 yr and later time = 1 yr.	INVALID_INPUT	Secondary compression (creep) settlement of a clay layer: the later time must exceed the end of primary consolidation.